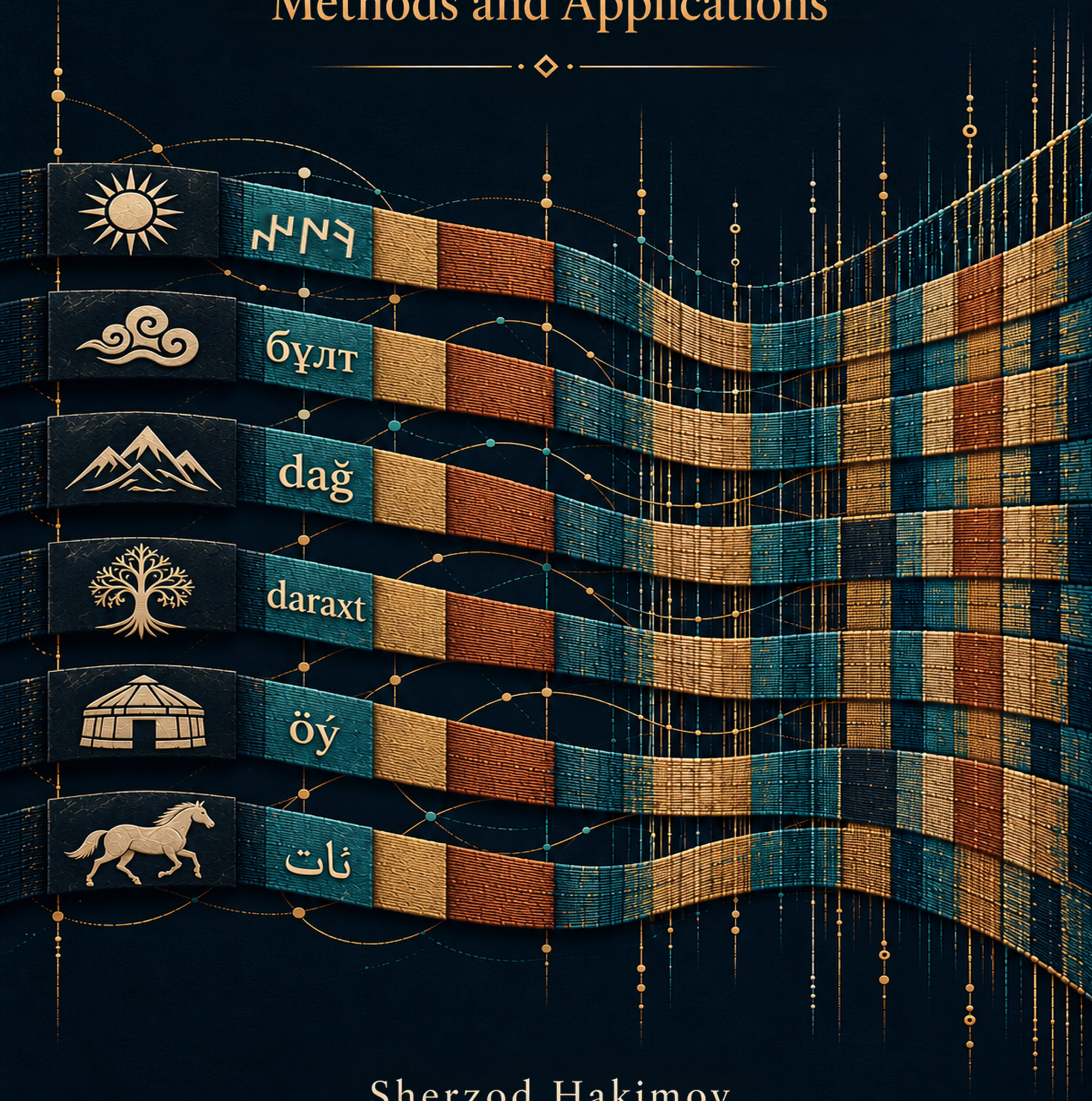
Natural Language Processing Across Turkic Languages: Methods and Applications



Sherzod Hakimov

Natural Language Processing Across Turkic Languages: Methods and Applications

Sherzod Hakimov
Version 0.1.0, 2026
DOI: 10.5281/zenodo.23324599

This book is intended for NLP researchers and practitioners, graduate students, and members of Turkic language communities who want to build or improve language technology.

© 2026 Sherzod Hakimov.

Unless otherwise noted, the text and figures in this book are licensed under the Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0) license. The program code in the book's repository is licensed under the Apache License 2.0.

https://creativecommons.org/licenses/by-sa/4.0/

Preface

Turkic languages are spoken from the Balkans and Anatolia to Central Asia, western China and north-eastern Siberia. They share much of their grammar, yet the data, tools and evaluations available for them differ greatly from one language and task to the next. This book explains how to build and assess language technology with both facts in view.

General NLP is treated briefly, with Jurafsky and Martin's *Speech and Language Processing* as the companion text. The chapters concentrate on what is specific to the family: agglutinative morphology and vowel harmony, the Latin, Cyrillic and Perso-Arabic scripts in which the languages are written, transfer between related languages, and the construction of corpora, treebanks and benchmarks where few exist. Johanson's *Turkic* is the linguistic reference throughout. Three questions recur: how morphology affects a task, what changes when the script changes, and which knowledge transfers between related languages.

The book is written for NLP researchers and practitioners, graduate students and members of Turkic language communities. The programming sections assume intermediate Python and basic machine learning; linguistic concepts are introduced as they are needed. Section 1.6 describes how the parts fit together.

Each chapter combines linguistic analysis with a practical section, most of them using TurkicNLP (version 0.5.0), an open toolkit that brings components for Turkic languages into one interface.

AI tools were used in preparing this book: for drafting and editing the text, reviewing code, checking the bibliography against publisher and index records, and style passes over the manuscript. The cover artwork was generated with an AI image tool and then edited by hand. All AI-assisted material was reviewed, corrected and checked manually.

The resources described here will change faster than the methods. The book aims to help its readers check what exists, measure what works, and build what their language communities need.

Table of contents

Preface 3

I The Language Family 21

1 The Turkic World: Languages, Speakers, and the Resource Reality 22
1.1 Geographic and Demographic Overview . . . 22
1.2 Genealogical Classification . . . 23
1.3 Mutual Intelligibility and What It Means Computationally . . . 25
1.4 Available Resources . . . 26
1.5 Key Institutions and Community Infrastructure . . . 28
1.6 Book Organisation and How to Use This Book . . . 28
1.7 Hands-On: A Procedure for Building a Task-Specific Inventory . . . 30
What to Take Forward . . . 31
Further Reading . . . 31
Exercises . . . 32

2 The Computational Phenotype of Turkic Languages 34
2.1 Agglutination and Productive Word Formation . . . 34
2.2 Vowel Harmony: Rules, Exceptions, and Computational Consequences . . . 39
2.3 The Case System as Syntactic Infrastructure . . . 43
2.4 SOV Base Order and Scrambling . . . 45
2.5 Non-Finite Subordination: Participial and Converbal Clauses . . . 47
2.6 Shared Patterns and Variation . . . 49
2.7 Hands-On: Counting Segments in Turkish, Kazakh and Turkmen . . . 51
What to Take Forward . . . 54
Further Reading . . . 54
Exercises . . . 54

II Writing Systems and Corpora 56

3 Scripts, Encoding, and Orthographic Engineering 57
3.1 A Map of Turkic Scripts . . . 57
3.2 The Active Transitions . . . 60
3.3 Unicode for Turkic Languages . . . 61
3.4 Bidirectional Transliteration . . . 67
3.5 Text Normalisation for Turkic . . . 72
3.6 A Common Script as an Interoperability Layer . . . 75
3.7 Language Identification and Corpus Construction . . . 80
3.8 Hands-On: Checking a Text Pipeline . . . 82

What to Take Forward . . . 83
Further Reading . . . 83
Exercises . . . 84

4 Building Turkic Corpora at Scale **86**
4.1 What Exists: Turkic Text in Multilingual Web Corpora . . . 86
4.2 What the Labels Hide: Auditing a Corpus . . . 95
4.3 A Pipeline for Turkic Text . . . 98
4.4 Choosing a Script for the Corpus . . . 101
4.5 Parallel and Comparable Data . . . 102
4.6 Licensing, Provenance, and Sovereignty . . . 102
4.7 Hands-On: Auditing a Corpus Sample . . . 103
What to Take Forward . . . 103
Further Reading . . . 104
Exercises . . . 104

III Morphology **105**

5 Morphological Structure: A Linguistic Foundation **106**
5.1 The Morpheme Inventory . . . 106
5.2 Morphophonological Alternations . . . 112
5.3 What a Full Morphological Analysis Looks Like . . . 115
5.4 Cross-Language Morphological Comparison . . . 119
5.5 Hands-On: Walking Through a Morphological Analysis . . . 122
What to Take Forward . . . 124
Further Reading . . . 124
Exercises . . . 124

6 Finite-State Morphology: The Rule-Based Approach **126**
6.1 Why Finite-State Methods for Turkic . . . 126
6.2 Two-Level Morphology: The Core Formalism . . . 128
6.3 The Apertium Turkic Project . . . 131
6.4 Language-Specific Tools . . . 135
6.5 Morphological Generation and Data Augmentation . . . 137
6.6 Limitations of the Finite-State Approach . . . 140
6.7 Hands-On: A Procedure for Comparing Rule-Based and Neural Analysis 141
What to Take Forward . . . 142
Further Reading . . . 142
Exercises . . . 143

7 Neural Morphology and the Tokenisation Problem **145**
7.1 Subword Tokenisation in Agglutinative Languages . . . 145
7.2 Morpheme-Aware Tokenisation . . . 149
7.3 Neural Morphological Analysis . . . 152
7.4 Morphological Disambiguation . . . 154
7.5 Zero-Shot Morphological Tagging for Unseen Languages . . . 156
7.6 Hands-On: A Fine-Tuning Procedure for Morphological Tagging with BERTurk . . . 159

What to Take Forward . . . 162
Further Reading . . . 162
Exercises . . . 163

IV Syntax **164**

8 Syntactic Structure and Universal Dependencies **165**
8.1 A Computational Sketch of Turkic Syntax . . . 165
8.2 The Universal Dependencies Framework for Turkic . . . 170
8.3 Turkic Treebanks . . . 175
8.4 Dependency Parsing for Turkic . . . 178
8.5 Cross-Lingual Parsing . . . 183
8.6 Building a Treebank for a New Turkic Language . . . 184
8.7 Hands-On: Parsing Turkish Text with TurkicNLP . . . 185
What to Take Forward . . . 189
Further Reading . . . 189
Exercises . . . 190

V Applications **192**

9 Sequence Labelling: POS Tagging, NER, and Chunking **193**
9.1 POS Tagging and Morphological Analysis . . . 194
9.2 Named Entity Recognition in Agglutinative Languages . . . 195
9.3 Cross-Lingual NER Transfer . . . 201
9.4 Other Sequence Labelling Tasks . . . 202
9.5 Pretrained Taggers and Recognisers in TurkicNLP . . . 203
9.6 Hands-On: A Fine-Tuning Procedure for Kazakh NER with XLM-R . . . 205
What to Take Forward . . . 212
Further Reading . . . 212
Exercises . . . 212

10 Machine Translation: Within and Beyond the Family **215**
10.1 The Challenges of Turkic MT . . . 215
10.2 Rule-Based MT: Apertium . . . 217
10.3 Neural MT for Turkic Languages . . . 219
10.4 Intra-Family Translation . . . 222
10.5 Low-Resource Neural MT Strategies . . . 224
10.6 Hands-On: A Fine-Tuning Procedure for Kazakh–Kyrgyz Translation with NLLB-200 . . . 225
What to Take Forward . . . 227
Further Reading . . . 227
Exercises . . . 228

11 Speech Technologies **230**
11.1 Phonological Sketch of Turkic Languages . . . 230
11.2 Automatic Speech Recognition . . . 232
11.3 Text-to-Speech . . . 234

11.4 Speech for Low-Resource Languages . . . 235
11.5 Hands-On: A Fine-Tuning Procedure for Kazakh ASR with Whisper . . . 237
What to Take Forward . . . 239
Further Reading . . . 239
Exercises . . . 239

VI Language Models **241**

12 Pretrained Language Models for Turkic **242**
12.1 Encoder-Only Models for Turkic Languages . . . 242
12.2 Generative Language Models for Turkic . . . 244
12.3 Building Turkic Language Models . . . 246
12.4 The Tokeniser Problem for Large Language Models . . . 248
12.5 Hands-On: A LoRA Fine-Tuning Procedure for Kazakh Text Classification with XLM-R . . . 250
What to Take Forward . . . 251
Further Reading . . . 251
Exercises . . . 252

13 Meaning: Embeddings, Semantic Resources, and Retrieval **253**
13.1 Lexical-Semantic Resources . . . 253
13.2 Word Embeddings and Morphology . . . 255
13.3 Sentence Encoders . . . 255
13.4 Keyword Search in Agglutinative Languages . . . 259
13.5 Retrieval across Scripts and Languages . . . 263
13.6 Retrieval-Augmented Generation for Turkic Languages . . . 264
13.7 Hands-On: A Procedure for Finding Translations Across the Family . . . 267
What to Take Forward . . . 268
Further Reading . . . 268
Exercises . . . 269

14 Generation, Dialogue, and Interactive Evaluation **270**
14.1 What Goes Wrong When Models Write Turkic Text . . . 270
14.2 Measuring Fidelity Automatically . . . 271
14.3 Generation Tasks and Data . . . 272
14.4 Evaluating Generated Text . . . 273
14.5 Interactive Evaluation with Dialogue Games . . . 274
14.6 Language Drift in Ordinary Chat . . . 276
14.7 Spoken Dialogue . . . 276
14.8 Hands-On: A Procedure for Kazakh Taboo with a Morphological Rule Checker . . . 276
What to Take Forward . . . 277
Further Reading . . . 277
Exercises . . . 278

15 Evaluation, Benchmarks, and Responsible Deployment **279**
15.1 Principles for Reading and Reporting Results . . . 279
15.2 Evaluation Challenges for Agglutinative Languages . . . 280

15.3 Existing Benchmarks . . . 281
15.4 Human Evaluation for Turkic NLP . . . 284
15.5 Responsible Deployment . . . 285
15.6 Hands-On: Evaluating a Multilingual Model on TUMLU . . . 286
What to Take Forward . . . 288
Further Reading . . . 288
Exercises . . . 288

VII Cross-Lingual Methods and the Resource Frontier 290

16 Cross-Lingual Transfer within the Turkic Family 291
16.1 Why Related Languages Can Help . . . 291
16.2 Script Conversion as an Experimental Condition . . . 292
16.3 Transfer Strategy by Available Evidence . . . 292
16.4 Multilingual Turkic Training . . . 293
16.5 Pivot Translation as Data Augmentation . . . 294
16.6 Code-Switched and Mixed Text . . . 296
16.7 Hands-On: Comparing Native Text and Script Conversion . . . 296
What to Take Forward . . . 297
Further Reading . . . 298
Exercises . . . 298

17 Endangered Languages, Digital Preservation, and Ethics 299
17.1 What a Resource Gap Means . . . 299
17.2 Digital Preservation Strategies . . . 300
17.3 Building a Useful First System . . . 301
17.4 Community, Consent, and Data Sovereignty . . . 302
17.5 Open Problems and Research Priorities . . . 303
17.6 Hands-On: Building a Turkic Language Resource Audit . . . 304
What to Take Forward . . . 305
Further Reading . . . 306
Exercises . . . 306

References 307

Appendices 320

A Mathematical Foundations 320
A.1 Linear Algebra . . . 320
A.2 Probability and Information Theory . . . 321
A.3 Optimisation . . . 323
A.4 Neural Network Basics . . . 324
Further Reading . . . 327
Worked Checks . . . 327

B Computational Resources and Tools 328
B.1 Environment Setup . . . 328

B.2 Key Libraries and Frameworks 329
B.3 Datasets and Corpora 331
B.4 Pretrained Models 335
B.5 Community and Infrastructure 336
B.6 Reproducing Results from This Book 336

C Turkic Language Reference **338**
C.1 Language Identifiers and Scope 338
C.2 NLP Resource Evidence 339
C.3 Script Reference 340
C.4 Typology and Computational Interpretation 341

D Glossary **343**
Gloss Abbreviations 343

List of Tables

1.1 Branches and subbranches of the Turkic language family, following Johanson (2021, 22–23) . . . 23
1.2 Selected living Turkic languages, with branch affiliation and script overview (Johanson 2021, 22–25, chapter 19); identifiers follow Appendix C. Regional script variants are not exhaustive. . . . 24
1.3 Named examples for a task-oriented resource audit. Treat it as a reading list. It neither exhausts the field nor ranks the work in it. . . . 27
1.4 The recurring example sentences. Each is a UD 2.17 test sentence; the treebanks are licensed CC BY-SA 4.0. Translations are the book's. Morphological glosses are in Section 5.4.4. . . . 30
1.5 A filled inventory row for one language and task. . . . 31

2.1 The Turkish word *evlerimizden* 'from our houses'. Gloss abbreviations follow the Leipzig Glossing Rules: PL plural, POSS possessive, ABL ablative, ACC accusative, GEN genitive, PASS passive, NEG negation, PST past, 3PL third person plural; the full list is in Table D.1. . . . 35
2.2 Surface pieces of *konuşturamadıklarımızdan*, Turkish. After the nominaliser *-DIK* the word continues with nominal inflection (plural, possessive-form agreement, case). The stem *konuş-* is treated as one unit, and *-a-ma-* is split under the inability convention introduced above. 36
2.3 Plural and accusative suffixes on the same stems ('house', 'horse', 'eye', 'arm/hand') in five Turkic languages, in standard spelling. The columns contrast front and back stems (*houses* vs. *horses*) and rounded stems (*eye*, *arm/hand*). . . . 40
2.4 Vowel inventories under the cited analyses. Symbols follow Johanson's Turkological notation, not a uniform IPA transcription. Orthographic correspondences do not imply identical pronunciation across languages. 41
2.5 The word "house" (*ev/yw/uy/öý*) in six cases across five Turkic languages. Note allomorphic variation due to vowel harmony and consonant assimilation: Turkmen writes a rounded vowel in the genitive (*öýüň*) but not in the accusative (*öýi*). Latin forms in parentheses are a broad transliteration (*i* = i, *ŭ* = y, *ң* = ñ), not the 2021 Kazakh Latin alphabet used by the toolkit. . . . 43
2.6 Recurring features of the Turkic profile (Johanson 2021, 34–39). . . . 50

3.1 Selected language–script relationships. Historical dates follow the cited accounts; script prevalence and current legal status are not inferred from them. . . . 58

3.2 Unicode blocks relevant to Turkic language NLP. Only the most frequently encountered ranges are shown. Note that the Latin schwa is split across two blocks (capital Ə in Latin Extended-B, small ə in IPA Extensions), and that the Cyrillic Ә (U+04D8) is a different character that looks the same. 62
3.5 Transliteration pairs and mapping standards in TurkicNLP 0.5.0. Ottoman Turkish (Latin transcription to Perso-Arabic) and Old Turkic (runes to Latin transcription) are one-way reconstructions, not reversible conversions. 68
3.6 CTS base alphabet and the additional Éé pair for Uyghur, from the frozen local TurkicNLP source identifying itself as 0.5.0 (13 October 2026). Source hash and all CTS mappings are recorded in `tables/tbl-cts-alphabet/source.json`; the package version alone does not identify this local source revision. 76
3.7 Fertility on the parallel challenge sample of Hakimov (2026). Group means use full-precision values, not the rounded displayed cells. The OpenAI column is the measured `o200k_base` encoding, not an inference-time model measurement. 78
3.8 Language identification for eight smaller Turkic languages: GlotLID restricted to Turkic labels, 20 parallel challenge sentences per language (Hakimov 2026). Counts refer to the most frequent recorded wrong label, not all errors. 81

4.1 Turkic subsets of HPLT v3.0 (sizes as labelled by the corpus's language identifier, not audited). Russian and English are shown for reference. Not present in HPLT v3.0: Gagauz, Ottoman Turkish, Karakalpak, Nogai, Kumyk, Karachay-Balkar, Sakha, Altai, Tuvan, Khakas, Chuvash, Khalaj, Old Turkic. Source: HPLT v3.0 manifest, accessed 12 October 2026. . 87
4.2 Documents per Turkic language in eight corpora (mC4: pages; Wikipedia: articles; CC-100 reports tokens only, so ✓ marks presence). HPLT v1.2 and v2.0: deduplicated releases; v3.0: sorted release. Sizes are as labelled by each corpus's language identifier and not audited. FineWeb2 sums all script subsets (train split, after filtering). Absent from all eight: Khalaj, Old Turkic. Sources: HPLT v1.2/v2.0/v3.0 download pages and manifest, FineWeb2 language-distribution CSV, CulturaX dataset card, mT5 and XLM-R papers (Table 6 each), MediaWiki statistics API; all accessed 12 October 2026. . . . 91
4.3 The Turkic subsets across three HPLT releases: v1.2 deduplicated; v2.0 deduplicated (dedup.) and cleaned; v3.0 sorted (Web Docs Scorer bins 5–10 only). v2.0's cleaned release drops, among others, documents with a language-identification probability below 0.5 or a Web Docs Scorer value below 5 (Burchell et al. 2025, sec. 4.2). Because v3.0 also contains only bins 5–10, v2.0's cleaned release is the closer comparison on that criterion; the other v3.0 filters were not checked here. v1.2 does not report characters. Sources: hplt-project.org/datasets/v1.2 and /v2.0, HPLT v3.0 manifest; accessed 12 October 2026. 93

4.4 Quality-filter thresholds that FineWeb2 estimated for the primary Turkic subsets, against the English defaults of the same datatrove filters. Avg word length: allowed range of a document's mean word length; alpha words: minimum share of words containing a letter (called `max_non_alpha_words_ratio` in datatrove); line punct.: minimum share of lines ending in punctuation (off = disabled; a threshold of 0.0 has the same effect); dup. lines: maximum share of duplicated lines; top 2-gram: maximum share of characters in the most frequent word bigram (values above 1 disable the rule); LID score: minimum language-ID confidence; stop words: size of the list (a document needs at least 2 of them). Sources: github.com/huggingface/fineweb-2 `configs/*.yml` and datatrove filter defaults (main branch), accessed 12 October 2026. 99

5.1 Selected derivational suffixes in Turkish 107
5.3 Turkish possessive suffixes after consonant- and vowel-final stems. *I* = four-way harmony (ı/i/u/ü), *A* = two-way harmony (a/e). With a plural noun the plural comes first: *ev-ler-im* 'my houses'. 110
5.5 Turkish verbal voice suffixes (allomorphs vary by phonological context) 111
5.6 Selected Turkish TAM suffixes. *A* = a/e, *I* = ı/i/u/ü, *D* = d/t according to voicing; *(y)* appears where required. 112
5.7 Turkish verbal agreement: the two main paradigms, shown with *gel-* 'come' (Johanson 2021, 439). 112
5.11 Three levels of representation for *evlerimizden*. The Apertium reading is `ev<n><pl><px1pl><abl>` (TurkicNLP 0.5.0). 115
5.14 Selected morphological contrasts across five Turkic languages, as illustrated in Chapter 2 and Chapter 5. 120

6.1 Small diagnostic probe results (Det% and non-empty-feature rate) for Apertium and multilingual neural morphological analysis on the 20-sentence parallel challenge set (Hakimov 2026). Not a measure of analysis accuracy. Det%: sentences in which the target UD feature is detected; Feat%: content words receiving at least one feature (coverage, not feature accuracy). †Zero-shot for the neural model. Bold: higher value per metric. Published rounded averages are preserved. 132
6.2 Selected Turkish morphological tools; repository code licences checked 14 October 2026 and again 19 October 2026. Model and data licences require separate inspection. Google's repository is archived. 137

7.1 Tokeniser fertility (mean tokens per word) of four LLM tokenisers on the 20 parallel challenge sentences of Hakimov (2026), each word tokenised separately. Small samples (61–88 words per language): read the table as a diagnostic, not a benchmark. 147
7.2 Tokens for the recurring sentences (Table 1.4) in the native script and in CTS, including punctuation and without special tokens; words are whitespace-separated. 148
7.3 A minimal evaluation suite for Turkic tokenisers. 151
7.4 Three readings of the Turkish form *yüz*, as the Apertium Turkish transducer returns them (it also returns substantivised and copular variants). Context selects among them. 154

7.5 Questions for evaluating cross-lingual transfer of morphological features. Annotation-schema compatibility (whether source and target treebanks annotate the same features the same way) is a separate question from linguistic similarity. 157

8.1 The six constituent orders of a simple Turkish transitive sentence. SOV is the neutral order; the other five occur in suitable discourse contexts. The accusative suffix *-ı* on *kitab-ı* ("book-ACC") identifies the object in each order. Pragmatic labels (topic, focus, backgrounding) are deliberately not assigned per row, because they depend on a discourse context that a table cannot show. 166

8.2 Core Universal Dependencies morphological features heavily used in Turkic treebanks. The VerbForm feature is especially important: participial (Part) and converbal (Conv) forms are the primary mechanisms for subordinate clauses. 170

8.3 Head direction of `compound:lvc` in the official UD 2.17 Turkic treebanks (all splits pooled; treebanks with no such arc omitted). "Nominal head" means any head whose UPOS is not VERB or AUX, so the light verb depends on the noun or adjective. Counts are an annotation-convention diagnostic, not a parser result. Generated by `tables/tbl-ud217-lvc-direction/generate.py` from hash-verified release files; URLs and hashes are in `data.csv`. 171

8.4 Basic dependency distance in selected official UD 2.17 test sets, measured in integer-ID word positions. Root and punctuation-dependent arcs are excluded; intervening punctuation still occupies a position. P90 uses nearest rank. All 25 treebanks and source hashes are in the accompanying CSV. These are corpus diagnostics, not parser scores or matched language comparisons. 173

8.5 UD 2.17 inventory, generated from the release CoNLL-U files by `tables/tbl-ud217-inventory/generate.py`; file hashes are recorded alongside the CSV. A zero means no sentences in that split. Word and token totals sum all splits; multiword-token ranges count once in the token total, and empty nodes count in neither. Historical and code-switched treebanks retain their UD language names. 175

8.6 UD 2.17 inventory, generated from the release CoNLL-U files by `tables/tbl-ud217-inventory/generate.py`; file hashes are recorded alongside the CSV. A zero means no sentences in that split. Word and token totals sum all splits; multiword-token ranges count once in the token total, and empty nodes count in neither. Historical and code-switched treebanks retain their UD language names. 176

8.7 POS tagging and dependency parsing from the frozen local manuscript table of 13 October 2026 (`tables/tbl-parsing-results/source.tex`), associated with Hakimov (2026). This snapshot differs from the public arXiv table in some cells and sample sizes. Both backends receive the same automatic tokenisation. These are *conditional* scores, not the official tokenisation-aware CoNLL 2018 LAS F1: UPOS is computed over the n words whose boundaries match gold, and UAS/LAS additionally require matched gold and predicted heads, so tokenisation errors drop out of the denominator. †Zero-shot with proxy embeddings: Karakalpak → Uzbek, Kumyk → Tatar, Sakha → Bashkir. ‡Silver test sets (translated and automatically annotated); these mainly measure agreement with the process that produced them. Tatar (29 words), Sakha (16), Kumyk (144) and Karakalpak (235) are qualitative examples, not grounds for ranking. Bold: higher score of the two backends. The source's aggregate is omitted because it mixes gold and silver rows. 180
8.8 Measured pretrained Stanza scores (%) on official UD 2.17 test files, using gold sentence and syntactic-word boundaries and predicted tags/features. The UD scorer includes punctuation in UAS/LAS; all denominators are retained. † Kazakh is a replay diagnostic with split-exposure risk, not held-out accuracy. These are not raw-text or newly trained results. Source: `run_parsing.py`. 182
8.9 Measured Stanza scores (%) on the same official UD 2.17 tests. Both conditions retain gold sentence boundaries; the automatic condition predicts tokenization/MWT and uses the official alignment-aware scorer. † Kazakh has split-exposure risk and is not a held-out score. Gold-word LAS is repeated from the preceding table. Source: run_parsing.py and run_parsing_auto.py. 182

9.1 The Kazakh city name *Алматы* in six cases (the instrumental *Алматымен* 'with Almaty' is a seventh). A recogniser needs to detect mentions in these inflected forms; resolving their identity is a separate task. In Turkish, the apostrophe (*Almatı'ya*, *Almatı'dan*) separates the name from its suffix in formal writing, but is often omitted in informal text (*Almatıya*, *Almatıdan*). 195
9.2 Three different outputs for one Kazakh mention. A stem-only span is not necessarily the canonical name (multiword names, aliases and inflected modifiers inside names need their own rules), and a canonical name is not yet an identity: ambiguous names need linking. 196
9.3 Turkic NER resources identified in this survey (searched 2026-10-13; not exhaustive). Human annotation, weak supervision and automatic projection are kept apart; fields the source does not state are marked as such. WikiANN sizes are those of the commonly used Hugging Face splits. 197
9.4 The 25 KazNERD entity classes (Yeshpanov, Khassanov, and Varol 2022), grouped for readability, with mention counts computed from the authors' release (IOB2; 90,228 / 11,167 / 11,307 training / validation / test sentences; 109,346 / 13,483 / 13,508 mentions). Rare classes such as NON_HUMAN have too few test mentions for a meaningful per-class F1. 199

9.5 Measured cached Stanza KazNERD checkpoint on the complete pinned test split, with gold word/sentence boundaries and exact entity spans and types. No fine-tuning was performed. Source: `run_ner.py`; Yeshpanov, Khassanov, and Varol (2022). 205

10.1 Training-set sizes for selected language pairs in the TIL corpus release (Mirzakhalov, Babu, Ataman, et al. 2021); counts from the release README (til-mt, commit 7d988cc), not the paper's appendix table, which gives different totals for some pairs. English–Turkish has about 370 times more training data than English–Uyghur. Karachay-Balkar, Khakas and Tuvan are covered, but with small pair sizes; Nogai is not covered. 216
10.2 Apertium MT pairs with a Turkic language, by the release stage in Apertium's list of language pairs (checked October 2026). The page was generated in April 2020, and its arrows describe planned directions. Check each repository's release and supported directions before installation; a trunk listing is no guarantee of current support in both directions. 218
10.3 Turkic varieties in NLLB-200 and their language/script codes. Note `uzn_Latn` and `azj_Latn`, not `uzb_Latn` or `aze_Latn`. 219
10.4 Complementary MT evaluation signals; no universal language-specific correlation ranking is assumed. 220
10.5 Measured NLLB-200 distilled 600M greedy translation on original FLORES-200 devtest. No fine-tuning or input truncation; explicit target language token, one beam and a 256-token generation ceiling. Exact SacreBLEU signatures and output audits are retained. Source: `run_translation.py`; NLLB Team et al. (2024). 221

12.1 The main BERTurk variants. The family also includes DistilBERTurk, ConvBERTurk, ELECTRA and T5 models, some trained on the Turkish part of mC4 (242 GB). 243
12.2 Approximate number of words that fit into a context window, computed as window size divided by the lowest and highest fertility measured among the four tokenisers in Table 7.1. 249

13.1 Selected lexical-semantic resources, checked October 2026. Sizes refer to the cited release or publication; this is not an exhaustive family inventory. 254
13.2 Native-script bitext top-1 accuracy, averaged equally over the other 12 target varieties. Each direction uses all 1,012 original FLORES-200 devtest rows as queries and candidates. Models are the pinned multilingual paraphrase MiniLM and XLM-R encoders, with their 128-token limit and recorded truncation. Source: `run_bitext.py`; NLLB Team et al. (2024). 257
13.3 Ways to reduce Turkic word forms for keyword search. 259

13.4 Measured nDCG@10 for source-passage retrieval derived from Belebele: 900 questions and 488 candidate passages per language. This is not Belebele multiple-choice accuracy. BM25 uses fixed k1=1.2 and b=0.75; dense encoders use their 128-token limit. Gold-score ties receive expected rank credit. Source: `run_retrieval.py`; Bandarkar et al. (2024). 261
13.5 Measured nDCG@10 on the same 900 Uzbek questions and 488 Latin passages. Cyrillic queries are generated by the pinned toolkit mapping, then optionally converted back to Latin. No native-speaker validation is claimed. Source: `run_retrieval.py` and `prepare_script_probe.py`; Bandarkar et al. (2024). 264
13.6 Cross-language passage-recovery nDCG@10: 900 questions and 488 aligned target-language passages per direction. Source: `run_retrieval.py`; Bandarkar et al. (2024). 264
13.7 Measured content of sentence-packed chunks with a 512-token ceiling on original FLORES-200 devtest (1,012 aligned sentence rows per variety). Ranges are the minimum and maximum means across four pinned tokenisers; extrema in different columns need not come from the same tokeniser. Words are whitespace-delimited units. The final tail is excluded from means, with all rows retained in the chunk audit. Fill is the mean percentage of the 512-token budget used. Source: experiment N2-4; NLLB Team et al. (2024). 265

15.1 Selected benchmarks covering Turkic languages. 283
15.2 Measured zero-shot constrained-letter Qwen score on 797 valid TUMLU-mini Kazakh test items; two malformed items of 799 are excluded and recorded. The letter-D baseline is selected from the 40 development items, not the test answers. Uniform accuracy is an expectation, not a model run. Source: `run_tumlu.py`; Isbarov et al. (2025). 287

16.1 Candidate comparisons, not measured resource thresholds. 293

17.1 Follow-up Hub tag queries from an isolated environment; repository counts do not measure usable task coverage. 305

B.1 Parallel corpora covering Turkic languages 331
B.2 Monolingual corpora for Turkic languages 332
B.3 Named entity recognition datasets for Turkic languages 333
B.4 Evaluation benchmarks for Turkic NLP 334
B.5 Speech corpora for Turkic languages 334
B.6 Pinned configurations checked 15 October 2026. Counts are generated from `AutoModel` on the meta device, without pretrained weights; a task head changes the count. Availability is not a quality evaluation. Sources, revisions and licence metadata: `tables/tbl-verified-models/data.csv`. 335
B.7 Speech models for Turkic languages 336

C.1 Selected contemporary languages in this appendix; 23 rows, distinct from toolkit coverage. Groupings summarise the book's linguistic discussion; script examples are not an exhaustive statement of legal status. Population estimates and unverified Glottocodes are omitted. 338
C.2 Resource evidence must retain task, source, release and evaluation status. Unknown and search failure are not "No". 339
C.3 Selected Cyrillic characters, checked against Unicode names and code points by the generator. These are encoding identities, not cross-language pronunciation claims. 340
C.4 Typological interpretation replaces unsupported numerical rankings. 341

D.1 Abbreviations used in the book's interlinear glosses. 343

List of Figures

1.1 Geographic distribution of Turkic languages. The family stretches from Turkey and the Balkans in the west through Central Asia to Siberia and western China. Map by GalaxMaps and Wikimedia Commons contributors, revision of 17 April 2024, CC BY-SA 4.0, source; reproduced unchanged. The distribution is schematic and does not indicate political borders or census totals. 23

2.1 Surface pieces of the Turkish form *konuş-tur-a-ma-dık-lar-ımız-dan*: 'from/among those whom we could not get to speak'. Each coloured block shows one surface piece with its gloss below. Lexical stem (coral), verbal suffixes (blue), nominaliser (gold), nominal suffixes (green). . . 37

2.2 The Turkic suffix template shown as two slot diagrams. Left: nominal template (Root → Derivation → Number → Possession → Case → Copula). Right: verbal template (Root → Voice → Ability/Negation → TAM → Person/Number). These are ordering constraints, not independent or necessarily overt slots; agreement can be zero. Ability and negation interact, and nominalisation changes the following template. 38

2.3 A qualitative comparison of harmony in four Turkic languages, separating suffix alternation, spoken rounding and orthographic representation (Johanson 2021, 305–15). This is not a vowel-inventory chart. 42

2.4 Basic UD dependencies for English (top) and Turkish (bottom). English has a post-nominal relative clause with who as the subject of came; Turkish Dün gelen adam gitti has a prenominal participial clause with no relative pronoun. Each word has exactly one head; enhanced coreference edges are omitted. 48

3.1 Selected Unicode blocks used in Turkic text, following Unicode 17.0. Latin Ə/ə and Cyrillic Ә/ә occupy different blocks; the zero-width non-joiner is in General Punctuation. Band widths are schematic. 63

3.2 Greedy longest-match lookup for a versioned transliteration mapping. At each position the longest matching non-empty key is consumed; otherwise one character is copied. The maximum key length comes from the mapping. Contextual spelling, normalisation and case policies are separate from this kernel. 70

3.3 Character-trigram overlap (Dice coefficient) between each Turkic language and Turkish on parallel sentences, in the native script and after conversion to CTS. Every Cyrillic- and Arabic-script language starts at zero. Data: `figures/fig-cts-overlap/data.csv`, from Hakimov (2026). 79

3.4 GlotLID accuracy (restricted to Turkic labels) on 20 parallel challenge sentences per language, with the most frequent wrong label for the weaker languages. Data: `figures/fig-lid-accuracy/data.csv`, from Hakimov (2026). 81

4.1 Size of the Turkic subsets of HPLT v3.0 in characters (logarithmic scale). Data: Table 4.1. 89
4.2 Characters per Gemma 3 token in HPLT v3: eleven primary Turkic language subsets plus English and Russian references (HPLT Consortium 2025). Values are ratios of corpus totals, not averages of document ratios or estimates of word counts. 90
4.3 Documents per Turkic language in eight corpora (log scale). Grey cells: absent. Hatched: present in CC-100, which reports tokens but not documents. Data: Table 4.2. 92
4.4 Share of pages per Turkic language in each Common Crawl snapshot (CLD2 primary language, log scale). In the legend: the share in the latest snapshot, CC-MAIN-2026-39. Source: Common Crawl crawl statistics (`plots/languages.csv`), accessed 12 October 2026. 94
4.5 Share of each Turkic subset of HPLT v3.0 (compressed bytes) per Web Docs Scorer quality bin, from 5 (lowest in the release) to 10 (highest). 97
4.6 Measured difference from English in mean letters per orthographic unit on original FLORES-200 devtest. Bars use the same 1,012 aligned sentence rows in each variety. Intervals are the 2.5th and 97.5th percentiles of 2,000 paired sentence bootstrap samples (seed 20261017); English is the zero reference. 100

5.1 Common nominal and verbal suffix sequences. Ability and negation interact; nominalised verbs enter nominal inflection. Boxes describe morphological domains, not a requirement for an overt suffix in each position. 109

7.1 Measured fertility of four LLM tokenisers on the parallel challenge sentences, sorted by worst-case fertility; non-Latin-script languages in bold. Data: Table 7.1. 148

8.1 Basic UD dependencies for English (top) and Turkish (bottom). English has a post-nominal relative clause with who as the subject of came; Turkish Dün gelen adam gitti has a prenominal participial clause with no relative pronoun. Each word has exactly one head; enhanced coreference edges are omitted. 168
8.2 Mean, median and 90th-percentile basic dependency distances for the 25 official UD 2.17 test sets in the book inventory. Genre, sentence length, translation and annotation choices differ across treebanks; these distributions do not isolate a language effect. 174

9.1 Measured exact span-and-type F1 for the pretrained Stanza KazNERD checkpoint. Each label shows the number of gold test entities for that type; small-support categories should not be interpreted as equally precise estimates. 206

13.1 Measured XLM-R paraphrase-encoder top-1 retrieval accuracy on original FLORES-200 devtest. Rows are query varieties and columns are target candidates; self-retrieval is omitted. Codes follow the dataset's language identifiers. Each cell uses 1,012 queries. 258
13.2 Exploratory sensitivity of BM25 source-passage retrieval to prefix length on the seven Belebele language collections. Each point is nDCG@10 over 900 questions. The test grid is descriptive; no best prefix is selected as a tuned result. 262
13.3 Measured mean words and aligned sentence rows per closed chunk under a 512-token ceiling, using four pinned tokenisers on original FLORES-200 devtest. Counts exclude the final tail. Exact inputs and per-chunk counts accompany the sentence-stream packing experiment. 266

16.1 An evaluation workflow for choosing a cross-lingual transfer strategy. Target task labels determine the initial comparison. Script conversion and an available finite-state transducer are options to test; final selection depends on held-out target performance, coverage, cost and error analysis. 295

Part I

The Language Family

1 The Turkic World: Languages, Speakers, and the Resource Reality

Learning Objectives

After reading this chapter, you will be able to:

- Identify the major branches of the Turkic language family and their key languages
- Explain why mutual intelligibility matters for cross-lingual NLP transfer
- Assess NLP resource availability for a Turkic language task by task, and explain why raw resource counts mislead
- Name the key institutions and community infrastructure supporting Turkic NLP research
- Find your way through the book and know what each part covers

1.1 Geographic and Demographic Overview

Turkic languages are spoken across Anatolia, the Balkans, Central Asia, Siberia and western China. Johanson calls this area *Turcia* (Johanson 2021, 16). Figure 1.1 gives a schematic overview.

Turkic-speaking communities also live in Iran, Iraq, Afghanistan, the Caucasus and the Volga-Ural region. Migration has established communities farther afield, including in Germany (Johanson 2021, 16).

Six states give official status to Turkic languages: Turkey (Turkish), Azerbaijan (Azerbaijani), Kazakhstan (Kazakh), Kyrgyzstan (Kyrgyz), Turkmenistan (Turkmen), and Uzbekistan (Uzbek). The family has more than a dozen standard languages (Johanson 2021, 16).

Historical varieties also matter for heritage digitisation and script conversion. The first known Old Turkic texts are the early-eighth-century inscriptions of the second Türk empire, especially those on the Orkhon River (Johanson 2021, 159). Ottoman Turkish served as the Ottoman Empire's official and literary language; its New Ottoman period lasted until 1928 (Johanson 2021, 156, 164). In Central Asia, writers cultivated the Persian-influenced literary language Chaghatay from the fifteenth century onwards (Johanson 2021, 162).

Language policy affects which corpora can be combined and reused. Different Kazakh Latin proposals, for example, require different conversion tables (Chapter 3). Funding

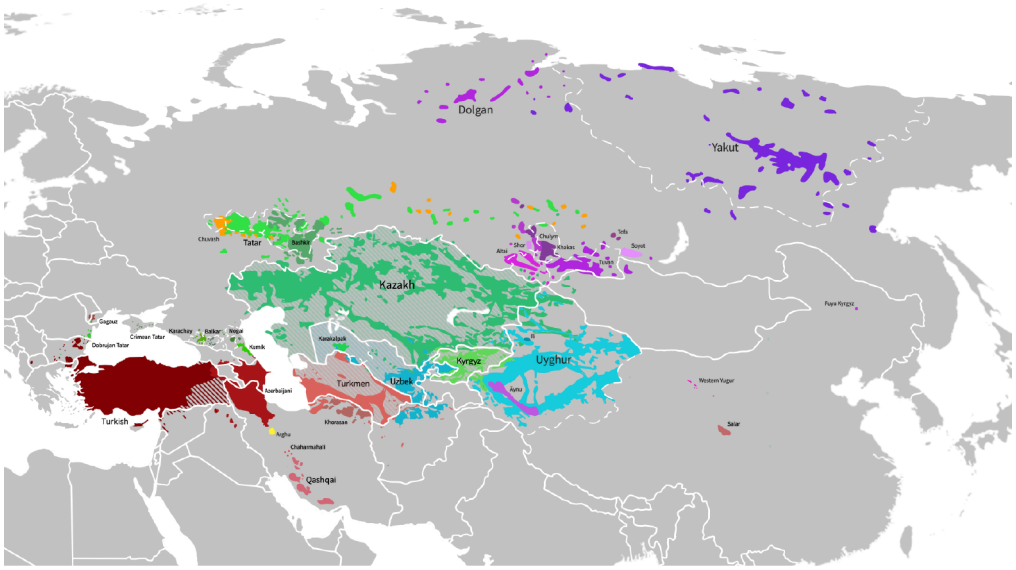


Figure 1.1: Geographic distribution of Turkic languages. The family stretches from Turkey and the Balkans in the west through Central Asia to Siberia and western China. Map by GalaxMaps and Wikimedia Commons contributors, revision of 17 April 2024, CC BY-SA 4.0, source; reproduced unchanged. The distribution is schematic and does not indicate political borders or census totals.

can support resource development. To judge what it has produced, inspect the quality, coverage and access conditions of each release.

1.2 Genealogical Classification

Genealogical classification groups languages by common descent. Historical-comparative linguistics supports the Turkic family and the reconstruction of a common ancestor, Proto-Turkic (Johanson 2021, 18).

Following Johanson (2021, 22–23), the present-day Turkic languages divide into six main branches:

Table 1.1: Branches and subbranches of the Turkic language family, following Johanson (2021, 22–23)

Branch	Subbranches	Major Languages
SW (Oghuz)	West Oghuz, South Oghuz, East Oghuz	Turkish, Azerbaijani, Gagauz, Turkmen
NW (Kipchak)	West, North, South, East Kipchak	Kazakh, Kyrgyz, Tatar, Bashkir, Karakalpak, Kumyk, Nogai, Crimean Tatar, Karachay-Balkar, South Altai
SE (Karluk)	West Karluk, East Karluk	Uzbek, Uyghur

Branch	Subbranches	Major Languages
NE (Siberian)	South, West, North Siberian	Sakha/Yakut, Tuvan, Khakas, Shor, North Altai, Dolgan, Chulym
Oghur	—	Chuvash
Arghu	—	Khalaj

These groups follow the classification in the cited reference; the northeastern (Siberian) group is genealogically less uniform than the others (Johanson 2021, 22). Johanson places South Altai with Kyrgyz in the eastern Kipchak subbranch and North Altai in the South Siberian subbranch (Johanson 2021, 22–23); the ISO code `alt` used by the toolkit and by corpora denotes Southern Altai. Oghuz includes Turkish, Azerbaijani, Turkmen and Gagauz; Kipchak includes languages of the Volga-Ural region, Caucasus and Central Asia. The Karluk group includes Uzbek and Uyghur, whose vowel systems and harmonic patterns must be described separately rather than treated as one contact-induced exception (Chapter 2). Chuvash and Khalaj represent the distinct Oghur and Arghu branches (Johanson 2021, 22–25).

Choosing a transfer language. Start with genealogical proximity when selecting donors, then test them on the target task. TurkicNLP uses proxy-language representations where training data are unavailable: Turkish for Gagauz and Crimean Tatar, Tatar for Kumyk, and Bashkir for Sakha (Hakimov 2026). Some of these choices cross branch boundaries. Their implementation alone tells us little about which donor would work best. Controlled comparisons are needed to separate the effects of genealogy, script and available training data.

Table 1.2 provides an overview of the major Turkic languages with selected scripts and language identifiers. The selection covers living languages only; the toolkit's coverage list is a separate matter. Speaker estimates are omitted here until Appendix C supplies a dated source, geographic scope and first-language versus total-speaker definition for each estimate.

Table 1.2: Selected living Turkic languages, with branch affiliation and script overview (Johanson 2021, 22–25, chapter 19); identifiers follow Appendix C. Regional script variants are not exhaustive.

Language	ISO Code	Branch	Script(s)	Country/Region
Turkish	tur	SW (Oghuz)	Latin	Turkey, Cyprus
Uzbek	uzb	SE (Karluk)	Latin, Cyrillic	Uzbekistan
Azerbaijani	aze	SW (Oghuz)	Latin, Cyrillic, Arabic	Azerbaijan, Iran
Kazakh	kaz	NW (Kipchak)	Cyrillic; Perso-Arabic (China); Latin proposals	Kazakhstan, China
Uyghur	uig	SE (Karluk)	Arabic (Perso-Arabic)	China (Xinjiang)
Turkmen	tuk	SW (Oghuz)	Latin; Perso-Arabic regionally	Turkmenistan

Language	ISO Code	Branch	Script(s)	Country/Region
Kyrgyz	kir	NW (Kipchak)	Cyrillic; Perso-Arabic (China)	Kyrgyzstan
Tatar	tat	NW (Kipchak)	Cyrillic	Russia (Tatarstan)
Bashkir	bak	NW (Kipchak)	Cyrillic	Russia (Bashkortostan)
Chuvash	chv	Oghur	Cyrillic	Russia (Chuvashia)
Karakalpak	kaa	NW (Kipchak)	Latin, Cyrillic	Uzbekistan
Crimean Tatar	crh	NW (Kipchak)	Latin, Cyrillic	Ukraine, Turkey
Kumyk	kum	NW (Kipchak)	Cyrillic	Russia (Dagestan)
Sakha (Yakut)	sah	NE (Siberian)	Cyrillic	Russia (Sakha Republic)
Karachay-Balkar	krc	NW (Kipchak)	Cyrillic	Russia (Caucasus)
Tuvan	tyv	NE (Siberian)	Cyrillic	Russia (Tuva)
Gagauz	gag	SW (Oghuz)	Latin	Moldova
Nogai	nog	NW (Kipchak)	Cyrillic	Russia (Caucasus)
Southern Altai	alt	NW (East Kipchak)	Cyrillic	Russia (Altai Republic)
Khakas	kjh	NE (Siberian)	Cyrillic	Russia (Khakassia)
Shor	cjs	NE (Siberian)	Cyrillic	Russia (Kemerovo Oblast)

The identifiers used by a toolkit need not match a dataset's variety codes. TurkicNLP 0.5.0 lists 24 entries: `alt, azb, aze, bak, chv, crh, gag, kaa, kaz, kir, kjh, klj, krc, kum, nog, ota, otk, sah, tat, tuk, tur, tyv, uig, uzb` (reproduce with `turkicnlp.list_languages()`). This includes historical Ottoman and Old Turkic and a separate Iranian Azeri entry; it does not include Shor, Dolgan or Chulym. Dataset identifiers such as `azj` and `uzn` designate narrower varieties than the toolkit's `aze` and `uzb`; record the mapping instead of silently equating them.

1.3 Mutual Intelligibility and What It Means Computationally

Mutual intelligibility within the Turkic family is "very limited within the family as a whole, but it is occasionally relatively high between neighbouring speaker groups" (Johanson 2021, 16). Johanson gives examples at both ends. Kumyk has a high degree of mutual intelligibility with Azeri (Johanson 2021, 61), and the increasing mutual intelligibility of Kyrgyz and Kazakh appears to result from recent convergence (Johanson 2021, 72).

Turkish and Turkmen are closer to each other than either is to Kazakh or Uyghur (Johanson 2021, 21). At the other end, Yakut (Sakha) and Dolgan are unintelligible to speakers of other Turkic languages (Johanson 2021, 24). Intelligibility between neighbouring varieties is therefore not a simple function of branch membership.

Written similarity. In TurkicNLP's small parallel sample, conversion to a common script increased character-trigram overlap between languages written in different scripts (Hakimov 2026). Spelling had hidden some shared forms. Human intelligibility and NLP task performance require separate measurements; Section 3.6 gives the overlap method and results.

Relatedness as a transfer resource. The Kardeş-NLU benchmark (Senel et al. 2024) evaluates cross-lingual language understanding for five lower-resourced Turkic languages (Azerbaijani, Kazakh, Kyrgyz, Uzbek and Uyghur) and shows that bringing in Turkish, their high-resource "cousin", during intermediate training and fine-tuning improves transfer in the evaluated settings. The practical lessons are:

- Before annotating data for a new Turkic language, check whether a related, better-resourced language can be used as an additional source in training.
- Compare native-script and common-script versions using the same data split; conversion can expose written similarities but may hurt a model trained on national orthographies (Chapter 3).
- Evaluate donors separately for Chuvash and for each Siberian language; the Siberian group is internally diverse, and branch labels alone do not predict transfer quality (Chapter 16).

Chapter 16 develops these transfer strategies in detail.

1.4 Available Resources

Resource availability depends on the task, variety, domain, script and access conditions. Count named releases and check what they support: a web crawl, a treebank and a speech corpus serve different purposes.

1.4.1 Named Resources and What They Enable

Turkish has established resources including the Turkish National Corpus (Aksan et al. 2012), the BOUN dependency treebank (Türk et al. 2022), BERTurk (Schweter 2020) and KeNet (Bakay et al. 2021). These support different activities: corpus description, supervised parsing, representation learning and lexical-semantic applications. Their existence does not guarantee coverage of a new domain or dialect.

Kazakh resources include KazNERD for named-entity recognition (Yeshpanov, Khassanov, and Varol 2022) and the speech work described in TurkicASR and TurkicTTS (Mussakhojayeva et al. 2023; Yeshpanov, Mussakhojayeva, and Khassanov 2023). Audit each named release and its documentation to establish what it offers.

For other languages, inspect dedicated resources alongside multilingual collections. Apertium's morphological analysers, parallel UD treebanks (Akhundjanova et al. 2025) and multilingual reading-comprehension benchmarks such as Belebele (Bandarkar et al. 2024) illustrate different kinds of coverage. Belebele's multiple-choice comprehension task should not be confused with extractive question answering. An unsuccessful search for a dedicated dataset is not evidence that no resource exists.

Table 1.3: Named examples for a task-oriented resource audit. Treat it as a reading list. It neither exhausts the field nor ranks the work in it.

Resource	Example coverage	What it enables	What still needs checking
Turkish National Corpus (Aksan et al. 2012)	Turkish written and spoken material	Corpus queries and usage analysis	Access terms, sampling and target-domain match
BOUN UD (Türk et al. 2022)	Turkish syntax	Supervised tagging and dependency parsing	Release, split, annotation conventions and genre
KazNERD (Yeshpanov, Khassanov, and Varol 2022)	Kazakh named entities	NER training and evaluation	Entity inventory, domain and licence
Parallel UD-Turkic (Akhundjanova et al. 2025)	Aligned examples across Turkic languages	Controlled cross-language parsing comparisons	Size, translation effects and development versus released data
Belebele (Bandarkar et al. 2024)	Multilingual reading comprehension, including Turkic varieties	Multiple-choice comprehension evaluation	Exact language/script identifier and translation effects
Apertium (J. Washington, Salimzyanov, and Tyers 2014; Khanna et al. 2021)	Language-specific morphological transducers	Analysis and generation without an annotated training corpus	Lexical coverage, ambiguity, supported categories and held-out accuracy

1.4.2 A Worked Resource Audit

Resource descriptions can conceal large differences in domain. In the documentation consulted on 13 October 2026, Turkish BOUN lists nonfiction and news. Azerbaijani TueCL consists of grammar examples: 148 sentences translated from Turkish. Both specify CC BY-SA 4.0. The book's inventory uses UD 2.17 (Chapter 8). TueCL supports a controlled parallel comparison, but performance on naturally occurring Azerbaijani news needs its own evaluation.

For each candidate resource, record its URL, release or commit, language/variety, script, domain, unit of size, licence, access conditions and intended task. Mark work in progress separately from released data. Then inspect a sample: does it suit the task you have in mind?

1.5 Key Institutions and Community Infrastructure

The following organisations maintain resources or provide venues for Turkic NLP work.

Apertium. This open-source platform provides morphological transducers and rule-based translation systems (Khanna et al. 2021; J. Washington, Salimzyanov, and Tyers 2014). A transducer can be built without a supervised training corpus. The work lies in its lexicon, linguistic rules, testing and maintenance. Chapter 6 and Chapter 10 distinguish analysers from released translation pairs.

SIGTURK. The ACL Special Interest Group on Turkic Languages supports workshops and community exchange. Consult its current announcements for releases, shared tasks and events; where a resource was published says little about how mature it is.

ISSAI. The Institute of Smart Systems and Artificial Intelligence at Nazarbayev University contributes datasets, models and evaluations for Kazakh and other Turkic languages, including the TurkicASR and TurkicTTS work cited above. Check each release's access and licence independently.

Universal Dependencies. UD (Nivre et al. 2016) is the annotation framework and community behind the Turkic treebanks, with shared conventions and versioned releases. Use the release documentation to distinguish released treebanks from development repositories. Chapter 8 covers the framework and how to build new treebanks.

Turkic Interlingua. This community provides machine-translation resources for Turkic languages: the TIL corpus, baselines (Mirzakhalov, Babu, Ataman, et al. 2021) and the X-WMT test sets, which cover eight Turkic languages plus English and Russian in 88 directions. Chapter 10 explains the data and evaluation coverage.

Turkic UD. This group works on harmonising annotation across Turkic UD treebanks. Its parallel treebanks annotate the same sentences in several Turkic languages, starting with Azerbaijani, Kyrgyz, Turkish and Uzbek (Akhundjanova et al. 2025), so that annotation choices can be compared directly.

TurkLang. The International Conference on Computer Processing of Turkic Languages has met since 2013. It is a regular venue for work on corpora, morphology, machine translation and other language technology for Turkic languages; check individual proceedings for peer-review and publication details.

Turkic Academy. This international organisation facilitates and encourages scientific and educational cooperation among Turkic countries and communities. It is a point of contact for research collaboration across the Turkic-speaking states rather than a source of NLP datasets.

1.6 Book Organisation and How to Use This Book

The book has 17 main chapters in seven parts, followed by four appendices. The parts cover the language family; writing systems and corpora; morphology; syntax; applications; language models; and cross-lingual methods and the resource frontier.

Choose a route according to your task:

- **Build a corpus:** start with scripts and orthography, then corpora at scale, and consult evaluation before treating crawl labels as ground truth.
- **Analyse words or sentences:** read the computational phenotype, morphological structure, finite-state morphology, neural morphology, and syntax and UD.
- **Build an application:** choose sequence labelling, translation or speech.
- **Adapt a language model:** connect pretrained models with meaning and retrieval, generation and dialogue, and evaluation.
- **Work with a language with limited resources:** pair cross-lingual transfer with endangered languages and ethics, including community priorities and data permissions from the outset.

1.6.1 Prerequisites

This book assumes familiarity with:

- **Python programming** at an intermediate level
- **Basic machine learning** concepts (supervised learning, train/test splits, evaluation metrics)
- **Basic linear algebra and probability** (vectors, matrices, conditional probability)

No NLP background beyond the introductory chapters of Jurafsky and Martin (2024) is assumed. For finite-state morphology (Chapter 6) and language models (Chapter 12), familiarity with formal automata theory and neural networks respectively will help, but the text introduces these concepts from first principles.

You need no prior knowledge of Turkic languages or of linguistics; each concept is introduced where it is first used. For a full linguistic reference on the Turkic family, see Johanson (2021) and Johanson and Csató (1998).

1.6.2 Hands-On Environment

Most chapters include a hands-on section with practical exercises; this chapter has a worked resource audit instead (Section 1.4). The code examples use:

- **Python 3.10+** as the primary programming language
- **PyTorch** for neural network implementations
- **Hugging Face Transformers** for pretrained model access and fine-tuning (Wolf et al. 2020)
- **Hugging Face Datasets** for loading Turkic NLP datasets
- **TurkicNLP** for morphological analysis and script processing (Hakimov 2026)
- **Apertium** for finite-state morphology demonstrations (Khanna et al. 2021)

The code in the hands-on sections uses TurkicNLP 0.5.0.

1.6.3 A Note on Language Coverage

Turkish, Kazakh, Uzbek, Azerbaijani and Turkmen provide recurring examples. Other languages appear where documented examples, data or tools are available. The comparisons show both shared patterns and language-specific differences.

1.6.4 Recurring Example Sentences

Four short sentences recur across the book, so that one text can be followed through script conversion, morphology, tokenisation and syntax (Table 1.4). They come from UD 2.17 test treebanks, cover Cyrillic, Latin and Perso-Arabic script, and were checked by the author. They explain operations; the experiments use their own datasets and splits.

Table 1.4: The recurring example sentences. Each is a UD 2.17 test sentence; the treebanks are licensed CC BY-SA 4.0. Translations are the book's. Morphological glosses are in Section 5.4.4.

Language (script)	Sentence	Translation	Source
Kazakh (Cyrillic)	*Қазақстан осы өңірдегі бейбітшілікті қолдайды.*	'Kazakhstan supports peace in this region.'	Kazakh-KTB `akorda-random.tagged.txt:1:0`
Kyrgyz (Cyrillic)	*Менин машинам бар.*	'I have a car.'	Kyrgyz-KTMU `001`
Uzbek (Latin)	*Toshkentda dzyudo bo'yicha Gran-pri musobaqasi bo'lib o'tmoqda.*	'A judo Grand Prix tournament is taking place in Tashkent.'	Uzbek-UT `55`
Uyghur (Perso-Arabic)	كۆرىدۇ. ھارۋىسىنى ئات بىر سىرتىدىكى شەھەر ئۇ	'He/She sees a horse cart outside the town.'	Uyghur-UDT `s103`

They appear in vowel harmony (Section 2.2), script conversion (Section 3.6), morphological analysis (Section 5.4.4), tokenisation (Table 7.2), dependency annotation (Section 8.2) and entity boundaries (Section 9.2).

1.7 Hands-On: A Procedure for Building a Task-Specific Inventory

Question. Which resources support one chosen language, domain and task?

Inputs. The named inventories and toolkit registry in Section 1.4.

Compute. CPU and network queries; record failed requests.

Output. A dated inventory of candidates, permissions and evaluation gaps.

Interpretation. Use the inventory to select what to investigate next.

Worked row. For Kyrgyz named entity recognition, figures recorded elsewhere in the book fill the fields as follows.

Table 1.5: A filled inventory row for one language and task.

Field	Kyrgyz NER
Labelled data	KyrgyzNER: 7,033 training and 3,867 test sentences, no development split (Sequence Labelling, Exercise 3); news articles from 24.kg (KyrgyzNER README)
Licence	Dataset CC BY-NC-SA 4.0 (KyrgyzNER README)
Related annotation	UD 2.17 Kyrgyz-KTMU (1,308 training and 1,222 test sentences) and Kyrgyz-TueCL (173 test sentences) (Section 8.3)
Ready-made models	No dedicated pretrained Kyrgyz NER model in TurkicNLP 0.5.0; its generic LLM backend is a separate option. Stanza's Kazakh KazNERD model reaches 39.9 micro F1 on the KyrgyzNER test set without Kyrgyz training data (PER/LOC/ORG, gold words; Section 9.3)
Evaluation gap	One news source, and no development split: model selection needs a held-out part of the training data

Status: procedure only. The fields above define the audit, and Table 1.5 fills them for one task; Exercises 1, 3 and 5 carry it out for selected languages.

What to Take Forward

The family map suggests languages to compare. A resource audit shows which comparisons are feasible. TurkicNLP 0.5.0 lists 24 languages, with different components and evaluation evidence for each (Section 1.4). Use a dated inventory of data, permissions and results to choose a task. Chapter 2 examines the linguistic patterns that these systems must handle.

Further Reading

- **Turkic language family (reference grammar)**: Johanson (2021) provides a systematic overview of the entire Turkic family covering structural, functional, historical, sociolinguistic, and literary aspects.
- **Turkic languages (individual descriptions)**: Johanson and Csató (1998) provides concise descriptions of individual languages.

- **NLP foundations**: Jurafsky and Martin (2024) is the standard NLP textbook and a companion to this book for general NLP concepts.
- **Turkish NLP survey**: Çöltekin, Doğruöz, and Çetinoğlu (2023) provides a critical survey of resources for Turkish natural language processing.
- **Turkish NLP (book)**: Oflazer and Saraçlar (2018) covers morphology, parsing, speech, NER and machine translation for Turkish with methods relevant to other Turkic languages.
- **Turkic NLP benchmark**: Isbarov et al. (2025) presents TUMLU, a native multiple-choice benchmark; Chapter 15 explains its language coverage and release counts.
- **ISSAI resources**: The ISSAI research group at Nazarbayev University publishes datasets, models, and papers at issai.nu.edu.kz.
- **Universal Dependencies**: The UD project at universaldependencies.org provides treebank documentation for all Turkic UD treebanks.
- **Apertium Turkic**: The Apertium Turkic community at turkic.apertium.org maintains morphological tools and MT systems for a range of Turkic languages; inspect the individual repository and release.

Exercises

1. **Language family exploration.** Choose a Turkic language you are unfamiliar with from Table 1.2. Using online resources, find its Wikipedia edition (if it exists) and estimate its digital footprint. How many Wikipedia articles does it have? Is it covered by OSCAR or FineWeb2? Does it have a Universal Dependencies treebank? Locate the language on the map in Figure 1.1 and identify its branch according to Table 1.1.

2. **Branch membership and transfer.** Using Table 1.1, identify: (a) which branch contains Uzbek; (b) which branch Kazakh and Kyrgyz both belong to; (c) which branch Chuvash belongs to and why it is unusual. Based on genealogical proximity alone, predict which language would be a better transfer source for Kyrgyz NLP: Turkish or Kazakh? Justify your answer.

3. **Resource gap analysis.** Choose a language, domain and community use case. Using the audit fields in Section 1.4, identify three resources needed for that use case, search for named releases, and record any gaps as unverified until the search scope is documented. Prioritise work by user needs, permissions and feasibility.

4. **Mutual intelligibility and NLP.** The Kardeş-NLU paper (Senel et al. 2024) studies transfer to five Turkic languages with Turkish as a high-resource relative. Before reading it, rank the five languages (Azerbaijani, Uzbek, Kyrgyz, Kazakh, Uyghur) by how much you expect them to benefit from Turkish, based on genealogy, script, and resources. Then compare your ranking with the paper's results. Which factors are confounded in this comparison, and what controlled experiment would distinguish them?

5. **What a toolkit offers per language.** Install TurkicNLP (`pip install "turkicnlp==0.5.0"`) and run:

```
import turkicnlp
for lang in ["tur", "kaz", "kir", "sah", "kum", "chv"]:
    print(lang, turkicnlp.list_processors(lang))
```

Which processors are available for each language, and from which backends? Compare the result with Table 1.3: which tasks have both a listed backend and suitable evaluation data?

6. **Community exploration.** Visit the SIGTURK website, the Apertium Turkic page, or the Universal Dependencies project. Find a recent shared task, release, or publication related to Turkic NLP. Summarise the contribution in 2–3 sentences and explain how it addresses one of the gaps identified in Section 1.4.

2 The Computational Phenotype of Turkic Languages

Learning Objectives

After reading this chapter, you will be able to:

- Explain agglutination and distinguish morphemes, morphs, words and model tokens
- Describe vowel harmony, how it creates suffix allomorphy, and which alternations the spelling shows
- Explain the role of the case system in Turkic syntax, including case ambiguity and differential object marking
- Characterise SOV base order, scrambling and pro-drop
- Describe participial and converbal clauses and why they create long-distance dependencies
- Identify which features are constant across the family and which vary, and turn each into a testable question about NLP systems

This chapter introduces the linguistic patterns used throughout the book: suffixation, vowel harmony, case, word order and non-finite clauses. It explains what each pattern means for text processing, while keeping linguistic facts separate from claims about model performance.

Most worked analyses in this chapter use Turkish, giving readers a consistent starting point. Comparisons with Kazakh, Kyrgyz, Uzbek, Uyghur, Turkmen and smaller languages show how far each pattern extends. Chapter 5 develops the morphology in detail; later chapters test the computational methods.

2.1 Agglutination and Productive Word Formation

2.1.1 What Agglutination Is

A **morpheme** is an abstract unit of grammatical or lexical structure; a **morph** is a surface form used to realise it. Different surface forms of the same morpheme are **allomorphs**: Turkish plural *-lar* and *-ler* realise one plural morpheme. An orthographic word, a morpheme and a language-model token are different units, and their boundaries need not match.

A language is **agglutinative** when words are built by chaining morphs, each typically expressing one grammatical function, with clear boundaries between them. Johanson (2021, 35) characterises the Turkic word structure as based on "a juxtaposing technique with clear-cut morpheme boundaries", with "mostly one-to-one correspondence of morphemes and their functions". This contrasts with:

- **Fusional** languages (German, Russian, Latin): a single morph can signal several functions at once (case + number + gender in one suffix)
- **Isolating** languages (Chinese, Vietnamese): grammatical relations expressed by word order and separate particles, with little inflection

They describe tendencies; a language can show several at once. **Polysynthesis** concerns how much grammatical and lexical information a word can contain; it is a separate dimension from whether its morphs have clear boundaries. A language can be both agglutinative and polysynthetic.

A short example shows the principle. The Turkish word *evlerimizden* 'from our houses' consists of four surface pieces:

Table 2.1: The Turkish word *evlerimizden* 'from our houses'. Gloss abbreviations follow the Leipzig Glossing Rules: PL plural, POSS possessive, ABL ablative, ACC accusative, GEN genitive, PASS passive, NEG negation, PST past, 3PL third person plural; the full list is in Table D.1.

Surface piece	Analysis	Contribution
ev	house	lexical stem
-ler	PL	plural
-imiz	POSS.1PL	first-person plural possessor
-den	ABL	source (ablative case)

The same structure appears across the family: Kazakh *үй-лер-іміз-ден*, Uzbek *uy-lar-imiz-dan*, with the same order of plural, possessive and case. A short verb works the same way: *yaz-ıl-dı-lar* 'they were written' is stem + passive + past + third person plural.

Turkic agglutination is productive: a single root can yield many well-formed word forms by adding suffixes in a constrained order.

2.1.2 The Suffix Template

Suffix order is constrained. A common verbal pattern is:

Root → Voice → Ability / Negation → Tense/Aspect/Mood → Person/Number

For nouns, it is:

Root → Derivation → Number → Possessive → Case

The slots in these templates interact. Ability and negation restrict one another, as the next example shows. Derivation can also change the inflection that follows: a nominalised

verb takes nominal suffixes such as plural, possessive and case. Each suffix operates on the form to its left, so order affects meaning.

Negative ability illustrates both points. Turkish *yaz-ıl-a-ma-dı-lar* ‘they could not be written’ adds *-a-ma-* between the passive and the past. Positive ability is a different suffix, *-(y)Abil-* (*yaz-ıl-abil-di-ler* ‘they could be written’); the negative form is not simply *-abil-* plus negation. In this book we segment *-a-ma-* as two surface pieces, ability and negation, and treat them together as the inability construction; other analyses treat *-AmA-* as a single morph. Whatever the convention, it has to be stated, because it changes every morpheme count.

A statistical subword tokeniser partitions the written form without changing suffix order. Its pieces may cross morphological boundaries or combine several morphemes, leaving the model to learn the grammatical structure from those pieces.

2.1.3 An Advanced Example

The Turkish word *konuşturamadıklarımızdan* combines verbal derivation with nominal inflection. It means “from/among those whom we could not get to speak”:

Table 2.2: Surface pieces of *konuşturamadıklarımızdan*, Turkish. After the nominaliser *-DIK* the word continues with nominal inflection (plural, possessive-form agreement, case). The stem *konuş-* is treated as one unit, and *-a-ma-* is split under the inability convention introduced above.

Surface piece	Gloss	Function
konuş-	speak	lexical verb stem
-tur-	CAUS	causative: make/get someone to speak
-a-	ABIL	ability (with the following negation : inability)
-ma-	NEG	negation
-dık-	NMLZ	non-future nominaliser/participle *-DIK*
-lar-	PL	plural referents: ‘those’
-ımız-	POSS.1PL	first-person plural possessive-form agreement: ‘we’ as the subject of the relative clause
-dan	ABL	ablative case: ‘from/among’

Under this convention the word has eight surface pieces, each counted as one morpheme: one stem and seven suffixes. There is no buffer consonant in this example. The form is a **headless relative clause**: the people referred to are understood rather than named by a following noun. The plural *-lar-* marks those people, whereas *-ımız-* identifies the subject as ‘we’; it does not express ownership here. The *k* of *-dık-* remains unchanged before the consonant-initial plural *-lar-*.

-DIK is analysed here as a nominaliser; its temporal reading (often past or present relative to the main clause) depends on the construction, so the gloss NMLZ is preferred over “past

participle” (Chapter 5 and Chapter 8 use the same analysis). The example is deliberately long; in running text most words carry far fewer suffixes. The long tail of rarer, longer forms is what makes the vocabulary open-ended.

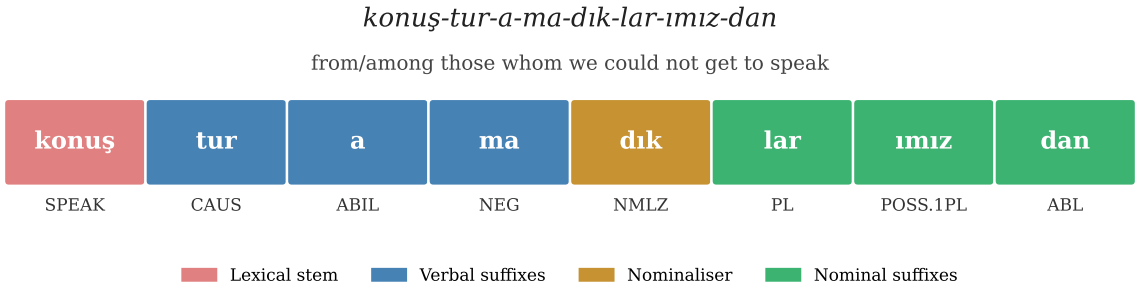


Figure 2.1: Surface pieces of the Turkish form *konuş-tur-a-ma-dık-lar-ımız-dan*: ‘from/among those whom we could not get to speak’. Each coloured block shows one surface piece with its gloss below. Lexical stem (coral), verbal suffixes (blue), nominaliser (gold), nominal suffixes (green).

The template summarises common patterns; Chuvash, for example, places possessive marking before plural (Section 5.4.5). Possessive agreement and evidentiality are expressed within suffix chains, while clitics such as Turkish question particle *mI* and auxiliary constructions such as Kazakh *оқу ал-* ‘be able to read’ extend across words. Chapter 5 explains these distinctions; Chapter 8 covers analytic predicates.

2.1.4 Unseen Word Forms

Productive suffixation creates well-formed words absent from a finite training corpus. A whole-word model needs a way to handle these unseen forms. Character-complete or byte-level tokenisers can encode them as smaller pieces; other subword tokenisers may return an unknown token. Comparing vocabulary sizes across languages requires matched corpus sizes, domains and tokenisation rules.

Morphemes per word and model tokens per word are different measures. The Hands-On section counts surface segments under an explicit convention. Section 3.6 measures how changing the script affects token counts for the same sentences. Neither, taken alone, is a measure of morphological complexity.

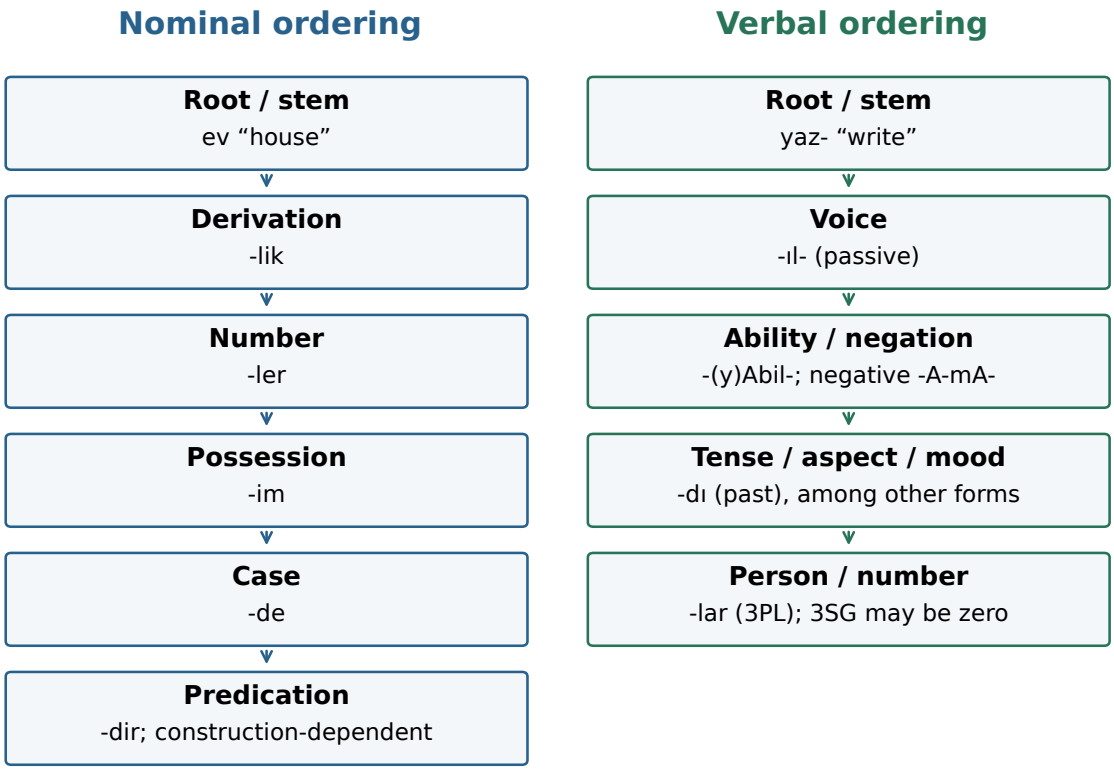


Figure 2.2: The Turkic suffix template shown as two slot diagrams. Left: nominal template (Root → Derivation → Number → Possession → Case → Copula). Right: verbal template (Root → Voice → Ability/Negation → TAM → Person/Number). These are ordering constraints, not independent or necessarily overt slots; agreement can be zero. Ability and negation interact, and nominalisation changes the following template.

2.2 Vowel Harmony: Rules, Exceptions, and Computational Consequences

2.2.1 The Basic Mechanics

Vowel harmony is the phenomenon by which the vowels in a word are constrained to agree in some phonological property. The most general constraint in Turkic languages is **front/back (palatal) harmony**. Johanson (2021, 38) describes "intrasyllabic [±front] harmony, which requires the segments of a syllable to be either front or back", and intersyllabic harmony, which neutralises the front/back distinction under the influence of a preceding syllable.

In Turkish, the eight vowels divide into two sets:

- **Front**: *e, i, ö, ü*
- **Back**: *a, ı, o, u*

Most native Turkish roots have vowels from one set, with exceptions such as *anne* 'mother', *elma* 'apple' and *kardeş* 'sibling'. Harmony within a stem is a strong lexical tendency. Suffix selection is productive: the plural takes *-ler* after a front vowel (*ev-ler*, houses) and *-lar* after a back vowel (*araba-lar*, cars). It follows the stem's last vowel even in disharmonic words such as *elma-lar*. Many other suffixes have four allomorphs, varying in both backness and rounding.

A second harmony axis, **labial (round) harmony**, governs high suffix vowels (*i/ı/ü/u*) in Turkish: after a rounded vowel they round, after an unrounded vowel they do not. This gives four-way alternation for suffixes such as the first-person singular possessive *-ım/-im/-um/-üm*.

2.2.2 Variation Across the Family

Vowel harmony is found throughout the family, but its scope, its exceptions and above all its *spelling* differ:

- **Turkish**: backness and rounding harmony in suffixes, both written; lexical exceptions in stems (*anne*, *elma*), suffixes with limited or no vowel alternation (*-yor*, *-ken*, *-ki*, *-leyin*), and loanwords whose final consonant triggers front suffixes (*saat-ler*, *rol-ü*, *hal-i*)
- **Kazakh**: backness harmony written; rounding harmony is present in pronunciation but **not written** in the standard orthography, whereas Kyrgyz writes it (Kazakh *көздi* vs. Kyrgyz *көздү*, 'eye-ACC')
- **Kyrgyz**: backness and rounding harmony written, including in low vowels (*үйлөр* 'houses')
- **Uyghur**: backness harmony in many suffixes, plus raising of stem vowels before suffixes, which the script writes out (*bala* 'child', *balilar* 'children')

- **Uzbek**: suffix vowels are largely fixed in standard spelling (*uylar*, *otlar*), while front/back harmony is weaker in many urban varieties and clearer in some northern varieties; standard Uzbek has no clear-cut rounding harmony (Johanson 2021, 307, 313)
- **Chuvash**: a vowel system that differs markedly from the other languages (Table 2.4)

Table 2.3 shows the same stems with the plural and accusative suffixes in five languages.

Table 2.3: Plural and accusative suffixes on the same stems ('house', 'horse', 'eye', 'arm/hand') in five Turkic languages, in standard spelling. The columns contrast front and back stems (*houses* vs. *horses*) and rounded stems (*eye*, *arm/hand*).

Language	Script	'houses'	'horses'	'eyes'	'eye' (ACC)	'arm/hand' (ACC)	What the spelling shows
Turkish	Latin	*evler*	*atlar*	*gözler*	*gözü*	*kolu*	backness and rounding written
Kazakh	Cyrillic	*үйлер*	*аттар*	*көздер*	*көзді*	*қолды*	backness written; rounding not written
Kyrgyz	Cyrillic	*үйлөр*	*аттар*	*көздөр*	*көздү*	*колду*	backness and rounding written (also in low vowels)
Uzbek	Latin	*uylar*	*otlar*	*koʻzlar*	*koʻzni*	*qoʻlni*	suffix vowels largely fixed
Turkmen	Latin	*öýler*	*atlar*	*gözler*	*gözi*	*goly*	backness written; rounding written in some suffixes (genitive *gözüň*) but not in the accusative

A vowel inventory records a language's contrasts. Harmony governs restrictions and alternations involving those contrasts. Uzbek suffixes illustrate how a language can retain a vowel after losing a productive harmonic alternation. For written-text NLP, ask which alternations appear in the spelling.

The recurring sentences (Table 1.4) show these patterns in running text. Kazakh *өңір-де-гі* 'in the region' and *бейбітшілік-ті* 'peace-ACC' take front suffix vowels after front stems, and the accusative begins with *т* after the voiceless *к*. Uyghur هارۋىسىنى (*harwisini*, 'its cart', accusative) shows stem-vowel raising: *harwa* 'cart' becomes *harwi-* before the suffix. Uzbek *Toshkent-da* and *musobaqa-si* keep the same suffix vowels whatever the stem. The full analyses are in Section 5.4.4.

Table 2.4: Vowel inventories under the cited analyses. Symbols follow Johanson's Turkological notation, not a uniform IPA transcription. Orthographic correspondences do not imply identical pronunciation across languages.

Language	Vowels (source transcription)	Spelling, in corresponding order	Scope	Source
Turkish	i ï ü u ä a ö o	i ı ü u e a ö o	Core short vowels; loan length treated separately	(Johanson 2021, 256, table 14.3)
Kazakh	a e ä o ö ï į ų ų̈	а е ә о ө ы і ұ ү	Near-high qualities use underdots; и and у are not additional simple vowels in this analysis	(Johanson 2021, 263)
Kyrgyz	a e i ï u ü ö o; a: e: u: ü: ö: o:	а е и ы у ү ө о; аа ээ уу үү өө оо	Eight short qualities and six long counterparts; no phonemic long i or ï	(Johanson 2021, pp. 263–264)
Uzbek	i ù u e ò o à å	i u u e o‘ o‘ a o	Johanson's eight-quality analysis, which he contrasts with the traditional six-phoneme analysis mirrored by the six vowel letters; realisations vary by variety	[Johanson (2021), pp. 273–275, table 14.22; traditional analysis p. 274]
Uyghur	i ü u e ö o ä a	ا ه و ۆ ې ۇ ۇ ى	Core qualities; hamza carrier and contextual forms omitted; length is not written	[Johanson (2021), p. 275, table 14.23; p. 413, table 19.6]
Chuvash	i ü ï u į į̈ ä a	и ӳ ы у ĕ ă е а	Short vowels; Russian-loan о and Upper Chuvash variants treated separately	(Johanson 2021, pp. 270–271, table 14.17)

The table uses Johanson's transcription, with its own symbol conventions. His eight-quality analysis of Uzbek distinguishes sounds that the six vowel letters do not consistently separate. Counts also depend on variety and on how loan vowels and length are treated.

Before using these symbols as IPA training labels, consult a phonetic description of the intended variety.

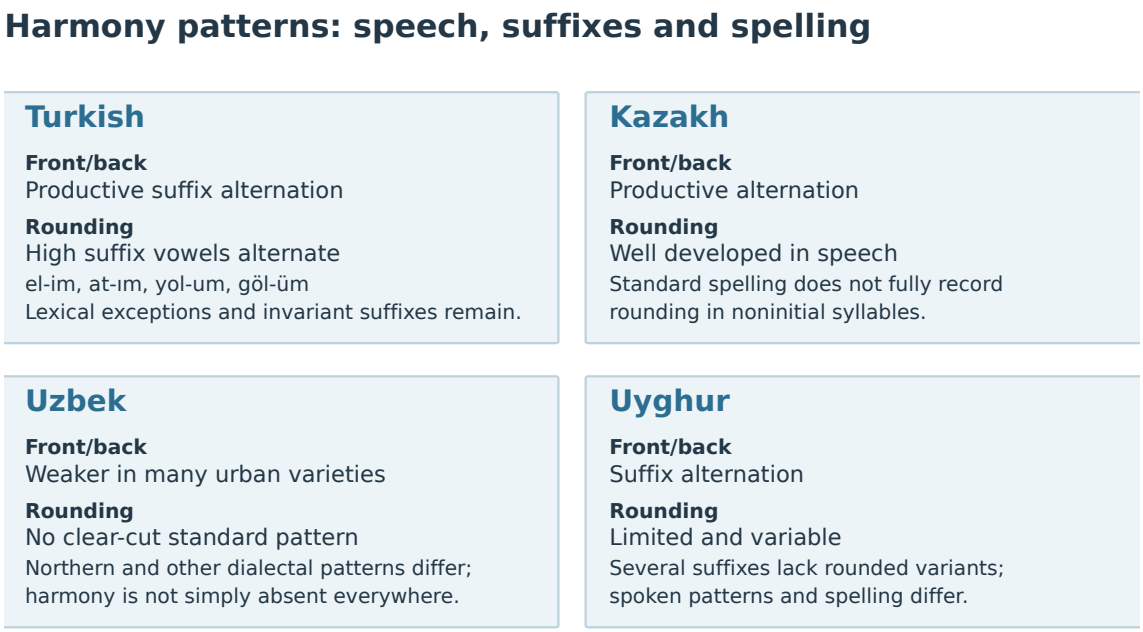


Figure 2.3: A qualitative comparison of harmony in four Turkic languages, separating suffix alternation, spoken rounding and orthographic representation (Johanson 2021, 305–15). This is not a vowel-inventory chart.

2.2.3 Exceptions

Suffix harmony is governed by the *last* vowel of the stem, so loanwords with mixed vowels such as *telefon* or *kitap* are regular (*telefon-lar, kitap-lar*). The genuine exceptions are lexical: stems whose final consonant is pronounced palatalised and therefore takes front suffixes despite a back vowel (*saat-ler* 'hours', *rol-ü* 'role-ACC', *hal-i* 'state-ACC'), and suffixes outside the regular pattern (*-yor*, *-ken*, and usually *-ki*). Turkish *-ki* has exceptions such as *dünkü* 'pertaining to yesterday' (Johanson 2021, 493).

An analyser needs lexical information about these exceptional stems. A statistical model must learn their behaviour from examples; a uniform harmony rule will miss them.

2.2.4 Computational Consequences

1. **Suffix allomorphy.** The same morpheme has two or four surface forms (more with consonant alternation). A morphological analyser must map *-ler* and *-lar* to one

plural; a machine translation system generating Turkish must produce the correct allomorph.

2. **What the representation exposes.** Character representations expose the letters relevant to written harmonic alternations, usually only a few characters apart. Subword models can also learn these regularities, but their token inventory and training examples determine how directly the relevant vowels are represented: when the stem vowel and the suffix vowel fall into different tokens, the model must learn the dependency between token identities. The usefulness of either representation should be evaluated on regular forms and on lexical exceptions separately.
3. **Allomorphs multiply vocabulary entries.** Forms such as *-lar/-ler* and *-dı/-di/-du/-dü/-tı/-ti/-tu/-tü* can be represented by different subword pieces, including pieces crossing the stem–suffix boundary. They need not each occupy a whole vocabulary entry. Morphological segmentation alone does not merge allomorphs: sharing a canonical unit or grammatical label requires an additional normalisation or analysis step (Section 7.2).
4. **Orthography differs from phonology.** Whether harmony is *written* matters more for text NLP than whether it is pronounced. Kazakh does not spell rounding harmony while Kyrgyz does, so the same suffix has more written variants in Kyrgyz, a difference a transfer model must absorb.

2.3 The Case System as Syntactic Infrastructure

2.3.1 The Case Inventory

Turkic languages use morphological case to mark the grammatical relations of noun phrases. Most Turkic languages have at least six cases: nominative, accusative, dative, locative, ablative and genitive. Some have more, such as an instrumental (Chuvash, Sakha, Kazakh), an equative or a directive. Sakha is the notable outlier: it has **no genitive case** but has partitive, comitative, comparative and instrumental cases [Johanson (2021), chapter 22; p. 463]. The cases are expressed by suffixes added after number and possessive marking.

Table 2.5: The word “house” (*ev/ɣŭ/uy/öý*) in six cases across five Turkic languages. Note allomorphic variation due to vowel harmony and consonant assimilation: Turkmen writes a rounded vowel in the genitive (*öýüň*) but not in the accusative (*öýi*). Latin forms in parentheses are a broad transliteration (*i* = i, *ŭ* = y, *ң* = ñ), not the 2021 Kazakh Latin alphabet used by the toolkit.

Case	Function	Turkish	Kazakh	Uzbek	Kyrgyz	Turkmen
Nominative	Unmarked nominal form	*ev*	*ɣŭ (üy)*	*uy*	*ɣŭ (üy)*	*öý*
Accusative	Specific direct object	*ev-i*	*ɣŭ-∂i (üy-di)*	*uy-ni*	*ɣŭ-∂ɣ (üy-dü)*	*öý-i*
Dative	Indirect object / goal	*ev-e*	*ɣŭ-ғe (üy-ge)*	*uy-ga*	*ɣŭ-ғө (üy-gö)*	*öý-e*

Case	Function	Turkish	Kazakh	Uzbek	Kyrgyz	Turkmen
Locative	Location	*ev-de*	*үй-дe* *(üy-de)*	*uy-da*	*үй-дө* *(üy-dö)*	*öý-de*
Ablative	Source / comparison	*ev-den*	*үй-дeн* *(üy-den)*	*uy-dan*	*үй-дөн* *(üy-dön)*	*öý-den*
Genitive	Possession	*ev-in*	*үй-дің* *(üy-diñ)*	*uy-ning*	*үй-дүн* *(üy-dün)*	*öý-üň*

2.3.2 How Case Does What Word Order Does in English

In English, “The dog bit the man” and “The man bit the dog” have opposite meanings because position determines grammatical role. In Turkish, the same distinction is carried largely by case suffixes:

- *Köpek adam-ı ısırdı.* (dog man-ACC bit: the dog bit the man)
- *Adam-ı köpek ısırdı.* (man-ACC dog bit: the dog bit the man, with a different information structure)
- *Adam köpeğ-i ısırdı.* (man dog-ACC bit: the man bit the dog; note *köpek* → *köpeğ-* before a vowel)

Here the accusative suffix marks the direct object in either position, and the nominative subject has no suffix. Case marking is an important cue to grammatical relations, but the relationship is not one-to-one, as the next two subsections show.

2.3.3 Differential Object Marking

Not every direct object carries the accusative. Turkish marks specific objects and leaves non-specific objects bare:

- *Ali kitap okudu.* ‘Ali read (a book / books)’: non-specific object, no suffix; the bare object normally stands directly before the verb.
- *Ali kitab-ı okudu.* ‘Ali read the book’: specific object, accusative *-ı*.

A parser must combine case, position and context. The accusative does not universally mark definiteness, and an unmarked noun can be a bare preverbal object as well as a subject.

2.3.4 Case Ambiguity

A written ending can have more than one morphological analysis. Turkish *ev-i* can be ‘house-ACC’ (*evi gördüm* ‘I saw the house’) or ‘house-POSS.3SG’ (*onun evi* ‘his/her house’); *ev-in* can be ‘house-GEN’ (*evin kapısı* ‘the door of the house’) or ‘house-POSS.2SG’ (*senin evin* ‘your house’). These are homophonous suffixes of different categories rather than syncretism within the case paradigm. A morphological analyser typically returns all readings, and the sentence context selects one: *onun* or *senin* before the noun, or the verb that governs it.

Case also differs from grammatical relation and from semantic role. A dative noun phrase can be a goal (*eve gitti* 'went home'), a recipient (*Ali'ye verdi* 'gave to Ali') or the experiencer-like argument of some verbs. In nominalised subordinate clauses the subject is often in the genitive rather than the nominative: *Ali'nin gel-diğ-i-ni biliyorum* 'I know that Ali came' (Ali-GEN come-NMLZ-POSS.3SG-ACC). Section 2.5 returns to these clauses.

2.3.5 Computational Consequences

These consequences are hypotheses about systems; later chapters test them.

1. **Argument detection.** English-trained parsers learn strong positional cues (subject before the verb, object after it). In Turkic languages, case, agreement and position all carry information, and their weights differ from English. Positional biases learned from English can therefore transfer poorly.
2. **Explicit versus implicit morphology.** A dependency parser can take predicted morphological features as input, predict them jointly with dependencies, or learn morphological cues within contextual representations. The biaffine parser (Dozat and Manning 2017), for example, scores learned token representations without requiring a separate morphological step. Compare otherwise matched models with and without predicted features to test their contribution (Chapter 8).
3. **Case suffixes in NER.** Named entities carry case suffixes in ordinary text: Kazakh *Алматы-дан* 'from Almaty', *Абай-дың* 'of Abai'. Mention detection must preserve the original span; mapping *Алматыдан* to the canonical name *Алматы* for linking or counting is a separate step, which can use a morphological analyser (Chapter 9).
4. **Machine translation.** Translation into a Turkic language must generate the correct case and agreement on the target side. Evaluate case and agreement errors directly rather than assuming that a separate syntactic analysis of the source is required (Chapter 10).

2.4 SOV Base Order and Scrambling

2.4.1 Default SOV Order

The usual unmarked constituent order in Turkic is **subject–object–verb** (SOV). Some contact varieties also have unmarked verb-medial orders (Chapter 8). Johanson (2021, 34) describes Turkic as possessing "a basic left-branching constituent order with dependents preceding their heads" with the unmarked order being "subject + direct object + predicate."

The same modifier-before-head principle places determiners, attributive adjectives and relative clauses before the head noun (Johanson 2021, 34), and Turkic uses postpositions rather than prepositions (Johanson 2021, 36).

2.4.2 Scrambling

While SOV is the default, constituents can be reordered for discourse purposes. The order is tied to information structure: in Turkish, the position directly before the verb is associated with focus, the sentence-initial position with topic, and material after the verb with backgrounded information (Hoffman 1994). The felicity of an order therefore depends on context, which question-answer pairs make visible:

- *Kitabı kim okudu?* 'Who read the book?' → *Kitabı **Ali** okudu.* (OSV; *Ali*, the answer, stands before the verb)
- *Ali ne yaptı?* 'What did Ali do?' → *Ali kitabı okudu.* (SOV)
- *Ali kitabı okudu mu?* 'Did Ali read the book?' → *Okudu Ali kitabı.* (VSO; the verb is in focus, the rest is given)

Other permutations, such as *Ali okudu kitabı* (SVO, with the book as an afterthought), also occur in suitable contexts; Chapter 8 lists all six orders. Not every noun phrase can move equally freely: bare, non-specific objects such as *kitap* in *Ali kitap okudu* normally stay directly before the verb.

2.4.3 Pro-Drop

Turkic languages are **pro-drop**: subjects and objects may be omitted when their referents are recoverable from the context, and person-number agreement largely makes subject pronouns redundant (Johanson 2021, 35). Person-number agreement can help identify an omitted subject, while an overt pronoun can still contribute contrast or discourse information:

- *Ben geliyorum.* (I am coming, with an explicit pronoun)
- *Geliyorum.* (am coming; the pronoun is dropped and *-um* marks 1SG)
- *Geliyor.* (is coming; 3SG, no overt subject and no agreement suffix)

In the third-person singular example *Geliyor*, neither a subject pronoun nor a person suffix is overt; the subject must be inferred from context. In the other persons, as in Spanish or Italian, the agreement suffix identifies the person and number of the dropped subject, but not its referent.

2.4.4 Computational Consequences

1. **Positional features carry different information.** Combine case and agreement with word order. English positional biases may transfer poorly, while Turkic patterns such as preverbal focus and adjacent bare objects can still help.
2. **Sequence labelling.** A tagger trained only on SOV-ordered text may over-rely on position relative to the verb. Training data with natural scrambling, and evaluation on non-canonical orders, show whether this happens.

3. **Pro-drop complicates coreference resolution.** With no overt subject, a system must recover the referent from context. Third-person singular forms may also lack an overt agreement suffix. Evaluate subject resolution in context, including zero arguments that systems designed around overt subjects may overlook.
4. **Pro-drop complicates MT into non-pro-drop languages.** When translating Kazakh or Turkish into English or French, the system must *generate* a pronoun absent in the source, which requires identifying the implicit subject from context.

2.5 Non-Finite Subordination: Participial and Converbal Clauses

2.5.1 Subordination Without Complementisers

English subordinates clauses with function words: “the man **who** came”, “I know **that** he left”, “**after** he arrived”. The Turkic strategy is different: dependent clauses are headed by non-finite forms (action nominals, participles and converbs) carrying bound markers that do the work of English subordinating conjunctions (Johanson 2021, 35). **The verb of the subordinate clause appears in a non-finite form**, and the clause type is marked morphologically on that verb. (Finite alternatives also exist, such as Turkish *ki*-clauses or quotative constructions with *diye* ‘saying’.) Johanson (2021, 34) notes that the “left-branching constituent order” with “dependents preceding their heads” extends to clause structure: embedded clauses appear to the left of their head.

2.5.2 Participial Relative Clauses

In English, “the man who came” contains the relative pronoun *who* and a finite verb *came*. In Turkish, the equivalent is *gelen adam*:

- *gel-* (come) + *-en* (participle) = *gelen* ‘who comes / who came’
- *gelen adam* = the man who came

The participle modifies *adam* ‘man’. The entire clause, including its own arguments and adverbs, appears inside the noun phrase, before the noun:

- *Dün buraya gelen adam* ‘the man who came here yesterday’ (yesterday here come.PTCP man)

Turkish participles differ according to what is relativised. In a common subject relative, the participle is *-(y)An*: *kitabı okuyan öğrenci* ‘the student who read the book’. A common non-subject relative, such as one relativising an object or location, uses *-DIK* for a non-future reading or *-(y)AcAK* for a future reading. The participle carries a possessive suffix agreeing with the genitive subject: *öğrencinin okuduğu kitap* ‘the book that the student read’ (student-GEN read-NMLZ-POSS.3SG book).

Kazakh and Uzbek place participles before the noun too, but without this subject/non-subject split. Both Kazakh *оқушы оқыған кітап* ‘the book that the student read’ and

кітапты оқыған оқушы ‘the student who read the book’ use *-ған*. Uzbek *-gan* works similarly, as in *o‘quvchi o‘qigan kitob*.

Turkmen uses *-An* in both *kitaby okan okuwçy* ‘the student who read the book’ and *okuwçynyň okan kitaby* ‘the book that the student read’. In the second construction, the possessive suffix agreeing with the genitive subject attaches to the head noun, *kitab-y*. Turkish puts that agreement on the participle, *okuduğu*. Adapting a Turkish analyser or annotation scheme therefore requires changing where this agreement is represented.

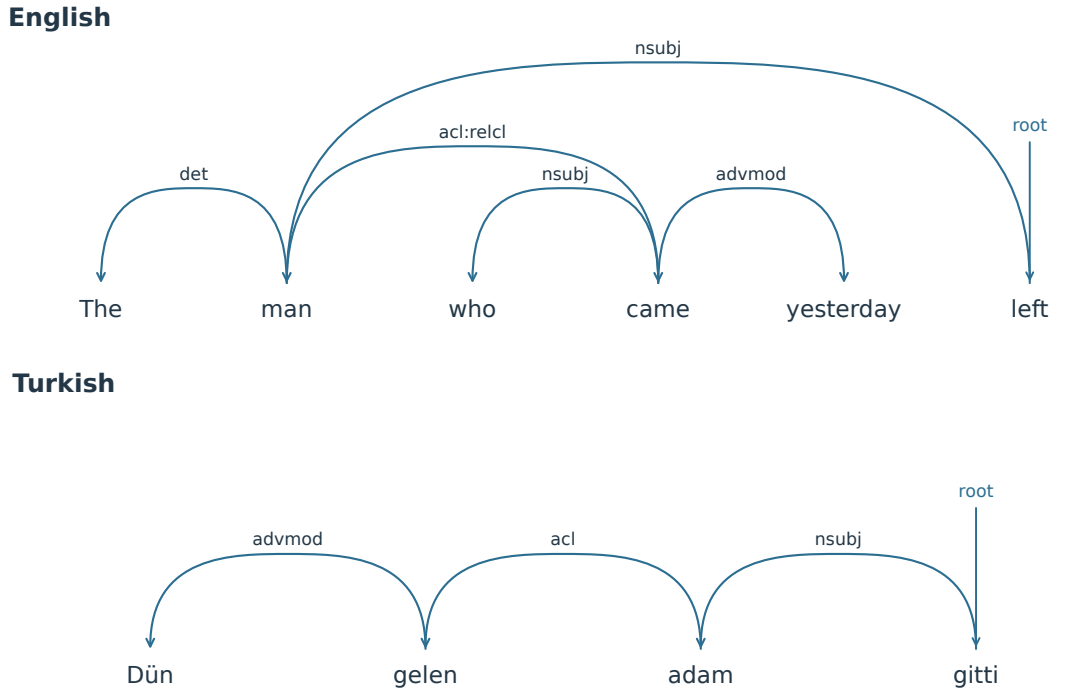


Figure 2.4: Basic UD dependencies for English (top) and Turkish (bottom). English has a post-nominal relative clause with who as the subject of came; Turkish Dün gelen adam gitti has a prenominal participial clause with no relative pronoun. Each word has exactly one head; enhanced coreference edges are omitted.

2.5.3 Converbs and Nominalised Clauses

For adverbial clauses (temporal, conditional, manner), Turkic uses **converbs**, non-finite verb forms that mark the adverbial relation morphologically, alongside nominalised forms followed by case suffixes or postpositions:

- *gel-ince* ‘when/as soon as (someone) came’: converb *-(y)IncA*
- *gel-erek* ‘by coming, while coming’: converb *-(y)ArAk*

- *gel-ip* ‘having come, and then’: converb *-(y)Ip*, which links clauses in a sequence
- *gel-dik-ten sonra* ‘after coming’: nominaliser *-DIK* + ablative + postposition *sonra*, not a converb but a nominalised clause
- *gel-se* ‘if (someone) came’: conditional *-sA*, which can also be fully finite

These constructions mark adverbial relations through verb morphology, sometimes together with a postposition such as *sonra*.

2.5.4 Long-Distance Dependencies

An embedded clause can make a noun phrase long even when the participle stays close to its head noun, as in *gelen adam*. Longer arcs connect the clause’s earlier dependents (*Dün*, *buraya*) to the participle, or connect the whole phrase’s head noun to the final main verb. A left-to-right processor must keep those dependents pending until it reaches their heads (Chapter 8). Postverbal and other dependents that follow their heads create a different processing pattern.

2.5.5 Computational Consequences

1. **Clause boundaries without a marker word.** Systems for event extraction, coreference and semantic role labelling that have been developed for English can use relative pronouns and complementisers as clause-boundary cues. In Turkic text the cue is a suffix on the embedded verb. Whether a given system misses non-finite clauses should be tested, for example by evaluating event recognition separately on finite and non-finite predicates.
2. **Participial forms are morphologically ambiguous.** The ending *-en* in Turkish can be a participle (*gelen* ‘who comes’) or part of other forms; the same string *-DIK* appears in relative clauses, complement clauses and adverbial clauses. Morphological disambiguation helps to identify clause boundaries.
3. **Information extraction must look inside noun phrases.** Events described in participial clauses are embedded inside noun phrases: *öğrencinin okuduğu kitap* contains a reading event. A system that only looks at finite main verbs misses them.

2.6 Shared Patterns and Variation

2.6.1 The Shared Computational Template

Turkic languages share broad structural tendencies, but individual constructions, varieties and contact settings require separate description. The table orients the reader to recurring patterns. Exceptions exist for almost every row:

Table 2.6: Recurring features of the Turkic profile (Johanson 2021, 34–39).

Feature	Status	Example
Agglutinative morphology	Widespread pattern	Multiple suffixes per word, clear boundaries
SOV default order	Widespread pattern	Reordering for information structure; SOV is unmarked
Vowel harmony (at least palatal)	Common, with exceptions	Weakened in standard Uzbek, whose spelling largely fixes suffix vowels (Johanson 2021, p p. 307, 313)
Six commonly compared cases	Common, with exceptions	Nominative, accusative, dative, locative, ablative, genitive; Sakha lacks the genitive
Pro-drop	Widespread pattern	Subject pronouns routinely omitted
Participial relative clauses	Widespread pattern	Embedded clause precedes head noun
Postpositions	Widespread pattern	Spatial relations after the noun
No grammatical gender	Widespread pattern	*o* means he/she/it in Turkish
No definite article	Widespread pattern	Specificity expressed through case marking, word order and context

2.6.2 What Varies

The shared patterns leave room for important differences:

Vowel harmony and its spelling. Turkish and Kyrgyz write backness and rounding harmony; Kazakh writes backness but not rounding; Uzbek suffix vowels are largely fixed; Turkmen writes rounding in some suffixes but not others (Table 2.3); Uyghur writes stem-vowel raising (see the list above). Harmony-based features useful for Turkish therefore cannot be applied identically to Uzbek.

Case inventory. Most languages have the six core cases plus some extras (an instrumental in Chuvash, Kazakh and Sakha, for example). Sakha's system differs most: it has no genitive but has partitive, comitative and comparative cases.

Morpheme inventories. While the template is shared, the specific suffixes differ: aspect markers, evidential categories and certain voice morphemes vary across branches. A morphological analyser built for Turkish cannot be applied to Sakha without new morphotactics, morphophonology and lexicon.

Script. As detailed in Chapter 3, contemporary Turkic languages are written in Latin, Cyrillic and Perso-Arabic scripts, and Old Turkic in a runic script. Kazakh has a planned transition from Cyrillic to Latin, and Uzbek uses Latin officially while Cyrillic remains in wide use. Script differences affect tokenisation and string overlap, so whether to convert scripts before cross-lingual transfer should be tested for each model and task.

Vocabulary. Related languages share vocabulary, but spelling can obscure shared forms. Section 3.6 measures written overlap before and after script conversion.

2.6.3 The Exceptional Cases: Chuvash and Khalaj

Chuvash is the only surviving descendant of the Oghur branch, which split from Common Turkic early (Johanson 2021, 20, 25). Its best-known divergences are rhotacism and lambdacism: where other Turkic languages have *z*, Chuvash has *r*, and where they have *š*, Chuvash has *l* (Chuvash *χi̯l* vs. Common Turkic *ḳïš* 'winter') (Johanson 2021, 23, 26). For NLP, the available evidence suggests testing Oghur transfer separately: on the TurkicNLP morphological challenge set, the rule-based Apertium analyser detected the target phenomena in more Chuvash sentences than the multilingual neural analyser, and the authors attribute this to Chuvash's divergent morphology limiting transfer from the other branches (Hakimov 2026). With 20 sentences per language, the set supports a diagnostic and nothing stronger; it cannot rank languages.

Khalaj (Arghu branch) is a small language of Iran with archaic features, including the retention of primary long vowels (Johanson 2021, 26). Primary long vowels are also retained systematically in Turkmen and Sakha, so vowel length is not unique to Khalaj within the modern family (Johanson 2021, 26; Gökçür 2016). Khalaj has very few NLP resources. In TurkicNLP it has no Apertium transducer; the multilingual neural analyser serves it through Turkish as a proxy language (Hakimov 2026).

2.6.4 Why This Matters: Relatedness as a Hypothesis

Related languages offer analyses, annotation conventions and modelling experience that can be reused. Apertium's Turkic analysers share a formalism and tag convention across about 20 languages; TurkicNLP packages 20 of them. Multilingual neural models can also serve languages with little training data through proxy-language representations (Hakimov 2026). Adaptation still requires a language-specific check of the lexicon, morphotactics, morphophonology, script and annotation assumptions.

Treat relatedness as a reason to test a donor, and use target-language evidence to decide whether it is useful. For a practitioner starting NLP work on a new Turkic language with minimal resources, a reasonable procedure is:

1. List candidate donors: closely related languages and the strongest available models, recording their training data, scripts and annotation conventions.
2. Set aside target-language development and test data before adapting anything.
3. Compare donors on the development set, with native-script input and, where supported, input converted to a common script (Chapter 3).
4. Adapt the selected donor, using separate training data where needed. Fix the system before evaluating it on the untouched test set.

The following chapters develop this procedure concretely.

2.7 Hands-On: Counting Segments in Turkish, Kazakh and Turkmen

Question. Where does automatic segmentation agree with the stated surface analysis?

Inputs. The Turkish, Kazakh and Turkmen sentences below; TurkicNLP 0.5.0 and the named transducers.

Compute. CPU; download the required analyser resources first.

Output. Per-word segments, exact agreement and segments per word.

Interpretation. Inspect boundary errors and analytic constructions before comparing averages.

Start by segmenting a few words by hand under a stated convention, then compare the analyser's output. This experiment reports surface segments for that sample and configuration. A larger corpus study would also report errors and coverage alongside the mean.

Counting convention. As in Table 2.2, count the stem and every overt affix. Attach buffer consonants to the following suffix (*dığ-ı-ndan*), stem-final alternations stay in the stem (*kitab-ı*), and zero morphemes (3SG agreement) are not counted. This is the convention of the TurkicNLP segmentation test set (Hakimov 2026); other conventions give other counts.

Setup. `pip install "turkicnlp[hfst,transformers]==0.5.0"`. The segmenter combines Apertium readings (`hfst`) with a multilingual neural analyser (`transformers`). It runs with only one of them, so the code prints which analysers were actually loaded; report this with any result. Before the first run, download each language's morphology resources with `turkicnlp.download("tur", processors=["morph"])` and the equivalent calls for `"kaz"` and `"tuk"`; model downloads require network access.

```
import turkicnlp
from turkicnlp.processors.morpheme_tokenizer import MorphemeTokenizer

# Reference segmentations under the exercise convention: stem + overt affixes; buffer
# consonants go with the following suffix; zero morphemes are not counted.
reference = {
    "tur": [("Ali", "Ali"), ("kitabı", "kitab-ı"), ("okuyamadığından",
     ↪  "oku-ya-ma-dığ-ı-ndan"),
            ("çok", "çok"), ("üzgündü", "üzgün-dü")],
    "kaz": [("Әли", "Әли"), ("кітапты", "кітап-ты"), ("оқи", "оқ-и"),
            ("алмағандықтан", "ал-ма-ған-дық-тан"), ("өте", "өте"),
            ("қайғылы", "қайғы-лы"), ("болды", "бол-ды")],
    "tuk": [("Ali", "Ali"), ("kitaby", "kitab-y"), ("okap", "oka-p"),
            ("bilmedigi", "bil-me-dig-i"), ("üçin", "üçin"), ("gaty", "gaty"),
            ("gamgyn", "gamgyn"), ("boldy", "bol-dy")],
}

print("turkicnlp", turkicnlp.__version__)
for lang, words in reference.items():
    seg = MorphemeTokenizer(lang=lang)
    seg.load()
    # private flags in 0.5.0: which analysers were actually loaded
    print(f"{lang}: neural={seg._neural_available} hfst={seg._hfst_available}")
    n_reference = n_auto = exact = 0
    for word, g in words:
        r = seg.segment(word)
        auto = "-".join(m for m, _ in r.labeled)
        n_reference += len(g.split("-")); n_auto += len(r.labeled); exact += auto == g
        print(f"  {word:<17} reference {g:<22} auto {auto:<22} [{r.source}]")
```

```
    n = len(words)
    print(f"  exact {exact}/{n}; segments per word: reference {n_reference/n:.2f}, automatic
     ↪  {n_auto/n:.2f}\n")
```

The output below was reproduced on 13 October 2026 using the local toolkit source reporting version 0.5.0, with only the Apertium transducers available (the neural model was not installed). The package version alone is not an exact source pin:

```
turkicnlp 0.5.0
tur: neural=False hfst=True
  Ali               reference Ali                    auto Ali                    [hfst]
  kitabı            reference kitab-ı                auto kitab-ı                [hfst]
  okuyamadığından   reference oku-ya-ma-dığ-ı-ndan   auto oku-ya-ma-dı-ğından    [hfst]
  çok               reference çok                    auto çok                    [hfst]
  üzgündü           reference üzgün-dü               auto üzgün-dü               [hfst]
  exact 4/5; segments per word: reference 2.40, automatic 2.20

kaz: neural=False hfst=True
  әли               reference әли                    auto әли                    [hfst]
  кітапты           reference кітап-ты               auto кітап-ты               [hfst]
  оқи               reference оқ-и                   auto оқ-и                   [hfst]
  алмағандықтан     reference ал-ма-ған-дық-тан      auto ал-ма-ған-ды-қтан      [hfst]
  өте               reference өте                    auto өт-е                   [hfst]
  қайғылы           reference қайғы-лы               auto қайғы-лы               [hfst]
  болды             reference бол-ды                 auto бол-ды                 [hfst]
  exact 5/7; segments per word: reference 2.14, automatic 2.29

tuk: neural=False hfst=True
  Ali               reference Ali                    auto Ali                    [none]
  kitaby            reference kitab-y                auto kitab-y                [hfst]
  okap              reference oka-p                  auto oka-p                  [hfst]
  bilmedigi         reference bil-me-dig-i           auto bil-me-di-gi           [hfst]
  üçin              reference üçin                   auto üçin                   [hfst]
  gaty              reference gaty                   auto gat-y                  [hfst]
  gamgyn            reference gamgyn                 auto gamgyn                 [hfst]
  boldy             reference bol-dy                 auto bol-dy                 [hfst]
  exact 6/8; segments per word: reference 1.75, automatic 1.88
```

The Turkmen lines were produced on 18 October 2026 with the same toolkit version and the Apertium Turkmen transducer (beta; SHA-256 of `tuk.automorf.hfst` begins `528202a5`). The Turkmen sentence *Ali kitaby okap bilmedigi üçin gaty gamgyn boldy* ‘Ali was very sad because he could not read the book’ parallels the other two.

One reference segmentation follows the spelling rather than the morphology: Kazakh *оқи* is the stem *оқы-* ‘read’ plus the converb *-й*, which the orthography writes as *и* (ы + й → и); the convention assigns *-и* to the suffix. The examples show two kinds of error. Boundaries are misplaced in the nominalised forms *okuyamadığından*, *алмағандықтан* and *bilmedigi*. Short words can also receive the wrong reading: Kazakh *өте* ‘very’ is split as a form of *өт-* ‘pass’, and Turkmen *gaty* ‘very’ as an accusative of *gat* ‘layer’.

Word boundaries also affect the average. Turkish expresses inability in one word here; Kazakh *оқи алмағандықтан* and Turkmen *okap bilmedigi* use two. Turkmen adds the postposition *üçin* where the other examples use a case suffix. The resulting averages describe these sentences under this counting convention, not the languages as a whole.

For larger samples, the TurkicNLP paper reports exact segmentation of 89% for the hybrid segmenter, against 79% for Apertium alone and 68% for the neural analyser alone,

on a test set of 483 words in 13 categories across 20 languages (Hakimov 2026). The authors note that the set is small, that the gold segmentations for nine languages are drafts awaiting native-speaker verification, and that it was used during development, so the scores are likely optimistic. Each result records which analysis produced it (`seg.segment(word).source` is `"hfst"`, `"neural"` or `"greedy"`).

Status: demonstration. The outputs above are recorded. Exercise 5 adds a related segmentation check.

What to Take Forward

Morphemes, written words and model tokens are different units, and a system's score depends on which one it counts. Harmony changes suffix forms; case, word order and context express grammatical relations; participles and converbs connect clauses. The segmentation comparison illustrates how finite-state and neural analyses can complement each other, though its small, development-used set still includes draft gold analyses. Before evaluating a system, decide which construction and counting unit the task needs. Chapter 3 turns to scripts and spelling conventions.

Further Reading

- **Turkic morphology and syntax**: Johanson (2021) (especially chapters 14–15 on phoneme systems and harmony) is the primary linguistic reference for the features described in this chapter.
- **Agglutination and NLP**: Oflazer (1994) gives a two-level description of Turkish morphology, an approach that later Turkic finite-state analysers build on.
- **Word order and information structure**: Hoffman (1994) relates Turkish word orders to topic and focus.
- **Subword tokenisation**: Sennrich, Haddow, and Birch (2016b) describes BPE; Rust et al. (2021) and Petrov et al. (2023) show how multilingual tokenisers disadvantage many languages; Hakimov (2026) measures this for 20 Turkic languages.
- **Kardeş-NLU**: Senel et al. (2024) shows that a high-resource relative (Turkish) helps cross-lingual transfer to five other Turkic languages.
- **Universal Dependencies and Turkic syntax**: Nivre et al. (2016) and Türk et al. (2022) cover the UD annotation framework applied to Turkish.

Exercises

1. **Segmenting words.** Segment *evlerimizden* and explain the contribution of each piece. Then compare *yazıldılar* and *yazılamadılar*. Use the analyses *yaz-ıl-dı-lar* and *yaz-ıl-a-ma-dı-lar* to identify passive voice, past and agreement, and explain how the second form adds negative ability. Under this chapter's surface-segmentation

convention, how many pieces does each word contain? Why need this count not equal the number of language-model tokens?

2. **Vowel harmony and consonant alternation.** Add the plural suffix to *ev*, *araba*, *göz* and *kol*. Separately, form the third-person singular past of *gel-*, *bak-*, *gör-*, *bul-* and *git-*. Identify which changes follow vowel harmony and which come from consonant assimilation. Compare a regular example with one of the exceptional stems in Section 2.2.

3. **Case and word order.** Translate *Kitabı Ayşe okudu* and *Ayşe kitabı okudu* into English. Give a question that each order would answer naturally. Then compare *Ayşe kitap okudu*: what changes when the accusative is absent? Write the sentence with the subject omitted (pro-drop) and explain what must be recovered from context.

4. **Participial clauses.** Translate *Dün şehre gelen adam* and *Ali'nin okuduğu kitap*. Identify the participle in each, say whether the head noun is the subject or a non-subject of the relative clause, and sketch the dependency structure.

5. **Checking a segmenter.** Run `MorphemeTokenizer(lang="tur")` on *yazıldılar* and *yazılamadılar* and on the Kazakh *жазылды* and *жазыла алмады*. Report which analysers were loaded. Where does the segmenter agree with your segmentation from Exercise 1, and where does it differ?

6. **Segments per word across the family.** Take the same short text in Turkish, Kazakh, Uzbek and Tatar (for example a FLORES-200 paragraph). Segment 30 words per language by hand under the chapter's convention, then compare with the automatic output as in the Hands-On section. Report exact agreement, the gold and automatic means, and the words where analytic constructions change the word count.

7. **Choosing a donor.** You need a Kyrgyz dependency parser and have no Kyrgyz training data. Propose Turkish and Kazakh donor systems, recording their training data, input scripts and annotation conventions, and explain why either might be preferable. Design a small target-language evaluation comparing the available systems and, where supported, native-script and transliterated input. Which result would justify the cost of script conversion?

Part II

Writing Systems and Corpora

3 Scripts, Encoding, and Orthographic Engineering

Learning Objectives

After reading this chapter, you will be able to:

- Identify which script each major Turkic language uses and when transitions occurred
- Explain the specific Unicode challenges posed by Turkic scripts, including the Turkish İ/I problem and Cyrillic homoglyphs
- Implement table-driven transliteration with a greedy longest-match algorithm, and explain where context rules and lexicons are needed
- Diagnose encoding problems, and normalise informal Turkic text (diacritic dropping, apostrophe variants, homoglyphs) without destroying genuine content
- Design a preprocessing pipeline that keeps the original text, decides language and script before language-specific repairs, and records every derived view
- Measure how script conversion changes tokenisation cost, and use a common script as a technical interoperability layer
- Identify closely related Turkic languages automatically, including when to abstain, and build a corpus that records language, script, and orthography version

A Turkic corpus may contain several scripts and spelling conventions for the same language. Historical collections add Old Turkic runic, Old Uyghur and earlier Perso-Arabic traditions (Johanson 2021, chap. 19). Processing these texts requires a clear record of the input script, the orthography and any conversion applied.

This chapter answers the practical question: given a corpus of Turkic text, what do you need to do before you can run any NLP tool on it?

3.1 A Map of Turkic Scripts

3.1.1 Contemporary Scripts and Historical Collections

Most contemporary Turkic text uses Latin, Cyrillic or Perso-Arabic script, with conventions that vary by language, region and period. Historical collections add Old Turkic runic, Old

Uyghur and other scripts. A preprocessing system must distinguish the script family, the language's alphabet and the orthographic reform on which a conversion table is based.

Latin. Contemporary Latin orthographies include Turkish, Azerbaijani, Uzbek, Turkmen, Crimean Tatar, Gagauz and Karakalpak. Several reflect reforms after the Soviet period; others have earlier histories (Johanson 2021, 421–25). Kazakhstan has published multiple Latin designs (Section 3.2). A Latin script label alone is insufficient: Turkish *ş* and Uzbek *sh*, for example, require different conversion rules.

Cyrillic. Cyrillic-based orthographies are used for Kazakh, Kyrgyz, Tatar, Bashkir, Chuvash, Sakha and several South Siberian languages. Cyrillic alphabets were introduced for nearly all Turkic languages of the Soviet Union between 1937 and 1940, but Chuvash and Sakha have earlier Cyrillic histories (Johanson 2021, 415–20). Each orthography has its own additional letters and spelling conventions.

Perso-Arabic. This script family is used for Uyghur in Xinjiang, Azerbaijani in Iran, and Kazakh and Kyrgyz in China, as well as historical Ottoman and Chaghatay collections (Johanson 2021, 411–13, 419, 423). Modern Uyghur writes vowel qualities explicitly; this facilitates conversion but does not make its orthography a complete phonetic transcription.

3.1.2 Language-to-Script Mapping

Table 3.1: Selected language–script relationships. Historical dates follow the cited accounts; script prevalence and current legal status are not inferred from them.

Language	Scripts and regional scope	Historical or implementation note	Source
Turkish	Latin; Perso-Arabic in Ottoman collections	Reform ordained in 1928; the reference discusses implementation in 1929	(Johanson 2021, pp. 411, 421)
Azerbaijani	Latin in Azerbaijan; Perso-Arabic in Iran; Cyrillic in historical material	An early Soviet Latin alphabet was adopted in 1922; this is not the date of exclusive use	(Johanson 2021, pp. 421, 423)
Uzbek	Latin and Cyrillic; Perso-Arabic in historical collections	Latin reform enacted in 1993 and finalised in 1995; this does not measure corpus uptake	(Johanson 2021, 424)
Kazakh	Cyrillic; Perso-Arabic in China; multiple Latin designs	National designs and the 2026 commission recommendation are distinguished in the section on active transitions	(Johanson 2021, pp. 413, 419, 422); government and TDK sources in the section on active transitions

Language	Scripts and regional scope	Historical or implementation note	Source
Uyghur	Perso-Arabic in Xinjiang; Cyrillic in Central Asia; Latin transliteration	Johanson dates the pinyin-based script experiment to 1964–1983	(Johanson 2021, pp. 413, 415, 422)
Turkmen	Latin; Cyrillic in older material	The source discusses successive post-Soviet designs; a single date would conceal revisions	(Johanson 2021, pp. 422–423)
Kyrgyz	Cyrillic in Kyrgyzstan; Perso-Arabic in Xinjiang	A 2026 commission Latin proposal is documented in the section on active transitions, not treated as national enactment	(Johanson 2021, 419); TDK source in the section on active transitions
Tatar	Cyrillic; historical Perso-Arabic and Latin	Soviet Latinisation and subsequent Cyrillic reform are separate stages	(Johanson 2021, pp. 415–418, 422)
Bashkir	Cyrillic; historical Perso-Arabic and Latin	Its Cyrillic spelling differs from Tatar despite shared letters	(Johanson 2021, pp. 418–419, 422)
Chuvash	Cyrillic	Yakovlev system introduced in 1871–1873; modified in 1938	(Johanson 2021, pp. 415, 420)
Sakha	Cyrillic; historical Latin-based systems	Roman-based designs of 1917 and 1922–1923 precede the modern Cyrillic norm	(Johanson 2021, pp. 415, 420–421)
Karakalpak	Latin and Cyrillic	Johanson records a 1997 Latin adoption; the toolkit implements a later 2016 mapping	(Johanson 2021, 425); (Hakimov 2026, sec. 4.2)
Crimean Tatar	Latin and Cyrillic; historical Perso-Arabic	Distinguish historical Latinisation from the mapping used by a current converter	(Johanson 2021, 422); (Hakimov 2026)

The dates in Table 3.1 refer to specific reforms or source accounts. Writers may change script at different times. Record legal adoption, alphabet publication, school implementation and corpus prevalence separately; one transition date cannot describe them all.

3.1.3 Historical Scripts

Historical collections involve two further scripts and two Perso-Arabic literary traditions:

- **Old Turkic runic (Orkhon script)**: used in inscriptions from the early eighth century onward in present-day Mongolia and the Yenisei Basin (Johanson 2021, 159); it has its own Unicode block
- **Old Uyghur script** (not to be confused with modern Uyghur Perso-Arabic): an alphabet modelled on the Soghdian script, used for Old Uyghur manuscripts (Johanson 2021, 410)
- **Chaghatay**: the Persian-influenced literary language of Central Asia from the fifteenth century on (Johanson 2021, 162), written with Perso-Arabic conventions; a language and literary tradition, not a separate script
- **Ottoman Turkish**: written with Perso-Arabic conventions in literature and administration until the 1928 reform

These historical scripts are relevant for heritage document digitisation and require specialised OCR (Section 17.2.2). For Ottoman Turkish, two UD treebanks exist (Chapter 8), and TurkicNLP converts Latin transcriptions into Perso-Arabic script and processes Ottoman text with a dedicated Stanza model (Hakimov 2026); Old Turkic runes can be converted to Latin transcription. An annotated Chaghatay manuscript corpus contains about 30,700 words (Steimel et al. 2020).

3.2 The Active Transitions

3.2.1 Kazakhstan: Published Designs and Implementation

Kazakh corpus processing must distinguish several published Latin designs. The following milestones explain the mappings encountered in this book; they are not a legal-status register:

- **2017**: A first version that marked the specifically Kazakh sounds with apostrophes after a base letter (e.g., *a’* for ә, *s’* for ш, *n’* for ң), while қ was already written *q* (*Qazaqstan*). Apostrophe-based letters are fragile in digital text: keyboard and typographic apostrophes are different Unicode characters, which breaks search and indexing.
- **2018**: The revised alphabet used acute accents such as *á*, *ǵ* and *ń*, but wrote *ш* with the digraph *Sh sh*, not *ś*; see the reproduced appendix to Decree No. 637 of 19 February 2018.
- **2021**: A further revision with Turkic-style diacritics, e.g. *ä* (ә), *ğ* (ғ), *ñ* (ң), *ö* (ө), *ş* (ш), *ū* (ұ), *ü* (ү), released as a draft for public discussion. The 2023–2029 language-policy concept makes the transition schedule depend on a further legal act. These documents show that a draft and a policy framework were published. The current legal status of any later alphabet has to be checked elsewhere.
- **2026**: The Turkish Language Association reports that the common-alphabet commission accepted Kazakh and Kyrgyz alphabet proposals as reference documents at its meeting on 15 June 2026. A commission recommendation is not national enactment. The toolkit mapping described below does not implement a newly inferred 2026 standard.

For Kazakh-to-Latin conversion, this book uses the toolkit's table based on the 2021 proposal. Record that mapping version with the corpus. It identifies how the text was converted, without implying national adoption or widespread use.

What this means for corpus building. A Kazakh corpus may contain: 1. Cyrillic 2. Latin text following one of the published Latin designs 3. Informal Latin, including ASCII approximations without diacritics (*a* for *ä*, *s* or *sh* for *ş*)

A Kazakh pipeline can convert text to a shared script in a derived view, or train on the scripts it expects to encounter. Compare the two approaches on the intended task. Assuming a single spelling convention will leave mismatches in a mixed corpus.

3.2.2 Uzbekistan: The Persistent Cyrillic

The transition to the Uzbek Latin alphabet was adopted in 1993 and revised in 1995 as a replacement for Cyrillic (Johanson 2021, 424). More than three decades later, both scripts are still in everyday use: the relevant corpus may include both Latin and Cyrillic documents. How widely writers in a given domain actually follow the legal standard is a separate, empirical question.

This duality creates a potential distribution shift for a model trained in one script. Compare native-script processing, transliteration and mixed-script training on a held-out sample from the intended domain before choosing a pipeline.

3.2.3 Corpus Implications

For any NLP practitioner working with Kazakh or Uzbek:

- **Detect the script** of each document, and of each span in mixed documents, before processing (Section 3.4 shows the toolkit functions)
- **Keep the original** and create converted text only as a derived view, or train script-robust models
- **Test transliteration** to use resources from one script variant for the other (Section 3.4)
- **Do not assume** that a web-crawled corpus is monoscript; even within a single document, informal text mixes scripts. A change of script is not necessarily a change of language: record script and language as separate metadata

3.3 Unicode for Turkic Languages

3.3.1 Relevant Unicode Blocks

Each Turkic script family occupies specific Unicode code point ranges:

Table 3.2: Unicode blocks relevant to Turkic language NLP. Only the most frequently encountered ranges are shown. Note that the Latin schwa is split across two blocks (capital Ə in Latin Extended-B, small ə in IPA Extensions), and that the Cyrillic Ә (U+04D8) is a different character that looks the same.

Script	Primary Block	Range	Key Characters
Latin	Latin Extended-A	U+0100–U+017F	ğ (U+011F), ş (U+015F), ı (U+0131), İ (U+0130)
Latin	Latin Extended-B	U+0180–U+024F	Ə (U+018F, capital schwa)
Latin	IPA Extensions	U+0250–U+02AF	ə (U+0259, small schwa)
Latin	Spacing Modifier Letters	U+02B0–U+02FF	ʻ (U+02BB), ʼ (U+02BC), used in Uzbek
Cyrillic	Cyrillic	U+0400–U+04FF	Ә ә (U+04D8/U+04D9), Ң (U+04A2), Ғ (U+0492), Қ (U+049A)
Cyrillic	Cyrillic Supplement	U+0500–U+052F	Additional characters for some languages
Arabic/Perso-Arabic	Arabic	U+0600–U+06FF	Core Arabic + Persian letters
Arabic	Arabic Presentation Forms-A/B	U+FB50–U+FDFF, U+FE70–U+FEFF	Legacy contextual glyph forms; apply a selective compatibility policy (Section 3.3)
Old Turkic	Old Turkic	U+10C00–U+10C4F	Orkhon and Yenisei runic characters

3.3.2 The Turkish İ/I Problem

Turkish has **four** distinct characters where English has two:

Character	Unicode	Name	Example
`İ`	U+0130	Latin Capital Letter I with Dot Above	*İstanbul*
`I`	U+0049	Latin Capital Letter I (dotless)	*Isparta*
`i`	U+0069	Latin Small Letter I (dotted)	*istanbul* (lowercase of *İstanbul*)
`ı`	U+0131	Latin Small Letter Dotless I	*ısparta* (lowercase of *Isparta*)

The pairs are: `İ` ↔ `i` (dotted I, both cases) and `I` ↔ `ı` (dotless I, both cases).

Python's `str.lower()` and `str.upper()` apply the language-independent Unicode case mappings. They are **not** locale-aware (unlike, e.g., Java's `toLowerCase(Locale)`). For Turkish this gives two different errors:

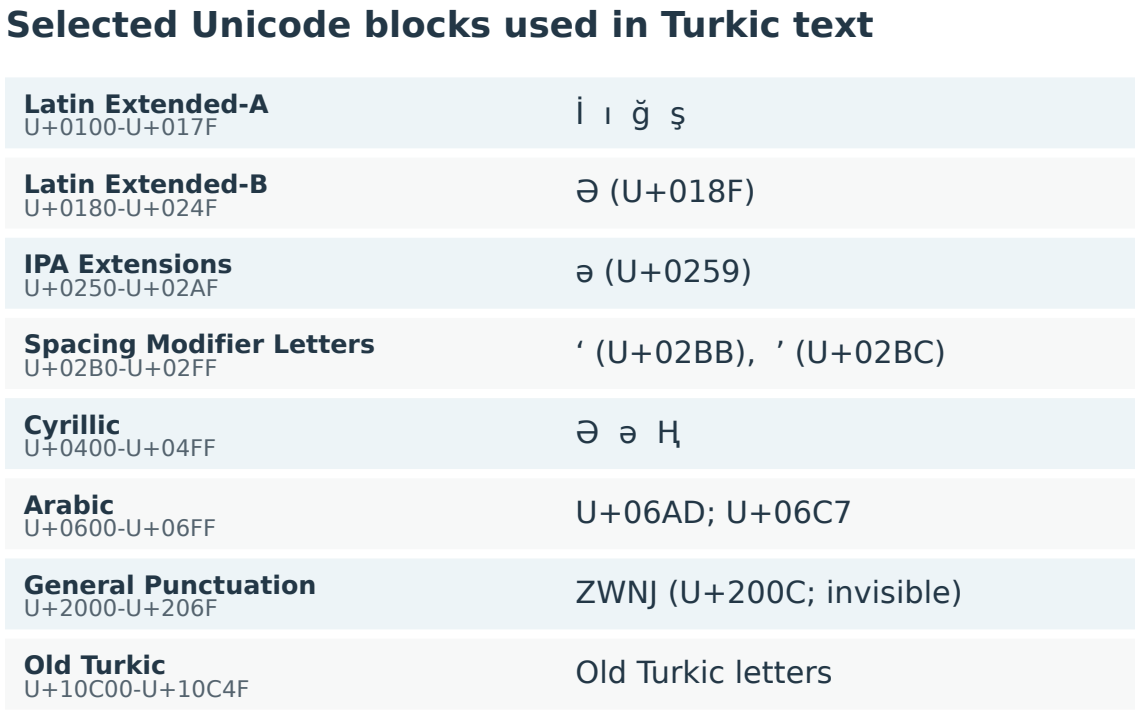


Figure 3.1: Selected Unicode blocks used in Turkic text, following Unicode 17.0. Latin Ə/ə and Cyrillic Ә/ə occupy different blocks; the zero-width non-joiner is in General Punctuation. Band widths are schematic.

```
"ISPARTA".lower()        # → "isparta"   WRONG: should be "ısparta" (I ↔ ı)
"İstanbul".lower()       # → "i stanbul"  WRONG: 'i' + U+0307 COMBINING DOT ABOVE
len("İ".lower())         # → 2  — the string even changes length
"istanbul".upper()       # → "ISTANBUL"  WRONG: should be "İSTANBUL"
```

A correct helper must also handle the decomposed spelling of *İ* (`I` followed by U+0307 COMBINING DOT ABOVE), which is canonically equivalent to U+0130. Normalising to NFC first takes care of it:

```
import unicodedata

def turkish_lower_nfc(text: str) -> str:
    """Turkish/Azerbaijani lowercasing for ordinary text; returns an NFC view."""
    text = unicodedata.normalize("NFC", text)
    text = text.translate(str.maketrans({"İ": "i", "I": "ı"}))
    return unicodedata.normalize("NFC", text.lower())

def turkish_upper_nfc(text: str) -> str:
    """Turkish/Azerbaijani uppercasing for ordinary text; returns an NFC view."""
    text = unicodedata.normalize("NFC", text)
    text = text.translate(str.maketrans({"i": "İ", "ı": "I"}))
    return unicodedata.normalize("NFC", text.upper())

assert turkish_lower_nfc("ISPARTA") == "ısparta"
assert turkish_lower_nfc("İSTANBUL") == "istanbul"
assert turkish_lower_nfc("I\u0307STANBUL") == "istanbul"      # decomposed İ
assert turkish_upper_nfc("istanbul ısparta") == "İSTANBUL ISPARTA"

# TurkicNLP ships the mapping step as a helper: it maps İ → i and I → ı
from turkicnlp.utils.unicode_utils import normalize_turkish_i
normalize_turkish_i("ISPARTA").lower()   # → "ısparta"
```

These helpers handle the ordinary Turkish dotted/dotless pairs and canonical normalisation. Unicode's SpecialCasing includes further context-sensitive rules. Apply the helpers only after identifying the language: even English or Russian material within a Turkish corpus can contain *INFO*, which they turn into *ınfo*. Casing may also change string length, so offsets must identify the text view they refer to.

The combining-dot result can be hard to spot: it survives into vocabularies and makes `"i̇stanbul" != "istanbul"`. The dotted/dotless distinction applies to Turkish, Azerbaijani, Crimean Tatar, Gagauz, Tatar Latin and the Kazakh Latin proposals. For these orthographies, calling `lower()` without an explicit mapping introduces systematic errors in case folding, vocabulary building and string comparison. Lowercasing itself is optional: cased models and annotation tasks often need the original case, so keep it unless the task requires folding.

3.3.3 Cyrillic/Latin Homoglyphs

Several characters appear visually identical in Cyrillic and Latin but have different Unicode codepoints:

Character	Cyrillic codepoint	Latin codepoint	Problem
`а`	U+0430	`a` U+0061	Sort order, script detection
`е`	U+0435	`e` U+0065	Word matching
`о`	U+043E	`o` U+006F	Vocabulary contamination
`р`	U+0440	`p` U+0070	Word matching
`ә`	U+04D9	`ə` U+0259	Azerbaijani/Kazakh: the Cyrillic and Latin schwa look identical
`і`	U+0456	`i` U+0069	Kazakh *i* typed with a Latin *i*
`һ`	U+04BB	`h` U+0068	Kazakh, Tatar, Bashkir, Sakha
`ј`	U+0458	`j` U+006A	Old Azerbaijani Cyrillic

These homoglyphs cause subtle failures:

- Mixing Cyrillic `а` and Latin `a` can change tokenisation and cause dictionary lookup failures
- Script detection based on character statistics may misclassify mixed-script text
- String comparison (`"alma" == "alma"` is false: the second string starts with Cyrillic U+0430)

Unicode normalisation does not help here: NFC and NFKC leave Latin `a` and Cyrillic `а` distinct, because they *are* different characters. To find them, inspect the characters themselves:

```
import unicodedata
for ch in "aa аә":
    if not ch.isspace():
        print(repr(ch), hex(ord(ch)), unicodedata.name(ch))
# 'a' 0x430 CYRILLIC SMALL LETTER A
# 'a' 0x61 LATIN SMALL LETTER A
# 'ә' 0x4d9 CYRILLIC SMALL LETTER SCHWA
# 'ə' 0x259 LATIN SMALL LETTER SCHWA
```

For a general script test, use the Unicode Script and Script_Extensions properties (UAX #24) rather than block ranges. Latin letters span several blocks. Many digits and punctuation marks have the Script value `Common`, and many combining marks have `Inherited`; others are script-specific. Script_Extensions records use across scripts. Character names, as above, are a teaching approximation. Repairing look-alikes is a separate, language-dependent decision (Section 3.5).

3.3.4 Arabic-Script Text: Uyghur and Ottoman

Uyghur text in the Perso-Arabic script is written right-to-left (RTL). Three issues matter in practice:

1. **Presentation forms**: older Uyghur web text and PDFs were often produced with fonts and converters that store contextual glyph shapes (Arabic Presentation Forms-A/B) instead of the base letters. Visually identical words then have different code points. NFKC normalisation maps them back to the base letters; NFC does not.
2. **Letter variants**: Uyghur-specific letters ۆ (ۇ) ۋ (ۈ) ې (ۋ) are in the basic Arabic block, but look-alike Persian, Arabic, and Kazakh/Kyrgyz Arabic letters (e.g., different yeh and kaf forms) occur in real text and need a mapping table.
3. **Joiners and punctuation**: zero-width joiners/non-joiners (U+200C/U+200D) and Arabic punctuation ،) ؛ (؟ must be handled by the tokeniser; TurkicNLP uses a separate Arabic-script tokeniser for this reason (Hakimov 2026).

Because the reformed Uyghur orthography marks vowels explicitly (Johanson 2021, 413), modern Uyghur does not have the same systematic missing-vowel problem as unvocalised Arabic or Persian. Missing vowels remain an issue for **Ottoman Turkish** and Chaghatay, whose Perso-Arabic orthography leaves many vowels unwritten.

3.3.5 Unicode Normalisation: NFC vs. NFD

Unicode allows characters to be represented in different ways. For example, the Turkish ş can be represented as:

- **Precomposed (NFC)**: a single code point U+015F
- **Decomposed (NFD)**: the base letter s (U+0073) followed by a combining cedilla (U+0327)

Much modern text is in NFC, but text from some sources (older databases, some operating systems and input methods) can contain NFD. An NLP system that encounters both will treat ş (NFC) and ş (NFD) as different strings. The failure is silent and hard to debug.

Best practice: keep the original text, and apply `unicodedata.normalize('NFC', text)` as the first step in building the *processing view* of it. NFC makes canonically equivalent strings identical; it does not correct spelling, repair look-alike letters or decide the language (UAX #15).

```
import unicodedata
from turkicnlp.utils.unicode_utils import normalize_nfc, strip_diacritics

def normalize_unicode(text: str) -> str:
    """Apply NFC normalisation as the first preprocessing step."""
    return unicodedata.normalize('NFC', text)     # same as normalize_nfc(text)

strip_diacritics("Türkçe")    # → "Turkce": a lossy view for a declared purpose; retain the
 ↪  original
```

When NFC is not enough: NFC leaves compatibility characters untouched: Arabic presentation forms, full-width Latin letters, ligatures such as *fi*. NFKC folds them, but it also folds characters you may want to keep (superscripts, some punctuation), including in the Latin and symbol content embedded in an Arabic-script document. A practical recipe is NFC for all text, plus NFKC applied only to the characters in the Arabic Presentation Forms blocks, and an explicit, context-aware treatment of apostrophe variants (Section 3.5).

3.3.6 Encoding, Display Order and Visible Characters

Three further problems come before any of the above.

Wrong decoding. Text decoded with the wrong encoding ("mojibake") is a different string, and no Unicode normalisation can repair it. Kazakh UTF-8 bytes read as Windows-1251 look like this:

```
s = "Қазақстан".encode("utf-8").decode("cp1251")
print(s)                                        # ТљР°Р·Р°Т›СЃС,Р°РЅ
print(s.encode("cp1251").decode("utf-8"))       # Қазақстан  (reversible only if no byte was
 ↪  lost)
```

Decode with the known encoding, record failures instead of silently dropping bytes, and check samples of every source; a UTF-8 byte-order mark (U+FEFF) at the start of a file should be removed on decoding (`encoding="utf-8-sig"`).

Logical versus visual order. Uyghur Perso-Arabic text is stored in *logical* order (the order in which it is read) and displayed right to left; numbers and embedded Latin words are displayed left to right inside it, following the Unicode Bidirectional Algorithm (UAX #9). The string "ئۇيغۇرچە Python يىلى-2024" is stored with ئ first, whatever it looks like on screen. Never reverse stored strings for NLP; only display software applies the visual order.

Characters versus what the reader sees. `len()` counts code points, not visible characters: decomposed *ş* is two code points, and an Arabic letter with a combining mark is one visible unit. User-facing offsets and truncation should use grapheme clusters (UAX #29; e.g. the `regex` module's `\X`).

3.4 Bidirectional Transliteration

3.4.1 Why Transliteration Matters Computationally

Shared morphology and vocabulary offer little direct help to a model if unfamiliar spelling hides them. A multilingual or byte-level model can accept Kyrgyz Cyrillic, yet a model fine-tuned on Turkish Latin may still represent it poorly. Transliteration offers one possible remedy. Test it against the same model with native-script input.

Transliteration maps one written representation to another; transcription, by contrast, aims to represent sounds. *Алматы* (Cyrillic) becomes *Almaty* (Latin). A good mapping

preserves the information in the spelling, but some mappings are lossy: where two source letters map to one target letter, the distinction is gone (see the hard cases below).

The TurkicNLP toolkit (Hakimov 2026) supports conversion for ten multi-script Turkic languages, following national standards or, for Kazakh, the 2021 proposal, plus conversion of 23 languages into the toolkit's Common Turkic Script (CTS) (Section 3.6). Two of the pairs (Ottoman Turkish, Old Turkic) work in one direction only:

Table 3.5: Transliteration pairs and mapping standards in TurkicNLP 0.5.0. Ottoman Turkish (Latin transcription to Perso-Arabic) and Old Turkic (runes to Latin transcription) are one-way reconstructions, not reversible conversions.

Language	ISO	Direction	Standard
Kazakh	kaz	Cyrl ↔ Latn	2021 proposal (toolkit mapping)
Uzbek	uzb	Cyrl ↔ Latn	Official 1995
Azerbaijani	aze	Cyrl ↔ Latn	Official 1991
Tatar	tat	Cyrl ↔ Latn	Zamanälif
Turkmen	tuk	Cyrl ↔ Latn	Official 1993 (revised)
Karakalpak	kaa	Cyrl ↔ Latn	Official 2016
Crimean Tatar	crh	Cyrl ↔ Latn	Latin standard
Uyghur	uig	Arab ↔ Latn ↔ Cyrl	ULY (Uyghur Latin Yéziqi), Uyghur Cyrillic
Ottoman Turkish	ota	Latn → Arab	Academic
Old Turkic	otk	Orkh → Latn	Turkological convention

3.4.2 The Greedy Longest-Match Algorithm

The core of a table-driven converter fits in a few lines:

```
def longest_match(text: str, mapping: dict[str, str]) -> str:
    """Table lookup only: no case handling, context rules or restoration."""
    if "" in mapping:
        raise ValueError("Mapping keys must be non-empty")
    keys = sorted(mapping, key=len, reverse=True)
    out, pos = [], 0
    while pos < len(text):
        for key in keys:
            if text.startswith(key, pos):
                out.append(mapping[key]); pos += len(key)
                break
        else:
            out.append(text[pos]); pos += 1          # unmatched: copied, and should be
 ↪  logged
    return "".join(out)

# A small demonstration table, not a complete Uzbek converter
demo = {"sh": "ш", "ch": "ч", "o‘": "ў", "s": "с", "h": "ҳ", "o": "о", "a": "а", "q": "қ",
 ↪  "i": "и", "l": "л"}
longest_match("sh", demo)                        # 'ш'   (digraph matched as a unit)
longest_match("sh", {"s": "с", "h": "ҳ"})         # 'сҳ'  (without the digraph entry)
longest_match("o‘qish", demo)                    # 'ўқиш'
longest_match("Shahar", demo)                    # 'Sҳаҳar'  (capitals and 'r' missing from
 ↪  the table)
```

An empty key would consume no input and loop forever, so the code rejects it. With no mappings, it copies the input; with no input, it returns an empty string. A practical converter also needs capitalisation rules, a complete table and a record of characters copied unchanged. Those copies show where conversion fell back to the source. Some letters need context rules too: Uzbek Cyrillic е becomes *ye* initially and after a vowel, but *e* elsewhere.

TurkicNLP extends this greedy longest-match algorithm. At each input position it tries the longest sequence first, up to four characters in this source version, then shortens the match. Uzbek sh, ch and o‘ can thus map as units to ш, ч and ў. Context rules handle position-dependent letters: Cyrillic е can be initial *ye* or elsewhere *e* in Uzbek, Turkmen and Tatar; Crimean Tatar ё/ю represent *ö*/*ü* after consonants (Hakimov 2026).

The algorithm handles capitalisation by checking whether the first character of the source sequence is upper-case and applying the same capitalisation to the output sequence; all-capital words are handled separately. Test any converter on title case, all capitals, digraphs versus separate letters (Uzbek *sh* versus *s’h* in *Is’hoq*), unknown characters, and decomposed input.

3.4.3 Code Example: Script Detection and Transliteration

The following example demonstrates automatic script detection and Kazakh Cyrillic↔Latin transliteration using the TurkicNLP toolkit (adapted from the companion library):

```
from turkicnlp.scripts import Script
from turkicnlp.scripts.detector import detect_script
from turkicnlp.scripts.transliterator import Transliterator

# --- Script Detection ---
cyrl_text = "Мен Алматыда тұрамын."  # Kazakh Cyrillic: "I live in Almaty."
print(detect_script(cyrl_text))       # Script.CYRILLIC

# --- Cyrillic → Latin (toolkit mapping based on the 2021 proposal) ---
t_cyrl_latn = Transliterator("kaz", Script.CYRILLIC, Script.LATIN)
latin_text = t_cyrl_latn.transliterate(cyrl_text)
print(latin_text)   # Men Almatyda tūramyn.   (ұ → ū in the 2021 alphabet)

# --- Latin → Cyrillic (reverse direction) ---
t_latn_cyrl = Transliterator("kaz", Script.LATIN, Script.CYRILLIC)
restored = t_latn_cyrl.transliterate(latin_text)
print(restored)     # Мен Алматыда тұрамын.

# --- Script bridging inside a pipeline ---
# The Kazakh models are trained on Cyrillic. For Latin input the pipeline
# transliterates to Cyrillic, runs the models, and maps the results back:
# word texts, lemmas and character offsets refer to the original Latin text.
import turkicnlp
nlp = turkicnlp.Pipeline("kaz", script="Latn", processors=["tokenize", "morph"],
                         tokenize_backend="rule", morph_backend="apertium")
doc = nlp("Men Almatyda tūramyn.")
print(doc._model_text)          # Мен Алматыда тұрамын.  (what the models saw)
for w in doc.words:
```

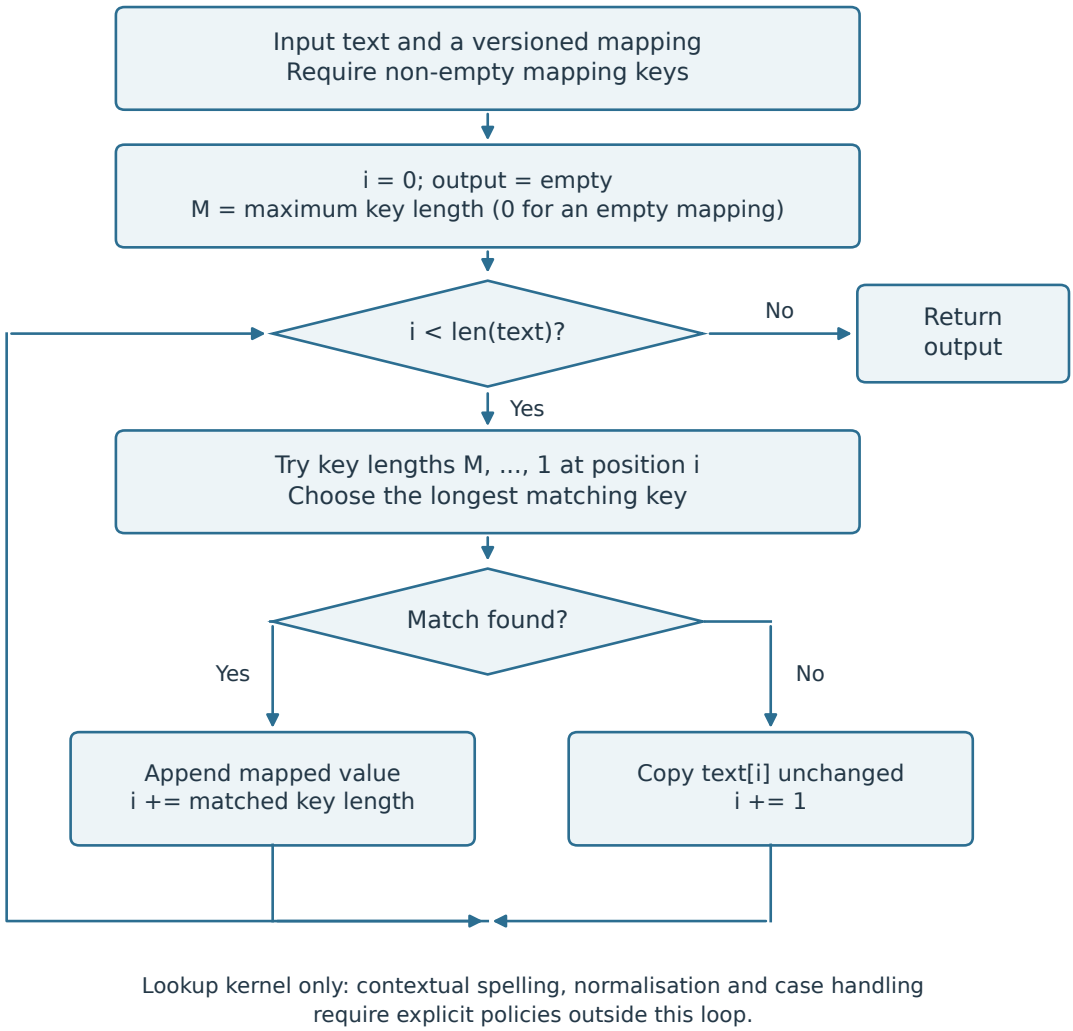


Figure 3.2: Greedy longest-match lookup for a versioned transliteration mapping. At each position the longest matching non-empty key is consumed; otherwise one character is copied. The maximum key length comes from the mapping. Contextual spelling, normalisation and case policies are separate from this kernel.

```
    print(w.text, w.lemma, w.upos, w.feats, w.start_char)
# Men      men      PRON  Case=Nom|Number=Sing|Person=1|PronType=Prs  0
# Almatyda Almaty   PROPN Case=Loc                                    4
# tūramyn  tūr      VERB  Aspect=Hab|Number=Sing|Person=1|Tense=Pres   13
# .        .        PUNCT _                                           20
```

The same mechanism covers Uzbek in Cyrillic, which this toolkit configuration routes to Latin-script models: `turkicnlp.Pipeline("uzb", script="Cyrl")` analyses the text with the Latin-script models. The output above was reproduced with the local 0.5.0 source, the rule-based tokeniser, and the Apertium analyser.

3.4.4 Hard Cases and Limitations

Many-to-one mappings (Kazakh). In the 2021 Kazakh Latin proposal both Cyrillic и and й are written i, and Cyrillic у can stand for a vowel or a glide. Converting Latin back to Cyrillic therefore needs context rules or a lexicon, and Russian loanwords (with ц, щ, ё, ъ, ь) need exception lists.

Information lost in one direction (Crimean Tatar). Crimean Tatar Cyrillic writes word-initial *ö* and *ü* as о and у, so *ötmek* comes back from a Latin → Cyrillic → Latin round trip as *otmek*. In the TurkicNLP evaluation on its parallel sample, this is the only language whose round trip did not recover every word (97.6% of words, versus 100% for the other languages and pivots) (Hakimov 2026). A perfect round trip on a sample shows that those inputs were recovered; it does not prove that a mapping is invertible on arbitrary text, nor that the converted spelling is the one a writer of the target script would use.

Perso-Arabic diacritics (harakat) in Ottoman Turkish. Ottoman Turkish texts sometimes include Arabic diacritics that provide vowel information absent from the base consonantal script. In TurkicNLP 0.5.0 the Ottoman converter runs only from Latin transcription to Perso-Arabic, so vowel diacritics in Perso-Arabic input are not used to recover Latin vowels.

Loanwords in the Latin→Cyrillic direction. For Uzbek, the Latin digraph sh maps unambiguously to Cyrillic ш, but Russian loanwords keep their Russian spelling in Cyrillic (*film* → *фильм*, with a soft sign the Latin form does not show). TurkicNLP's Uzbek conversion agrees with the independent UzTransliterator on 99.4% of words (Cyrillic→Latin) and 98.4% (Latin→Cyrillic); the remaining differences are such loanwords (Hakimov 2026). Agreement between two converters is not accuracy against checked spellings, since both can make the same mistake. Evaluate forward conversion, reverse conversion and round-trip recovery separately, on text with loanwords, names and mixed case. An exception lexicon or learned model may resolve some differences; check its output against independently verified spellings.

3.5 Text Normalisation for Turkic

3.5.1 Spell-Checking in Agglutinative Languages

A spell-checker that looks words up in a list of full word forms is impractical for Turkish or Kazakh: productive suffixation creates forms absent from a finite word list. Practical checkers therefore model morphology. Hunspell combines a stem list with affix rules, and **morphological spell-checking** accepts a word if a morphological analyser can parse it; Zemberek-NLP does this for Turkish, and Apertium transducers can be used the same way for other Turkic languages.

Acceptance by the analyser is not the same as correctness, in either direction. A real-word error passes: Turkish *evde* 'at home' typed for *ev de* 'the house too' is a valid word, and only context shows the mistake. An unknown proper name or a new loanword fails, although it is spelled correctly, because it is missing from the lexicon. Correction candidates (all strings within edit distance 1 or 2 that the analyser accepts) must be ranked with context, typically by a language model.

3.5.2 Orthographic Variation in Web-Crawled Text

Web-crawled corpora for Turkic languages contain significant orthographic variation:

Turkish apostrophe usage. Turkish orthography separates certain inflectional suffixes on proper names with an apostrophe, as in *Ankara'dan* (from Ankara), *Türkiye'de* (in Turkey). This is not a rule for every suffix or every named expression; derivation and institution names require separate treatment. In informal digital text, this apostrophe is frequently omitted (*Ankaradan*, *Türkiyede*), producing forms that a morphological analyser may not recognise as proper noun inflections.

Cyrillic/Latin mixing. Uzbek and Kazakh online text can mix scripts within a post or even within a sentence, for example Latin-script loanwords or technical terms in Cyrillic text. Script detection must operate at the word level, not just the document level, for such text.

Punctuation and whitespace. Arabic-script Uyghur text uses Arabic punctuation conventions (distinct comma, semicolon and question mark characters). Uyghur web text often mixes Arabic and ASCII punctuation, which can change punctuation attachment and token boundaries depending on the tokeniser.

3.5.3 Diacritic Dropping

A frequent form of informal orthographic variation is **diacritic dropping**: omitting the diacritics that distinguish some characters from their base letters. Typical substitutions are:

- **Turkish**: `ğ→g`, `ş→s`, `ç→c`, `ö→o`, `ü→u`, `ı→i` (the dotless I becomes a regular i)

- **Kazakh (2021 Latin)**: ä→a, ğ→g, ş→s, ñ→n, ū→u, ü→u, ö→o; in Cyrillic, Kazakh-specific letters are replaced by their Russian look-alikes (қ→к, ң→н, ү→у, ұ→у, ө→о, і→и) on keyboards without a Kazakh layout
- **Uzbek**: o‘→o, g‘→g

The Uzbek apostrophe problem. The official spelling rules distinguish the letters *o‘*, *g‘* and the separating sign *tutuq belgisi* (*ma’no*), but do not prescribe Unicode code points. A common convention uses U+02BB (‘) for the letter mark and U+02BC (’) for the separating sign. Toolkit output and some linguistic examples in this book retain ASCII U+0027.

Real text also uses backticks, curly quotes and acute accents for these marks. Normalise them according to a documented corpus convention, using word and quotation context. A mark after *o* or *g* may belong to a letter, but a mark at a word boundary may be quotation punctuation. Global replacement can erase that distinction. TurkicNLP 0.5.0 uses ASCII apostrophes in Uzbek conversion; adapt its output if the corpus uses another convention.

Diacritic-dropped text occurs where keyboards or habits make the special letters inconvenient. A morphological analyser built for correctly diacritised text may fail to parse such forms; measure how often on a sample of the target data.

Diacritic restoration, recovering the correct form from the dropped version, can be approached as a sequence-to-sequence task. For Turkish, vowel harmony provides strong constraints on the restored vowels. Kazakh also needs lexical and contextual evidence, because the dropped distinctions (*к/қ*, *ү/ұ*) are not all predictable from harmony.

3.5.4 Ordering the Pipeline

Preserve the original, diagnose its encoding, and build a conservative processing view. Then assess script and language. Turkish casing, Kazakh look-alike repairs and Uzbek apostrophe handling depend on those decisions. Each subsequent change should produce a view that can be inspected and traced back to its source.

1. **Preserve the source.** Save the source identifier, acquisition date, licence information and the original bytes. Decode with the known encoding and record decoding failures.
2. **Build an NFC view.** Apply NFC; apply NFKC only to Arabic presentation-form characters. Keep the previous view.
3. **Assess script and language.** Detect scripts per span, and identify the language with an unrestricted label set or an explicit “unknown” option (Section 3.7). Use source metadata as evidence (a Kazakh news site), and abstain when metadata and text disagree.
4. **Split mixed content.** Treat spans in different languages or scripts separately; protect URLs, code and quoted foreign text. Choose a language and orthography version only for spans the evidence supports.
5. **Propose language-specific repairs.** Casing, apostrophes and look-alike letters, as candidates with their positions. Keep uncertain cases for review, and re-run language identification if a repair changes the evidence it used.

6. **Restore and convert.** Apply a restoration model only in the representation it was trained on (a model trained on noisy and corrected Cyrillic expects Cyrillic). Convert to the downstream model's script, if a task comparison supports conversion.
7. **Tokenise** with the downstream model's tokeniser, keeping view and offset metadata, and evaluate on held-out original-language examples.

Restoration and transliteration. Suppose a writer types Kazakh ҝ for қ. A Cyrillic-to-Latin converter produces k; recovering the intended q requires restoration. A model tolerant of noisy spelling may need neither repair. When restoration is useful, apply it in the representation the restoration model expects: before conversion for a model trained on Cyrillic, afterwards for one trained on Latin. A joint model may perform both operations.

Code Example: Script Spans and Look-Alike Letters

The toolkit's `detect_script(text)` returns the dominant script of a whole text and raises `ValueError` if the text contains no characters recognised as letters in a supported script. `detect_script_segments(text)` returns `(text, script)` pairs:

```
from turkicnlp.scripts.detector import detect_script, detect_script_segments

detect_script("Мен Алматыда тұрамын.")                # Script.CYRILLIC
detect_script_segments("Мен iPhone сатып алдым, бағасы 500 $.")
# [('Мен ', CYRILLIC), ('iPhone ', LATIN), ('сатып алдым, бағасы 500 $.', CYRILLIC)]
detect_script_segments("«Salom» — dedi u.")
# [('Salom» — dedi u.', LATIN)]   the leading « is not in any segment
```

Neither function gives character offsets, and leading neutral characters can be dropped, so the segments are not a complete map of the input. Do not reconstruct the original by joining them; retain the original and track offsets, including neutral characters, if annotations must be mapped back. *iPhone* is genuine Latin text in a Cyrillic sentence and must not be "repaired".

Look-alike letters inside a Cyrillic word are a different case. The function below only *suggests* a repair, and only when the repaired form is a known word; it never edits a document:

```
import unicodedata

_LATIN_TO_CYRILLIC = {"a": "а", "e": "е", "o": "о", "p": "р", "c": "с",
                      "x": "х", "y": "у", "i": "і", "h": "һ", "ə": "ә"}
_LATIN_TO_CYRILLIC.update({k.upper(): v.upper() for k, v in
 ↪  list(_LATIN_TO_CYRILLIC.items())})

def is_cyrillic(ch: str) -> bool:
    return "CYRILLIC" in unicodedata.name(ch, "")

def suggest_cyrillic_token(token: str, trusted_forms: set[str]) -> str | None:
    """Suggest a repair for a mixed-script token; never edits a document."""
    token = unicodedata.normalize("NFC", token)
    if not token or not all(ch.isalpha() for ch in token):
        return None                                   # URLs, numbers, punctuation
    if not any(is_cyrillic(ch) for ch in token):
        return None                                   # all-Latin: a name, a loanword,
         ↪  English
```

```
    if any(not is_cyrillic(ch) and ch not in _LATIN_TO_CYRILLIC for ch in token):
        return None                                  # a Latin letter without a look-alike
    candidate = "".join(_LATIN_TO_CYRILLIC.get(ch, ch) for ch in token)
    if candidate == token:
        return None                                  # nothing to repair
    return candidate if candidate in trusted_forms else None

trusted = {"Алматы", "Қара"}                         # e.g. from a lexicon or the analyser
assert suggest_cyrillic_token("Алматы", trusted) == "Алматы"   # Latin A at the start
assert suggest_cyrillic_token("Қара", trusted) == "Қара"       # two Latin letters in a row
assert suggest_cyrillic_token("Алматы", trusted) is None       # already clean
assert suggest_cyrillic_token("web", trusted) is None          # genuine Latin word
assert suggest_cyrillic_token("https://example.org", trusted) is None
assert suggest_cyrillic_token("Қара", set()) is None           # unknown form: no suggestion
```

The small confusable table proposes candidates for review. A known word can still be wrong in context, and an absent lexicon entry leaves the repair unsupported. Store each candidate's offsets, replacement and acceptance decision. Apply accepted edits in a new view with a span map, so annotations such as NER spans can be traced back to the original.

> Tip
>
> **Practice note.** Synthetic pairs made by removing diacritics from clean text can train a restoration model. Test on real informal text as well: artificial corruption does not capture every typing error. Apply language-specific repair only after identifying the language.

3.6 A Common Script as an Interoperability Layer

3.6.1 The Common Turkic Alphabet

The **Common Turkic Alphabet** began as a 34-letter Latin proposal from a 1991 symposium in Istanbul (Devlet 1992; AKIpress 2024). In 2024 the Turkic World Common Alphabet Commission agreed on a revised version. Its declaration presents recommendations to member states rather than a binding orthography (Turkic World Common Alphabet Commission 2024).

The toolkit implements its own common-script representation, called **CTS** in this book. The local source snapshot reports version 0.5.0 and supplies CTS-target mappings for 23 languages (Hakimov 2026). Table 3.6 lists the case pairs and code points. The accompanying snapshot also records the full source hash and mapping tables, since local changes can leave the package version unchanged. Keep this implementation distinct from both alphabet proposals.

Table 3.6: CTS base alphabet and the additional Éé pair for Uyghur, from the frozen local TurkicNLP source identifying itself as 0.5.0 (13 October 2026). Source hash and all CTS mappings are recorded in `tables/tbl-cts-alphabet/source.json`; the package version alone does not identify this local source revision.

Letters	Code points (upper / lower)	Letters	Code points (upper / lower)
Aa	U+0041 / U+0061	Mm	U+004D / U+006D
Ää	U+00C4 / U+00E4	Nn	U+004E / U+006E
Bb	U+0042 / U+0062	Ññ	U+00D1 / U+00F1
Cc	U+0043 / U+0063	Oo	U+004F / U+006F
Çç	U+00C7 / U+00E7	Öö	U+00D6 / U+00F6
Dd	U+0044 / U+0064	Pp	U+0050 / U+0070
Ee	U+0045 / U+0065	Rr	U+0052 / U+0072
Ff	U+0046 / U+0066	Ss	U+0053 / U+0073
Gg	U+0047 / U+0067	Şş	U+015E / U+015F
Ğğ	U+011E / U+011F	Tt	U+0054 / U+0074
Hh	U+0048 / U+0068	Uu	U+0055 / U+0075
Xx	U+0058 / U+0078	Üü	U+00DC / U+00FC
Iı	U+0049 / U+0131	Vv	U+0056 / U+0076
İi	U+0130 / U+0069	Ww	U+0057 / U+0077
Jj	U+004A / U+006A	Yy	U+0059 / U+0079
Kk	U+004B / U+006B	Zz	U+005A / U+007A
Qq	U+0051 / U+0071	Éé	U+00C9 / U+00E9
Ll	U+004C / U+006C		

The implementation uses *ä* (U+00E4), not Azerbaijani *ə* (U+0259), and *ñ* (U+00F1), not *ŋ* (U+014B). It distinguishes dotless *ı* (U+0131) and dotted *i* (U+0069), with capitals *I* and *İ* (U+0130). Uyghur additionally uses *é* (U+00E9) to distinguish its closed vowel from *e*. Kazakh *ұ* maps to CTS *u*, while the national-2021 conversion uses *ū*; CTS also maps Kazakh *ү* to *u*, so this conversion is lossy. These choices describe the toolkit's implemented mapping; whether it conforms exactly to the 2024 commission's recommendation has not been checked. Unsupported input is copied by the converter, so arbitrary output need not contain only letters from this alphabet.

The mapping gives some corresponding letters a shared representation: Kazakh *Қазақстан* becomes *Qazaqstan*, Azerbaijani *Azərbaycan* becomes *Azärbaycan*, Turkmen *baý* becomes *bay*. This works only as far as the source spelling goes: phoneme inventories differ across the family, and a distinction a national orthography does not write (Kazakh rounding harmony, Section 2.2) is not recovered by conversion.

```
from turkicnlp.scripts import Script
from turkicnlp.scripts.transliterator import Transliterator

Transliterator("kaz", Script.CYRILLIC, Script.COMMON_TURKIC).transliterate("Қазақстан")
# 'Qazaqstan'
Transliterator("aze", Script.LATIN, Script.COMMON_TURKIC).transliterate("Azərbaycan dili")
# 'Azärbaycan dili'
Transliterator("tuk", Script.LATIN, Script.COMMON_TURKIC).transliterate("Türkmen dili
 ↪  baýdyr")
# 'Türkmen dili baydır'
```

For the recurring sentences (Table 1.4), TurkicNLP 0.5.0 gives these CTS views:

```
kaz  Qazaqstan osı öñirdegi beybitşilikti qoldaydı.
kir  Menin maşinam bar.
uzb  Toşkentda dzyudo böyiça Gran-pri musobaqasi bölib ötmoqda.
uig  u şeher sirtidiki bir at harvisini köridu.
```

Two details matter for a pipeline. Only the Uyghur conversion has a reverse table in this version: CTS→Perso-Arabic restores the original exactly, while the Kazakh, Kyrgyz and Uzbek CTS views cannot be converted back with the toolkit, so the original must be stored. CTS is also not the treebank's own romanisation: Uyghur-UDT transliterates the same sentence as *u sheher sirtidiki bir at harwisini köridu* (ULY), with *sh* and *w* where CTS has *ş* and *v*.

CTS gives models and string-matching tools a common written input. Keep the original orthography, store CTS as a derived view and make conversion an explicit option. Community decisions about writing systems belong to speakers (Section 17.4).

3.6.2 How Script Conversion Changes Tokenisation Cost

Chapter 7 shows that multilingual LLM tokenisers need far more tokens per word for some Turkic languages than for others. Is that because of morphology? On a parallel sample, Turkish needs 1.95–2.33 tokens per word across four tokenisers while Uyghur needs 3.38–6.85. Latin-script languages with few resources, such as Crimean Tatar and Gagauz, are as cheap as Turkish; the expensive languages are those written in Cyrillic or Arabic script (Hakimov 2026).

Converting the same sentences to CTS lets us compare their tokenisation cost under two written representations (Table 3.7). Conversion may lose distinctions: Kazakh *y* and *ɣ*, for example, both become *u*. The comparison therefore says nothing by itself about meaning preservation.

Fertility is the total token count divided by the number of Unicode alphanumeric runs. Python's `re.findall(r"[^\W_]+", text)` extracts those runs from the concatenated sample, excluding underscores. Each run is tokenised separately, without special tokens. The word count is recomputed after conversion and may change if separators change; averages are macro-averages over languages.

Across the 13 Cyrillic- and Arabic-script languages, fertility drops by 22–41%, depending on the tokeniser. The gap between Latin-script and non-Latin-script languages falls from 0.62–1.70 to at most 0.03 tokens per word. CTS has roughly the compactness of the national Latin orthographies. Under Qwen3.5 it uses fewer tokens than the 2021 Kazakh Latin alphabet (2.28 vs. 2.48 per word), but slightly more than Uzbek Latin (2.30 vs. 2.12).

Table 3.7: Fertility on the parallel challenge sample of Hakimov (2026). Group means use full-precision values, not the rounded displayed cells. The OpenAI column is the measured `o200k_base` encoding, not an inference-time model measurement.

Language	Script	Qwen3.5 (native → CTS)	LLaMA 3 (native → CTS)	OLMo 3 (native → CTS)	o200k_base (native → CTS)
Kazakh	Cyrl	2.86 → 2.28	3.51 → 2.32	4.05 → 2.55	2.37 → 2.29
Kyrgyz	Cyrl	2.84 → 2.16	3.48 → 2.31	3.80 → 2.59	2.52 → 2.12
Tatar	Cyrl	3.13 → 2.37	4.13 → 2.59	4.46 → 2.71	2.51 → 2.40
Bashkir	Cyrl	3.35 → 2.33	4.68 → 2.57	5.00 → 2.68	2.54 → 2.35
Nogai	Cyrl	2.92 → 2.02	3.08 → 2.08	3.48 → 2.27	2.65 → 1.98
Kumyk	Cyrl	2.76 → 2.10	3.06 → 2.23	3.61 → 2.42	2.56 → 2.08
Karachay-Balkar	Cyrl	2.25 → 1.85	2.52 → 1.88	2.94 → 2.01	2.23 → 1.80
Uyghur	Arab	4.51 → 2.26	5.62 → 2.38	6.85 → 2.64	3.38 → 2.26
Sakha	Cyrl	3.65 → 2.35	4.21 → 2.43	4.78 → 2.63	3.48 → 2.37
Altai	Cyrl	2.83 → 2.06	3.08 → 2.18	3.63 → 2.38	2.63 → 1.97
Tuvan	Cyrl	2.67 → 2.11	3.09 → 2.18	3.59 → 2.23	2.44 → 2.02
Khakas	Cyrl	2.79 → 2.13	3.16 → 2.16	3.94 → 2.37	2.67 → 2.10
Chuvash	Cyrl	3.52 → 2.03	3.66 → 2.18	4.09 → 2.29	3.42 → 2.03
Average (13 languages)		**3.08 → 2.16**	**3.64 → 2.27**	**4.17 → 2.45**	**2.72 → 2.14**

The frozen full-precision results and a copy of the experiment method are stored beside the table. The historical run does not record tokeniser commit hashes or the mapping revision used at execution time, so these assets reproduce the reported table, not a byte-exact rerun of tokenisation. The measured OpenAI encoding is `o200k_base`; the table names the encoding directly.

A common script can also expose cognates to string-based methods. Here, overlap is computed on each language's concatenated sample: regex-extracted words are lowercased with ordinary Python `str.lower()` and joined with spaces; the character-trigram Dice score uses sets of trigrams, including those crossing word boundaries. Word overlap is Jaccard similarity of lowercased word sets. Cross-script summaries average unordered language pairs, not aligned sentence-level scores. For language pairs written in different national scripts, character-trigram overlap on the parallel sentences rises from 0 to 0.30 on average after conversion to CTS (Turkish–Kumyk: 0.42), and word-type overlap more than doubles (Hakimov 2026).

Figure 3.3 shows written overlap after conversion. Gagauz has the highest overlap with Turkish (0.77), followed by Crimean Tatar (0.54) and Azerbaijani (0.46); Chuvash is lowest (0.13). This ordering is not a genealogy: the sample, shared vocabulary and conversion conventions all affect it.

The sample contains only 20 sentences per language. What it shows is a change in token cost for the tested mappings and tokenisers. Separating the causal contributions of script and morphology would need a different design. Lower fertility also need not improve task accuracy: a model trained on native orthographies may handle CTS poorly.

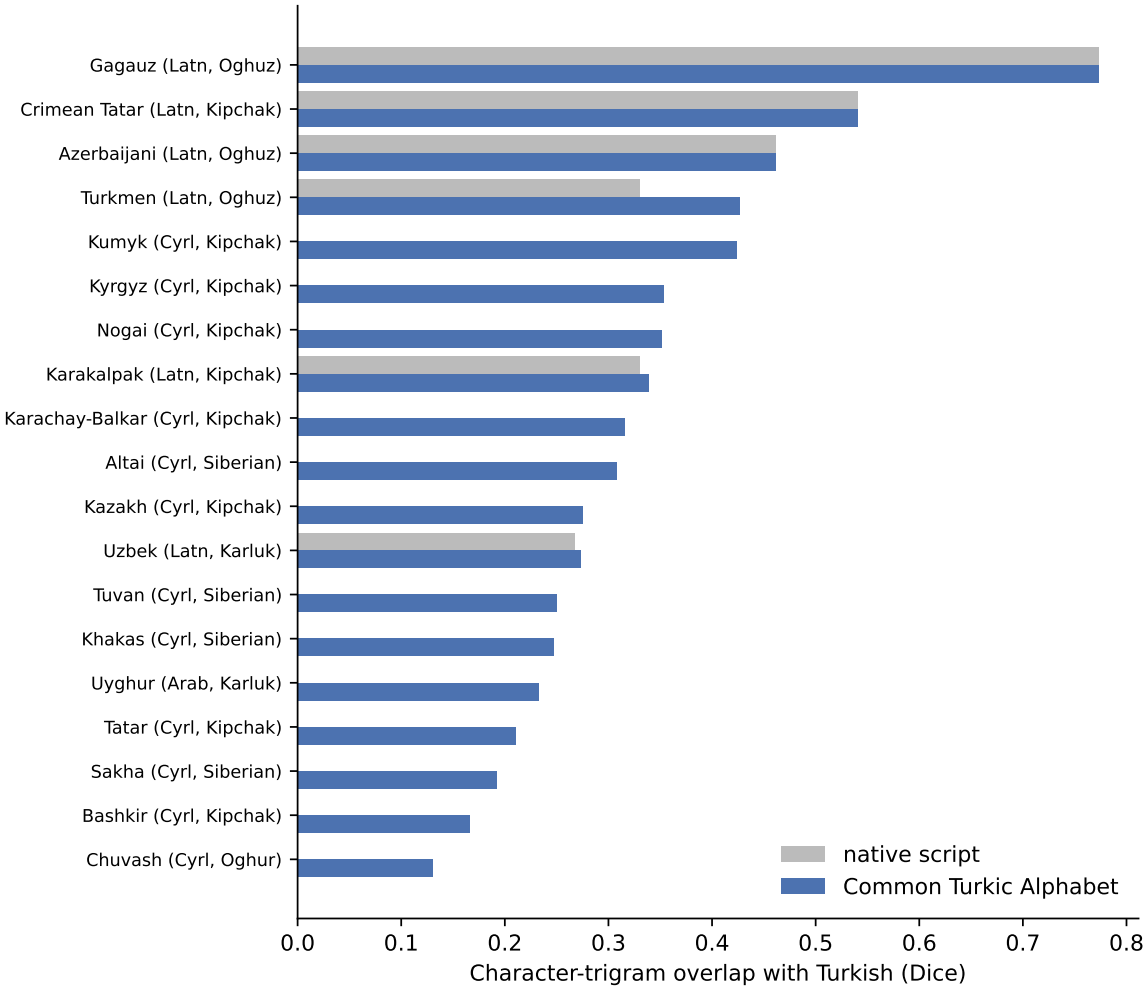


Figure 3.3: Character-trigram overlap (Dice coefficient) between each Turkic language and Turkish on parallel sentences, in the native script and after conversion to CTS. Every Cyrillic- and Arabic-script language starts at zero. Data: `figures/fig-cts-overlap/data.csv`, from Hakimov (2026).

To test CTS for a new model, train native-script and converted conditions with matched data and splits. Keep all versions of a document in the same split, and report task quality alongside token cost (Chapter 7 and Chapter 16).

3.7 Language Identification and Corpus Construction

3.7.1 Why Identification Is Hard Inside the Family

Language identification is difficult when neighbouring languages share function words, suffixes and scripts. TurkicNLP wraps GlotLID (Kargaran et al. 2023), which has more than 2,000 labels. By default, `LanguageDetection()` restricts predictions to the toolkit's Turkic labels and renormalises their scores. Russian text, English text and boilerplate must then receive a Turkic label too. The score expresses a preference within that restricted set. Use it only for input already known to be Turkic; for a web crawl, start with the unrestricted model:

```
# pip install "turkicnlp[lid]==0.5.0"   # GlotLID needs the lid extra (fastText + Hub
 ↪  download)
import turkicnlp
from turkicnlp.scripts.detector import detect_script

lid = turkicnlp.LanguageDetection()            # restricted to Turkic labels
text = "salam, hemmelere!"

def identify_with_abstention(text, lid, *, min_full_score, min_margin):
    """Thresholds must be calibrated on labelled target-domain text."""
    full_labels, full_probs = lid.model.predict(text.replace("\n", " "), k=3)
    supported = set(lid.available_labels())
    accepted = (len(full_labels) >= 2
                and full_labels[0] in supported
                and float(full_probs[0]) >= min_full_score
                and float(full_probs[0] - full_probs[1]) >= min_margin)
    if not accepted:
        return {"language": None, "full_labels": full_labels, "full_scores": full_probs}
    labels, probs = lid.predict(text, k=3)
    return {"language": labels[0], "labels": labels, "restricted_scores": probs,
            "full_labels": full_labels, "full_scores": full_probs}

# Inspect the unrestricted candidates before choosing thresholds.
# Invoke the function only with thresholds selected on held-out calibration data.
print(lid.model.predict(text.replace("\n", " "), k=3))

# Alternatively, keep the contact languages in the restricted label set,
# so that Russian or Persian text is not forced into a Turkic label:
lid_contact = turkicnlp.LanguageDetection(
    languages=lid.available_labels() + ["__label__rus_Cyrl", "__label__pes_Arab"])
```

On the sentences of the parallel UD Turkic test treebanks, GlotLID restricted to Turkic labels is correct for 85.7% of sentences; on the short parallel challenge sentences it reaches only 69.2% (Hakimov 2026). The two figures come from test sets with different language compositions and sentence lengths, so they should not be read as a difference between "natural" and "artificial" text. Per-language results for the challenge sample

appear in Figure 3.4. The errors are systematic: smaller languages are absorbed by a close neighbour written in the same script (Figure 3.4, Table 3.8).

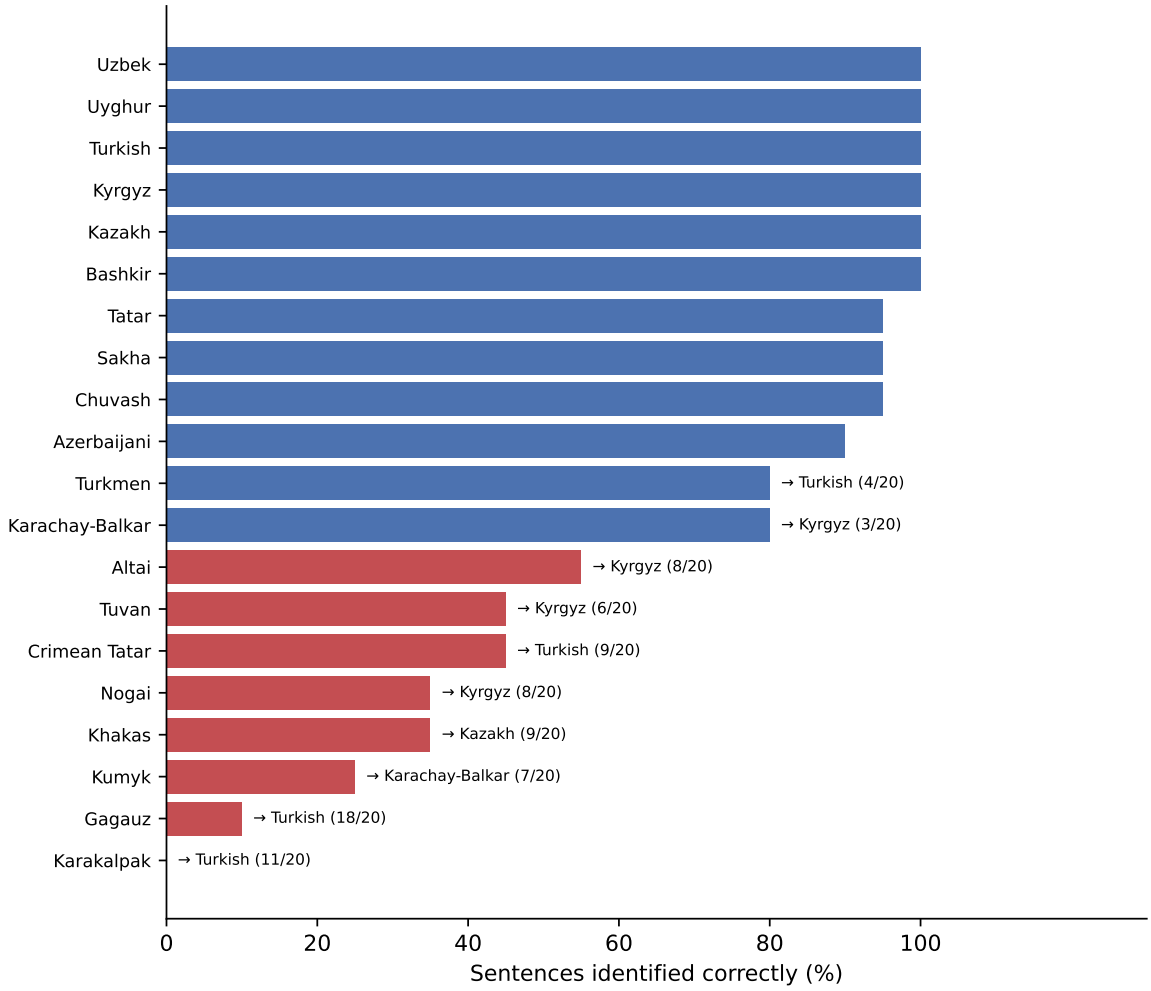


Figure 3.4: GlotLID accuracy (restricted to Turkic labels) on 20 parallel challenge sentences per language, with the most frequent wrong label for the weaker languages. Data: `figures/fig-lid-accuracy/data.csv`, from Hakimov (2026).

Table 3.8: Language identification for eight smaller Turkic languages: GlotLID restricted to Turkic labels, 20 parallel challenge sentences per language (Hakimov 2026). Counts refer to the most frequent recorded wrong label, not all errors.

Language	Most frequent wrong label (sentences)	Challenge-set accuracy
Gagauz	Turkish (18/20)	10%
Crimean Tatar	Turkish (9/20)	45%

Language	Most frequent wrong label (sentences)	Challenge-set accuracy
Karakalpak (2016 Latin)	Turkish (11/20)	0%
Nogai	Kyrgyz (8/20)	35%
Altai	Kyrgyz (8/20)	55%
Tuvan	Kyrgyz (6/20)	45%
Khakas	Kazakh (9/20)	35%
Kumyk	Karachay-Balkar (7/20)	25%

Three practical lessons follow:

1. **Identify before you transliterate.** On text that has been converted to another script, accuracy drops to 48.1% (Hakimov 2026). Conversion changes the native-script letter sequences used by the identifier. This result does not isolate which individual cues caused the drop.
2. **Restrict the label space only for text known to be Turkic.** On the Turkic test sets, restricting GlotLID to Turkic labels gave small gains (+0.6 to +2.0 points) (Hakimov 2026), because it removes confusions with unrelated languages written in the same script. On a web crawl the same restriction mislabels every non-Turkic document. Establish the family first, keep the top-k candidates, allow abstention, and calibrate thresholds on labelled target-domain text that includes Russian and English distractors, short strings, names, boilerplate and code-switching.
3. **Do not trust language labels in web corpora for the small languages.** Large multilingual web corpora assign languages with identifiers of this kind, so a subset may contain another language, while genuine target-language text may be assigned elsewhere. These error rates require a corpus audit; the challenge set does not measure them. Inspect samples, and for the smaller languages prefer sources with language evidence at the document level (checked news articles, Wikipedia pages or digitised books); a national website may also publish in Russian or other languages.

3.7.2 From Single Texts to Corpora

These lessons apply to every document that enters a corpus. Section 4.3 turns them into an ordered pipeline for building a corpus: provenance, NFC, language identification before transliteration, script and orthography metadata, quality filtering, deduplication across scripts, and conversion only in a derived copy.

3.8 Hands-On: Checking a Text Pipeline

Question. Which text changes are justified, and what information do they lose?

Inputs. The Unicode and conversion examples above, with the declared toolkit mappings.

Compute. CPU; some exercises download language-identification models or task data.

Output. Original and derived strings, conversion failures and language-ID decisions.

Interpretation. Round trips and downstream scores answer different questions, so inspect each on its own.

Worked step. Replace the first Cyrillic *а* (U+0430) in the recurring Kazakh sentence (Table 1.4) with a Latin *a* (U+0061), a common copy-and-paste corruption, and run the checks (output abridged):

```
clean      NFC unchanged: True; Latin letters: none
           detect_script: Cyrl
           CTS: Qazaqstan osı öñirdegi beybitşilikti qoldaydı.
corrupted  NFC unchanged: True; Latin letters: ['a U+0061']
           detect_script: Cyrl
           segments: [('Қ', Cyrl), ('a', Latn), ('зақстан осы өңірдегі бейбітшілікті
            ↪  қолдайды.', Cyrl)]
           CTS: Qazaqstan osı öñirdegi beybitşilikti qoldaydı.
strings equal: False | CTS views equal: True
```

NFC normalisation and whole-text script detection both miss the corruption; only the per-segment check finds it. The CTS view hides it entirely, because Latin *a* and Cyrillic *а* map to the same letter. Run homoglyph checks on the original text, before any conversion, and keep the original.

Status: demonstration. The checks above are recorded output. Exercises 1–5 work through text handling; Exercises 6, 8 and 9 extend the comparison to language identification and tagging.

What to Take Forward

A reliable text pipeline keeps the original and records every derived view, with the direction and convention of each conversion. Conversion can reduce token counts: for the 13 Cyrillic- and Arabic-script languages in the TurkicNLP measurements, CTS lowers tokeniser fertility by 22–41% (Hakimov 2026). It can also lose information, which is why round trips and downstream scores are checked separately. Convert only when the task benefits and the loss is recorded, and let the language identifier abstain when it is uncertain. Chapter 4 makes these decisions part of corpus metadata.

Further Reading

- **Script history**: Johanson (2021, chap. 19) covers the history of Turkic writing systems from Old Turkic runic to the 20th century.
- **Unicode standard**: The Unicode Consortium's *The Unicode Standard* (current version) documents all relevant code blocks. Use the Old Turkic code chart for character identities and code points.
- **Turkish casing**: Unicode's SpecialCasing file lists the language-specific dotted/dotless I mappings for Turkish and Azerbaijani; Çöltekin, Doğruöz, and Çetinoğlu (2023) surveys Turkish NLP resources more generally.

- **TurkicNLP toolkit**: Hakimov (2026) describes the transliteration, script detection, CTS conversion and language identification used in this chapter, with their evaluation.
- **Language identification**: Kargaran et al. (2023) describes GlotLID.
- **Transliteration for transfer**: Muller et al. (2021) show for Uyghur, among other languages, that transliterating an unseen script into Latin helps multilingual models.
- **Apertium transliteration**: Khanna et al. (2021) covers the Apertium approach to script conversion as part of MT pipelines.

Exercises

1. **Unicode investigation.** Run `"İstanbul".lower()`, `"ISPARTA".lower()` and `turkish_lower_nfc("I\u0307STANBUL")` from Section 3.3. Print `repr`, `len` and the code points of each result. Explain why the standard `str.lower()` is incorrect for Turkish, and why the helper normalises to NFC first.

2. **Homoglyph detection.** Write a Python function `detect_homoglyphs(text)` that returns a list of (position, character, codepoint) tuples where the character is a Cyrillic/Latin homoglyph (use the table in Section 3.3). Test it on a string that mixes Cyrillic `а` (U+0430) and Latin `a` (U+0061).

3. **Transliteration by hand.** Using the toolkit's Kazakh mapping (based on the 2021 proposal; print the table from `Transliterator("kaz", Script.CYRILLIC, Script.LATIN)`), convert *Алматы*, *Қазақстан*, *Астана* and *киім* ('clothes') by hand, then check your answers with the toolkit. Identify one mapping whose reverse is ambiguous and explain what additional information would be needed to convert back. Why does implementing a mapping say nothing about its legal status?

4. **Pipeline ordering.** You have Kazakh text in Cyrillic and Latin, some of it diacritic-dropped, and a restoration model trained on noisy Cyrillic paired with corrected Cyrillic. Design a pipeline that preserves the original and records the script of each span. Where can that restoration model be applied, and why? How would the design change if the restoration model had been trained on Latin text instead?

5. **Round trips.** Take 50 Uzbek sentences in Latin script (for example from FLORES-200, `uzn_Latn`). Convert them to Cyrillic and back with `Transliterator("uzb", …)`. Which sentences do not survive the round trip unchanged? Classify the differences (apostrophe variants for *o'*/*g'*, the tutuq belgisi, loanwords, capitalisation), and say which ones matter for a corpus.

6. **Identify, then transliterate.** Choose 20 Kyrgyz sentences. Run `turkicnlp.LanguageDetection()` on them as they are, then again after converting them to CTS. How does accuracy change, and which wrong labels appear? Relate the result to Section 3.7.

7. **Script transition implications.** *In this hypothetical exercise*, a researcher has scraped Kazakh web text in which 60% of the documents are in Cyrillic and 40% in Latin (following the 2021 proposal plus informal variants). Describe three specific problems this mixed-script corpus will cause if used without preprocessing, and propose a concrete solution for each. Would your answer change if the 60/40 split counted words instead of documents?

8. **Abstention in language identification.** Build a sample of 100 short texts: 60 in Turkic languages, 20 in Russian, 10 in English and 10 mixed Kazakh–Russian. Run `LanguageDetection()` restricted to Turkic labels and the unrestricted model. For a range of thresholds, report precision for each Turkic language and the share of non-Turkic texts that are accepted. Which threshold and abstention rule would you choose for a crawl, and why?
9. **Does conversion help the task?** Choose a task with data in a Cyrillic-script Turkic language (for example POS tagging of Kazakh). Train and evaluate the same model on native-script and CTS versions of the same split. Report task accuracy and tokens per word for both. Is the conversion worth it?

4 Building Turkic Corpora at Scale

Learning Objectives

After reading this chapter, you will be able to:

- Describe the large multilingual web corpora that contain Turkic text, and compare their coverage of the family
- Explain why a language label in a web corpus is a claim to be checked rather than a fact, and name the contamination types that are specific to Turkic text
- Audit a sample of a corpus for one Turkic language with language identification, script detection, and morphological-analyser coverage
- Design a cleaning pipeline for Turkic text, including normalisation and deduplication across scripts
- Decide how to store and release a corpus for a language written in more than one script
- Apply licensing and provenance rules when building a corpus for a small Turkic language

A web-corpus label is an automatic prediction. This chapter shows how to compare corpus releases, check a sample and build a processing pipeline. The first checks are simple: is the text in the intended language, and which script does it use?

The chapter has three parts. The first surveys what the large web corpora contain for each Turkic language and how much (Section 4.1). The second shows how to check those claims by auditing a sample (Section 4.2). The third describes how to build and release a cleaner corpus: the processing pipeline, script choice, parallel data, and licensing (Section 4.3 to Section 4.6).

4.1 What Exists: Turkic Text in Multilingual Web Corpora

4.1.1 Successive web-corpus releases

The collections below draw on multilingual web archives, including Common Crawl. They differ in how they extract text, which language identifier they use, how they filter for quality, and how they remove duplicates:

- **CC-100** (Conneau et al. 2020) was built to train XLM-R. It includes Turkish, Azerbaijani, Kazakh, Kyrgyz, Uzbek, and Uyghur, but not Tatar, Bashkir, Turkmen, Chuvash, or Sakha.
- **mC4** (Xue et al. 2021) was built to train mT5. It has five Turkic languages: Turkish, Azerbaijani, Kazakh, Kyrgyz, and Uzbek.
- **CulturaX** (Nguyen et al. 2024) combines and re-cleans mC4 and OSCAR, another Common Crawl corpus. It lists 14 Turkic languages; ten of them have fewer than 250,000 documents, and Tuvan has 23.
- **MADLAD-400** (Kudugunta et al. 2023) covers more than 400 languages, and its authors audited samples of each language by hand.
- **HPLT** is represented here by three releases: v1.2 (Gibert et al. 2024), v2.0 (Burchell et al. 2025), and v3.0 (HPLT Consortium 2025; Oepen et al. 2026). Each re-extracts text from web archives. The language identifier changed between releases: CLD2 for v1.2, a modified OpenLID (Burchell et al. 2023) for v2.0, and OpenLID-v2 for v3.0.
- **FineWeb2** (Penedo et al. 2025) adapts its filtering pipeline to each language. Its language identifier, GlotLID (Kargaran et al. 2023), has many more labels than OpenLID, and FineWeb2 provides the language/script subsets listed in the inventory below, several of them in more than one script.

4.1.2 How much text: HPLT v3.0 as a worked example

HPLT v3.0 publishes a machine-readable manifest with the size of every language subset: documents, characters, and tokens under the Gemma 3 tokeniser. Table 4.1 lists the Turkic subsets, and Figure 4.1 shows their sizes on a logarithmic scale.

Table 4.1: Turkic subsets of HPLT v3.0 (sizes as labelled by the corpus's language identifier, not audited). Russian and English are shown for reference. Not present in HPLT v3.0: Gagauz, Ottoman Turkish, Karakalpak, Nogai, Kumyk, Karachay-Balkar, Sakha, Altai, Tuvan, Khakas, Chuvash, Khalaj, Old Turkic. Source: HPLT v3.0 manifest, accessed 12 October 2026.

Language	Subset	Documents (M)	Characters (B)	Gemma 3 tokens (B)	Characters per token	Compressed-byte share in bins 8–10
Turkish	`tur_Latn`	159.47	512.61	149.98	3.42	56%
Azerbaijani (North)	`azj_Latn`	11.07	41.26	15.96	2.59	72%
Kazakh	`kaz_Cyrl`	5.12	17.21	7.34	2.35	69%
Uzbek	`uzn_Latn`	1.88	6.51	2.34	2.79	62%
Kyrgyz	`kir_Cyrl`	1.49	3.80	1.54	2.48	67%
Tatar	`tat_Cyrl`	1.26	3.69	1.58	2.33	67%

Language	Subset	Documents (M)	Characters (B)	Gemma 3 tokens (B)	Characters per token	Compressed-byte share in bins 8–10
Uyghur	`uig_Arab`	0.65	2.16	1.20	1.80	1%
Turkmen	`tuk_Latn`	0.38	0.90	0.37	2.44	60%
Bashkir	`bak_Cyrl`	0.28	0.80	0.39	2.04	65%
Crimean Tatar	`crh_Latn`	0.12	0.32	0.13	2.47	62%
South Azerbaijani	`azb_Arab`	0.09	0.30	0.13	2.20	48%
Russian (reference)	`rus_Cyrl`	3,304.96	15,950.00	4,402.59	3.62	—
English (reference)	`eng_Latn`	18,057.79	72,336.99	16,280.22	4.44	—

Three observations follow.

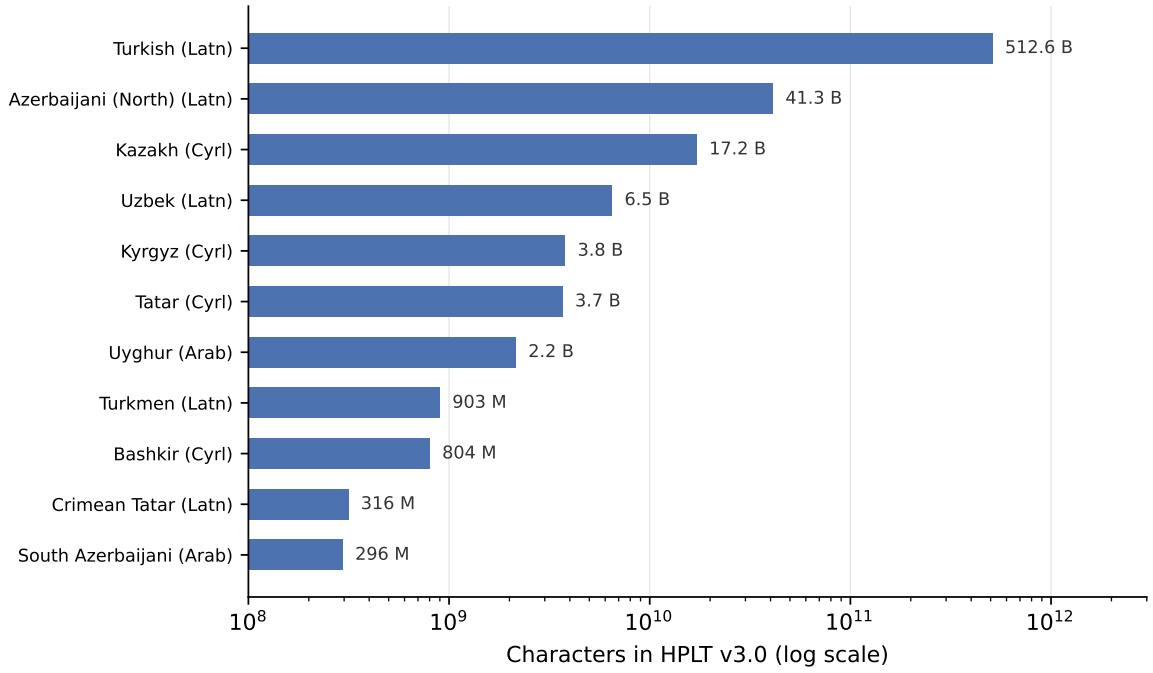
The sizes span more than three orders of magnitude. Turkish has about 1,600 times as many characters as Crimean Tatar and 1,700 times as many as South Azerbaijani. Kazakh has 3% of the Turkish total. Russian, the main contact language of Kazakh, Kyrgyz, Tatar and Bashkir, has about 900 (Kazakh) to 20,000 (Bashkir) times as many characters in the same release.

Count subsets before counting languages. The manifest separates North (`azj`) and South Azerbaijani (`azb`) and lists eleven Turkic subsets altogether. A language without its own subset may still have pages in the crawl, assigned another label, filtered out or left unclassified. Define the release-specific mapping from subsets to languages before reporting coverage.

Characters per token vary across the HPLT language subsets. Dividing character totals by Gemma 3 token totals gives 1.80 for Uyghur, 3.42 for Turkish and 4.44 for English (Figure 4.2). Latin and non-Latin ranges overlap. This measures text length per token in different corpora; it is not the tokens-per-word fertility measure of Section 7.1 and does not isolate a causal script effect. Corpus composition, orthographic conventions and tokeniser behaviour can all affect the ratio.

4.1.3 Comparing corpora and corpus generations

Table 4.2 and Figure 4.3 compare eight sources by document count, using pages for mC4 and articles for Wikipedia. Most sources report this unit, whereas their word and token counts use incompatible definitions. Document boundaries also vary: one corpus may split a page that another retains whole.



Not present: Gagauz, Ottoman Turkish, Karakalpak, Nogai, Kumyk, Karachay-Balkar, Sakha, Altai, Tuvan, Khakas, Chuvash, Khalaj, Old Turkic

Figure 4.1: Size of the Turkic subsets of HPLT v3.0 in characters (logarithmic scale). Data: Table 4.1.

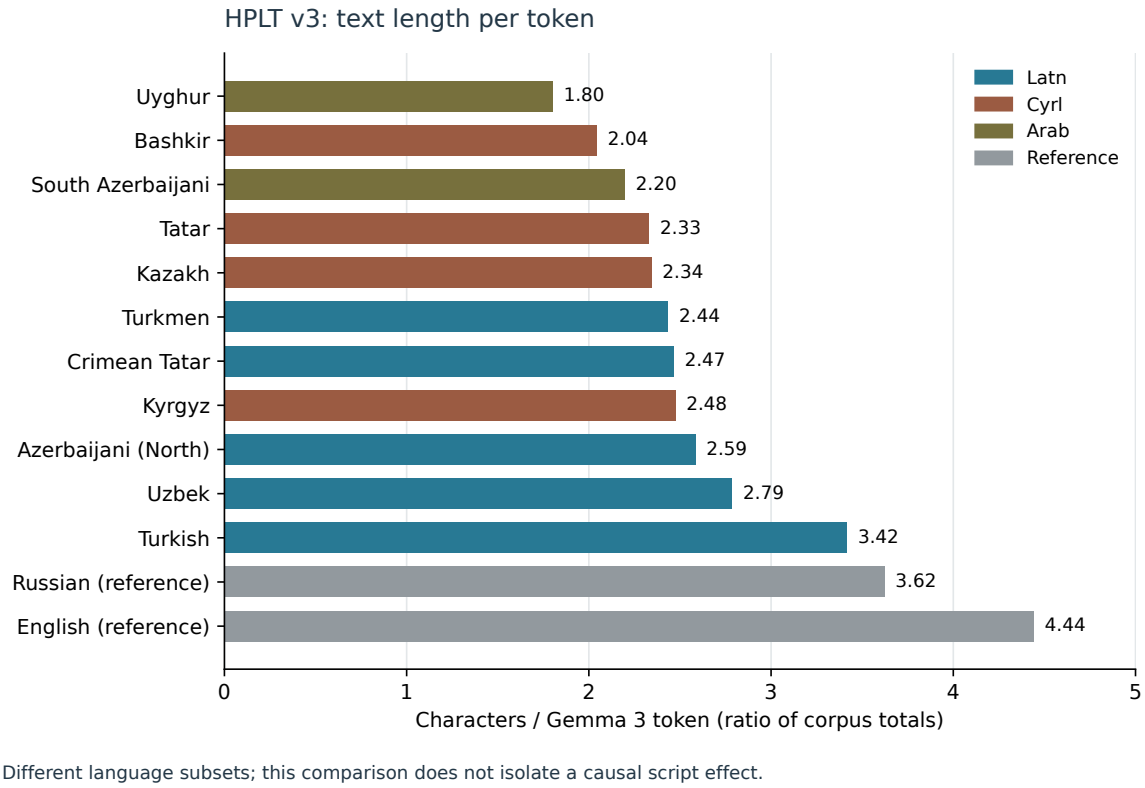


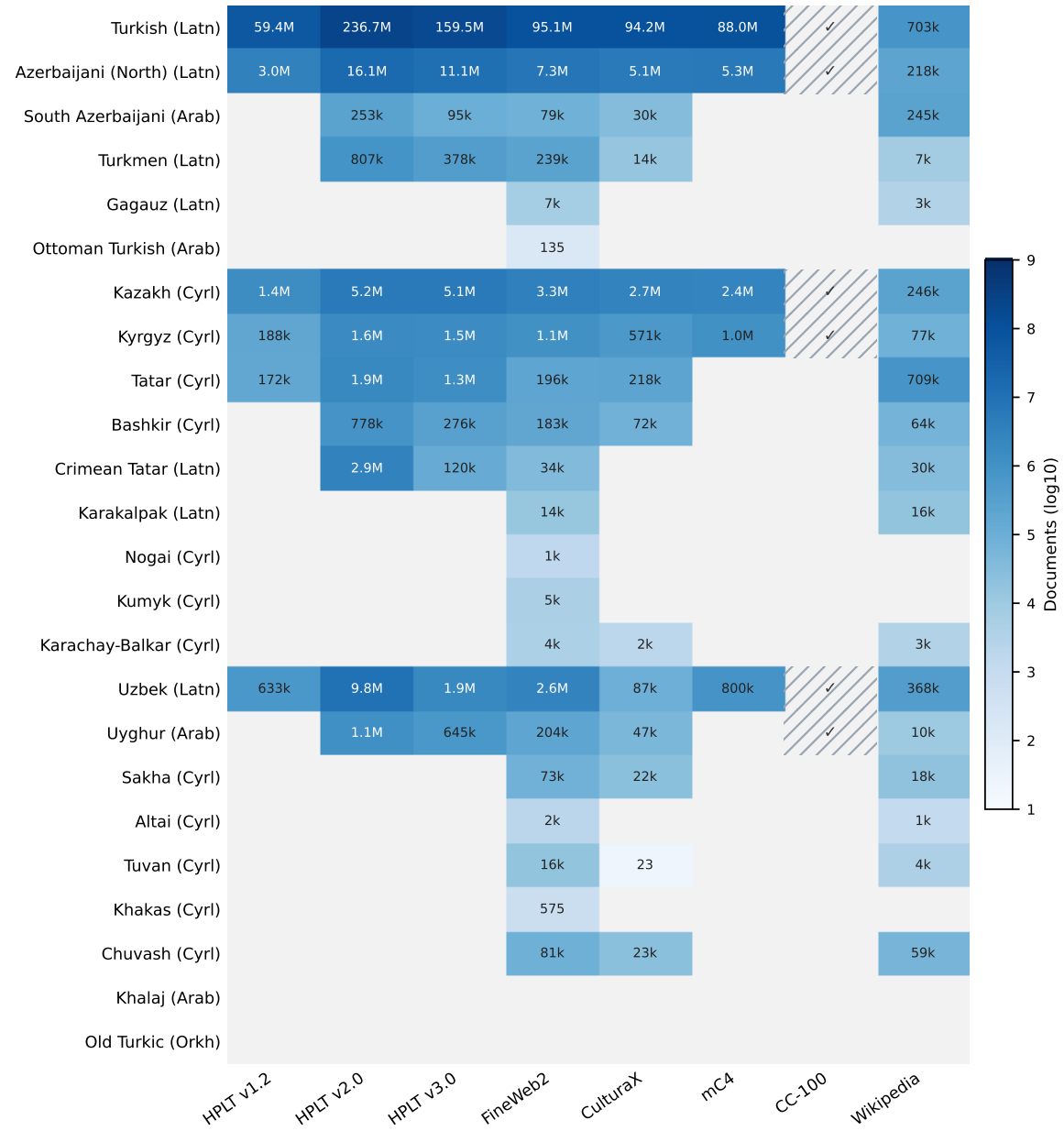
Figure 4.2: Characters per Gemma 3 token in HPLT v3: eleven primary Turkic language subsets plus English and Russian references (HPLT Consortium 2025). Values are ratios of corpus totals, not averages of document ratios or estimates of word counts.

Table 4.2: Documents per Turkic language in eight corpora (mC4: pages; Wikipedia: articles; CC-100 reports tokens only, so ✓ marks presence). HPLT v1.2 and v2.0: deduplicated releases; v3.0: sorted release. Sizes are as labelled by each corpus's language identifier and not audited. FineWeb2 sums all script subsets (train split, after filtering). Absent from all eight: Khalaj, Old Turkic. Sources: HPLT v1.2/v2.0/v3.0 download pages and manifest, FineWeb2 language-distribution CSV, CulturaX dataset card, mT5 and XLM-R papers (Table 6 each), MediaWiki statistics API; all accessed 12 October 2026.

Language	Branch	HPLT v1.2	HPLT v2.0	HPLT v3.0	FineWeb2	CulturaX	mC4	CC-100	Wikipedia
Turkish	Oghuz	59.4M	236.7M	159.5M	95.1M	94.2M	88.0M	✓	703k
Azerbaijani (North)	Oghuz	3.0M	16.1M	11.1M	7.3M	5.1M	5.3M	✓	218k
South Azerbaijani	Oghuz	—	253k	95k	79k	30k	—	—	245k
Turkmen	Oghuz	—	807k	378k	239k	14k	—	—	7k
Gagauz	Oghuz	—	—	—	7k	—	—	—	3k
Ottoman Turkish	Oghuz (historical)	—	—	—	135	—	—	—	—
Kazakh	Kipchak	1.4M	5.2M	5.1M	3.3M	2.7M	2.4M	✓	246k
Kyrgyz	Kipchak	188k	1.6M	1.5M	1.1M	571k	1.0M	✓	77k
Tatar	Kipchak	172k	1.9M	1.3M	196k	218k	—	—	709k
Bashkir	Kipchak	—	778k	276k	183k	72k	—	—	64k
Crimean Tatar	Kipchak	—	2.9M	120k	34k	—	—	—	30k
Karakalpak	Kipchak	—	—	—	14k	—	—	—	16k
Nogai	Kipchak	—	—	—	1k	—	—	—	—
Kumyk	Kipchak	—	—	—	5k	—	—	—	—
Karachay-Balkar	Kipchak	—	—	—	4k	2k	—	—	3k
Southern Altai	Kipchak	—	—	—	2k	—	—	—	1k
Uzbek	Karluk	633k	9.8M	1.9M	2.6M	87k	800k	✓	368k
Uyghur	Karluk	—	1.1M	645k	204k	47k	—	✓	10k
Sakha	Siberian	—	—	—	73k	22k	—	—	18k
Tuvan	Siberian	—	—	—	16k	23	—	—	4k
Khakas	Siberian	—	—	—	575	—	—	—	—
Chuvash	Oghur	—	—	—	81k	23k	—	—	59k

Which languages appear depends on the language identifier. Five languages (Turkish, Azerbaijani, Kazakh, Kyrgyz, Uzbek) are in all eight sources. CC-100 and HPLT v1.2 each have six Turkic languages, mC4 five. FineWeb2, whose identifier has the largest label set, is the only one of these web corpora with Nogai, Kumyk, Khakas, Altai, Gagauz, and Karakalpak. Their subsets are small (575 documents for Khakas, 1,393 for Nogai), but they exist.

The same language differs by an order of magnitude between corpora. Uzbek has 9.8 million documents in the deduplicated HPLT v2.0 release, 2.6 million in FineWeb2,



Grey: absent. Hatched ✓: present, documents not reported (CC-100). mC4 = pages, Wikipedia = articles.

Figure 4.3: Documents per Turkic language in eight corpora (log scale). Grey cells: absent. Hatched: present in CC-100, which reports tokens but not documents. Data: Table 4.2.

and 87,000 in CulturaX. Filtering, deduplication and language labels all differ between these corpora; the audit in Section 4.2 is needed to tell how much each contributes.

Release stages change the comparison. Table 4.3 follows eleven HPLT subsets across three releases. Crimean Tatar appears to shrink from 2.9 million documents (2.0 billion characters) in deduplicated v2.0 to 120,000 (0.3 billion) in v3.0. Yet the cleaned v2.0 release already contains just 123,000 Crimean Tatar documents. That cleaning stage filters low language-identification probabilities, low Web Docs Scorer values and other signals (Burchell et al. 2025, sec. 4.2).

Most of the apparent decline therefore occurred within v2.0. We have not established which filter removed those documents or what languages they contained. Against cleaned v2.0, every other Turkic subset grows in v3.0. Uzbek rises from 707,000 documents to 1.9 million.

Table 4.3: The Turkic subsets across three HPLT releases: v1.2 deduplicated; v2.0 deduplicated (dedup.) and cleaned; v3.0 sorted (Web Docs Scorer bins 5–10 only). v2.0's cleaned release drops, among others, documents with a language-identification probability below 0.5 or a Web Docs Scorer value below 5 (Burchell et al. 2025, sec. 4.2). Because v3.0 also contains only bins 5–10, v2.0's cleaned release is the closer comparison on that criterion; the other v3.0 filters were not checked here. v1.2 does not report characters. Sources: hplt-project.org/datasets/v1.2 and /v2.0, HPLT v3.0 manifest; accessed 12 October 2026.

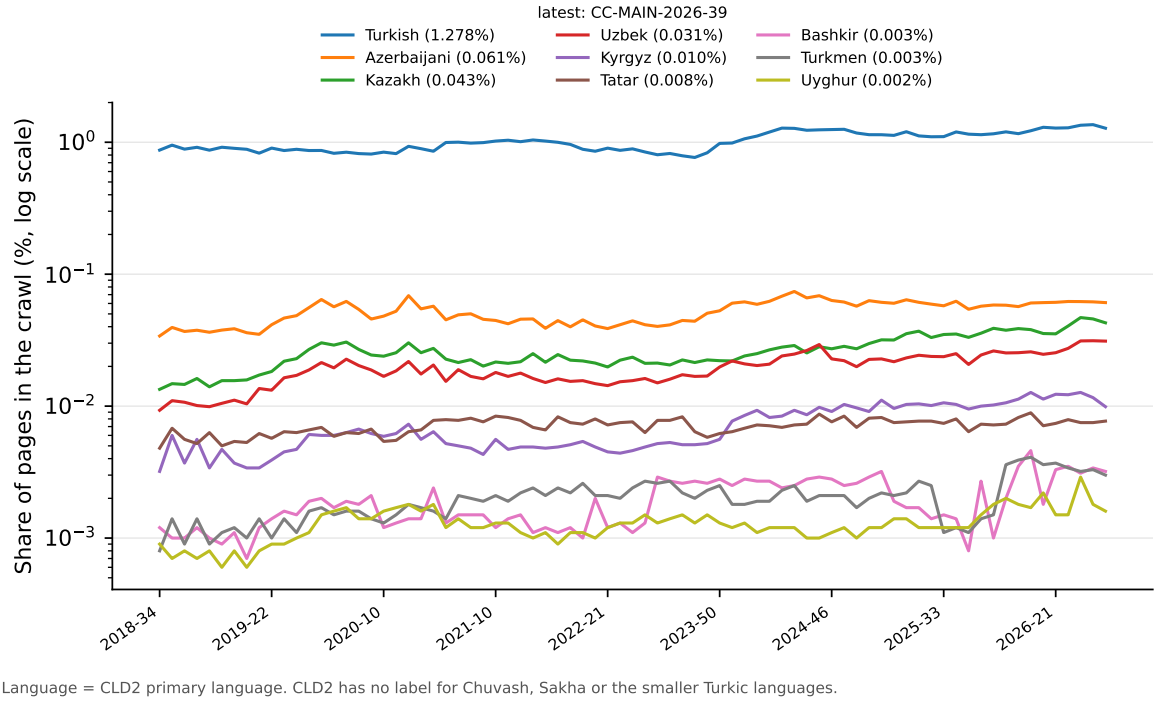
Language	Subset	Docs v1.2	Docs v2.0 dedup.	Docs v2.0 cleaned	Docs v3.0	Chars v2.0 dedup.	Chars v2.0 cleaned	Chars v3.0
Turkish	tur_Latn	59.4M	236.7M	116.6M	159.5M	544.0B	389.8B	512.6B
Azerbaijani (North)	azj_Latn	3.0M	16.1M	6.5M	11.1M	34.7B	19.6B	41.3B
Kazakh	kaz_Cyrl	1.4M	5.2M	2.6M	5.1M	15.4B	11.1B	17.2B
Uzbek	uzn_Latn	633k	9.8M	707k	1.9M	10.3B	2.8B	6.5B
Kyrgyz	kir_Cyrl	188k	1.6M	676k	1.5M	2.6B	1.9B	3.8B
Tatar	tat_Cyrl	172k	1.9M	631k	1.3M	3.2B	2.2B	3.7B
Uyghur	uig_Arab	—	1.1M	442k	645k	2.5B	1.7B	2.2B
Turkmen	tuk_Latn	—	807k	171k	378k	1.6B	570.2M	902.8M
Bashkir	bak_Cyrl	—	778k	171k	276k	1.0B	558.7M	803.7M
Crimean Tatar	crh_Latn	—	2.9M	123k	120k	2.0B	281.2M	315.9M

South Azerbai-jani	azb_Arab	—	253k	66k	95k	445.3M	260.3M	296.1M

Article counts tell only part of the story. Tatar Wikipedia has 709,000 articles, compared with 196,000 Tatar documents in FineWeb2. South Azerbaijani Wikipedia has 245,000 articles against 79,000 FineWeb2 documents. To compare running text, also measure article lengths and the share of automatically generated articles.

4.1.4 Common Crawl itself

Common Crawl publishes, for every snapshot, the number of pages per primary language as detected by CLD2. Figure 4.4 shows the Turkic shares in all 77 snapshots from August 2018 to September 2026.



Language = CLD2 primary language. CLD2 has no label for Chuvash, Sakha or the smaller Turkic languages.

Figure 4.4: Share of pages per Turkic language in each Common Crawl snapshot (CLD2 primary language, log scale). In the legend: the share in the latest snapshot, CC-MAIN-2026-39. Source: Common Crawl crawl statistics (`plots/languages.csv`), accessed 12 October 2026.

Turkish makes up 1.28% of the pages in the latest snapshot. Every other Turkic language is below 0.1%: Azerbaijani 0.061%, Kazakh 0.043%, Uzbek 0.031%, and Uyghur 0.0016%. Russian, for comparison, has 6.7%. Between the first and the latest snapshot, the reported

shares of Kazakh, Uzbek, and Kyrgyz roughly tripled, while Turkish grew by about half. The statistics report only nine Turkic CLD2 labels, so the rest of the family is invisible in them: those pages are counted under other labels or as unknown.

4.1.5 National and curated corpora

Web corpora are not the only source. Several Turkic languages have national reference corpora, built by language institutes with documented sources and annotation. The Turkish National Corpus is one example (Aksan et al. 2012). Wikipedia is another candidate source: its provenance is known, but language and quality still require checking, and its article counts (Table 4.2) do not measure running text.

4.2 What the Labels Hide: Auditing a Corpus

4.2.1 Why labels fail for Turkic languages

A language label in a web corpus is the output of an automatic classifier applied to each document. Kreutzer et al. (2022) audited samples of several multilingual corpora by hand and found that, for many low-resource languages, a large share of the text was in the wrong language or was not language at all. For Turkic languages, seven sources of error need checking:

1. **A close neighbour gets the label.** Shared spellings can make related languages hard to distinguish. On a 20-sentence challenge set, GlotLID restricted to Turkic labels assigned Turkish to 18 of 20 Gagauz sentences; its most frequent wrong label was Karachay-Balkar for Kumyk and Kyrgyz for Nogai, Altai, and Tuvan (Table 3.8, Figure 3.4).
2. **Russian inside Cyrillic pages.** Pages from Kazakhstan, Kyrgyzstan, Tatarstan, Bashkortostan, and Sakha can mix Russian and the Turkic language: bilingual navigation, comments, and quotations. A script detector sees only Cyrillic. Only a word- or sentence-level language check finds the Russian.
3. **Several scripts and orthographies for one language.** Uzbek is written in Latin and Cyrillic; Kazakh in Cyrillic and in Latin text that may follow different proposals or informal conventions; Uyghur in Arabic script, Latin, and Cyrillic; Azerbaijani in Latin and, in Iran, Arabic script (Section 3.2). A corpus may keep only one script, mix them silently, or split them into separate subsets.
4. **Lookalike letters.** Special letters such as Kazakh *ə ғ қ ң ө ұ ү һ і* or Bashkir *ҙ ҫ* can be replaced in web text by Latin lookalikes or other substitutes (Section 3.3).
5. **Informal Latin typing.** Text in a Cyrillic-script Turkic language may be typed in an ad hoc Latin transcription, and Turkish may be typed without its special letters.
6. **Machine-translated and automatically generated text.** Machine-translated pages and automatically generated articles can enter web subsets and Wikipedias. Their share in Turkic corpora has not been measured here.
7. **Arabic-script confusion.** South Azerbaijani, Uyghur, Persian, and Ottoman Turkish share a script.

4.2.2 A Method for Auditing

Define the sampling unit (document), corpus revision, subset and complete population before sampling. Keep original IDs and raw text. A document sample estimates document shares, not token shares. Human labels from qualified evaluators are the reference; automatic scores are screening signals.

The functions in corpus.py report script shares over letters, sentence-level target/Russian/other shares, and raw FST coverage over alphabetic word tokens. Empty denominators return `None`. A recognised word needs a finite, non-failure FST reading. A copied surface form, truthy lemma or nonempty UD field is insufficient. The helper also excludes `<unk>` guesses, such as the Bashkir analyser's `Weezy<np><unk>`. These choices define lookup coverage; analysis accuracy and language labels still need independent reference data.

Construct the tokeniser, sentence splitter, unrestricted language identifier and target-language FST once and reuse them. Pass the exact expected label (for example `__label__kir_Cyrl`) rather than a prefix test. Rebuild all language-dependent objects when changing language; changing only a global `LANG` leaves the old analyser active. Newlines are not automatically sentence boundaries.

```
from corpus import audit  # code/ch04-corpus on PYTHONPATH

def audit_document(row, splitter, tokenize, lid_model, fst_lookup, target_label):
    return {"id": row["id"], **audit(
        row["text"], splitter(row["text"]), target_label,
        predict_all=lambda s: lid_model.predict(s, k=1)[0][0],
        lookup=fst_lookup, words=tokenize(row["text"]))}
```

`lid_model` here is the underlying unrestricted model, not a Turkic-only candidate wrapper. Otherwise Russian text is forced into a Turkic label and a target-share estimate becomes misleading. Save model/transducer revisions, segmentation policy, raw labels and denominators in the audit manifest. Compare signals on separately labelled development and test samples before choosing thresholds.

4.2.3 HPLT's quality bins

HPLT v3.0 sorts the documents of each language into Web Docs Scorer quality bins; the release contains bins 5 to 10, with higher bins for higher scores (Oepen et al. 2026). Figure 4.5 shows how the data of each Turkic subset is distributed across the bins.

Nine of the eleven subsets place 56–72% of their compressed bytes in bins 8–10. South Azerbaijani has 48%. Uyghur has just 1%, with almost all its data in bins 5–7. Several explanations remain possible: scorer calibration on Arabic-script text, domain composition or extraction quality. If calibration is responsible, a strict threshold could discard useful Uyghur text. If the subset is noisier, more cleaning may help. Inspect samples to distinguish these cases. Compression ratios vary too, so the byte shares cannot be read as document or token shares.

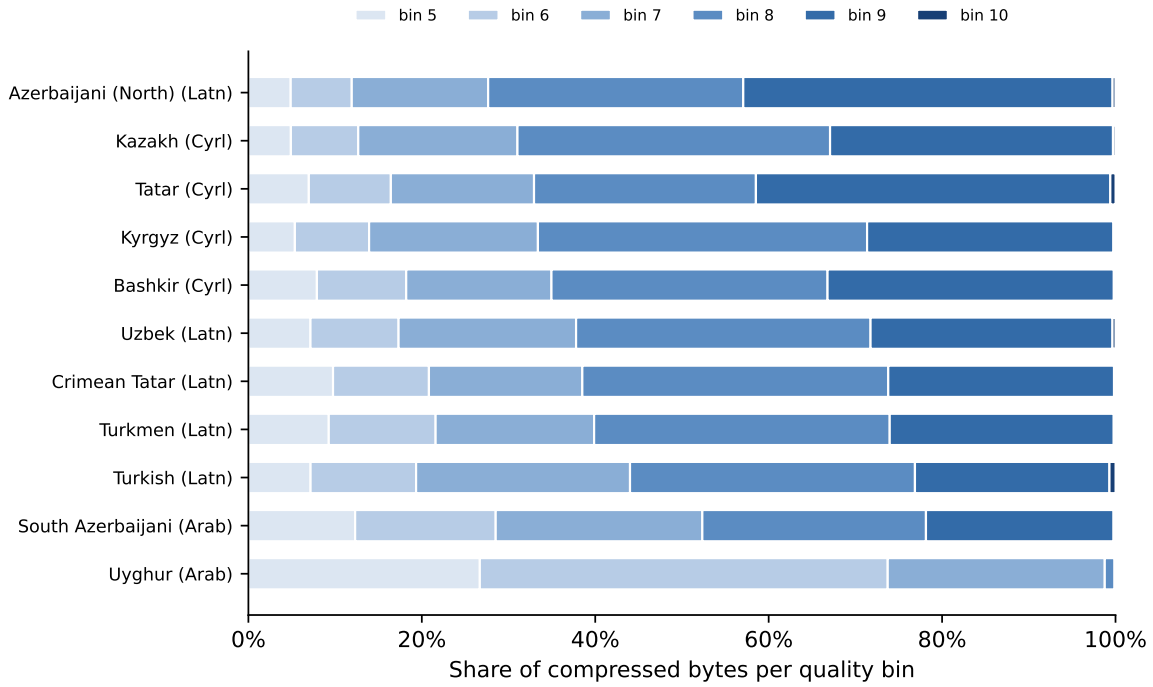


Figure 4.5: Share of each Turkic subset of HPLT v3.0 (compressed bytes) per Web Docs Scorer quality bin, from 5 (lowest in the release) to 10 (highest).

4.3 A Pipeline for Turkic Text

4.3.1 Order matters

The steps of a corpus pipeline interact. The following order avoids most of the problems discussed here and in Chapter 3:

1. **Collect** from sources whose language and orthography are documented, and keep the URL, date, licence or terms, and the original bytes.
2. **Decode and normalise** to a known encoding and create an NFC copy, with selective NFKC for Arabic presentation-form characters (Section 3.3). Keep the raw text; delay language-specific repairs (lookalike letters, apostrophes) until step 4.
3. **Identify the language before any transliteration**, at sentence or document level. An unrestricted model can predict Russian or English; it may still assign noise a language label. Add an explicit rejection policy and use a Turkic-only label set only for text already established as Turkic (Section 3.7). Record candidates and scores, and route uncertain text to an "unknown" bin. Conversion changes native-script cues: on converted text, GlotLID accuracy fell to 48.1% in the TurkicNLP evaluation (Hakimov 2026).
4. **Detect the script per document and per span**, and record the orthography version as metadata (for example Kazakh Cyrillic, 2018 Latin or 2021 Latin; Uzbek Cyrillic or Latin) rather than silently converting (Section 4.4).
5. **Handle mixed-language segments and boilerplate** under the collection's declared inclusion policy; Russian quotations are not automatically corrupt text.
6. **Filter for quality**, with thresholds set per language (below).
7. **Deduplicate with a derived key** that unifies apostrophe variants and, where needed, converts to one script, so that the same text in two variants counts once (below). Make the train/dev/test split at the level of duplicate clusters or sources, *before* creating script variants, so that a native and a converted copy cannot end up on opposite sides of the split.
8. **Normalise orthography and convert scripts in a derived copy only**: the Turkish *İ/ı* casing rules, the Uzbek apostrophe variants for *oʻ*, *gʻ* and the tutuq belgisi, and optionally conversion to a common script (Section 3.5). The archival copy keeps the original.
9. **Document permissions and personal data**: record the licence or terms of each source, and remove or mask personal data before releasing a derived corpus (Section 4.6).

4.3.2 Adapting English-Derived Quality Filters

Some web-corpus filters use word length, common words, repetition and symbol counts. The Gopher rules (Rae et al. 2021), as implemented in datatrove, include a mean word length of 3–10 characters and at least two words from an eight-word English list. These defaults need checking before use on Turkic text. FineWeb2 estimates language-specific thresholds (Penedo et al. 2025).

The aligned FLORES measurement below compares word lengths without mixing release and development treebanks. How many web documents a filter would reject is another matter. In particular, a rule requiring English stop words can reject valid Turkic text. FineWeb2 instead estimates thresholds and word lists per language (Table 4.4).

Some estimated stop-word lists suggest which domains to inspect. Among the first eight entries for Cyrillic Azerbaijani, Cyrillic Gagauz and Khakas are *јеһова* 'Jehovah', *иисус* 'Jesus' or *худай* 'God'; Cyrillic Crimean Tatar includes *рабби* 'Lord'. Kazakh, Bashkir and Chuvash lists contain words for 'water' and 'river' (*су*, *өзен*; *һыу*, *йылға*; *шыв*, *юханшыв*, respectively). They tell you where to sample; they say nothing about how large each domain is. FineWeb2 derived the lists from its own data; this chapter supplies no independent audit.

Table 4.4: Quality-filter thresholds that FineWeb2 estimated for the primary Turkic subsets, against the English defaults of the same datatrove filters. Avg word length: allowed range of a document's mean word length; alpha words: minimum share of words containing a letter (called `max_non_alpha_words_ratio` in datatrove); line punct.: minimum share of lines ending in punctuation (off = disabled; a threshold of 0.0 has the same effect); dup. lines: maximum share of duplicated lines; top 2-gram: maximum share of characters in the most frequent word bigram (values above 1 disable the rule); LID score: minimum language-ID confidence; stop words: size of the list (a document needs at least 2 of them). Sources: github.com/huggingface/fineweb-2 `configs/*.yml` and datatrove filter defaults (main branch), accessed 12 October 2026.

Subset	Avg word length	Alpha words (min)	Line punct.	Dup. lines	Top 2-gram	LID score	Stop words
English defaults	3–10	0.8	0.12	0.3	0.2	—	8
rus_Cyrl	3–11	0.713	0.231	0.322	0.184	0.9	22
tur_Latn	3–21	0.773	0.091	0.272	0.214	0.875	16
azj_Latn	3–13	0.725	0.188	0.879	0.514	0.3	22
azb_Arab	3–14	0.719	0.111	0.379	0.403	0.3	14
tuk_Latn	1–24	0.769	0.0	0.276	0.406	0.666	22
gag_Latn	2–24	0.778	0.0	0.536	0.628	0.3	19
kaz_Cyrl	3–12	0.694	0.176	0.304	0.172	0.844	24
kir_Cyrl	4–12	0.623	0.133	0.311	0.247	0.756	31
tat_Cyrl	3–11	0.692	0.333	0.356	0.26	0.86	8
bak_Cyrl	3–11	0.72	0.222	0.217	0.221	0.9	25
crh_Latn	3–19	0.679	0.167	0.774	0.595	0.3	20
kaa_Latn	2–17	0.714	0.05	0.275	0.844	0.3	14
nog_Cyrl	3–13	0.688	off	0.331	0.783	0.357	13
kum_Cyrl	3–11	0.744	off	0.434	0.62	0.65	15
krc_Cyrl	2–11	0.743	0.0	0.498	0.578	0.3	27
uzn_Latn	3–13	0.691	0.25	0.254	0.871	0.631	21
uig_Arab	2–15	0.782	0.0	0.293	0.2	0.874	16
sah_Cyrl	1–13	0.711	0.143	0.401	0.547	0.3	20
alt_Cyrl	4–9	0.671	0.286	0.7	1.337	0.3	22
tyv_Cyrl	2–11	0.682	0.167	0.285	0.327	0.642	8
kjh_Cyrl	3–12	0.7	off	0.594	1.052	0.3	42

Subset	Avg word length	Alpha words (min)	Line punct.	Dup. lines	Top 2-gram	LID score	Stop words
chv_Cyrl	3–11	0.68	0.116	0.243	0.205	0.732	50

4.3.3 A source-aligned orthographic-length check

A complementary measurement uses all 1,012 aligned rows of **original FLORES-200 devtest** in eleven Turkic varieties, English and Russian (NLLB Team et al. 2024). The count covers Unicode letters in each whitespace-delimited unit containing at least one letter, after NFC normalisation. It averages units within each sentence and then weights sentences equally. This definition excludes punctuation from the length count; it does not segment morphemes.

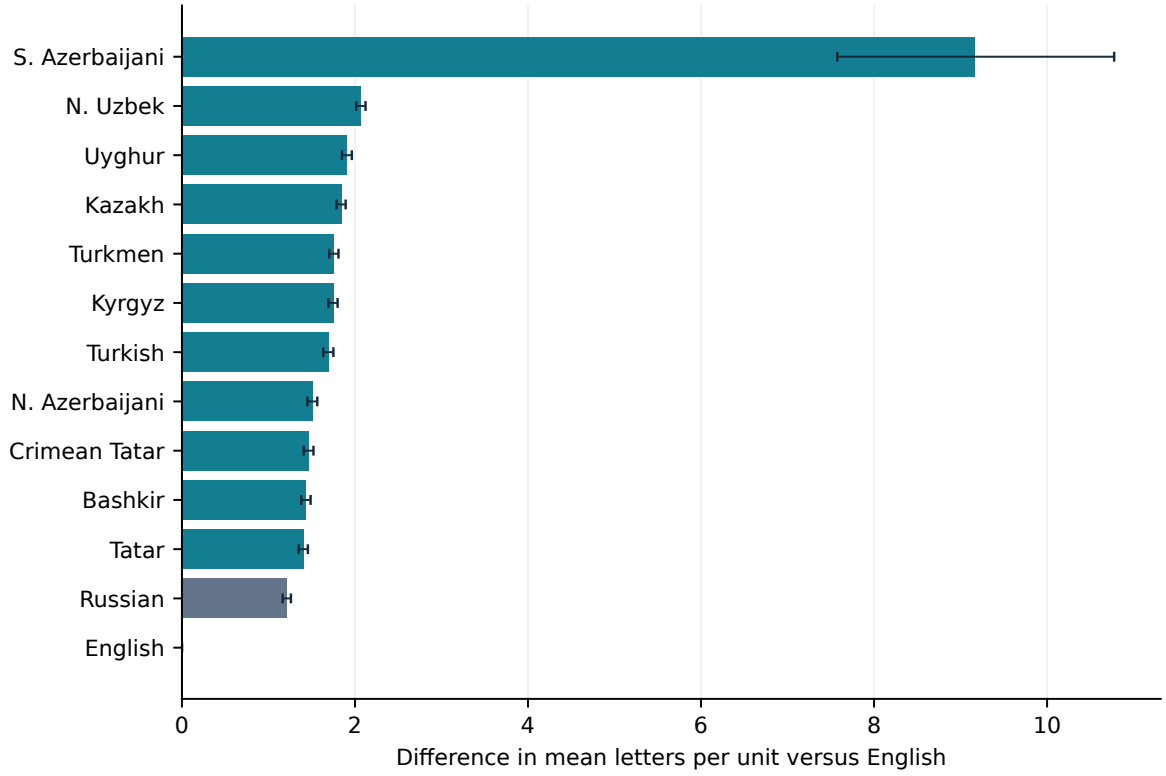


Figure 4.6: Measured difference from English in mean letters per orthographic unit on original FLORES-200 devtest. Bars use the same 1,012 aligned sentence rows in each variety. Intervals are the 2.5th and 97.5th percentiles of 2,000 paired sentence bootstrap samples (seed 20261017); English is the zero reference.

Spacing anomalies dominate the South Azerbaijani bar: 83 of 1,012 rows contain only one whitespace unit, and 128 average more than 30 letters per unit (`audit_spacing.py`).

Row 198, for example, has 227 letters without an internal space. The audit retains and lists these rows. The bar thus shows how formatting affects the measurement; it cannot serve as a language-level word-length estimate.

Matching the translated source rows removes one source of content variation, but translations and orthographic conventions still differ. The intervals describe resampling these sentences, not uncertainty over the population of web documents; source-article clustering is not modelled. The saved results also include a contiguous-letter-run definition for sensitivity checking. Neither definition measures morphological complexity, and neither validates a quality filter. That requires independently labelled documents.

4.3.4 Deduplication across scripts

Character-overlap methods can miss duplicates published in different scripts. An Uzbek story in Latin and Cyrillic may share little written text even when the content is the same. After both versions are converted to one script, the overlap can be measured. The same holds for Kazakh text in Cyrillic and in one of the Latin designs. Figure 3.3 showed the effect of conversion for related languages; its effect on duplicate detection for the same language in two scripts can be measured on labelled document pairs.

After a versioned script conversion, a deduplication key can normalise Uzbek *o/g* letter marks with `uzbek_letter_apostrophes` in corpus.py. The helper maps apostrophe variants after *o* or *g* to U+02BB, while TurkicNLP uses U+0027 (Section 3.5). Either convention can work if applied consistently. Keep tutuq signs and quotation punctuation distinct.

MinHash requires further choices: character shingles, hash count and similarity threshold, tuned on labelled pairs. Since conversion can merge different originals, treat key matches as candidates and measure false merges. Build duplicate/source clusters before assigning train/dev/test splits.

4.3.5 Personal data

Web text contains names, phone numbers, addresses, and identity numbers. Turkic-specific patterns include national identification numbers with fixed formats, such as the Turkish *T.C. kimlik numarası* and the Kazakh individual identification number (IIN). Remove or mask these before releasing any derived corpus.

4.4 Choosing a Script for the Corpus

For a language written in more than one script, the corpus builder has to decide what to store and what to release. Three rules work well:

- **Keep the original.** Store every document in the script it was published in, with the detected script and orthography version as metadata. Conversion can lose information: in the toolkit's CTS mapping, for example, Kazakh *ұ* and *ү* both become *u* (Section 3.6), so the original cannot always be recovered from the converted text.
- **Release conversions as derived data.** A version in a common script, such as the toolkit's CTS mapping (Section 3.6) or an official Latin alphabet, is useful for training. Release it next to the original, not instead of it.
- **Say what was merged.** If Uzbek Latin and Cyrillic are merged into one training set, or Kazakh text in several Latin designs is normalised to the 2021 design, document the conversion and the mapping version.

4.5 Parallel and Comparable Data

Parallel text expresses corresponding content in two languages. It provides supervision for machine translation (Chapter 10) and cross-lingual sentence embeddings (Section 13.3), though both also have methods that learn without sentence-aligned data. For Turkic languages, useful sources include:

- **Curated collections**: OPUS (Tiedemann 2012), the TIL corpus with its X-WMT test sets (Mirzakhalov, Babu, Ataman, et al. 2021), and FLORES-200 (NLLB Team et al. 2024).
- **Bilingual institutional websites**: governments and news agencies that publish in a Turkic language and in Russian or English.
- **Religious texts**: translated texts are one possible source; FineWeb2's word lists suggest checking for them in some subsets (Table 4.4). Check their licences before including them.
- **Comparable text**, such as Wikipedia articles on the same topic, from which parallel sentences can be mined.

Mining parallel sentences usually relies on multilingual sentence embeddings such as LaBSE or LASER (Feng et al. 2022; Heffernan, Çelebi, and Schwenk 2022), scored with a margin criterion (Artetxe and Schwenk 2019). For very close pairs, such as Turkish–Gagauz or Kazakh–Kyrgyz, string similarity after conversion to a common script (Figure 3.3) is a candidate retrieval signal or extra filter; its precision has to be checked by hand (Exercise 3).

4.6 Licensing, Provenance, and Sovereignty

Check the terms for both the corpus release and its source material. A licence on the compilation may not cover redistribution of every source page.

- **Prefer releasing IDs and labels.** When an audit or a filtered subset is shared, consider sharing document identifiers, scores and labels when the text cannot be redistributed. It does not remove rights or privacy obligations.
- **Check religious and literary texts individually.** A text that is free to read online may come with terms that forbid text and data mining or redistribution.

- **National law differs.** Copyright exceptions for text and data mining differ between the countries where Turkic languages are spoken.
- **Document the corpus.** A datasheet records sources, collection dates, filters, known problems, and intended uses. For a small language, record the community's position on its use as well (Section 17.4).

4.7 Hands-On: Auditing a Corpus Sample

Question. What kinds of unwanted text do the audit signals detect?

Inputs. A documented sample, unrestricted language identification, script checks and an analyser.

Compute. CPU inference and resource downloads; size the sample before processing a full corpus.

Output. Per-document signals, reviewed errors and a provenance manifest.

Interpretation. Calibrate decisions against reviewed documents before selecting thresholds.

Worked step. Exercise 1 first runs the audit on documents of known composition: five-sentence X-WMT documents in Kyrgyz, in its neighbour Kazakh, in Russian, mixed (three Kyrgyz and two Russian sentences), and Kyrgyz converted to CTS, 20 of each. The recorded output uses GlotLID v3 and the Apertium Kyrgyz analyser; the illustrative "kept" rule requires a target share and an analyser coverage of at least 0.6.

document type	target	Russian	FST cov.	Cyrillic	kept
target	1.00	0.00	0.83	0.98	20/20
neighbour	0.00	0.00	0.28	0.97	0/20
Russian	0.00	1.00	0.09	0.97	0/20
target 3 + Russian 2	0.60	0.40	0.52	0.97	4/20
target in CTS	0.00	0.00	0.00	0.00	0/20

Language identification separates whole documents cleanly, and analyser coverage agrees with it. Mixed documents are the hard case: the rule keeps only 4 of 20, although 60% of their sentences are Kyrgyz. Kyrgyz in CTS is lost entirely: GlotLID labels it mostly as Turkish or Gagauz, and the Cyrillic analyser rejects it. Audit before converting. These are constructed documents; a real corpus needs a hand-labelled sample to set thresholds.

Status: demonstration. The table is recorded output of Exercise 1 on constructed documents; an audit of a real web corpus is a procedure to run on its own sample.

What to Take Forward

Start an audit from the corpus label, then inspect what it contains. The Uzbek counts in Section 4.1 show how widely collections can differ. Sample documents for the intended task, preserve their provenance through processing, and check permissions before release. Chapter 5 introduces the word analyses used in later text-processing methods.

Further Reading

- Kreutzer et al. (2022) is the reference audit of multilingual web corpora and the model for Section 4.2.
- The corpus papers describe their pipelines and language identifiers: CC-100 (Conneau et al. 2020), mC4 (Xue et al. 2021), CulturaX (Nguyen et al. 2024), MADLAD-400 (Kudugunta et al. 2023), HPLT (Gibert et al. 2024; Burchell et al. 2025; Oepen et al. 2026), and FineWeb2 (Penedo et al. 2025).
- GlotLID (Kargaran et al. 2023) and OpenLID (Burchell et al. 2023) are used by the corpora discussed here.
- Glot500 (Imani et al. 2023) built a corpus and a model for more than 500 languages; its language list shows which smaller Turkic languages it covers.

Exercises

1. **Audit another language.** Use the method in Section 4.2 to audit samples for two Turkic languages in the same corpus. Which contamination type dominates in each, and how do they compare?
2. **Lookalike letters.** For Kazakh or Bashkir, count the tokens in a corpus sample that mix Latin and Cyrillic letters. Build a repair table for the 20 most frequent substitutions, and measure how much the repair raises analyser coverage.
3. **Mining with a shared script.** Mine Kazakh–Kyrgyz sentence pairs from comparable Wikipedia articles in two ways: character overlap after conversion to CTS, and LaBSE similarity. Check 100 pairs from each method by hand. Which is more precise, and where does each fail?
4. **Absent languages.** Choose one language that is absent from HPLT v3.0 (Table 4.1). Find three web sources for it, estimate how much text they hold, and run GlotLID on a sample. Under which labels would this text end up in a web corpus?

Part III

Morphology

5 Morphological Structure: A Linguistic Foundation

Learning Objectives

After reading this chapter, you will be able to:

- Distinguish derivational from inflectional morphology and explain their interaction with suffix ordering
- Describe the nominal and verbal suffix sequences of Turkic languages, and the restrictions a template diagram leaves out
- Explain how vowel harmony, consonant assimilation, and stem alternations create allomorphic variation
- Distinguish surface segmentation, morphological analysis and a UD feature bundle, and read each of them
- Identify morphological ambiguity and explain why disambiguation requires contextual information
- Compare the morphological profiles of Turkish, Kazakh, Uzbek, Uyghur, Turkmen and Chuvash and explain key divergences

Morphological analysis identifies the stem and grammatical information in a word. Turkish *okuyamıyor*, for example, expresses reading, inability and progressive aspect. An NLP system may model this structure explicitly or learn it from data. This chapter explains suffix ordering, alternations and ambiguity; the next two chapters show computational methods.

The basic definitions of agglutination, case and harmony are in Chapter 2. Here we use them to read complete analyses, compare suffix sequences and resolve ambiguity.

5.1 The Morpheme Inventory

5.1.1 Derivational vs. Inflectional Morphology

A useful starting distinction is between derivation and inflection.

Derivational morphemes change the lexical category or meaning of the base. They create a new word, often with a new part of speech. Examples from Turkish:

Table 5.1: Selected derivational suffixes in Turkish

Input	Suffix	Output	Function
ev (house, N)	*-li*	*evli* (married, Adj)	denominal adjective
yaz- (write, V)	*-ı*	*yazı* (writing, N)	deverbal noun
güzel (beautiful, Adj)	*-leş-*	*güzelleş-* (become beautiful, V)	deadjectival verb
git- (go, V)	*-mek*	*gitmek* (to go, V→infinitive)	infinitive nominalisation

In the simple pattern illustrated here, derivational morphemes precede the outer inflection and can be stacked: *güzelleştir-* (to make beautiful) adds a causative to the deadjectival verb. The resulting derived stem behaves as a new root for subsequent affixation.

Inflectional morphemes mark grammatical categories such as number, case, tense and agreement. They usually follow derivation in the patterns shown here. The boundary is not absolute: a category change can introduce another inflectional domain, and individual stems or constructions restrict which suffixes can follow.

An analyser often needs lexical entries for derived meanings: composition alone cannot tell it that *ev-li* means ‘married’. Inflection is more systematic, so rules can generate many forms without listing each one. Grammars and treebanks draw the boundary differently, especially for infinitive *-mAk*, voice suffixes and copular forms. State which convention the analyser follows.

5.1.2 The Suffix Template

Turkic suffix ordering follows regular patterns. Johanson (2021, 37) notes that the high degree of synthesis “allows combinations of a large number of bound denominal and deverbal markers to form long sequences”. In the common patterns below, derivational material comes before inflectional material, and each suffix applies to the form on its left. For noun inflection, “Common Turkic follows the ordering number-possessive-case”; Chuvash is the exception, with possessive before number (Johanson 2021, 452) (Section 5.4.5).

Nominal template (for nouns and adjectives):

$$\text{Root} \rightarrow [\text{Derivation}]^{*} \rightarrow \text{Number} \rightarrow \text{Possessive} \rightarrow \text{Case} \rightarrow [\text{Copula}]$$

- **Derivation** (optional, repeatable): changes stem meaning or category
- **Number** (*-lar/-ler* plural): absence of plural marking need not mean singular reference
- **Possessive** (1sg, 2sg, 3sg, 1pl, 2pl, 3pl): marks possessor agreement
- **Case** (nominative, accusative, dative, locative, ablative, genitive)
- **Copula** (optional): predicative forms *-dır/-dir/-dur/-dür* or evidential *-(y)mış*

Example: Turkish *ev-ler-imiz-den* (from our houses):

Slot	Morpheme	Gloss
Root	*ev*	house
Number	*-ler*	PL
Possessive	*-imiz*	POSS.1PL
Case	*-den*	ABL

Verbal template (for verbs):

Root → [Derivation]* → Voice → Ability/Negation → TAM → Person/Number

- **Derivation** (optional): lexical derivation such as *güzel-leş-* 'become beautiful'
- **Voice** (passive *-Il/-In*, causative *-DIr/-t*, reciprocal *-Iş*, reflexive *-In*). Some grammars treat voice as derivation, because it changes the valency of the stem; this book lists it as a separate slot. Apertium encodes voice with tags, while UD uses features such as `Voice=Cau` and `Voice=Pass`
- **Potential** (*-(y)a/-(y)e*, the ability marker that precedes negation in *gel-e-me-* "be unable to come"; the positive *-(y)abil/-(y)ebil* "can" goes back to the same marker plus the verb *bil-* "know")
- **Negation** (*-ma/-me*)
- **TAM** (tense, aspect, mood: one slot, complex inventory)
- **Person/Number** (agreement with subject: 1sg, 2sg, 3sg, 1pl, 2pl, 3pl; two paradigms, see below)

Read the diagrams as summaries of common sequences. Which suffix can follow depends on the stem, the construction and the morphology already present. TAM may have several exponents (*gel-iyor-du* 'was coming', progressive + past), agreement has several paradigms, and a nominalised verb takes nominal inflection (*oku-duk-lar-ı-n-dan*). Chapter 6 shows how continuation classes encode the combinations licensed by a language.

5.1.3 Nominal Morphology in Detail

Number. The plural is marked by *-lar/-ler* (the two forms differ by vowel harmony). The form without a plural suffix is unmarked for number, and its reference can include more than one entity: Turkish *Ali gazete okudu* can mean that Ali read a newspaper or newspapers (Johanson 2021, 945). In Turkish, a noun after a numeral normally takes no plural suffix: *beş kitap* (five book), not *beş kitaplar*. A system that reads plural marking as a count of referents will therefore be wrong in both directions.

Possessive. The Turkish possessive suffixes have different shapes after consonant-final and vowel-final stems. In the abstract notation used here, *I* stands for the four-way harmonising vowel (ı/i/u/ü) and *A* for the two-way one (a/e):

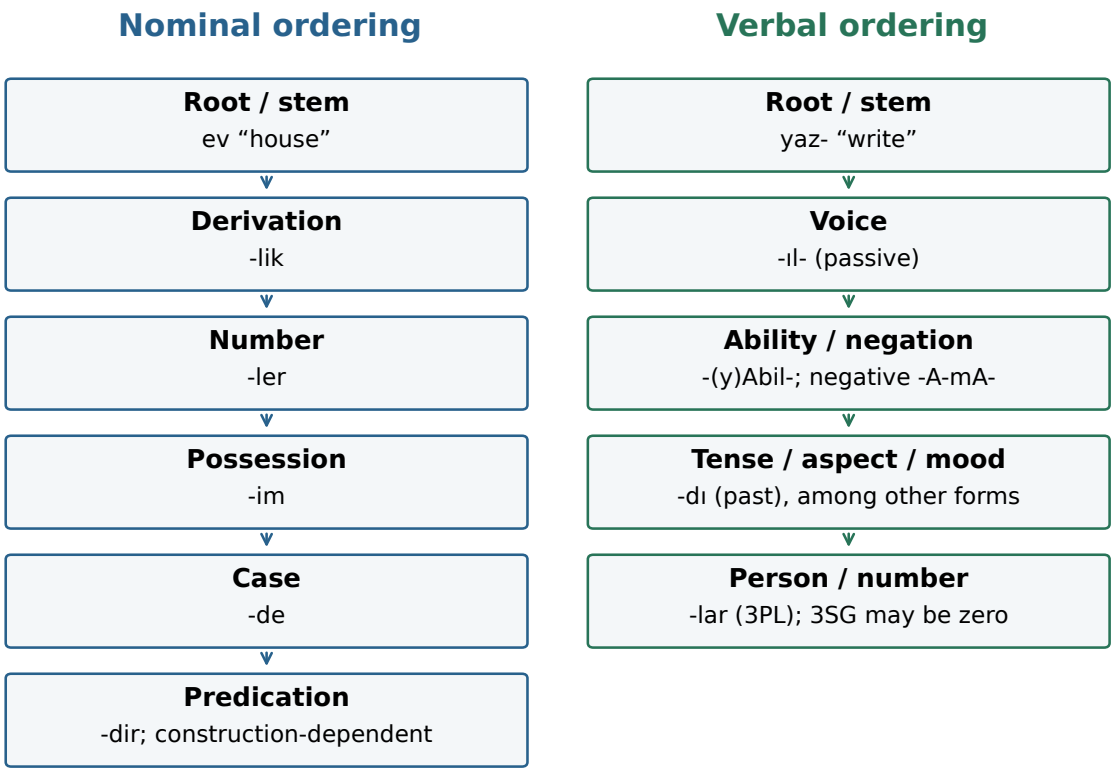


Figure 5.1: Common nominal and verbal suffix sequences. Ability and negation interact; nominalised verbs enter nominal inflection. Boxes describe morphological domains, not a requirement for an overt suffix in each position.

Table 5.3: Turkish possessive suffixes after consonant- and vowel-final stems. *I* = four-way harmony (ı/i/u/ü), *A* = two-way harmony (a/e). With a plural noun the plural comes first: *ev-ler-im* ‘my houses’.

Possessor	After a consonant	After a vowel	Examples
1sg	*-Im*	*-m*	*ev-im*, *araba-m* ‘my house/car’
2sg	*-In*	*-n*	*ev-in*, *araba-n*
3sg	*-I*	*-sI*	*ev-i*, *araba-sı*
1pl	*-ImIz*	*-mIz*	*ev-imiz*, *araba-mız*
2pl	*-InIz*	*-nIz*	*ev-iniz*, *araba-nız*
3pl	*-lArI*	*-lArI*	*ev-leri*, *araba-ları*

The third person plural is a source of ambiguity: *evleri* can be *ev-leri* ‘their house’ (POSS.3PL) or *ev-ler-i* ‘his/her houses’ or ‘their houses’ (PL + POSS.3, with a singular or plural possessor), and Turkish does not write a second plural for ‘their houses’. Johanson describes the same three-way reading for the type ‘horse-PL-POSS3’ and notes that a form with two plurals is illicit (2021, 459). A possessive pronoun resolves it for nouns: *onların evi* only means ‘their house’, and *onların evleri* only ‘their houses’ (2021, 459). The number of the possessed noun and the number of the possessor cannot be read off as independent slots.

Possessive suffixes come between number and case, and a third-person possessive is followed by a buffer *n* before a case suffix: *araba-sı-n-ı* ‘his/her car’ (ACC), *araba-sı-n-da* ‘in his/her car’. For segment counts, this book attaches the buffer to the following suffix, as in Section 2.7: *araba-sı-nı* has three counted pieces. A finer notation, *araba-sı-n-ı*, shows the buffer separately but does not count it as another morpheme.

Case. Six commonly compared cases (not present in every language) are introduced in Chapter 2 (Section 2.3). Their suffixes in Turkish are:

Case	Suffix	Function
Nominative	∅	Subject; predicate nominal
Accusative	*-(y)ı/i/u/ü*	Specific direct object
Dative	*-(y)a/e*	Indirect object; goal; experiencer
Locative	*-da/de/ta/te*	Location; point in time
Ablative	*-dan/den/tan/ten*	Source; material; comparison
Genitive	*-(n)ın/in/un/ün*	Possession; complement of some postpositions after pronouns (*benim için* ‘for me’)

Note the consonant alternation in locative and ablative: *-da* after voiced consonants and vowels, *-ta* after voiceless consonants. This is consonant assimilation, discussed in Section 5.2.

Copula. Nominal predicates in Turkic languages can take a copular suffix directly on the noun: Turkish *doktor-um* (I am a doctor), *doktor-sun* (you are a doctor), *doktor-dur* (he/she is a doctor). Third-person present predication can also be unmarked (*O doktor*, “He/she is a doctor”); *-DIr* is not an obligatory third-person ending and can add assertive

or inferential force. Identifying a nominal predicate therefore requires constructional context (Johanson 2021, chap. 27, pp. 555–571).

5.1.4 Verbal Morphology in Detail

Voice. The following table illustrates an unmarked active stem and four voice-related patterns:

Table 5.5: Turkish verbal voice suffixes (allomorphs vary by phonological context)

Voice	Suffix	Example	Gloss
Active	∅	*sev-*	love
Passive	*-il/-ın*	*sev-il-*	be loved
Reflexive	*-In*	*yıka-n-*	wash oneself
Reciprocal	*-Iş*	*gör-üş-*	see each other, meet
Causative	*-dir/-tir/-ir/-t*	*sev-dir-*	make (s.o.) love

Voice suffixes can be stacked: *sev-dir-il-* (to be made to love) places passive outside causative, and *öl-dür-t-* ("have someone killed") contains successive causatives. No single exhaustive ordering rule follows from such examples. What can come next depends on the stem, valency and construction. This stacking produces the long verb chains discussed in Chapter 2.

Negation. The negation suffix *-ma/-me* follows voice morphemes and the potential marker and precedes TAM in the ordinary synthetic forms illustrated here: *sev-me-* (not love), *sev-il-me-* (not be loved), *sev-dir-me-* (not cause to love). The template covers verbal *-mA-* only; other negative constructions work differently. Its surface shape also varies: before progressive *-Iyor*, its vowel is absent (*gel-m-iyor*).

Ability and inability. Turkish expresses these meanings with suffixes; other Turkic languages also use auxiliary constructions:

- *gel-ebil-* (to be able to come): the stem *gel-* + ability suffix *-ebil-* (historically involving *bil-* "know")
- *gel-e-me-* (to be unable to come): the stem *gel-* + *-e-* + negative *-me-*; the segmentation convention distinguishes the ability component from negation

The sequence *-(y)eme-/-(y)ama-* (potential + negation) is formally distinct from the bare negative *-me-/-ma-*, so inability and simple negation are not confused: *gel-me-di* "did not come" vs. *gel-e-me-di* "could not come".

Tense, Aspect, and Mood (TAM). TAM occupies a single slot in the verbal template but with a complex inventory. Turkish distinguishes:

Table 5.6: Selected Turkish TAM suffixes. *A* = a/e, *I* = ı/i/u/ü, *D* = d/t according to voicing; *(y)* appears where required.

Category	Suffix	Meaning
Simple past	*-DI*	Past; not restricted to witnessed events
Indirective/perfect	*-mIş*	Inference, report or perfect reading, depending on construction
Aorist	*-Ar/-Ir/-r* (stem-dependent)	Habitual/generic present
Present progressive	*-Iyor* (with stem-vowel adjustments)	Ongoing action now
Future	*-(y)AcAk*	Future tense
Conditional	*-sA*	Conditional mood
Optative	*-(y)A*	Wish/exhortation

Kazakh, Kyrgyz, and other Kipchak languages share many of these distinctions but with different suffix shapes, and many Turkic languages express aspect partly through converb + auxiliary constructions with verbs such as ‘lie’ (Kazakh *оқып жатыр* ‘is reading’, Turkmen *okap ýatyr*; Johanson (2021), p. 646). Some of these have fused into one word (Uzbek *oʻqiyapti* ‘is reading’, historically from *oʻqi-b yot-*). Analysers must treat them as multi-word or grammaticalised forms.

Person/Number agreement. The final slot marks the agreement of the verb with its subject. Turkish has two main agreement paradigms. Suffixes of the “pronominal type” follow the aorist, the progressive, *-mIş*, the future and the necessitative, and also mark nominal predicates; suffixes of the “possessive type” follow the past *-DI* and the conditional *-sA* (Johanson 2021, 439, 555):

Table 5.7: Turkish verbal agreement: the two main paradigms, shown with *gel-* ‘come’ (Johanson 2021, 439).

Person	After the aorist (*gel-ir-*)	After the past (*gel-di-*)
1sg	*gelir-im* (*-(y)Im*)	*geldi-m* (*-m*)
2sg	*gelir-sin* (*-sIn*)	*geldi-n* (*-n*)
3sg	*gelir* (∅)	*geldi* (∅)
1pl	*gelir-iz* (*-(y)Iz*)	*geldi-k* (*-k*)
2pl	*gelir-siniz* (*-sInIz*)	*geldi-niz* (*-nIz*)
3pl	*gelir-ler* (*-lAr*)	*geldi-ler* (*-lAr*)

In the two paradigms shown, third person singular has no overt agreement suffix, a fact with consequences for pro-drop and coreference (see Section 2.4 and Chapter 8).

5.2 Morphophonological Alternations

The same morpheme appears in multiple surface forms depending on phonological context. Three types of alternation are most important for Turkic NLP: vowel harmony in suffixes,

consonant assimilation, and stem alternations.

5.2.1 Vowel Harmony in Suffixes

Covered in Section 2.2, the harmonising vowels of Turkish suffixes commonly alternate in two-way or four-way sets. Invariant suffixes and consonantal alternations require separate rules. The pattern is:

- **2-way suffixes** (back vs. front only): e.g., Turkish plural *-lar/-ler*
  - *kitap-lar* (books, back vowel root)
  - *ev-ler* (houses, front vowel root)
- **4-way suffixes** (back/front × round/unround): e.g., Turkish 1sg possessive *-ım/-im/-um/-üm*
  - *kol-um* (my arm, back round)
  - *kitab-ım* (my book, back unround)
  - *göz-üm* (my eye, front round)
  - *ev-im* (my house, front unround)

For most stems, native or borrowed, the last vowel of the stem determines the harmony class of the suffix, even when earlier syllables mix classes (*kitap-lar*, *telefon-lar*). The exceptions listed in Section 2.2 must be marked in the lexicon: stems whose final consonant selects front suffixes (*saat-ler* 'hours', *rol-ü* 'role-ACC') and suffixes outside the regular pattern (*-yor*, *-ken*, and usually *-ki*). A regular stem and an exceptional one can look alike: *saat* and *sabah* both have a back vowel in their final syllable, but only *sabah-lar* follows the regular rule.

For morphological analysis, harmony is a strong constraint: if an analyser sees *evlerde*, it can rule out analyses in which the locative should be *-da*, because *ev* is a front-vowel root. Rule-based analysers encode the constraint explicitly; neural analysers can learn it from data.

5.2.2 Consonant Alternations

Beyond vowel harmony, consonant alternations are common at morpheme boundaries.

Voicing assimilation. The Turkish past tense suffix is *-di/-dı/-du/-dü* after voiced consonants and vowels, but *-ti/-tı/-tu/-tü* after voiceless consonants:

Verb	Past form	Rule
gel- (come)	*gel-di*	*l* is voiced → *d*
git- (go)	*git-ti*	*t* is voiceless → *t*
oku- (read)	*oku-du*	*u* is a vowel → *d*
iç- (drink)	*iç-ti*	*ç* is voiceless → *t*

The same assimilation applies to the locative (*-da/-ta*), ablative (*-dan/-tan*), and several other suffixes. For the morphological analyser, this means tracking the voicing of the

final consonant of the stem. That information is phonologically transparent in the surface form.

Buffer consonants. Turkish uses *y* before vowel-initial accusative and dative suffixes after a vowel-final stem. The *n* following a third-person possessive is a separate pattern: it also occurs before a consonant-initial case suffix, as *ev-i-n-de* shows:

Form	Analysis
araba-y-ı	car-(y)-ACC ‘the car’ (not *araba-ı*)
araba-sı-n-ı	car-POSS.3SG-(n)-ACC ‘his/her car’ (object)
ev-i-n-de	house-POSS.3SG-(n)-LOC ‘in his/her house’

Stem-final consonant alternation. Turkish *kitap* (book) alternates with *kitab-* before a vowel-initial suffix: *kitab-ı* (book-ACC). This can be represented with an underlying voiced consonant and a devoicing rule, or with stem allomorphs. It is lexically conditioned; not every stem-final obstruent alternates. The *kitap/kitab-* alternation must therefore be tracked in the morphological lexicon as a property of the stem.

Other stem patterns. Some stems alternate before suffixes; others remain unchanged:

Root	Suffixed form	Pattern
renk (colour)	*renk-li / reng-in*	*k* → *g* in *rengin*
yüz (face)	*yüzü / yüz-de*	no alternation in this root
ağız (mouth)	*ağzı*	vowel deletion: underlying /ağız/ + ACC → *ağz-ı*

The vowel deletion in *ağız* → *ağz-* is a stem alternation triggered by the attachment of a vowel-initial suffix. An analyser must identify which stems license this alternation; it cannot apply deletion to every matching vowel sequence.

5.2.3 Why Productive Generalisation Matters

The alternations described above mean that the same morpheme can appear in multiple surface forms, and conversely that similar surface strings can correspond to different underlying morphemes. Consider:

- *kitap-ta* (book-LOC) vs. *kitab-ı* (book-ACC): the same stem surfaces as *kitap-* and *kitab-*
- *ev-de* (house-LOC) vs. *at-ta* (horse-LOC): the same locative morpheme surfaces as *-de* and *-ta*

A whole-word vocabulary stores these forms separately. Their relationships must be learned from context or supplied as features, and new texts can always contain unlisted forms. Finite-state rules (Chapter 6) and character or subword representations (Chapter 7) offer different ways to generalise. The next chapters evaluate their coverage, handling of ambiguity and usefulness for particular tasks.

5.3 What a Full Morphological Analysis Looks Like

5.3.1 From Surface Form to Feature Bundle

A morphological analysis maps a surface word form to a triple: (lemma, part-of-speech, morphological features). In Universal Dependencies notation:

$$\text{surface form} \rightarrow (\text{lemma, UPOS, FEATS})$$

The FEATS field is a set of attribute-value pairs using the Universal Dependencies morphological feature vocabulary (Marneffe et al. 2021). For Turkic languages, the relevant feature categories include:

- **Number**: `Sing`, `Plur`
- **Case**: `Nom`, `Acc`, `Dat`, `Loc`, `Abl`, `Gen`
- **Person**: `1`, `2`, `3`
- **Tense**: `Past`, `Pres`, `Fut`, `Pqp` (pluperfect, Turkish IMST)
- **Aspect**: `Perf`, `Imp`, `Prog`, `Hab`, `Prosp`
- **Mood**: `Ind`, `Cnd`, `Opt`, `Imp`, `Pot` (ability), `Nec`, `Des`, `Gen`
- **Evident**: `Fh` (firsthand), `Nfh` (non-firsthand; used for *-mIş* in these annotations)
- **Voice**: `Pass`, `Cau`, `Rcp`, `Rfl`, and combinations such as `CauPass`
- **Polarity**: `Pos`, `Neg`
- **VerbForm**: `Fin`, `Part`, `Conv`, `Vnoun` (Turkish), `Ger`, `Inf` (Kazakh KTB)
- **Person[psor]**, **Number[psor]**: possessor person and number (distinct from `Poss=Yes`)
- **NumType**, **PronType**, etc.

The values beyond the core set were counted in the Turkish IMST and BOUN training files and the Kazakh KTB test file (UD GitHub master branch, accessed 19 October 2026). Treebanks of the same language do not use the same subset: BOUN uses `Evident=Fh` and `Voice=Rfl`, IMST does not.

5.3.2 Three Levels of Representation

A UD feature bundle describes selected grammatical properties of a syntactic word. A transducer may additionally expose derivational steps, surface boundaries and competing readings. We will compare three representations: surface segmentation, morphological analysis (lemma and analyser tags), and UD annotation. For *evlerimizden* ‘from our houses’:

Table 5.11: Three levels of representation for *evlerimizden*. The Apertium reading is `ev<n><pl><px1pl><abl>` (TurkicNLP 0.5.0).

Surface piece	Analysis (Apertium tag)	UD feature
ev	lemma *ev*, noun (`<n>`)	`UPOS=NOUN`
-ler	plural (`<pl>`)	`Number=Plur`

Surface piece	Analysis (Apertium tag)	UD feature
-imiz	possessor 1st person plural (`<px1pl>`)	`Person[psor]=1`, `Number[psor]=Plur`
-den	ablative (`<abl>`)	`Case=Abl`

The UD line, as TurkicNLP 0.5.0 outputs it:

```
form:   evlerimizden
lemma:  ev
upos:   NOUN
feats:  Case=Abl|Number=Plur|Number[psor]=Plur|Person[psor]=1
```

Note `Number[psor]` and `Person[psor]`: UD distinguishes the number of the noun itself (`Number=Plur`, ‘houses’) from the person and number of the possessor (‘our’).

5.3.3 An Advanced Example: Ambiguity in a Long Form

Consider *okuyamadıklarından*, a nominalised verb with several possessive readings. Three levels must be kept apart: the readings the grammar allows, the readings the analyser returns, and the features a treebank records.

Grammatical readings. The form is *oku-ya-ma-dık* ‘read-ABIL-NEG-NMLZ’ followed by possessive agreement and the ablative. The nominal can head a headless relative clause, ‘from/among the things that … could not read’, or denote the event; an ablative event nominal commonly expresses a cause, ‘because … could not read’. The possessive identifies the subject of the clause (Section 2.1): *-lar-ın* PL + 2SG (‘you’, with plural *-lar* marking the things), *-lar-ı* PL + 3 (‘he/she’ or ‘they’), or *-ları* 3PL (‘they’, with a singular nominal).

Analyser readings. The Turkish Apertium transducer returns these three readings (ignoring readings with a copula added):

```
oku<v><tv><abil><neg><ger_past><pl><px2sg><abl>    -lar-ın   PL + POSS.2SG   'from what you
 ↪  could not read'
oku<v><tv><abil><neg><ger_past><pl><px3sp><abl>    -lar-ı-n  PL + POSS.3     'from what
 ↪  he/she/they could not read'
oku<v><tv><abil><neg><ger_past><px3pl><abl>        -ları-n   POSS.3PL        'because they
 ↪  could not read'; 'from what they could not read'
```

The boundaries differ according to whether *-lar* marks the nominal’s plural or belongs to third-person plural agreement, and whether *-n-* is a buffer or part of the second-person possessive. Context narrows the choice. *Senin okuyamadıklarından* selects the second person (‘from what you could not read’); *onun okuyamadıklarından* selects a plural nominal with a singular possessor (‘from the things he/she could not read’). *Onların okuyamadıklarından* can still mean ‘from the things they could not read’ or, with *-ları* as third-person plural agreement, ‘because they could not read’.

Apertium lists the morphological possibilities but assigns the same analysis to relative-clause and event readings. Its tag `ger_past` names *-DIK*, whose temporal interpretation depends on the construction. Turkish UD treebanks annotate such forms as participles with `Aspect=Perf|Tense=Past|VerbForm=Part`. Their treatment of ability differs: IMST

gives *anlayamadığı* (sentence `00006231_29`) `Mood=Pot`, while BOUN gives *damgalayamadığını* (sentence `news_128`) no `Mood` feature (Chapter 8). The following illustrates an IMST-style bundle for the POSS.3PL reading:

```
form:   okuyamadıklarından
lemma:  oku
upos:   VERB
feats:  Aspect=Perf|Case=Abl|Mood=Pot|Number[psor]=Plur|Person[psor]=3|Polarity=Neg|Tense=P
 ↪  ast|VerbForm=Part
```

5.3.4 The Same Analysis Across Three Languages

To illustrate the shared template, consider the parallel: “from our houses” in Turkish, Kazakh, and Uzbek:

Language	Form	Morphemes
Turkish	*ev-ler-imiz-den*	house-PL-POSS.1PL-ABL
Kazakh	*үй-лер-іміз-ден* (üi-ler-ımız-den)	house-PL-POSS.1PL-ABL
Uzbek	*uy-lar-imiz-dan*	house-PL-POSS.1PL-ABL

Latin forms in parentheses use the toolkit’s Kazakh transliteration (Section 3.2). The Apertium analysers for the three languages return the same tag sequence for these forms (`<n><pl><px1pl><abl>`, TurkicNLP 0.5.0).

The order of plural, possessive and case is the same in all three languages, as in Common Turkic generally (Johanson 2021, 452). The differences are in the surface forms: Kazakh *-іміз* harmonises with front stems, Uzbek *-imiz* is invariant, and the ablative suffix varies. An analyser design for Turkish can therefore be reused for Kazakh and Uzbek by adjusting harmony rules, suffix shapes and lexicon; whether the result works must be validated on the target language, as the next comparison shows.

For a more demanding comparison, consider the Turkish sentence *Kitabı okuyamıyorum* (I cannot read the book), its Kazakh equivalent *Кітапты оқи алмаймын* (Kıtapty oqi almaimyn) and its Turkmen equivalent *Kitaby okap bilmeýärin*:

	Turkish	Kazakh	Turkmen
Object	*kitab-ı* (book-ACC)	*кітап-ты* (kıtap-ty, book-ACC)	*kitab-y* (book-ACC)
Verb	*oku-ya-m-ıyor-um* (read-ABIL-NEG-PROG-1SG)	*оқи ал-ма-й-мын* (*оқы-* ‘read’ + converb, take-NEG-PRS-1SG; analytic)	*oka-p bil-me-ýär-in* (read-CVB know-NEG-PRS-1SG; analytic)

Turkish packs inability into *okuyamıyorum*. Kazakh uses a converb and auxiliary *ал-* ‘take’: *оқи алмаймын*, literally ‘reading I do not take’. Turkmen also uses an auxiliary, *bil-* ‘know’, cognate with Turkish *-(y)Abil-*. Johanson lists ‘take’, ‘know’ and ‘become’ as ability auxiliaries, including Kazakh *žaz-a al-* ‘be able to write’ and Uzbek *o̧ki-y-ål-* ‘be able to read’ (2021, 613).

The Kazakh analyser returns `ал<vaux><neg><aor><p1><sg>` as one of two readings of *алмаймын*. Its tense tag is aorist, where our gloss uses present. Both the word structure and the annotation convention need checking when adapting an analyser across languages.

5.3.5 Morphological Ambiguity

A single surface form can have multiple valid analyses. This is **morphological ambiguity**, and it is pervasive in Turkic text. Two useful distinctions are:

Analysis ambiguity: The same segmentation can carry different features. Turkish *evi* is: - *ev-i* = house-ACC ('the house', as a specific object) - *ev-i* = house-POSS.3SG ('his/her house')

The boundaries are identical; only the analysis differs. *evleri* has three readings without a + continuation in the Turkish Apertium transducer: *ev-ler-i* (house-PL-ACC 'the houses'), *ev-ler-i* (house-PL-POSS.3 'his/her houses' or 'their houses'; Apertium `<px3sp>`), and *ev-leri* (house-POSS.3PL 'their house'). Here the readings also differ in where the boundary falls.

Category ambiguity: The same form can belong to different categories. Turkish *yazar* can be: - A noun: "author" - A verb: *yaz-ar* "writes" (aorist, 3SG)

and *okuma* can be the verbal noun "reading" or the negative imperative "don't read!".

On the Turkish TueCL test file (UD 2.17), 608/730 eligible tokens (83%) receive multiple raw readings; 399/730 (55%) still do after excluding every reading containing a + continuation. On the Kazakh-KTB test file (UD 2.17), a different and larger sample of mostly encyclopaedic and literary sentences, the counts are 6,313/7,667 (82%) and 5,071/7,667 (66%). The denominator consists of integer-ID syntactic words except `PUNCT`, `NUM` and `SYM`, including unknown words (41 Turkish and 56 Kazakh). Thus "without +" is a reproducible filter, not a linguistic test for copulas alone. Lookup uses NFC surface spelling, then Turkish-aware lowercasing for Turkish or ordinary lowercasing for Kazakh only if the surface has no reading. `code/ch05-ambiguity/` contains the script, exact counts, per-token readings and SHA-256 hashes of the two test files and transducers. Because the samples differ in genre and size, the Turkish–Kazakh difference is not a language comparison; read the figures as properties of these samples. They do not generalise to Turkish or Kazakh text at large. Disambiguation requires context, typically the surrounding words and their likely analyses.

5.3.6 The Disambiguation Problem

Given a set of possible analyses for each word in a sentence, the disambiguation problem is to select the correct analysis for each word. The constraints are mutual: the analysis of one word affects the analyses of its neighbours. For example, a noun in the accusative makes a following transitive verb more likely.

Classic approaches train statistical models over sequences of morphological tags on manually disambiguated corpora; for Turkish, Hakkani-Tür, Oflazer, and Tür (2000)

model the tags of each word as a sequence of inflectional groups (see also Oflazer and Saraçlar (2018), chapter 3). More recent work uses neural taggers trained on Universal Dependencies data (Qi et al. 2020). Supervised taggers need labelled training data, but those labels can come from manual annotation or automatic methods with different error rates. A target-language learning curve shows how additional annotation affects accuracy. This motivates the cross-lingual transfer strategies discussed in Chapter 7.

5.4 Cross-Language Morphological Comparison

5.4.1 The Shared Template

Use the templates in Section 5.1 to organise comparisons, then check each construction. Chuvash changes nominal suffix order (Section 5.4.5); Kazakh expresses the inability example above analytically. These differences guide what can be reused in the transfer methods of Chapter 6 and Chapter 16.

What differs across languages is:

1. The **inventory** of morphemes in each slot (which suffixes mark which distinctions)
2. The **allomorphic rules** (how harmony and assimilation apply)
3. The **productivity** of some slots (which suffixes combine freely and which are restricted to particular stems)
4. The **degree of analyticity** (how much is expressed by suffixes vs. auxiliary verbs)

5.4.2 Where Uzbek and Uyghur Diverge

Uzbek and Uyghur (Karluk branch) diverge from the Oghuz-Kipchak norm most visibly in vowel harmony. In standard written Uzbek, suffix vowels are largely fixed: the Uzbek plural is always *-lar* regardless of root vowel (unlike Turkish *-lar/-ler*), and standard Uzbek lacks a clear-cut rounding harmony (Johanson 2021, 313). The writing understates the pronunciation, however: front/back harmony has not vanished in spoken Uzbek, and Johanson calls the impression that it has ceased to function a “false impression” supported by the orthography (2021, 245, 307). For text processing, the written form is what an analyser sees: fewer written vowel alternations, even where pronunciation varies.

Uyghur writes front/back suffix variants (*iš-lar* ‘works’ vs. *bir-gä* ‘to one’; Johanson (2021), p. 245). Its Perso-Arabic orthography, unlike Arabic or Persian, writes every vowel, which makes conversion into Latin or Cyrillic largely table-driven (Chapter 3). The complication for analysis is rather vowel raising in stems (*bala* ‘child’ → *balilar* ‘children’; cf. *ati-si* ‘his/her father’ ← *ata*, Johanson (2021), p. 27), which creates stem alternants an analyser must list or derive. The Apertium Uyghur analyser reads بالىلار (*balilar*) as <n><pl><nom>ﻻڭ.

Uzbek, Uyghur and the Kipchak languages express some verbal categories, particularly aspect and modality, with converb + auxiliary constructions rather than suffix stacking (Kazakh *оқи ал-* ‘be able to read’, Uzbek *o‘qiy ol-* ‘be able to read’, Turkmen *okap bil-* ‘be

able to read’; Uzbek after Johanson (2021), p. 613). A form that Turkish expresses as a single verb may require two words.

5.4.3 Selected Cross-Language Differences

Table 5.14 lists only contrasts that are illustrated in this chapter or in Chapter 2; each cell refers to the standard written language.

Table 5.14: Selected morphological contrasts across five Turkic languages, as illustrated in Chapter 2 and Chapter 5.

Feature	Turkish	Kazakh	Uzbek	Turkmen	Kyrgyz
Backness harmony in suffixes, written	✓	✓	largely fixed vowels	✓	✓
Rounding harmony in suffixes, written	high-vowel classes	generally not represented	no productive alternation	some suffixes (GEN *-ŭň*), not ACC	broader than Turkish
Plural *-lAr*	✓	✓ (*-лар/-мар/ -дар*…)	*-lar* only	✓ (*-lar/-ler*)	✓

Row sources: backness examples in Table 2.3; rounding patterns in Johanson (2021, 311–15); plural harmony in Section 2.2.

Differential object marking affects parsing and translation. Turkish usually marks specific objects with the accusative (*kitabı okudum* ‘I read the book’) and leaves non-specific objects bare (*kitap okudum* ‘I read a book / I did book-reading’). A specific indefinite can also take the accusative: *bir kitabı okudum* ‘I read a certain book’. Kazakh (*кітап оқыдым* vs. *кітапты оқыдым*) and Uzbek (*kitob oʻqidim* vs. *kitobni oʻqidim*) show the same basic contrast.

The conditions depend on language, discourse and construction. For parsing, accusative marking is one cue to objecthood; word order and context help identify bare objects. For translation, preserve the intended reference. English articles cannot be mapped directly onto Turkic case suffixes.

5.4.4 The Recurring Examples Analysed

The recurring sentences of Table 1.4 show several of these contrasts together. Hyphens separate morphs, and glosses follow the Leipzig conventions (abbreviations in Table D.1); ATTR marks the attributive suffix (Kazakh *-ГІ*, Uyghur *-diki*). The Uyghur line uses the treebank’s ULY transliteration.

```
Қазақстан   осы   өңір-де-гі        бейбітшілік-ті  қолда-й-ды
Kazakhstan  this  region-LOC-ATTR   peace-ACC       support-PRS-3

Мен-ин  машина-м      бар
I-GEN   car-POSS.1SG  existent

Toshkent-da   dzyudo  bo‘yicha  Gran-pri    musobaqa-si         bo‘l-ib  o‘t-moqda
Tashkent-LOC  judo    on        Grand.Prix  competition-POSS.3  be-CVB   pass-PROG.3

u    sheher  sirt-i-diki              bir  at     harwi-si-ni      kör-i-du
3SG  town    outside-POSS.3-LOC.ATTR  a    horse  cart-POSS.3-ACC  see-PRS-3
```

- **Locative + attributive.** Kazakh *өңір-де-гі* and Uyghur *sirt-i-diki* turn a locative into a noun modifier with the same suffix order.
- **Possessive marking in compounds.** Uzbek *Gran-pri musobaqa-si* and Uyghur *at harwi-si* mark the head of a noun compound with the 3rd-person possessive and no genitive. In Kyrgyz *Мен-ин машина-м*, by contrast, a genitive possessor agrees with a 1st-person possessive suffix.
- **Analytic predicates.** Kyrgyz expresses possession with the existential *бар*, without a verb ‘have’; Uzbek *bo‘l-ib o‘t-* ‘take place’ is a converb + verb combination of the kind described above.
- **Person without number.** Kazakh *-ды* and Uyghur *-du* mark the 3rd person but not number.

The treebank annotations differ from these analyses in places: Kyrgyz-KTMU tags *машинам* as genitive and *бар* as a verb with tense and aspect; Uyghur-UDT tags *sirtidiki* as nominative and *bir* as an ordinal; Kazakh-KTB marks only the locative of *өңірдегі*. A released annotation documents a convention and can contain errors; Section 8.2 returns to this point.

5.4.5 Chuvash: The Exceptional Case

Chuvash is agglutinative, with phonology and morphotactics that differ from Common Turkic. Oghur sound changes give Chuvash *r* where Common Turkic has *z* (rhotacism) and *l* where Common Turkic has *š* (lambdacism) (Johanson 2021, 25). Its plural is *-сем* (*-säm*), becoming *-сен* before case suffixes (Johanson 2021, 453). Possessive marking comes first: *кил-ĕм-сен-чен* ‘from my houses’ (house-POSS.1SG-PL-ABL), compared with Turkish *ev-ler-im-den* (Johanson 2021, 452, 455). A transducer must encode this order along with Chuvash rules and vocabulary.

On the 20-sentence-per-language TurkicNLP challenge set, Apertium detected the target phenomenon in 80% of the Chuvash sentences and the multilingual neural analyser in 60% (Hakimov 2026). The authors suggest that divergent Oghur morphology limits transfer. A comparison this small cannot settle the cause, so test transfer to Chuvash directly.

5.4.6 Practical Guidance: Starting from a Close Relative

An existing analyser can provide a starting design for a related language. Choose it by documented structural similarities and tool quality, then check the following parts:

1. **Reuse the tag set and the morphotactic design** of the relative's analyser, and check each continuation against the target language.
2. **Build a new lexicon** for the target language; cognate roots speed this up but must be checked one by one.
3. **Write the target language's own alternation rules**: harmony dimensions (and whether they are written), consonant assimilation, stem alternations.
4. **Adjust suffix shapes.** The Kazakh locative *-da/-de/-ta/-te* matches Turkish, but the Kazakh accusative *-ны/-ні/-ды/-ді/-ты/-ті* differs from Turkish *-(y)ı/i/u/ü*.
5. **Add language-specific morphemes and constructions** not present in the relative.

The Apertium Turkic analysers were built in this spirit: each language has its own lexicon and rule files, but they follow shared formalisms and tag conventions (J. Washington, Salimzyanov, and Tyers 2014). Chapter 6 develops this architecture in detail.

5.5 Hands-On: Walking Through a Morphological Analysis

Question. How does the analyser represent ambiguity and choose a reading?

Inputs. The Turkish and Kazakh sentences below and their Apertium resources.

Compute. CPU with TurkicNLP and downloaded transducers.

Output. Selected lemmas, parts of speech, feature bundles and alternative readings.

Interpretation. Compare each field with the sentence context and inspect processor provenance.

The TurkicNLP toolkit (Hakimov 2026) provides rule-based morphological analysis via Apertium transducers for 20 Turkic languages and a multilingual neural analyser for 22. The following example analyses a Turkish and a Kazakh sentence with TurkicNLP 0.5.0 and the Apertium backend (`pip install "turkicnlp[hfst]==0.5.0"`; fetch the transducers with `turkicnlp.download(lang, processors=["morph"])` for each language before running) and prints the selected analysis of each token. Missing values are printed as `_`.

```
from turkicnlp import Pipeline

# --- Turkish ---
tr_pipeline = Pipeline("tur", processors=["tokenize", "morph"], tokenize_backend="rule",
 ↪  morph_backend="apertium")
doc_tr = tr_pipeline("Evlerimizden çok uzaklaşmıştık.")

print("Turkish morphological analysis:")
for word in doc_tr.words:
    print(f"  {word.text:<16} lemma={word.lemma or '_':<10} upos={word.upos or '_':<6}
     ↪  feats={word.feats or '_'}")

# --- Kazakh (Cyrillic) ---
kz_pipeline = Pipeline("kaz", processors=["tokenize", "morph"], tokenize_backend="rule",
 ↪  morph_backend="apertium")
doc_kz = kz_pipeline("Үйлерімізден өте алыс кеткенбіз.")
```

```
print("\nKazakh morphological analysis:")
for word in doc_kz.words:
    print(f"  {word.text:<16} lemma={word.lemma or '_':<10} upos={word.upos or '_':<6}
     ↪  feats={word.feats or '_'}")

# --- All readings of an ambiguous word (private API in 0.5.0) ---
apertium = next(p for p in tr_pipeline._processors if hasattr(p, "_analyze"))
for reading in apertium._analyze("evleri"):
    print(reading["raw"])
```

Output (local TurkicNLP 0.5.0 checkout and Apertium backend):

```
Turkish morphological analysis:
  Evlerimizden     lemma=ev         upos=NOUN
   ↪  feats=Case=Abl|Number=Plur|Number[psor]=Plur|Person[psor]=1
  çok              lemma=çok        upos=ADV    feats=_
  uzaklaşmıştık    lemma=uzaklaş    upos=VERB
   ↪  feats=Evident=Nfh|Number=Plur|Person=1|Tense=Past
  .                lemma=.          upos=PUNCT  feats=_

Kazakh morphological analysis:
  Үйлеріміздең     lemma=үй         upos=NOUN
   ↪  feats=Case=Abl|Number=Plur|Number[psor]=Plur|Person[psor]=1
  өте              lemma=өте        upos=ADV    feats=_
  алыс             lemma=алыс       upos=NOUN   feats=_
  кеткенбіз        lemma=кет        upos=VERB   feats=Number=Plur|Person=1|Tense=Past
  .                lemma=.          upos=PUNCT  feats=_
ev<n><pl><acc>
ev<n><pl><px3sp><nom>
ev<n><pl><px3sp><nom>+i<cop><aor><p3><sg>
ev<n><px3pl><nom>
ev<n><px3pl><nom>+i<cop><aor><p3><sg>
```

The pipeline selects one analysis per word using transducer weights and heuristic context. The transducer can offer several: for *evleri*, plural accusative, plural with a third-person possessor, a third-person plural possessor, and copular continuations. Check the selected reading against a manually reviewed reference and discuss mismatches, as the exercises do.

With the PyPI release of TurkicNLP 0.5.0 and the pure-Python `pyhfst` reader, the readings of *evleri* are printed in a different order and *алыс* gets `feats=Case=Nom`; the selected lemmas and parts of speech are the same.

Evlerimizden and *Үйлеріміздең* receive parallel FEATS bundles for the same structure in different surface forms. Other outputs reveal errors. In *uzaklaş-mış-tı-k*, *-mIş-DI* expresses the pluperfect, ‘we had moved far away’. The transducer instead reads *-mIş* as evidential past (`Evident=Nfh|Tense=Past`, Section 5.1); Turkish IMST uses `Tense=Pqp`. Kazakh *алыс* ‘far’ is tagged NOUN despite its adverbial use here. The toolkit chooses among transducer readings with heuristics, without a trained syntactic disambiguator (Chapter 7). Inspect `word._provenance` and `doc._processor_log` to see which processor supplied each field.

Status: demonstration. The displayed output is recorded and includes the backend and version differences. Exercise 3 develops the ambiguity comparison.

What to Take Forward

A morphological analysis links a stem and its affixes to grammatical functions, and context selects among competing readings. Ambiguity is the normal case: in the UD test samples above, 55% of eligible Turkish tokens and 66% of Kazakh ones keep several readings even after continuation readings are removed. Surface segments, analyser tags and UD features answer different questions, so state which representation a tool produces before comparing it with another. Chapter 6 shows how finite-state analysers produce these readings, and Chapter 7 how neural models select them.

Further Reading

- **Turkic morphology**: Johanson (2021), chapter 20 (pp. 431–439), introduces morphology; chapter 22 (pp. 452–478) treats noun inflection, and chapter 27 (pp. 555–571) treats copular devices.
- **Turkish morphological analysis**: Oflazer (1994) is the foundational computational paper on Turkish two-level morphology; Oflazer and Saraçlar (2018, chaps. 2–3) gives a modern treatment of morphological processing and disambiguation for Turkish, and the rest of the book covers parsing, semantic processing and speech.
- **Universal Dependencies morphological features**: Marneffe et al. (2021) defines the UD feature inventory; the UD guidelines for Turkic languages at universaldependencies.org provide language-specific annotation decisions.
- **Cross-lingual morphology**: Hakimov (2026) compares finite-state and multilingual neural analysers on a parallel 20-language challenge set, including languages without training data.

Exercises

1. **Template exercise.** Analyse *kitaplarımıza* as *kitap-lar-ımız-a*. Write a proposed UD feature bundle, then run the analyser of the Hands-On section and compare. Distinguish the noun's number, the possessor's person and number, and case. Explain any mismatch, and record the toolkit version and backend.
2. **Allomorphy tables.** For the Turkish dative suffix (*-a/-e*) and the locative suffix (*-da/-de/-ta/-te*), fill in the correct allomorph for each of the following roots: *ev* (house), *araba* (car), *göz* (eye), *kapı* (door), *çiçek* (flower), *kuş* (bird). State the rule governing each allomorph choice.
3. **Morphological ambiguity.** List all readings the Turkish transducer returns for *yazar*, *çalışır* and *evleri* (use the readings call of the Hands-On section, or Zemberek). For each word: (a) gloss each reading; (b) write two sentences in which different readings are correct; (c) say which reading the pipeline selects in each sentence, and whether it is right.

4. **Cross-language comparison.** Segment the Kazakh word *кетпегендіктен* ‘because (someone) did not leave’ as *кет-пе-ген-дік-тен* (leave-NEG-PTCP-NMLZ-ABL): it contains both a participle *-ген* and a nominaliser *-дік*. (a) Give an equivalent Turkish construction and compare the order of the pieces. (b) Propose a UD annotation under the conventions of the Kazakh KTB treebank, and explain which pieces the UD bundle does not show. Compare it with what the Apertium Kazakh analyser returns for the word. (c) Compare Turkmen *gitmedigi üçin* ‘because (someone) did not go’ (*git-me-dig-i üçin*, go-NEG-PTCP-POSS.3 for), which expresses the cause with a postposition instead of the ablative.

5. **Morphophonological rule writing.** Write a formal rule (in prose or as a conditional) for Turkish consonant alternation in the locative suffix (*-da/-ta*): state the trigger (what phonological property of the preceding consonant) and the two output forms. Then apply your rule to the following stems: *el* (hand), *kap* (lid), *göl* (lake), *at* (horse), *iç* (inside).

6 Finite-State Morphology: The Rule-Based Approach

Learning Objectives

After reading this chapter, you will be able to:

- Explain why finite-state transducers suit Turkic morphological analysis, and where they fall short
- Describe the two-level morphology formalism, and compile and test a small lexc/twolc grammar with HFST
- Use the Apertium Turkic analysers via the HFST Python interface
- Interpret diagnostic evaluations of the Apertium Turkic analysers, and design an evaluation that measures coverage and accuracy
- Use the TurkicNLP toolkit to run rule-based morphological analysis (20 languages) and neural analysis (22 languages), and Apertium generators for generation
- Explain morphological generation and its role in data augmentation for low-resource NLP

Chapter 5 described Turkic morphology. This chapter implements part of it with finite-state transducers, following work such as Oflazer's Turkish analyser (Oflazer 1994) and the Apertium Turkic modules (J. Washington, Salimzyanov, and Tyers 2014). A small grammar shows how to encode suffix order, vowel harmony and consonant alternation.

Finite-state morphology is a practical option when explicit linguistic knowledge is available and labelled data are scarce. Multilingual neural and hybrid approaches (Chapter 7) are alternatives, and their costs and errors should be compared on the language at hand.

6.1 Why Finite-State Methods for Turkic

6.1.1 The Transparency Argument

A finite-state transducer (FST) relates two string representations. A morphological transducer maps surface forms to analyses and can be inverted for generation. Its source lexicon and rules can be inspected and tested when a form is missing or has an unwanted analysis. Tracing a compiled path back to a source rule may still take work.

For a language without annotated training data, a linguist can build a transducer from grammars and speaker consultation. When it fails, the analysis can be traced to a rule or

lexicon entry. Error analysis also helps with learned models, although it rarely identifies an explicit rule in the same way.

6.1.2 The Reversibility Argument

An FST relates two sets of strings. Inverting it swaps input and output, allowing the same relation to support analysis and generation. Ambiguity survives the swap: one analysis may have several surface forms, and one surface form may have several analyses. Composing the relation with its inverse therefore need not give the identity relation. In practice:

- **Analysis**: surface form *ev-ler-imiz-den* → `ev<n><pl><px1pl><abl>`
- **Generation**: `ev<n><pl><px1pl><abl>` → surface form *evlerimizden*

This chapter writes analyses in the Apertium tag format, as the Turkish transducer returns them.

Generation supports spell-checking, data augmentation and machine translation. Acceptance means that a form is licensed by the implemented grammar; the spelling or the choice of word can still be wrong in context (Section 3.5).

6.1.3 The Low-Data Argument

Neural morphological analysers require training data: annotated word forms with their morphological analyses. For Turkish, Kazakh, and a few others, such data exists (Universal Dependencies treebanks provide morphological annotation). For a new target language, inspect separately the available treebanks, labelled paradigms and unannotated text; absence of a treebank does not imply absence of every kind of morphological supervision.

Building an FST draws on linguistic knowledge: the suffix inventory, harmony rules and lexicon. Grammar descriptions, fieldwork and speaker consultation can supply it without an annotated training corpus. The work still takes time, and independent test data are needed to assess the resulting rules and coverage.

6.1.4 The Coverage Argument

An FST generalises to every combination licensed by its lexicon and rules, including forms absent from the development texts. That behaviour is inspectable. Missing stems and constructions cause failures; overly permissive rules can admit forms speakers would reject. A neural model may handle the test distribution well while making less predictable errors on unusual forms.

6.2 Two-Level Morphology: The Core Formalism

A widely used formalism for finite-state morphology in Turkic NLP is **two-level morphology**, introduced by Koskenniemi (1983) and applied to Turkish by Oflazer (1994). The name refers to the two levels of representation:

1. **Lexical level**: the canonical, underlying form (morphemes listed in the lexicon)
2. **Surface level**: the realised form (what appears in actual text)

The two levels are related by **two-level rules** that map lexical sequences to surface sequences, encoding phonological alternations.

6.2.1 The Lexicon (lexc)

The lexicon file (written in the `lexc` formalism) defines:

- **Lexemes**: root morphemes with their lexical representation and continuation class
- **Continuation classes** (also called *sublexica*): the sets of suffixes that can follow a given form
- **Morpheme entries**: individual suffix morphemes, each with its lexical representation, surface representation (if different), and next continuation class

The following schematic notation explains the architecture; it is not a compilable HFST grammar (a complete, tested one follows below). A simplified excerpt of a Turkish noun lexicon:

```
LEXICON NounRoot
ev    NounInfl;    ! "house", continues to noun inflection
araba NounInfl;    ! "car"
göz   NounInfl;    ! "eye"

LEXICON NounInfl
<PL>:ler  NounCase;   ! plural suffix (front), continues to case
<PL>:lar  NounCase;   ! plural suffix (back)
          NounCase;   ! singular (empty suffix)

LEXICON NounCase
<LOC>:de  #;          ! locative (front voiced)
<LOC>:da  #;          ! locative (back voiced)
<LOC>:te  #;          ! locative (front voiceless)
<LOC>:ta  #;          ! locative (back voiceless)
<ABL>:den #;          ! ablative ...
...
```

The `#` symbol marks the end of a word. The continuation class mechanism encodes the suffix template: `NounRoot` continues to `NounInfl`, which continues to `NounCase`, which terminates. A suffix from `NounCase` cannot appear before one from `NounInfl` because the continuation classes do not allow it.

6.2.2 Phonological Rules (twolc)

The `twolc` component constrains pairs of lexical and surface symbols in context. For example, an abstract suffix vowel can surface as *a* or *e* according to harmony, and an abstract suffix consonant as *d* or *t* according to voicing.

The schematic lexicon above lists the allomorphs separately. The complete grammar below instead writes each suffix once with **archiphonemes**, symbols that stand for its alternating sounds. The two-level rules select the permitted surface forms.

6.2.3 A Complete Toy Grammar

The following grammar compiles with HFST and is small enough to read in full. It has four stems, the plural and two cases. The lexicon writes the harmonising vowel as `{A}` and the suffix-initial *d/t* as `{D}`; two rules choose their surface forms. The files are in `code/ch06-toy-fst/` of the book repository.

```
! tr-toy.lexc — upper side = analysis (lemma + tags), lower side = morphophonemic form
! {A} = harmonising vowel a/e;  {D} = suffix-initial d/t

Multichar_Symbols
%<n%> %<pl%> %<loc%> %<abl%> %{A%} %{D%}

LEXICON Root
Nouns ;

LEXICON Nouns
ev%<n%>:ev          Number ;   ! house
araba%<n%>:araba    Number ;   ! car
göz%<n%>:göz        Number ;   ! eye
kitap%<n%>:kitap    Number ;   ! book

LEXICON Number
%<pl%>:l%{A%}r      Case ;     ! plural -lAr
                    Case ;     ! singular: no suffix

LEXICON Case
                        # ;    ! nominative: no suffix
%<loc%>:%{D%}%{A%}      # ;    ! locative -DA
%<abl%>:%{D%}%{A%}n     # ;    ! ablative -DAn

! tr-toy.twolc — backness harmony of {A} and voicing of {D}

Alphabet
a b c ç d e f g ğ h ı i j k l m n o ö p r s ş t u ü v y z
%{A%}:a %{A%}:e %{D%}:d %{D%}:t ;

Sets
FrontV = e i ö ü ;
Cons = b c ç d f g ğ h j k l m n p r s ş t v y z ;
Voiceless = ç f h k p s ş t ;

Rules

"{A} is e after a front vowel, otherwise a"
%{A%}:e <=> :FrontV [ :Cons ]* _ ;
```

```
"{D} is t after a voiceless consonant, otherwise d"
%{D%}:t <=> :Voiceless _ ;
```

The harmony rule looks back over any number of consonants to the nearest vowel; because the surface vowel of a preceding {A} counts, *ev-l{A}r-{D}{A}* becomes *evlerde* in one pass. The Python bindings compile and test the grammar (`pip install hfst`; tested with HFST 3.16.0.1). Run this inline block from `code/ch06-toy-fst/`:

```
import hfst

lexicon = hfst.compile_lexc_file("tr-toy.lexc")              # analysis:morphophonemic
hfst.compile_twolc_file("tr-toy.twolc", "tr-toy.twol.hfst")
stream = hfst.HfstInputStream("tr-toy.twol.hfst")
rules = []
while not stream.is_eof():
    rules.append(stream.read())
stream.close()

generator = hfst.HfstTransducer(lexicon)
generator.compose_intersect(tuple(rules))                    # analysis:surface
generator.minimize()
analyser = hfst.HfstTransducer(generator)
analyser.invert()                                            # surface:analysis
analyser.minimize()
# Optimized-lookup format, the format of Apertium's .automorf/.autogen files
analyser.convert(hfst.ImplementationType.HFST_OLW_TYPE)
generator.convert(hfst.ImplementationType.HFST_OLW_TYPE)

def analyse(word):
    return sorted(a for a, _ in analyser.lookup(word))

def generate(analysis):
    return sorted(s for s, _ in generator.lookup(analysis))

accept = {"evler": "ev<n><pl>", "arabalar": "araba<n><pl>", "evlerde": "ev<n><pl><loc>",
          "arabalarda": "araba<n><pl><loc>", "gözden": "göz<n><abl>", "kitapta":
           ↪  "kitap<n><loc>",
          "kitaplardan": "kitap<n><pl><abl>"}
reject = ["evlar", "arabaler", "evlarda", "kitapda", "arabade"]
for word, gold in accept.items():
    assert analyse(word) == [gold], (word, analyse(word))      # the analysis, not just
     ↪  acceptance
for word in reject:
    assert analyse(word) == [], (word, analyse(word))
assert generate("göz<n><pl><loc>") == ["gözlerde"]
print("all tests passed")
```

The displayed checks cover representative cases. From any working directory, `python code/ch06-toy-fst/build_and_test.py` verifies all 24 cells of the four-stem paradigm in both directions. Accepted forms must receive the expected analysis; harmony and voicing errors must be rejected. The grammar leaves out the *kitap/kitab-* alternation before vowels. Exercise 2 adds it with an accusative suffix.

Apertium grammars extend this design with larger lexicons and language-specific rules. Throughput depends on transducer format and hardware, so measure it on your own text when speed matters. As checked on 19 October 2026, PyPI's `hfst` package had wheels

for Python 3.11 but none for Python 3.13. The pure-Python `pyhfst` can read compiled transducers; it cannot compile lexc or twolc.

6.2.4 The Analysis Operation

Given the compiled FST, analysis works by **composition**: input the surface string and read out all lexical paths that the transducer accepts. For *evlerimizden*:

1. The FST traverses the transitions that match *e-v-l-e-r-i-m-i-z-d-e-n*
2. At each step, valid continuation classes constrain which transitions are available
3. All accepting paths (paths that reach a final state after consuming all input characters) are returned as analyses

The output is a set of analysis strings from accepting paths; multiple paths can yield the same string. The Apertium Turkish transducer returns two (shown in Section 6.3):

```
ev<n><pl><px1pl><abl>
ev<n><pl><px1pl><abl>+i<cop><aor><p3><sg>
```

If the word is unrecognised (no accepting path), the FST returns an empty set. The word may be a loanword or a typo, or it may use a suffix or root missing from the lexicon.

6.2.5 The Generation Operation

Generation uses the inverse relation: swap the input and output labels, then look up the lexical analysis to obtain its surface form(s). This preserves string order; it does not reverse the transitions or the spelling. For `ev<n><pl><px1pl><abl>`:

1. The harmony rules select *-ler* (front plural, since *ev* has a front vowel) and *-imiz* (front unrounded, after *-ler*)
2. The voicing rule selects *-den*, because *-imiz* ends in the voiced consonant *z*
3. The transducer returns *evlerimizden*

A fully specified lexical analysis often has one standard surface realisation, although the relation can encode variants (spelling variants, or optional forms that the grammar allows), in which case generation returns several surface forms.

6.3 The Apertium Turkic Project

6.3.1 Overview

Apertium is an open-source rule-based machine translation platform that includes, as one of its core components, a collection of morphological analysers and generators implemented as FSTs (Forcada et al. 2011; Khanna et al. 2021). TurkicNLP loads Apertium transducers for **20 Turkic languages** (Hakimov 2026).

The analysers use the `lexc/twolc` formalism compiled with HFST. Each language module consists of:

- A language-specific lexicon file (*apertium-tur.tur.lexc*, *apertium-kaz.kaz.lexc*, etc.)
- A phonological rule file (*apertium-tur.tur.twolc*, etc.)
- Depending on the module, disambiguation rules and test suites

Bilingual dictionaries and transfer rules for machine translation live in separate *pair* modules (e.g., *apertium-kaz-tat*). Although every language has its own lexicon and rules, the Turkic modules follow common formalisms and tag conventions, and several were developed together (J. Washington, Salimzyanov, and Tyers 2014); J. N. Washington, Tyers, and Kuyrukçu (2020) describe multi-script transducers and transcribers for seven of them. This shared design is what makes it realistic to start a new language from the module of a close relative.

6.3.2 How Good Are the Analysers?

The 20 analysers differ widely in lexicon size and maturity. Because gold morphological annotation is missing for most Turkic languages, the TurkicNLP evaluation uses a parallel challenge set: the same 20 sentences translated into 20 languages, each targeting one phenomenon (five cases; past, progressive, future, negation, question, imperative, conditional, evidential, passive; plural, possessive, pronoun, reciprocal, numeral, converb). It compares the Apertium analysers with the toolkit's multilingual neural analyser (Hakimov 2026).

Table 6.1: Small diagnostic probe results (Det% and non-empty-feature rate) for Apertium and multilingual neural morphological analysis on the 20-sentence parallel challenge set (Hakimov 2026). Not a measure of analysis accuracy. Det%: sentences in which the target UD feature is detected; Feat%: content words receiving at least one feature (coverage, not feature accuracy). †Zero-shot for the neural model. Bold: higher value per metric. Published rounded averages are preserved.

Language	Apertium Det%	Apertium Feat%	Neural Det%	Neural Feat%
Turkish	**90**	87	80	**90**
Azerbaijani	50	42	**75**	**79**
Turkmen	**80**	51	75	**90**
Gagauz	**85**	77	75	**94**
Kazakh	**80**	78	70	**82**
Kyrgyz	**80**	81	60	**92**
Tatar	65	50	**75**	**89**
Bashkir	75	60	75	**89**
Crimean Tatar	**85**	75	75	**89**
Karakalpak†	**85**	60	50	**86**
Nogai†	**65**	61	55	**93**
Kumyk	70	74	70	**95**
Karachay-Balkar†	**80**	82	70	**97**
Uzbek	**85**	83	55	**87**
Uyghur	**80**	**85**	55	60

Language	Apertium Det%	Apertium Feat%	Neural Det%	Neural Feat%
Sakha	**70**	68	60	**85**
Altai	40	33	**75**	**89**
Tuvan	70	73	**75**	**89**
Khakas	55	23	**60**	**89**
Chuvash	**80**	63	60	**92**
Average	**74**	65	67	**88**

Each metric answers a narrow question. Det% records whether the target feature appears anywhere in a sentence. Feat% records any non-empty feature bundle, regardless of correctness. Assess full-analysis accuracy on representative running text separately, along with coverage, recall of the gold reading among candidates and contextual disambiguation.

On this diagnostic, mean target-feature detection is 74% for the transducers and 67% for the neural analyser; the corresponding non-empty-feature rates are 65% and 88%. The ordering varies by language (Table 6.1). Analyser maturity, correct disambiguation and the reasons for a cross-language gap all lie beyond what these measures can show. A full evaluation of an analyser would annotate tokens in held-out natural sentences per language, retain legitimate alternative readings, and report unknown-word rate, gold-candidate recall and selected-reading accuracy.

The Kazakh and Kyrgyz transducers were developed by the community that also produced the Kazakh UD treebank and its UD feature mapping (J. Washington, Salimzyanov, and Tyers 2014; Tyers and Washington 2015; J. Washington, Ipasov, and Tyers 2012); TurkicNLP converts module tags into UD-shaped feature strings. This does not guarantee agreement with every treebank's annotation conventions; audit the mapping before scoring.

6.3.3 Using Apertium FSTs via HFST Python Bindings

The HFST toolkit (Lindén et al. 2011) provides Python bindings that allow Apertium FST binaries to be loaded and queried without a system-level Apertium installation (`pip install turkicnlp[hfst]`). The TurkicNLP toolkit wraps these bindings with a unified API, downloads the GPL-licensed transducers separately at run time, converts input to the script each transducer expects, and scores competing readings with transducer weights and heuristics (Hakimov 2026).

6.3.3.1 Direct HFST Access

For users who want direct control over the FST:

```
import hfst
from turkicnlp.resources.registry import ModelRegistry

# Folder of the Apertium Turkish transducers fetched by turkicnlp.download("tur")
tur_dir = ModelRegistry.get_model_path("tur", "morph", "apertium", script="Latn")
```

```
def load(path):
    stream = hfst.HfstInputStream(str(path))
    fst = stream.read()
    stream.close()
    return fst

analyser = load(tur_dir / "tur.automorf.hfst")
print(hfst.__version__ if hasattr(hfst, "__version__") else "", tur_dir)

for word in ["evlerimizden", "yüzü", "xyzqw"]:
    for analysis, weight in analyser.lookup(word):     # (output string, weight) pairs
        print(f"{word}\t{analysis}\t{weight}")

# evlerimizden  ev<n><pl><px1pl><abl>                       0.0
# evlerimizden  ev<n><pl><px1pl><abl>+i<cop><aor><p3><sg>  0.0
# yüzü          yüz<n><acc>                                 0.0
# yüzü          yüz<n><px3sp><nom>                          0.0
# yüzü          yüz<num><subst><acc>                        0.0
# ...           (six readings in total)
# xyzqw         (no output: unknown word)
```

`lookup()` returns each analysis with its weight; the input is already known. *evlerimizden* has two readings: the noun ‘from our houses’, and the same noun followed by the copula, ‘it is from our houses’. *yüzü* has six: ‘face’ or the numeral ‘hundred’, each as an accusative, a third-person possessive, or a possessive plus copula. An unknown word returns nothing. Choosing between readings is the disambiguation problem of Chapter 7. The output above was produced with HFST 3.16.0.1 and the Apertium Turkish transducer distributed with TurkicNLP 0.5.0.

6.3.3.2 Via TurkicNLP

The TurkicNLP pipeline routes morphological analysis through the Apertium backend automatically, converting the Apertium-native tag format to Universal Dependencies features:

```
from turkicnlp import Pipeline

# Kazakh morphological analysis with UD feature output
pipeline = Pipeline("kaz", processors=["tokenize", "morph"], tokenize_backend="rule",
 ↪  morph_backend="apertium")   # "morph" = Apertium FST

doc = pipeline("Ол кітабымды оқып жатыр.")

for word in doc.words:
    print(f"{word.text:<12} {word.lemma or '_':<8} {word.upos or '_':<6} {word.feats or
     ↪  '_'}")
```

Output (TurkicNLP 0.5.0):

```
Ол           ол       PRON   Case=Nom|PronType=Dem
кітабымды    кітап    NOUN   Case=Acc|Number[psor]=Sing|Person[psor]=1
оқып         оқы      VERB   VerbForm=Conv
жатыр        жат      AUX    Number=Plur|Person=3|Tense=Pres
.            .        PUNCT  _
```

The analyser correctly recognises *оқып жатыр* ‘is reading’ as a converb followed by auxiliary *жат-* ‘lie’. Its selected `Number=Plur` is wrong for the subject *Ол* ‘he/she’. Kazakh third-person forms do not distinguish singular from plural here, so the transducer returns both `жат<vaux><pres><p3><sg>` and `жат<vaux><pres><p3><pl>`. The toolkit chooses the plural reading. Compare this decision with Stanza or Glot500 (Chapter 7) and check the feature mapping against the intended treebank.

6.3.4 Extending Coverage to a New Language

The common formalism and tag conventions make it feasible to extend Apertium Turkic coverage to new languages. The process, documented in the Apertium developer documentation and described practically for Kypchak languages by J. Washington, Salimzyanov, and Tyers (2014), involves:

1. **Create a new language directory** following the Apertium naming convention.
2. **Write a new lexicon file** with the target language’s root morphemes. For Kipchak languages, many roots are shared with Kazakh or Kyrgyz, reducing the lexicographic work.
3. **Model the suffix inventory on a close relative’s lexicon** (continuation classes and tags), then replace the allomorphs with the target language’s own.
4. **Write or adapt the phonological rules** for the target language’s harmony system.
5. **Compile and test** against known forms. The test suite (a list of surface forms with expected analyses) is the primary quality assurance mechanism.

The work includes lexicon building, grammatical description, implementation and speaker validation. Which part takes most time depends on the available resources.

6.4 Language-Specific Tools

Beyond Apertium, several language-specific morphological tools exist for the best-resourced Turkic languages.

6.4.1 Zemberek-NLP (Turkish)

Zemberek-NLP (Akın and Akın 2007) is a Java library for Turkish NLP; the 2007 paper describes the earlier Zemberek framework from which it developed. Its repository states that the project “is now in slow maintenance mode” (checked 19 October 2026). It provides:

- **Morphological analysis**: a Java-native implementation of Turkish morphology that implements Turkish suffix rules and includes a contextual disambiguator
- **Morphological generation**: generates forms from a dictionary item and requested morphological tags
- **Spell-checking**: based on morphological analysis rather than dictionary lookup

- **Tokenisation**: handles Turkish-specific patterns including acronyms, numbers, and clitic punctuation

Zemberek operates independently of Apertium and uses its own internal rule representation rather than `lexc/twolc`. It is written in Java; a separate project provides a Python implementation.

Zemberek integrates **morphological disambiguation**, selecting an analysis in sentence context. Evaluate candidate generation and contextual selection separately; having a disambiguator says nothing yet about its accuracy on a new domain.

6.4.2 TRmorph (Turkish)

TRmorph (Çöltekin 2010) is an open-source finite-state morphological analyser for Turkish. The 2010 paper describes an implementation in SFST; the current repository is a rewrite in lexc/xfst developed with foma, with a new lexicon, a revised tag set, a stemmer, a segmenter and an unknown-word guesser (README, checked 19 October 2026). Results reported for one version do not carry over to the other.

6.4.3 Google's Turkish Morphological Analyser

Ozturel, Kayadelen, and Demirsahin (2019) describe a "syntactically expressive" finite-state analyser for Turkish developed at Google and released as open source (`google-research/turkish-morphology`). Its analyses are designed to be compatible with Universal Dependencies-style syntactic annotation, which makes it a useful companion to UD parsers.

6.4.4 Analysers for Other Turkic Languages

Kazakh has published work on morphological disambiguation, including a voted-perceptron approach (Tolegen, Toleu, and Mussabayev 2020). Check tool availability against a dated resource inventory; a language's size is a poor guide. Apertium and TurkicNLP provide useful starting points, but this overview is not an exhaustive list of analysers.

6.4.5 Comparison of Available Turkish Analysers

Table 6.2: Selected Turkish morphological tools; repository code licences checked 14 October 2026 and again 19 October 2026. Model and data licences require separate inspection. Google's repository is archived.

Analyser	Language	Interface	Disambiguation	Licence
Apertium + HFST	Turkish	Python (via TurkicNLP)	Weights + heuristics in TurkicNLP	GPL-3.0
TRmorph	Turkish	foma / command line	None built-in	MIT (current rewrite)
Google Turkish morphology	Turkish	Python	None built-in	Apache 2.0 (original code)
Zemberek-NLP	Turkish	Java (+ Python port)	Statistical	Apache 2.0
TurkishDelightNLP	Turkish	Python / web	Neural	Component-specific; see repositories linked in Alecakir, Bölücü, and Can (2022)
VNLP	Turkish	Python	Neural	AGPL-3.0

6.5 Morphological Generation and Data Augmentation

6.5.1 Generation Mode

The FST analysers described above are bidirectional: Apertium builds a generator transducer from the same sources as the analyser, mapping an analysis string (lemma plus Apertium tags) to surface forms. TurkicNLP uses these generators internally (Hakimov 2026); the example below calls the Turkish generator directly through HFST, reusing `load()` and `tur_dir` from Section 6.3, which makes the mechanism visible.

```
gen = load(tur_dir / "tur.autogen.hfst")          # Apertium Turkish generator

def generate(analysis: str) -> list[str]:
    return sorted({form for form, _ in gen.lookup(analysis)})

for case in ["nom", "acc", "dat", "loc", "abl", "gen"]:
    print(case, generate(f"ev<n><{case}>"))
```

Output:

```
nom ['ev']
acc ['evi']
dat ['eve']
loc ['evde']
abl ['evden']
gen ['evin']
```

TurkicNLP exposes the same generator through its Apertium processor, which also converts the script where the transducer expects a different one:

```
from turkicnlp.processors.morphology import ApertiumMorphProcessor

kaz_dir = ModelRegistry.get_model_path("kaz", "morph", "apertium", script="Cyrl")
kaz = ApertiumMorphProcessor(lang="kaz")
kaz.load(kaz_dir)
print([kaz.generate("мектеп", ["n", case]) for case in ["nom", "acc", "dat", "loc", "abl",
 ↪  "gen"]])
# ['мектеп', 'мектепті', 'мектепке', 'мектепте', 'мектептен', 'мектептің']
```

6.5.2 Data Augmentation via Morphological Generation

One practically useful application of generation is **morphological data augmentation**. For low-resource Turkic languages where labelled training data is scarce, generating inflected forms of known lemmas expands the variety of surface forms a model sees.

Changing an entity's case can change its role or make a sentence ungrammatical. *Мен Алматыдан келдім* means 'I came from Almaty'; replacing *Алматыдан* with *Алматыға* gives 'I came to Almaty', a grammatical sentence with a different meaning. For entity substitution, keep the original case and number and generate a different entity of the same type in that form. The resulting sentence still needs contextual checking:

```
gen_kaz = load(kaz_dir / "kaz.autogen.hfst")      # Apertium Kazakh generator
ana_kaz = load(kaz_dir / "kaz.automorf.hfst")

def substitute_entity(tokens, labels, idx, new_lemma, allowed_lemmas, ana=None, gen=None):
    """Conservative single-token B-LOC replacement; None means abstain."""
    ana, gen = ana or ana_kaz, gen or gen_kaz       # Kazakh transducers by default
    if len(tokens) != len(labels) or not 0 <= idx < len(tokens):
        raise ValueError("unaligned tokens/labels or invalid index")
    if labels[idx] != "B-LOC" or (idx + 1 < len(labels) and labels[idx + 1] == "I-LOC"):
        return None                                 # no partial replacement of an entity
    if new_lemma not in allowed_lemmas or any(c.isspace() for c in new_lemma):
        return None                                 # require an independently checked
         ↪  gazetteer
    readings = {a for a, _ in ana.lookup(tokens[idx]) if "<np><top>" in a and "+" not in a}
    if len(readings) != 1:
        return None
    _, tags = readings.pop().split("<", 1)
    requested = f"{new_lemma}<{tags}"
    forms = {f for f, _ in gen.lookup(requested)}
    if len(forms) != 1:
        return None
    form = forms.pop()
    if not form or any(c.isspace() for c in form):
        return None
    if requested not in {a for a, _ in ana.lookup(form)}:
        return None                                 # generator/analyser round-trip check
    new_tokens = list(tokens); new_tokens[idx] = form
    return new_tokens, list(labels)

tokens = ["Мен", "Алматыдан", "келдім", "."]
labels = ["O", "B-LOC", "O", "O"]
```

```
# Demonstration candidates; a production gazetteer needs independent validation.
cities = {"Астана", "Шымкент", "Тараз"}
for city in ["Астана", "Шымкент", "Тараз", "Париж"]:
    print(substitute_entity(tokens, labels, 1, city, cities))
# Inspect the accepted forms; Париж is rejected because it is not in cities.
```

The same function works for Turkmen with the Apertium Turkmen transducers (beta quality):

```
tuk_dir = ModelRegistry.get_model_path("tuk", "morph", "apertium", script="Latn")
gen_tuk = load(tuk_dir / "tuk.autogen.hfst")
ana_tuk = load(tuk_dir / "tuk.automorf.hfst")

tokens = ["Men", "Aşgabatdan", "geldim", "."]     # 'I came from Ashgabat'
labels = ["O", "B-LOC", "O", "O"]
cities = {"Mary", "Daşoguz", "Türkmenabat"}
for city in ["Mary", "Daşoguz", "Türkmenabat", "Parij"]:
    print(substitute_entity(tokens, labels, 1, city, cities, ana=ana_tuk, gen=gen_tuk))
```

Output (18 October 2026):

```
(['Men', 'Marydan', 'geldim', '.'], ['O', 'B-LOC', 'O', 'O'])
(['Men', 'Daşoguzdan', 'geldim', '.'], ['O', 'B-LOC', 'O', 'O'])
(['Men', 'Türkmenabatdan', 'geldim', '.'], ['O', 'B-LOC', 'O', 'O'])
None
```

Parij ‘Paris’ is rejected because it is not in the vetted list (the transducer does not know it either). The Turkmen transducer is incomplete in other places: asked for the past participle of *oka-* ‘read’, it produces *okaen* where the standard form is *okan*, which is exactly why generated forms need checking.

A well-formed request can still yield a questionable word. Re-analysis tests consistency with the generator’s own grammar. Independent linguistic checks are needed to judge that grammar.

Keep the original analyser’s tag sequence, reject ambiguity and require a vetted replacement lexicon. This example handles pretokenised, single-token location entities only. If converting the result back to raw text, rebuild character offsets and verify tokenisation before attaching spans; equal token counts do not preserve character offsets. Split source documents into train, development and test sets before augmenting, and augment training data only. Report manual grammaticality checks and performance on unchanged held-out text. Enumerating a lemma’s cases demonstrates the paradigm. It does not produce training sentences that can stand in for one another.

6.5.3 Back-Off Strategies

FST analysers fail to recognise words that are not in their lexicon: loanwords, proper nouns not in the gazetteer, neologisms, and misspellings. Three back-off strategies are used in practice:

Heuristic morphological guessing. When the analyser returns no analysis, one can strip final characters and re-analyse the remainder, guessing that the stripped characters

are suffixes. Such heuristics can produce wrong guesses and need evaluation on the target language; mark their output as low-confidence and allow the system to abstain. Guesser transducers, below, are a more principled version of the same idea.

Morphological guesser FSTs. A guesser transducer matches unknown words against suffix patterns without a lexicon entry and returns the identified suffixes with an unknown stem. TRmorph ships such a guesser (Çöltekin 2010); for Apertium languages, check whether the module provides one.

Neural fall-back. For words the FST cannot analyse, a neural analyser can be used as a fall-back. TurkicNLP's morpheme segmenter takes this hybrid approach: the Apertium reading that accounts for the whole word is preferred, and the neural analyser breaks ties and handles unknown words. On a constructed 483-word set covering 20 Turkic languages and 13 morphological categories, the hybrid reaches 89% exact segmentation, versus 79% for Apertium alone and 68% for the neural analyser alone (Hakimov 2026, Appendix E). These are diagnostic results: the gold segmentations are rated high, medium or low confidence by language, and the words are constructed test items, not an independent natural-text benchmark.

6.5.4 Generation for Machine Translation

Translation into Turkish requires both a suitable lemma and its inflected form. Case, number and the appropriate accusative allomorph all depend on the sentence. Morphological generation can supply the surface form once those choices have been made.

This morphological realisation step can be decomposed into two sub-tasks: (1) determine the required feature bundle from the source-side and target-side context; (2) generate the correct surface form from the lemma and features using the FST generator. This decomposition is used in the Apertium rule-based MT system (Forcada et al. 2011). For neural MT, Tamchyna, Weller-Di Marco, and Fraser (2017) train a system to output lemmas and morphological tags and generate the inflected forms in a separate deterministic step, for Czech and German targets.

6.6 Limitations of the Finite-State Approach

The finite-state approach is powerful but has known limitations that motivate the neural methods in Chapter 7.

Lexicon coverage. A lexicon-based FST without guessing generally requires a licensed stem; productive compound rules and unknown-word guessers can extend that coverage. Loanwords, proper nouns, neologisms, and highly technical vocabulary are frequently unrecognised, and lexicon size varies greatly across the Turkic modules. A low feature-fill rate in Table 6.1 could also reflect tag mapping or annotation choices, so diagnose failures before attributing them to lexicon size.

Lexicon maintenance. Keeping the lexicon current requires ongoing manual effort, since new loanwords and names keep entering the written language. A static FST lexicon becomes increasingly incomplete over time without active maintenance.

Disambiguation is separate. The FST returns the analyses licensed by its implemented grammar for an ambiguous form; it does not choose among them. Disambiguation requires additional components (constraint-grammar rules, statistical taggers or neural rerankers) and, for the learned ones, training data. TurkicNLP falls back on transducer weights and heuristic scores (syntactic position, part-of-speech priority, question particles) where no trained disambiguator exists (Hakimov 2026).

Limited contextual information. The morphological grammars described here do not by themselves model sentence-level semantic plausibility. A statistical or neural disambiguator can use context to choose among their analyses.

Cross-lingual reuse is partial. While the suffix template is shared, each language still requires a new lexicon file and careful adjustment of phonological rules. For a completely new, undocumented language, building an FST requires substantial linguistic fieldwork.

Despite these limitations, finite-state morphology remains a foundation of Turkic morphological processing, particularly for languages with no annotated data. The two approaches can be combined, as in the TurkicNLP segmenter described above, where the transducer and the neural analyser cover different failures.

6.7 Hands-On: A Procedure for Comparing Rule-Based and Neural Analysis

Question. How do rule-based and neural analyses differ on the same input?

Inputs. The three sentences below, both morphological backends and the Turkish generator.

Compute. CPU inference is possible; the neural checkpoint requires a separate model download and more memory than the FST.

Output. Side-by-side analyses and generated Turkish forms.

Interpretation. Check disagreements against a linguistic analysis; use generated paradigms to inspect alternations.

This example analyses the same sentence in Turkish, Kazakh, and Kyrgyz with both of TurkicNLP's morphological backends, the Apertium transducers (`morph`) and the multilingual neural analyser (`morph_neural`). It then generates Turkish forms with the Apertium generator.

```
# pip install "turkicnlp[hfst,transformers]==0.5.0"   # Apertium and neural backends
import turkicnlp

sentences = {
    "tur": "Kitaplarımızdan birini okuyamadı.",
    "kaz": "Кітаптарымыздың бірін оқи алмады.",
    "kir": "Китептеribизден бирин окуй алган жок.",
}

for lang, sentence in sentences.items():
    turkicnlp.download(lang, processors=["tokenize", "morph", "morph_neural"])
```

```
    for proc in ["morph", "morph_neural"]:
        nlp = turkicnlp.Pipeline(lang, processors=["tokenize", proc],
↪   tokenize_backend="rule")
        doc = nlp(sentence)
        print(f"\n--- {lang} / {proc} ---")
        for w in doc.words:
            print(f"  {w.text:<28} {w.lemma or '_':<14} {w.upos or '_':<6} {w.feats or
                ↪  '_'}")
```

Compare the two backends word by word. Where do they agree? Which words does the transducer leave unanalysed, and which features does the neural analyser assign that are wrong? Repeat with a language that has only a prototype transducer (e.g., `alt` or `kjh`) and relate what you see to Table 6.1.

Then generate with the Turkish generator, using `generate()` from Section 6.5:

```
for tags in ["<n><nom>", "<n><acc>", "<n><px1sg><nom>", "<n><px1sg><abl>"]:
    print(tags, generate("kitap" + tags))
# <n><nom> ['kitap']
# <n><acc> ['kitabı']
# <n><px1sg><nom> ['kitabım']
# <n><px1sg><abl> ['kitabımdan']
```

Note the consonant alternation: *kitap* (bare) vs. *kitab-* before the vowel-initial suffixes shown here. (Tag names follow Apertium conventions; inspect the analyser's output for a word to see the exact tag sequence the generator expects.)

Status: procedure only. The full three-language comparison is not reported here. Exercises 4 and 5 record related generation and backend comparisons.

What to Take Forward

A finite-state analyser combines a lexicon, allowed morph sequences and spelling rules. Inverting the relation supports generation. On the TurkicNLP diagnostic, transducers detect the target feature more often than the neural analyser (74% against 67%), while supplying non-empty feature bundles less often (65% against 88%). Coverage and correctness need separate measures: use lookup failures to find gaps, annotated examples to assess readings, and regression tests after rule changes. Chapter 7 turns to neural tokenisation and tagging.

Further Reading

- **Two-level morphology (foundational)**: Koskenniemi (1983) is the original thesis introducing the formalism; required reading for anyone implementing a morphological analyser from scratch.

- **Turkish two-level morphology**: Oflazer (1994) is the landmark application of Koskenniemi's formalism to Turkish, an early computational treatment of Turkish morphology.
- **HFST toolkit**: Lindén et al. (2011) describes the Helsinki Finite-State Toolkit that compiles and runs the `lexc/twolc` formalism used in Apertium.
- **Apertium platform**: Forcada et al. (2011) introduces Apertium as an open-source rule-based MT platform, with morphological processing as a core component; Khanna et al. (2021) provides a later platform update.
- **Apertium Turkic coverage**: J. Washington, Salimzyanov, and Tyers (2014) describe the finite-state transducers for three Kipchak languages, one from each Kipchak sub-branch (Kazakh, Tatar, Kumyk); J. Washington, Ipasov, and Tyers (2012) describe the Kyrgyz transducer.
- **Kazakh FST and UD**: the Kazakh transducer is described with its Tatar and Kumyk siblings in J. Washington, Salimzyanov, and Tyers (2014); Tyers and Washington (2015) relate it to the Kazakh Universal Dependencies treebank.
- **Other Turkish analysers**: Çöltekin (2010) (TRmorph) and Ozturel, Kayadelen, and Demirsahin (2019) (Google's open-source analyser).
- **Rule-based vs. neural analysis across 20 Turkic languages**: Hakimov (2026).
- **Zemberek-NLP**: Akın and Akın (2007) introduces the Turkish-specific library; practical documentation is available at `github.com/ahmetaa/zemberek-nlp`.
- **Turkish NLP**: Oflazer and Saraçlar (2018, chaps. 2–3) covers Turkish morphological processing and disambiguation.

Exercises

1. **FST architecture.** Draw a state diagram for a transducer that analyses Turkish nouns with singular/plural number and nominative/dative case: four cells per root (*ev*, *ev-ler*, *ev-e*, *ev-ler-e*), one of which is the bare root. Fix your assumptions first (alphabet, whether tags are separate symbols, one stem or a lexicon of stems). Under those assumptions, how many states do you need, and which transitions? How does the diagram change if the suffixes contain the archiphoneme `{A}` of the toy grammar?

2. **Extending the toy grammar.** Add the accusative *-(y)I* to `code/ch06-toy-fst/`: a new archiphoneme for the four-way vowel, a buffer *y* after vowel-final stems (*arabayı*), and the *kitap/kitab-* alternation before vowels (*kitabı*). Add accepted and rejected test forms for each change, and run the tests.

3. **Coverage comparison.** Run the TurkicNLP Apertium analyser on 20 words of your choice in Turkish and in Altai. Count how many words receive at least one analysis vs. how many return empty (with HFST, `analyser.lookup(word)` returns an empty tuple; in a `morph` pipeline, unknown words get `upos == "X"`). Compare the results. What types of words does the Altai analyser fail on most often? Does the neural analyser (`morph_neural`) fill these gaps correctly?

4. **Generation.** With the Apertium Turkish generator, generate the six case forms of *kitap* in singular and plural, and explain the final-consonant alternation. Then generate the case forms of the place name *Ankara* (tag `<np><top>`): how does vowel-final attachment work, and does the generator write the apostrophe that Turkish

spelling requires after proper names (*Ankara'da*)? Finally, use `substitute_entity()` (adapted to Turkish) on three sentences, and check every generated sentence by hand.

5. **Comparing rule-based and neural analysis.** Use both the TurkicNLP Apertium analyser and a neural morphological tagger (e.g., Stanza for Turkish via `Pipeline("tur", processors=["tokenize","pos","lemma"])`) to analyse the sentence *Gidebilseydik ne güzel olurdu*. Compare the analyses: where do they agree? Where do they differ? What does this tell you about the respective strengths and failure modes of each approach?

7 Neural Morphology and the Tokenisation Problem

Learning Objectives

After reading this chapter, you will be able to:

- Interpret tokeniser fertility under a stated sampling convention and explain its implications for model input length
- Compare BPE, WordPiece, and Unigram tokenisation algorithms and reason about their suitability for Turkic text
- Describe morpheme-aware tokenisation and design a comparison with a subword baseline
- Contrast character-level, classifier-based, and hybrid FST-constrained neural morphological analysers
- Explain the morphological disambiguation task and compare HMM, CRF, and transformer-based solutions
- Design an evaluation of zero-shot morphological transfer across the Turkic family, including native-script and transliterated conditions

Neural morphology involves two related tasks: representing word forms and choosing their analyses in context. This chapter first examines subword tokenisation, then neural tagging, disambiguation and transfer to languages without task-labelled training data.

7.1 Subword Tokenisation in Agglutinative Languages

7.1.1 Three Algorithms

Many neural NLP models operate on **subword tokens** learned from training text. An individual token may represent a whole word, part of a word, a character or bytes; character- and byte-level models also exist. Three common subword algorithms are described here.

Byte-Pair Encoding (BPE) (Sennrich, Haddow, and Birch 2016b) begins with a character-level vocabulary and iteratively merges the most frequent adjacent pair of symbols. After a fixed number of merge operations, the resulting vocabulary covers frequent word forms as single tokens and decomposes rare forms into shorter pieces. BPE and its byte-level variants are used by many generative LLMs, among them the GPT model series and LLaMA 3.

WordPiece learns a vocabulary of reusable pieces. At inference, BERT tokenisers apply greedy longest-match segmentation within pretokenised words, using continuation markers such as ##; this differs from applying a ranked BPE merge list (BERT implementation). WordPiece is the tokeniser underlying BERT and its derivatives (Devlin et al. 2019), including BERTurk (Schweter 2020).

Unigram reverses the direction: it starts with a large initial vocabulary and iteratively *removes* the tokens whose removal hurts corpus likelihood least. **SentencePiece** (Kudo and Richardson 2018) is the library that popularised it; it implements both Unigram and BPE and operates on raw text without pre-tokenisation. mT5 and XLM-R (Conneau et al. 2020) use SentencePiece Unigram models, as does Glot500, which extends the XLM-R vocabulary; the SentencePiece model distributed with NLLB-200 is a BPE model (its `model_type`, checked 19 October 2026). LLaMA 1 and 2 used SentencePiece BPE, while LLaMA 3 switched to a byte-level BPE tokeniser of the tiktoken family.

All three algorithms can split rare, long suffix-chain forms into many pieces. Training data, sampling and vocabulary size determine the learned inventory. An English-heavy inventory reflects those choices, rather than an inherent property of the algorithm; monolingual Turkish tokenisers illustrate the difference. **Fertility** measures the extent of this splitting.

7.1.2 The Fertility Metric

Fertility is the mean number of subword tokens per input word. It equals 1.0 when a tokeniser assigns one token to each word, and the average word length for a character-level tokeniser. Two other measures help describe tokenisation: the share of words split into several tokens, and the 95th percentile of tokens per word. The latter captures the upper tail without relying on a single maximum. Here, words are Unicode alphanumeric runs matched by `[^\W_]+`, encoded separately without special tokens. This follows the protocol in Chapter 3; it differs from counting whitespace-separated words.

Table 7.1 and Figure 7.1 show measured fertility for four widely used LLM tokenisers on the same 20 parallel sentences in 20 Turkic languages and English (Hakimov 2026). All four tokenisers segment English at 1.08–1.09 tokens per word. Turkish needs between 1.95 (Qwen3.5 and `o200k_base`) and 2.33 (OLMo 3). For other languages the cost is considerably higher, although these languages share Turkish's productive suffixation: 3.38 (`o200k_base`) to 6.85 (OLMo 3) for Uyghur, 3.48 to 4.78 for Sakha, and 2.54 to 5.00 for Bashkir. The 95th percentile reaches 11 tokens per word for Uyghur and Tatar under OLMo 3, compared with 2 for English under every tokeniser. Averaged over the 20 Turkic languages, `o200k_base` is the most efficient tokeniser (2.51), followed by Qwen3.5 (2.75), LLaMA 3 (3.16), and OLMo 3 (3.58).

Table 7.1: Tokeniser fertility (mean tokens per word) of four LLM tokenisers on the 20 parallel challenge sentences of Hakimov (2026), each word tokenised separately. Small samples (61–88 words per language): read the table as a diagnostic, not a benchmark.

Language	Branch	Script	Qwen3.5	LLaMA 3	OLMo 3	o200k_base
English (reference)	—	Latn	1.09	1.09	1.09	1.08
Turkish	Oghuz	Latn	1.95	2.03	2.33	1.95
Azerbaijani	Oghuz	Latn	2.56	3.05	3.25	2.42
Turkmen	Oghuz	Latn	2.21	2.35	2.50	2.16
Gagauz	Oghuz	Latn	2.03	2.12	2.34	2.03
Kazakh	Kipchak	Cyrl	2.86	3.51	4.05	2.37
Kyrgyz	Kipchak	Cyrl	2.84	3.48	3.80	2.52
Tatar	Kipchak	Cyrl	3.13	4.13	4.46	2.51
Bashkir	Kipchak	Cyrl	3.35	4.68	5.00	2.54
Crimean Tatar	Kipchak	Latn	1.98	2.03	2.30	1.98
Karakalpak	Kipchak	Latn	2.08	2.21	2.38	2.08
Nogai	Kipchak	Cyrl	2.92	3.08	3.48	2.65
Kumyk	Kipchak	Cyrl	2.76	3.06	3.61	2.56
Karachay-Balkar	Kipchak	Cyrl	2.25	2.52	2.94	2.23
Uzbek	Karluk	Latn	2.12	2.18	2.18	2.11
Uyghur	Karluk	Arab	4.51	5.62	6.85	3.38
Sakha	Siberian	Cyrl	3.65	4.21	4.78	3.48
Altai	Siberian	Cyrl	2.83	3.08	3.63	2.63
Tuvan	Siberian	Cyrl	2.67	3.09	3.59	2.44
Khakas	Siberian	Cyrl	2.79	3.16	3.94	2.67
Chuvash	Oghur	Cyrl	3.52	3.66	4.09	3.42

The source files called the OpenAI column “GPT-5.2”, but the script measured the `o200k_base` encoding. The table uses the encoding name; it does not test a live model’s tokeniser (Section 3.6).

Script conversion changes the measured cost. Latin-script Crimean Tatar and Gagauz have fertility close to Turkish on this sample. Converting the Cyrillic- and Perso-Arabic-script samples to CTS substantially reduces the average gap (Section 3.6). This comparison concerns 20 short sentences per language and the tested mappings. Script and tokeniser training exposure remain confounded, and downstream gains are untested. Tokenising words in isolation also gives different counts from tokenising running text.

7.1.3 Token Cost and Morphological Boundaries

Token counts affect context capacity and computation. Boundary alignment raises a separate question about how explicitly the input represents morphology.

1. Context window depletion. Transformer models operate within a fixed context length measured in tokens, not words. Under OLMo 3, the least efficient of the four tokenisers, a 2,048-token window holds about 1,880 English words (1.09 tokens per word), about 880 Turkish words (2.33) and only about 300 Uyghur words (6.85) (Table 12.2 in Chapter 12 gives more values). These divisions are illustrative capacity estimates:

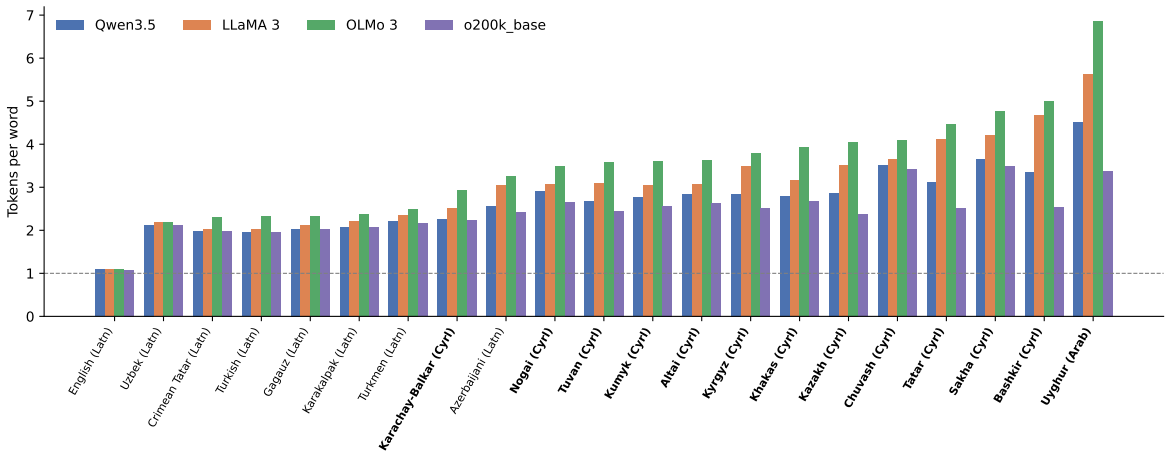


Figure 7.1: Measured fertility of four LLM tokenisers on the parallel challenge sentences, sorted by worst-case fertility; non-Latin-script languages in bold. Data: Table 7.1.

running-text tokenisation, punctuation and special tokens change the actual number of words that fit.

2. Split morphemes. Frequency-based subwords are not designed to follow morpheme boundaries. A form such as Turkish *geldiklerinden* (“because they came”, *gel-dik-ler-i-n-den*) might be segmented as `gel ##dik ##ler ##inden`, which cuts correctly after the plural but lumps the possessive, the pronominal *n* and the ablative into one piece; other tokenisers cut through the middle of a suffix. The model must then reconstruct morphological structure from fragments that do not align with it. (The segmentation shown is illustrative; Exercise 2 measures real boundary agreement.)

The recurring sentences give a measured case. XLM-R splits Kazakh *бейбітшілікті* ‘peace-ACC’ into `▁бей біт шілік ті`, with the accusative as its own piece, while Llama 3 splits it into eight byte-level pieces of one or two letters. For Uyghur ھارۇيسىنى ‘its cart (ACC)’, Llama 3 produces eight pieces, two of which are partial bytes of the letter ۇ. Converting to CTS changes the counts in opposite directions for the two tokenisers (Table 7.2).

Table 7.2: Tokens for the recurring sentences (Table 1.4) in the native script and in CTS, including punctuation and without special tokens; words are whitespace-separated.

Sentence	Words	Llama 3, native	Llama 3, CTS	XLM-R, native	XLM-R, CTS
Kazakh	5	32	20	11	19
Kyrgyz	3	7	8	5	7
Uzbek	7	32	24	21	20
Uyghur	7	40	17	13	14

Llama 3 needs fewer tokens after conversion for three of the four sentences. XLM-R, whose vocabulary was learned from native-script text, needs more for Kazakh and Kyrgyz. Four sentences show the mechanism; Table 7.1 measures it on a larger sample.

3. Inconsistent units across related forms. Frequency-based merges can segment the same suffix differently after different stems. Allomorphs such as *-lar* and *-ler* may also receive different token sequences. The model has to learn the grammatical connection between these sequences.

4. Inference latency and cost. More input tokens increase prefill work and consume context capacity; ordinary autoregressive generation then takes one decoding step per *output* token. Input tokens are not each separate decoding steps. At a fixed per-token price, twice as many billed tokens cost twice as much, but latency also depends on batching, caching, hardware and sequence length. Isolated-word fertility alone does not measure runtime or running-text billing.

7.2 Morpheme-Aware Tokenisation

7.2.1 Morphological Pre-Segmentation

One way to preserve morph boundaries is to segment words *before* applying the standard tokeniser. An FST (Chapter 6), neural segmenter or hybrid supplies the morphs: for example, *evlerimizden* → `ev ler imiz den`. BPE or Unigram then learns a vocabulary over these units, with a constraint that prevents tokens from crossing the supplied boundaries.

Pre-segmentation protects the supplied morph boundaries; it does not necessarily reduce token counts. With a fixed vocabulary, forbidding pieces that cross a boundary cannot lower the minimum possible count. Separately trained tokenisers learn different vocabularies, so their fertility must be measured.

A morph may still need several tokens, and a wrong boundary remains an error. Turkish studies report competitive results for morphology-level tokenisation (Toraman et al. 2023), but whether large generative models benefit in the same way is still open.

7.2.2 Vocabulary Size as a Lever

Increasing vocabulary size can reduce fertility by storing more word forms as single tokens. BERTurk (Schweter 2020) was released with 32K and 128K vocabularies. At a fixed embedding dimension, more entries require more embedding parameters. Compare token counts and downstream scores rather than assuming that a larger vocabulary is better. Exercise 1 measures tokenisation for the two BERTurk vocabularies.

7.2.3 Designing and Evaluating a Turkic Tokeniser

Designing a tokeniser for a new Turkic model involves three choices. Test each against the intended task.

1. **Script.** Train on the native script(s), or on a shared representation such as the toolkit's Common Turkic Script (CTS) (Section 3.6)? For a multilingual Turkic tokeniser, a shared script lets cognate stems and suffixes share vocabulary entries; the cost is a conversion step at input and output time and a representation whose pretraining coverage needs checking.
2. **Segmentation constraint.** Unconstrained BPE/Unigram, morph-pre-segmented training as above, or a hybrid in which morph boundaries are preferred but not enforced.
3. **Vocabulary and sampling.** A monolingual vocabulary (BERTurk's base model, for comparison, has 32K entries) or one multilingual vocabulary; for the latter, temperature sampling over languages (Section 16.4) can reduce the imbalance in training exposure.

TurkicNLP's `MorphemeTokenizer` supplies surface segments. The strict example below uses the same word-level segmenter for corpus preparation and inference. Run from the book root, with a supported TurkicNLP backend installed; `load()` may acquire neural resources. Record which backends loaded, rather than assuming an HFST-only run. The helpers in `code/ch07-neural/boundary_tokenizer.py` reject changed surface forms and reserved markers, and check decoding of token IDs, including byte fallback. Their whitespace sidecar preserves the original spacing; the point is to demonstrate boundaries, and the result is not a complete LM input format.

```
from pathlib import Path
import sys
import sentencepiece as spm
from turkicnlp.processors.morpheme_tokenizer import MorphemeTokenizer
sys.path.insert(0, "code/ch07-neural")
from boundary_tokenizer import BOUNDARY, training_lines, encode_text, decode_text

segmenter = MorphemeTokenizer(lang="kaz")
segmenter.load()  # loads available HFST/neural backends; record which succeeded
segment_word = lambda word: segmenter.segment(word).segments
text = Path("kaz.txt").read_text(encoding="utf-8")
Path("kaz.seg.txt").write_text("\n".join(training_lines(text, segment_word)) + "\n",
 ↪  encoding="utf-8")
spm.SentencePieceTrainer.train(
    input="kaz.seg.txt", model_prefix="kaz_morph", vocab_size=16000,
    model_type="unigram", character_coverage=1.0, hard_vocab_limit=False,
    user_defined_symbols=[BOUNDARY], byte_fallback=True,
    normalization_rule_name="identity", remove_extra_whitespaces=False,
    add_dummy_prefix=False,
)
sp = spm.SentencePieceProcessor(model_file="kaz_morph.model")
sample = "Мен мектептерімізде оқыдым"
encoded = encode_text(sample, sp, segment_word)
assert decode_text(encoded, sp) == sample
print(encoded)
```

A user-defined | marker is encoded separately. Identity normalisation prevents compatibility rewriting, while byte fallback covers characters absent from the training alphabet. Literal | and SentencePiece's ▁ in source words are rejected; an application needing them must define and test an escape convention. No piece may span the supplied boundary and decoding must reproduce the segmented surface. The SentencePiece training options document these settings.

The model needs a vocabulary large enough for its byte entries, characters and special symbols; increase the budget if training reports that it is too small. Report fertility including and excluding the inserted boundary tokens, and specify the whitespace representation. Preserve the model, training-data hashes, segmenter source and transducer hashes together. Toolkit corpus and pipeline wrappers can have different cleaning, pretokenisation or fallback behaviour; do not assume their output matches this contract without testing it. `python code/ch07-neural/check_examples.py` exercises the contract with a tiny trained SentencePiece model, synthetic boundaries, unseen Unicode characters and repeated whitespace. It checks mechanics; expert boundary accuracy is a separate evaluation (M2).

Evaluate any candidate tokeniser on held-out text with at least:

Table 7.3: A minimal evaluation suite for Turkic tokenisers.

Metric	What it measures
Fertility and split rate	Cost and context capacity (Section 7.1)
95th-percentile tokens per word	Upper-tail fragmentation of long forms
Boundary precision/recall	Agreement with manually checked morph boundaries; unreviewed analyser output measures agreement with that analyser
Cross-lingual token overlap	Share of tokens used by more than one Turkic language (for multilingual tokenisers)
Downstream score	The task you actually care about — evaluated alongside cost and error types

7.2.4 Choosing a Use Case

Morphological tagging and dependency parsing are useful tasks for testing pre-segmentation because their labels can depend on suffixes. Compare matched models with and without the segmentation step; supplied boundaries can be wrong, and more tokens can increase computation.

For an existing language model, replacing the tokeniser also changes its input vocabulary. Vocabulary extension is one adaptation option (Chapter 12). For a new model, compare tokenisation designs using Table 7.3. Where the segmenter has poor coverage, report its fallback behaviour and errors alongside downstream scores.

7.3 Neural Morphological Analysis

7.3.1 The Task and the Trade-Off

Morphological analysis predicts a lemma, part of speech and morphological features, as described in Chapter 5. FSTs use an explicit lexicon and rules (Chapter 6). Neural models learn from data and can generalise to forms absent from an FST lexicon; contextual models also use the surrounding sentence. Supervised training needs annotated examples, which may come from another language in a transfer setting. Both approaches can return wrong lemmas or features.

Two neural paradigms are currently in use, with a third hybrid approach combining their respective advantages.

7.3.2 Character-Level Sequence-to-Sequence Analysis

A character-based neural approach treats morphological analysis as a **string transduction** problem: the input is the character sequence of the surface form, and the output is the analysis string (lemma plus morphological tag sequence). An encoder-decoder architecture (an LSTM or a transformer, for example) learns this transduction from annotated data.

For example, the input character sequence `g-e-l-d-i-k-l-e-r-i-n-d-e-n` is transduced to an analysis string such as `gel+VERB+VerbForm=Part|Number[psor]=Plur|Person[psor]=3+Case=Abl` (one of its readings, Section 5.3). The model must learn to identify the stem (*gel-* "come"), to strip and identify each suffix layer, and to emit the correct tag sequence in the right format.

Character-level models can combine familiar stems and suffix patterns. A model trained on *gittikleri* and *geldim*, for example, may be able to analyse *geldikleri*. Both its stem and participial suffix pattern are familiar. But some surface forms allow several readings. A word-only model lacks the surrounding sentence needed to choose between them and may learn a preference that differs from the most frequent reading.

7.3.3 BERT-Based Morphological Tagging

A contextual tagger treats morphological analysis as **sequence labelling**. BERT-style embeddings let it use the surrounding sentence when choosing an analysis.

A standard token classifier first splits each word into subwords. It takes the first subword's contextual BERT embedding as the word representation, then uses a linear head to predict a morphological feature bundle. Fine-tuning requires a morphologically annotated corpus, typically a Universal Dependencies treebank with complete features in the FEATS column.

The Hands-On section implements this BERT-style architecture. TurkishDelightNLP is a different neural design: its morphology/syntax model uses stacked LSTMs, and its

morphological tagger combines character and word-context encoders (Alecakir, Bölücü, and Can 2022, 19–21). It should not be cited as a BERTurk token classifier.

7.3.4 Error Types of Neural Morphological Analysers

Evaluation should distinguish the following errors in neural and rule-based systems:

- **Wrong contextual reading**: a licensed analysis, but not the one the sentence requires (*evi* as possessive where the accusative is meant)
- **Incompatible feature combination**: a predicted bundle not licensed for the word's category and construction. Check the annotation scheme: stacked voice markers and underspecified values can legitimately produce multiple values.
- **Wrong lemma**: a lemma that does not match the word, for example a stem cut at the wrong place
- **Missing gold candidate** (for FST-based systems): the correct analysis is not among the candidates at all

A lemma missing from a dictionary is not necessarily an error: it can be a valid new word. These errors can be invisible to surface-level evaluation, because a plausible but wrong analysis still looks like a valid analysis string.

7.3.5 Hybrid FST-Constrained Analysis

In a hybrid system, the FST supplies a **candidate set** of analyses for each word. A neural model **reranks** them using the surrounding sentence. It can select only from this set. The FST may overgenerate or omit the correct reading, so report candidate recall before reranker accuracy: how often does the set contain the gold analysis?

TurkicNLP's rule-based backend follows this pattern in a simple form: Apertium FSTs provide candidates for 20 languages, and transducer weights plus heuristic scores (syntactic position, part-of-speech priority, question particles) select among them (Hakimov 2026). A learned reranker is a natural extension: fine-tune a model to score each (word-in-context, FST-candidate) pair and select the highest-scoring candidate, combining the grammar's constraints with the model's contextual sensitivity.

Which side should lead depends on the transducer. Where the FST is mature, it should generate the candidates; where it is a prototype with a small lexicon, a neural analyser may assign analyses to more words, whose correctness still needs checking (Table 6.1), and the FST is better used as a validator for the words it does know.

7.3.6 A Multilingual Neural Analyser for the Family

TurkicNLP uses one neural morphological analyser for 22 languages (Hakimov 2026). It fine-tunes the top three layers of Glot500 (Imani et al. 2023) and adds script-specific adapters (Latin, Cyrillic, Arabic), a language embedding and a shared two-layer BiLSTM. The outputs are UPOS, 69 UD feature–value pairs and lemmas represented as edit scripts.

Training draws on UD-format treebanks for ten languages (including silver treebanks for Azerbaijani, Turkmen and Bashkir), UniMorph paradigms for 13 languages (≈1.9 million entries) and Wiktionary paradigms for 20 languages (≈4.8 million forms). Temperature sampling ($T = 2$) reduces the imbalance in language exposure. Paradigm tables provide supervision for languages that lack annotated running text.

Languages without a treebank use a related *proxy* language's embedding: Turkish for Crimean Tatar, Gagauz and Khalaj; Bashkir for Sakha, Tuvan, Khakas, Altai and Chuvash; and Tatar for Kumyk. Their paradigm data also adapts the shared layers. Three further languages receive purely zero-shot coverage: Karakalpak uses Uzbek as its proxy, while Karachay-Balkar and Nogai use Tatar.

7.4 Morphological Disambiguation

7.4.1 The Task

Morphological disambiguation is the problem that motivates the hybrid approach. As established in Section 5.3, a single Turkic surface form may have several valid morphological analyses, because morphological categories overlap at the surface. In context, usually one analysis is intended, though genuine ambiguity and alternative annotation conventions also occur. Selecting the analysis is useful for parsing, NER and other downstream tasks, whether done explicitly or implicitly by the model.

On the UD 2.17 test samples and denominator defined in Section 5.3, 399/730 Turkish TueCL tokens (55%) and 5,071/7,667 Kazakh-KTB tokens (66%) have multiple readings even after removing all + continuations. The excluded continuation class is broader than copular readings. Exact counts and per-token outputs are in `code/ch05-ambiguity/`.

A representative example: among the readings of the Turkish form *yüz* are these three (the table illustrates selected readings, not all lexical senses):

Table 7.4: Three readings of the Turkish form *yüz*, as the Apertium Turkish transducer returns them (it also returns substantivised and copular variants). Context selects among them.

Analysis	Part of speech	Lemma	Meaning
`yüz<n><nom>`	N	*yüz*	face
`yüz<num>`	NUM	*yüz*	one hundred
`yüz<v><iv><imp><p2><sg>`	V	*yüz-*	swim! (2sg imperative)

The same form *yüz* occurs in all three contexts: *yüz metre* ("one hundred metres") selects the numeral; *Havuzda yüz!* ("Swim in the pool!") selects the verb; *güzel bir yüz* ("a beautiful face") selects the noun. Nothing inside the word itself resolves the ambiguity.

7.4.2 Classical Approaches

Hidden Markov Models (HMM) model the joint probability of a morphological tag sequence and the observed words:

$$P(\mathbf{t}, \mathbf{w}) = \prod_{i=1}^{n} P(t_i \mid t_{i-1}) \cdot P(w_i \mid t_i)$$

where t_i is the morphological tag at position i and w_i is the observed word. This is a first-order (bigram) model; the Viterbi algorithm recovers the most probable tag sequence in $O(n \cdot |T|^2)$ time. Second-order (trigram) models condition on two previous tags. For Turkish, Hakkani-Tür, Oflazer, and Tür (2000) disambiguate with statistical models of this kind over sequences of inflectional groups. A separately documented statistical approach for Kazakh uses a voted perceptron with Viterbi decoding (Tolegen, Toleu, and Mussabayev 2020); it should not be labelled an HMM.

The limitation is the short history: the probability of tag t_i depends only on the previous one or two tags. In Turkic sentences, agreement and case government can disambiguate a form several words away.

Conditional Random Fields (CRF) model the conditional distribution $P(\mathbf{t} \mid \mathbf{w})$ directly. A linear-chain CRF can use features of the *whole input* sequence (character n-grams of the current word to capture suffix patterns, surrounding words), while the dependence between *output labels* stays limited to neighbouring tags. These are two different axes: broad input context, local output structure.

7.4.3 Neural Approaches

BiLSTM-CRF replaces hand-crafted features with learned representations from a bidirectional LSTM, while retaining a CRF output layer for structured prediction over the tag sequence. The character-level BiLSTM is particularly valuable for Turkic: it encodes the morphological form of each word (vowel sequences, suffix patterns) as an implicit feature, allowing the model to generalise to unseen surface forms.

Transformer-based disambiguation uses a fine-tuned BERT model as the encoder. For each word in a sentence, the word's contextual BERT embedding is passed through a linear classification head predicting the full morphological feature bundle. The Hands-On section uses this closed-bundle classifier; TurkishDelightNLP instead uses LSTM encoders and a tag decoder (Section 7.3). Both BiLSTMs and transformers can use context on either side of a word. Attention provides direct interactions within the input window; whether this improves disambiguation must be measured on the same data.

7.4.4 Evaluation

Morphological disambiguation is evaluated at two granularities, and the distinction is consequential.

Per-feature accuracy measures individual categories, such as case or tense. State how absent features and extra predictions are counted.

Full-bundle accuracy requires every feature on a word to match. It reveals errors hidden by individual feature scores, but excludes lemma and POS unless those are explicitly included.

The official UD evaluator reports `UFeats` for feature-bundle agreement and `AllTags` for joint UPOS, XPOS and feature agreement. Per-feature scores require a separate calculation. Report both bundle and per-feature results, with the scorer version and word-alignment policy. Which errors matter for a parser or another application only downstream evaluation can tell.

7.5 Zero-Shot Morphological Tagging for Unseen Languages

7.5.1 Setup and Motivation

For a target language without task-labelled training data, zero-shot transfer offers a baseline. Check treebanks and paradigm resources separately before asserting that a language has no morphological annotation. Shared categories such as case, number, person and tense motivate transfer from Turkish or Kazakh. Their forms, ordering and annotation still differ across languages, so transfer quality needs testing (Chapter 5).

The zero-shot morphological transfer setup is:

1. Train a morphological tagger (BERT-based or BiLSTM) on a language with annotated data (typically Turkish or Kazakh)
2. Apply the tagger to a related language with no annotated data, without fine-tuning

Shared morphological patterns motivate testing transfer within the Turkic family against transfer across families. Two findings are relevant, though they concern different tasks. Kardeş-NLU reports improved sentence-level transfer to Azerbaijani, Kazakh, Kyrgyz, Uzbek and Uyghur when Turkish serves as a high-resource relative (Senel et al. 2024).

For morphology, TurkicNLP's multilingual analyser covers three languages without task-labelled or paradigm training data. On the parallel challenge set, it detects the target feature in 50% of Karakalpak sentences, 55% of Nogai sentences and 70% of Karachay-Balkar sentences. It assigns some feature to 86–97% of their content words (Hakimov 2026). Those coverage figures count non-empty bundles, including potentially incorrect ones. Here, *zero-shot* refers to the absence of target-language task-labelled training data; Glot500's pretraining text may still contain these languages.

7.5.2 The Script Barrier

Before applying a monolingual model such as BERTurk to Cyrillic Kyrgyz, check its tokenisation and task performance: the scripts share little of its subword vocabulary. A multilingual encoder pretrained on both scripts can support cross-script transfer, as the Glot500-based analyser above illustrates. Transliteration offers another option (Chapter 3). Converting Kyrgyz Cyrillic into CTS with the specified toolkit mapping increases shared spelling, but changes the input relative to the model's pretraining text.

Compare, rather than assume:

> Compare native-script multilingual transfer with a specified transliteration pipeline. Shared spelling can increase token overlap, yet more overlap and lower fertility need not translate into better tagging.

Evaluate the same target test items with native-script multilingual input, transliterated input and a target-supervised model. Treat the supervised model as a baseline whose advantage also needs to be measured.

7.5.3 Questions for a Transfer Evaluation

Table 7.5 turns the linguistic comparisons into evaluation questions. Shared categories motivate testing transfer; they do not predict its accuracy. Exercise 6 tests these questions.

Table 7.5: Questions for evaluating cross-lingual transfer of morphological features. Annotation-schema compatibility (whether source and target treebanks annotate the same features the same way) is a separate question from linguistic similarity.

Feature	Question	Basis
Noun case (Nom, Acc, Dat, Loc, Abl, Gen)	Does accuracy differ for shared and target-specific cases?	Cognate affixes in most languages; Sakha lacks the genitive and has partitive, comitative and comparative forms (Johanson 2021, p p. 463, 477–478)
Number (Sing / Plur)	Does the model recognise different plural markers?	*-lAr* or cognates in many languages; Chuvash *-сем* is a counterexample (Johanson 2021, 453)

Feature	Question	Basis
Verbal person agreement	Which agreement forms transfer?	Cognate agreement suffixes, two paradigms (Section 5.1)
Tense and aspect	How are auxiliary constructions handled?	Core categories shared; analytic aspect constructions differ
Voice (passive, causative, reflexive)	Which suffix patterns cause errors?	Present in all branches; formation varies
Derivational categories	Does the source label inventory cover the target?	Productivity and lexicalisation vary

7.5.4 What Requires Language-Specific Annotation

A closed classifier can only emit labels included in its output inventory, regardless of script normalisation.

Language-specific grammatical categories. A closed classifier cannot emit labels absent from its output inventory. A Turkish-only case inventory, for example, omits Sakha's partitive, comitative and comparative cases (Johanson 2021, 477–78). Chuvash's possessive–plural order presents a different problem: the model must handle a different sequence of markers, even when the output labels are familiar (Section 5.4.5).

Uzbek surface forms. Uzbek suffix vowels are largely fixed (Section 2.2), so Uzbek has *-da* and *-lar* where Turkish alternates *-da/-de*, *-lar/-ler*. UD labels do not encode allomorphs, but the changed input strings can still affect a Turkish-trained tagger. They also matter for surface generation.

Language-specific derivational productivity. Some derivational morphemes are highly productive in one language but lexicalised (non-compositional) in another. A tagger trained on Turkish cannot infer from its training data which derivational morphemes are productive in Kyrgyz.

Zero-shot transfer can supply an initial analysis of the shared inflectional patterns, useful, for example, for pre-annotating text before human correction. Measure its quality separately for each language. To adapt the model to language-specific features, consider paradigm tables from UniMorph or Wiktionary and a small amount of annotated running text.

7.6 Hands-On: A Fine-Tuning Procedure for Morphological Tagging with BERTurk

Question. Can a word-level classifier predict UPOS and complete feature bundles?

Inputs. Official Turkish UD splits, BERTurk and the training recipe below.

Compute. Model downloads and training memory are required; a GPU is useful for the full fine-tuning run.

Output. A saved tagger, per-word predictions, unseen-bundle counts and development/test scores.

Interpretation. Compare against the supplied analysers on the same words and retain unseen labels in the denominator.

This tutorial fine-tunes BERTurk for morphological feature prediction on Turkish UD data. Morphological tagging is framed as token classification: for each word, predict one composite label combining UPOS and the full feature bundle. A closed label set has a consequence the tutorial makes explicit: a bundle that never occurs in training cannot be predicted, and such words must still be scored as errors.

7.6.1 Setup and Data

Run from the book root. Use the official train/dev/test files of **UD 2.17 Turkish-IMST** without reshuffling sentences. Set `UD_IMST_DIR` to that release's `UD_Turkish-IMST` directory. The encoder and tokeniser are pinned to the same Hugging Face revision. The complete code is also in `code/ch07-neural/fine_tune.py`.

```
# pip install "transformers==5.3.0" "torch==2.10.0" "accelerate==1.12.0"
from pathlib import Path
import hashlib, importlib.metadata, json, os, sys
import torch
from transformers import (AutoTokenizer, AutoModelForTokenClassification, TrainingArguments,
                          Trainer, DataCollatorForTokenClassification, set_seed)
sys.path.insert(0, "code/ch07-neural")
from morph_tagging import (load_conllu, training_rows, predict_words,
                           score_predictions, export_conllu)

SEED, MAX_LEN = 13, 512
set_seed(SEED)
MODEL_NAME = "dbmdz/bert-base-turkish-cased"
MODEL_REVISION = "b6e1de16c983e0f2c70664591ea3f22810072608"
data_dir = Path(os.environ["UD_IMST_DIR"])
paths = {split: data_dir / f"tr_imst-ud-{split}.conllu" for split in ["train", "dev",
 ↪  "test"]}
train_sents, train_labels = load_conllu(paths["train"])
dev_sents, dev_labels = load_conllu(paths["dev"])
test_sents, test_labels = load_conllu(paths["test"])
```

The reader uses integer-ID syntactic words, skipping multiword-token headers and empty nodes. Feature attributes and multivalued features are sorted before constructing a label,

so equivalent FEATS strings have the same class. Punctuation remains in this evaluation's denominator, unlike the ambiguity diagnostic in Section 5.3.

7.6.2 Tokenising without Losing Words

Only the first subword of each word receives a training label. Special tokens and continuation pieces receive `-100`. The label vocabulary comes from training data alone. Long sentences are partitioned into non-overlapping chunks at **whole-word boundaries**, with special tokens included in the length check. Every word appears exactly once; a word too long to fit by itself raises an error. This loses context across chunk boundaries, which should be reported, but does not silently discard evaluation words.

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, revision=MODEL_REVISION,
 ↪  use_fast=True)
all_labels = sorted({label for sent in train_labels for label in sent})
label2id = {label: i for i, label in enumerate(all_labels)}
id2label = {i: label for label, i in label2id.items()}
train_dataset = training_rows(train_sents, train_labels, tokenizer, label2id, MAX_LEN)
assert sum(sum(label != -100 for label in row["labels"]) for row in train_dataset) ==
 ↪  sum(map(len, train_sents))
print(f"training sentences: {len(train_sents)}; chunks: {len(train_dataset)}")
```

7.6.3 Fine-Tuning

This baseline uses a fixed five-epoch budget and saves the final checkpoint. Development data are scored with the same all-word metric as test data; they are not passed through a loss that masks unknown gold classes. This avoids undefined losses on batches containing only unseen bundles. To select a checkpoint or tune hyperparameters, use development accuracy and freeze the choice before opening test results.

```
model = AutoModelForTokenClassification.from_pretrained(
    MODEL_NAME, revision=MODEL_REVISION, num_labels=len(label2id),
    id2label=id2label, label2id=label2id)
training_args = TrainingArguments(
    output_dir="berturk-morph-tagger", num_train_epochs=5, seed=SEED,
    per_device_train_batch_size=16, learning_rate=2e-5, warmup_ratio=0.1,
    eval_strategy="no", save_strategy="no", report_to="none",
)
trainer = Trainer(model=model, args=training_args, train_dataset=train_dataset,
                  processing_class=tokenizer,
                  data_collator=DataCollatorForTokenClassification(tokenizer))
trainer.train()
trainer.save_model("berturk-morph-tagger/final")
tokenizer.save_pretrained("berturk-morph-tagger/final")
```

7.6.4 Inference, Scoring and Export

Inference uses the same chunking function, the model's actual device and evaluation mode. The scorer checks sentence and word counts instead of silently shortening mismatched

lists with `zip`. Every gold word contributes: a class absent from training necessarily counts as an error.

```
results = {}
for split, sents, labels in [("dev", dev_sents, dev_labels), ("test", test_sents,
 ↪  test_labels)]:
    predictions = [predict_words(words, model, tokenizer, id2label, MAX_LEN) for words in
 ↪  sents]
    results[split] = score_predictions(predictions, labels, label2id)
    export_conllu(paths[split], predictions, f"berturk-morph-tagger/{split}.conllu")
    print(split, results[split])
manifest = {
    "ud_release": "2.17", "model": MODEL_NAME, "revision": MODEL_REVISION,
    "seed": SEED, "max_length": MAX_LEN, "epochs": 5,
    "checkpoint_policy": "fixed final epoch; no selection on test",
    "data_sha256": {split: hashlib.sha256(path.read_bytes()).hexdigest() for split, path in
     ↪  paths.items()},
    "versions": {name: importlib.metadata.version(name) for name in ["torch",
     ↪  "transformers", "accelerate"]},
    "training_sentences": len(train_sents), "training_chunks": len(train_dataset),
    "scores": results,
}
Path("berturk-morph-tagger/run.json").write_text(json.dumps(manifest, indent=2) + "\n")
```

The score is exact UPOS+FEATS agreement, excluding lemma and XPOS. The export preserves CoNLL-U comments, multiword-token rows and empty nodes, replacing only UPOS/FEATS on integer-ID rows. Its other fields remain source annotations, so use the official UD evaluator's UPOS and UFeats scores; do not report its lemma, parsing or AllTags outputs as this model's predictions.

The pinned tokeniser was also checked on the local UD 2.17 IMST release: all 37,522 training, 10,542 development and 10,032 test words retain alignment. There are 981 training labels, with 79 development and 112 test words whose gold composite labels are absent from training (`code/ch07-neural/ud-alignment.json`). No sentence in this release exceeds 512 tokens; synthetic long sentences exercise the overflow branch. These counts describe the release snapshot and are not tagging scores.

The helper checks exercise loss masking, word-preserving overflow, a tiny random BERT training step, inference, unseen labels and CoNLL-U export. For the full tutorial, fix a training budget, save the complete manifest and report development and test scores.

Compare the fine-tuned tagger with the two analysers that TurkicNLP ships, the Apertium transducer and the multilingual Glot500 analyser:

```
import turkicnlp

for proc in ["morph", "morph_neural"]:
    nlp = turkicnlp.Pipeline("tur", processors=["tokenize", proc], tokenize_backend="rule")
    for w in nlp("Ali kitabı dün okudu.").words:
        print(f"{proc:<13} {w.text:<8} {w.upos or '_':<6} {w.feats or '_'}")
```

This model can then be applied without further training to Azerbaijani or other Latin-script Oghuz languages as a zero-shot morphological tagger (Section 7.5); Exercise 6 asks you to measure how well that works.

> **Extending to Kazakh**
>
> To tag Kazakh text with the same architecture, substitute `dbmdz/bert-base-turkish-cased` with a Kazakh or multilingual encoder and train only on the Kazakh KTB training split. Keep its test set out of training and model selection (Chapter 8). For zero-shot transfer to Kyrgyz or Uzbek, compare native-script input to a multilingual encoder with transliterated input (Section 7.5); do not assume that either is better.

Status: procedure only. The full BERTurk fine-tuning run is not reported here; the mechanics checks and the separate exercise baselines are described above.

What to Take Forward

Subword tokenisation sets a model's input units; morphological analysis predicts linguistic structure. The fertility comparison in Table 7.1 shows how token counts vary across the sampled languages and tokenisers. Use fertility and boundary agreement as diagnostics, then choose a tokeniser using downstream performance, context use and computational cost. Evaluate an analyser's overall performance and report unseen bundles separately. Chapter 8 moves from the word to the sentence, where morphology and position together signal dependency structure.

Further Reading

Tokeniser fertility is measured across 20 Turkic languages and four LLM tokenisers, in native scripts and in CTS, by Hakimov (2026). Rust et al. (2021) and Petrov et al. (2023) analyse tokeniser quality and the resulting cost disparities across languages more generally; Toraman et al. (2023) compare tokenisation granularities for Turkish language models. Schweter (2020) describes BERTurk and its 32K and 128K vocabularies.

The SentencePiece paper (Kudo and Richardson 2018) describes the library; the Unigram algorithm it implements was introduced by Kudo (2018). For Turkish NLP more broadly, Oflazer and Saraçlar (2018) covers morphological analysis and disambiguation (chapters 2–3), parsing and language modelling. The TurkishDelightNLP system (Alecakir, Bölücü, and Can 2022) demonstrates joint LSTM-based morphological and syntactic processing for Turkish.

For multilingual morphological analysis across the family, including zero-shot languages, see Hakimov (2026); Glot500 is described by Imani et al. (2023). For sentence-level transfer with Turkish as a high-resource relative, see Kardeş-NLU (Senel et al. 2024). Chapter 16 extends the treatment to the full cross-lingual transfer methodology.

Exercises

1. **Fertility audit.** Take a Turkish news article of at least 500 words and tokenise it with `dbmdz/bert-base-turkish-cased` (32K), `dbmdz/bert-base-turkish-128k-cased` (128K) and `xlm-roberta-base`, without special tokens. Compute fertility twice: words tokenised one at a time, and running text (tokens of the whole text divided by its words). State how you count words and punctuation. What proportion of words are single tokens? What is the 95th-percentile tokens per word?

2. **Boundary agreement.** For a Turkish word, obtain morph boundaries (from the TurkicNLP segmenter or Zemberek) and token boundaries (from a tokeniser), both as character offsets. Automatically supplied morph boundaries measure agreement with that analyser; manually check them before treating them as gold. Count two different errors separately: a *token that spans a morph boundary* (a morph boundary falls inside a token), and a *morph split by a token boundary* (a token boundary falls inside a morph). Apply the script to the 100 most frequent nouns of a corpus and report both rates for each tokeniser.

3. **Disambiguation in context.** The Turkish form *yüz* has three readings in Table 7.4. Collect at least 60 sentences per reading containing the bare form *yüz* (from several documents), and split them by document into 40 training, 10 development and 10 test sentences per reading. Fine-tune a Turkish BERT classifier. What accuracy does it achieve on the test set? Which contextual cues (a following noun, a preceding adjective, sentence-final position) signal each reading?

4. **Constrained FST reranking.** Implement the hybrid FST-constrained neural approach described in Section 7.3. For a Turkish sentence, retrieve the Apertium FST candidates for each word (with `analyser.lookup(word)` on the transducer that TurkicNLP downloads, as in Section 6.3). For each candidate, embed the tag string as a character sequence and concatenate with the word's BERT contextual embedding. Train a linear classifier to select among candidates. First report candidate recall (how often the gold analysis is among the candidates), then compare full-bundle accuracy against (a) always taking the FST's first analysis, and (b) the unconstrained BERT tagger from the Hands-On section.

5. **Glot500 as a baseline.** Compare TurkicNLP's `morph_neural` analyser and the BERTurk tagger on the IMST test set. Supply the same gold word boundaries where supported; otherwise use an explicit alignment policy and report segmentation errors and unmatched words. Score UPOS and the full feature bundle under the same convention. How does a multilingual analyser serving 22 languages compare with the monolingual tagger? On which features does each win?

6. **Zero-shot transfer evaluation.** Apply the BERTurk tagger from the Hands-On section to the whole Azerbaijani TueCL UD test set (Akhundjanova et al. 2025), which translates the same sentences as the Turkish TueCL set; note whether any of those Turkish sentences were used during development. Report full-bundle accuracy and per-feature accuracy by category, and check whether the two treebanks annotate the same features in the same way. Which categories transfer best? What do the results show for the questions in Table 7.5?

Part IV

Syntax

8 Syntactic Structure and Universal Dependencies

Learning Objectives

After reading this chapter, you will be able to:

- Explain why English-style positional shortcuts are unreliable for Turkic languages and which signals (position, morphology, lexical selection, context) parsers combine instead
- Describe the role of participial relative clauses in creating long-distance dependencies
- Summarise the Universal Dependencies framework and identify the Turkic-specific annotation choices it leaves open
- Use the UD 2.17 inventory to identify Turkic treebanks and release-specific gaps
- Contrast transition-based and graph-based (biaffine) dependency parsing and reason about their fit with SOV languages
- Design and evaluate a zero-shot cross-lingual parsing pipeline for a Turkic language with no treebank
- Plan the construction of a new Turkic treebank, from sentence selection and pre-annotation to validation and release

This chapter examines how Turkic words combine into sentences and how Universal Dependencies (UD) represents that structure. The main topics are case-marked arguments, flexible word order, omitted subjects and participial clauses. Parsers must use these cues together; position alone is insufficient.

The chapter then surveys the UD 2.17 treebanks, explains dependency parsing and compares documented parser results. The final sections cover transfer and the construction of a new treebank. Throughout, annotation conventions matter: two treebanks can assign different heads or labels to the same kind of construction.

8.1 A Computational Sketch of Turkic Syntax

8.1.1 Case-Marked Arguments and Word Order

English declarative clauses often place the subject before the verb and the object after it. These are useful cues, but English parsers also need lexical, morphological and contextual

information.

Turkish makes this strategy unreliable. Consider the sentence *Ali kitabı okudu* (“Ali read the book”). The canonical, rhetorically neutral order is subject–object–predicate, and scrambling rearranges it for pragmatic purposes, for example to object–subject–verb or verb–subject–object (Johanson 2021, sec. 56.3, pp. 935–936). With an accusative-marked object, all six permutations of subject, object and verb can be used in a suitable discourse situation (Hoffman 1994, 117). This does not mean that every sentence permutes freely: orders other than SOV are marked, and order is more rigid in subordinate than in main clauses (Johanson 2021, 936):

Table 8.1: The six constituent orders of a simple Turkish transitive sentence. SOV is the neutral order; the other five occur in suitable discourse contexts. The accusative suffix *-ı* on *kitab-ı* (“book-ACC”) identifies the object in each order. Pragmatic labels (topic, focus, backgrounding) are deliberately not assigned per row, because they depend on a discourse context that a table cannot show.

Order	Turkish	Status
SOV	*Ali kitabı okudu*	neutral, default
OSV	*kitabı Ali okudu*	marked; context-dependent
SVO	*Ali okudu kitabı*	marked; context-dependent
OVS	*kitabı okudu Ali*	marked; context-dependent
VSO	*okudu Ali kitabı*	marked; context-dependent
VOS	*okudu kitabı Ali*	marked; context-dependent

The family-wide pattern also has exceptions. Johanson describes contact-induced verb-medial order in Gagauz, Karaim and Cypriot Turkish (Johanson 2021, 936). The Turkish table is therefore not a word-order rule for every variety.

How often each order occurs in edited text is measured in Exercise 1: among BOUN training sentences whose root has an overt subject and an overt object, 87.1% are SOV, followed by OSV (6.7%), OVS (3.3%) and SVO (2.5%); VSO and VOS occur three times in 957 sentences.

The accusative suffix *-ı* identifies *kitabı* as the direct object in each order. The unmarked *Ali* is the subject, identified here by verbal agreement and the accusative marking on the other argument. Lack of a suffix alone would be insufficient: non-specific objects can also be unmarked, and possessive *-ı/-i* can resemble the accusative. Word order contributes both grammatical and pragmatic information, including topic, focus and backgrounding. An SVO shortcut would mislabel some of these reordered arguments.

To test whether explicit morphology helps, compare the same parser with and without predicted features. A contextual encoder may already recover some of this information from word forms.

8.1.2 Scrambling and Information Structure

The reordering illustrated in Table 8.1 is not random. It follows regular principles of **information structure**: what is topical (given information, often fronted), what is focused

(new or contrastive information, often immediately pre-verbal in Turkish), and what is backgrounded (often after the verb) (Johanson 2021, sec. 56.4, pp. 937–949; Hoffman 1994, 117).

A transferred model must account for these links between discourse role and position. A sentence-initial topic in Turkish may be a subject, object or adjunct. The constituent immediately before the verb often carries focus, whatever its grammatical role.

This matters concretely for tasks like machine translation (where reordering must preserve pragmatic meaning, not just grammatical structure), sentiment analysis (where focus-marked constituents carry heightened emphasis), and information extraction (where the topic is often what the sentence is *about*, not what the syntax labels as subject).

8.1.3 Pro-Drop: The Absent Subject

Turkic languages generally allow a referential subject pronoun to be omitted when its referent is recoverable, though the conditions and frequency vary by language and discourse (Johanson 2021, sec. 54.7, pp. 851–853). In Turkish main clauses null subjects are much more frequent than overt ones, whereas Uyghur and Yakut use anaphoric pronouns more often (Johanson 2021, 851–52). Agreement constrains person and number; it does not identify an antecedent or encode grammatical gender. Turkish *Okudu* can mean “he/she/it read” in context, with third-person singular verbal agreement.

An overt pronoun may still be used for contrast or topic shift. Exercise 3 measures subject omission under an explicit eligibility definition rather than treating every missing subject arc as pro-drop. In the IMST training set, 43.8% of finite, active, non-imperative verbal predicates have an overt subject: 56.9% in the third person but only 13.5% in the first and 15.1% in the second, where the agreement suffix marks person and number, but does not identify the referent.

Pro-drop creates several downstream problems:

Coreference resolution must track implicit subjects. A system that only links overt mentions needs an additional representation for implicit subjects.

Machine translation into languages such as English, French or German often needs an overt subject where the Turkic source omits one. Choosing a pronoun or noun phrase requires the referent to be recovered from agreement and discourse context. A target pronoun may also require a gender distinction absent from the source verb.

Syntactic annotation must decide how to represent the absent subject. Universal Dependencies handles this by recording person and number in the verb’s FEATS column and leaving no subject arc in the parse graph when no overt subject is present.

8.1.4 Participial Relative Clauses: The Pre-Nominal Clause

Prenominal participial clauses differ from the relative-clause pattern illustrated by the English example below. In English, a relative clause follows the head noun. It may begin with a relative pronoun (*who*, *which*) or marker (*that*), though some relatives have no overt marker:

the man **who** came yesterday

In Turkish, the relative clause **precedes** the head noun, contains no complementiser, and marks clause type on the verb with a participial suffix rather than a separate word:

dün gelen adam yesterday come.PTCP man “the man who came yesterday”

The participial form *gelen* (come.PTCP) is a non-finite verbal form that modifies *adam* (“man”) directly. When the relative clause is complex, with its own arguments, adjuncts and embedded clauses, the entire structure builds to the left of the head noun:

[Atatürk’ün 1928’de başlattığı] dil reformu Atatürk-GEN 1928-LOC begin-CAUS-NMLZ-POSS.3 language reform-POSS.3 “the language reform [that Atatürk started in 1928]”

(The *-u* on *reform-u* is the possessive marker of the Turkish noun compound *dil reform-u*, not an accusative.) Figure 8.1 shows the two constructions as dependency trees.

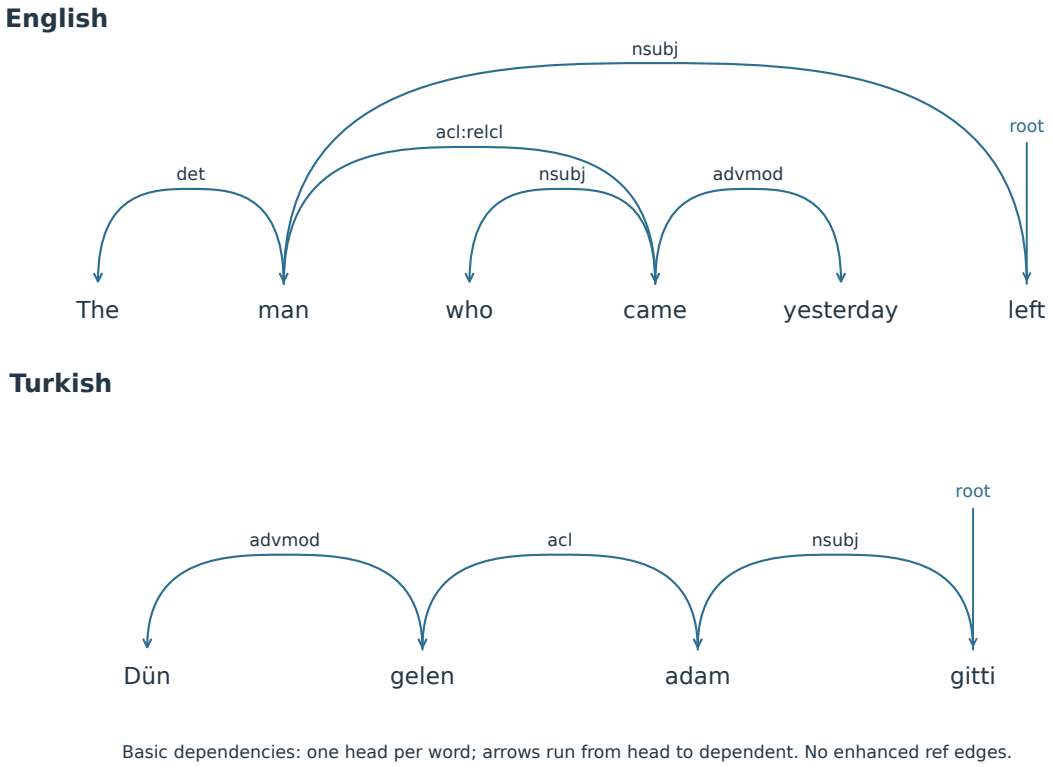


Figure 8.1: Basic UD dependencies for English (top) and Turkish (bottom). English has a post-nominal relative clause with who as the subject of came; Turkish Dün gelen adam gitti has a prenominal participial clause with no relative pronoun. Each word has exactly one head; enhanced coreference edges are omitted.

Three distinctions matter for parsing:

Where the long arcs are. Three different distances are easy to confuse:

1. *Clause width*: how many words the relative clause spans (*Atatürk'ün 1928'de başlattığı*: three words).
2. *Dependent-to-participle distance*: how far the clause's own dependents are from the participle that heads them (*başlattığı* → *Atatürk'ün*: two positions; *başlattığı* → *1928'de*: one).
3. *Participle-to-noun distance*: how far the participle is from the noun it modifies. The participle is normally the *last* word of the relative clause, so this edge is short whatever the clause width: *reformu* → *başlattığı* (`acl`) spans two words here, with only the compound modifier *dil* in between.

A wide relative clause can still have a short participle-to-noun edge. Long arcs may instead connect the participle to an early dependent, or the sentence-final main verb to the subject phrase. Turkic heads usually follow their dependents, with exceptions such as postverbal constituents and some coordination structures. A left-to-right transition-based parser must therefore keep dependents on its stack until their head appears. Throughout this chapter, arrows run from head to dependent.

Clause-boundary cues. The Turkish construction uses a participial suffix where the English example has a relative pronoun. A parser can use a morphological analyser or learn this cue from word representations; an explicit analysis is not required by every architecture.

Left-branching accumulation. A Turkic noun phrase can contain several participial modifiers before its head noun. Each clause may also contain embedded clauses, creating a long sequence before the noun. Exercise 2 measures these sequences in BOUN training data. For nouns that head a participial clause, the median number of preceding words is 4 in biographical, instructional and popular-culture text, 5 in essays and 6 in news. The 90th percentiles range from 10 to 13 words; the maximum is 75.

Despite these long examples, the median participial clause in these samples is shorter than the median relative clause in EN-EWT: 4 versus 6 words. Its participle is also usually adjacent to the noun, with a median distance of 1, compared with 3 for English `acl:relcl`.

8.1.5 Postpositions and the Case-Postposition System

Turkic languages use **postpositions**, which follow rather than precede the noun phrase. A postposition selects a specific case on its nominal complement. For example, Turkish *için* ("for") takes the nominative with nouns (but the genitive with most pronouns: *benim için* "for me"); *sonra* ("after") takes the ablative; *göre* ("according to") takes the dative. The case suffix is carried by the nominal, not by the postposition.

Some dependency traditions treat the postposition as the head of its phrase, but UD does not. In UD, the nominal is the head of an ordinary adpositional phrase. The postposition attaches to the nominal with `case`; the nominal's external relation depends on the construction. Its morphological case and the postposition's identity jointly help identify that construction. For example, in *Ali için geldim* ("I came for Ali"):

```
geldim  --obl-->   Ali
Ali     --case-->  için
```

The verb *geldim* is the root; *Ali* attaches to it as `obl`; *için* attaches to *Ali* as `case`. Inside a noun phrase the nominal would attach as `nmod` instead of `obl`.

8.2 The Universal Dependencies Framework for Turkic

8.2.1 UD's Core Representational Choices

Universal Dependencies (Marneffe et al. 2021; Nivre et al. 2016) represents basic syntax as directed arcs from heads to dependent **syntactic words**. These need not coincide with orthographic tokens: CoNLL-U can use a range ID for a multiword token and integer IDs for its component words. UD v2 defines 37 universal relations; labels such as `acl:relcl` are subtypes, not additional universal relations. The root word is encoded with `HEAD=0` and `DEPREL=root`, where 0 is the artificial root (CoNLL-U specification).

Each word has a universal POS tag (UPOS), an optional language-specific POS tag (XPOS), and morphological feature–value pairs (FEATS). For illustration, a finite verb might have `Mood=Ind|Number=Sing|Person=3|Tense=Past|VerbForm=Fin`. A participle may carry `VerbForm=Part`, tense and possessor agreement; a case feature is appropriate only if that form is actually case-inflected. The exact bundle must follow the language and treebank guidelines.

Table 8.2 lists the morphological features most heavily used in Turkic UD treebanks.

Table 8.2: Core Universal Dependencies morphological features heavily used in Turkic treebanks. The VerbForm feature is especially important: participial (Part) and converbal (Conv) forms are the primary mechanisms for subordinate clauses.

Feature	Values (sample)	Notes
Case	Nom, Acc, Dat, Loc, Abl, Gen	Selected values; inventories differ by language
Number	Sing, Plur	Dual absent in most modern Turkic
Person	1, 2, 3	Verbal agreement; possessor features use the [psor] layer
Tense	Past, Pres, Fut	Interacts with Aspect and Mood
Aspect	Perf, Imp, Prog	Not always distinguished from Tense in UD
Mood	Ind, Cnd, Opt, Imp, Pot, Nec	Indicative, conditional, optative, imperative, potential (ability), necessitative

Feature	Values (sample)	Notes
Voice	Act, Pass, Cau, Rcp, Rfl	Voice categories (and their combinations, e.g. `CauPass`)
VerbForm	Fin, Part, Conv, Vnoun	Finite, Participial, Converbal, Verbal noun
Polarity	Neg	Morphological negation on verbs
Number[psor], Person[psor]	Sing/Plur; 1/2/3	Possessor agreement on nominals

8.2.2 Turkic-Specific Annotation Decisions

The UD guidelines specify the representation for many common constructions but leave certain language-specific cases to individual treebank documentation. Several of these are common in Turkic text.

Light verb constructions. Turkish and other Turkic languages frequently form complex predicates from a nominal and a light verb: *okuma yaptı* ("did reading" = "read"), *yardım etti* ("did help" = "helped"). In the documented UD analysis the light verb is the head and the nominal is its dependent with `compound:lvc`:

```
etti  --compound:lvc-->  yardım
```

This matches the Turkish UD relation documentation, but the released data need to be checked independently. In UD 2.17, examples of *yardım et-* in BOUN (`news_578`), IMST (`00170166_6`) and PUD (`n01134015`) have the **opposite head direction**: the noun *yardım* heads the verb with `compound:lvc` (sentence IDs and POS tags in `tables/tbl-ud217-inventory/light-verbs.csv`). Table 8.3 shows how systematic the convention is: the pattern holds well beyond a few stray examples. In BOUN, IMST, PUD and the two Ottoman Turkish treebanks, nearly every `compound:lvc` arc runs from the nominal to a light verb on its right. In the Turkish and Azerbaijani TueCL sets, in Kazakh-KTB, Tatar-NMCTT and the Uzbek treebanks the verb is the head, as the documentation describes.

Table 8.3: Head direction of `compound:lvc` in the official UD 2.17 Turkic treebanks (all splits pooled; treebanks with no such arc omitted). "Nominal head" means any head whose UPOS is not VERB or AUX, so the light verb depends on the noun or adjective. Counts are an annotation-convention diagnostic, not a parser result. Generated by `tables/tbl-ud217-lvc-direction/generate.py` from hash-verified release files; URLs and hashes are in `data.csv`.

Treebank (all splits)	`compound:lvc` arcs	Verb/AUX head	Nominal head	Dependent before head
Azerbaijani-TueCL	17	17	0	17
Kazakh-KTB	7	7	0	7

Treebank (all splits)	compound:lvc arcs	Verb/AUX head	Nominal head	Dependent before head
Kyrgyz-TueCL	2	2	0	2
Ottoman Turkish-BOUN	248	2	246	0
Ottoman Turkish-DUDU	905	1	904	14
Tatar-NMCTT	36	36	0	36
Turkish-BOUN	1,265	29	1,236	0
Turkish-IMST	523	16	507	0
Turkish-PUD	201	1	200	0
Turkish-TueCL	5	5	0	5
Turkish German-SAGT	94	94	0	94
Uyghur-UDT	268	244	24	263
Uzbek-TueCL	15	15	0	15
Uzbek-UT	215	213	2	211
Uzbek-UzUDT	21	20	1	18

Neither direction is a grammatical fact about Turkish: both are annotation conventions for the same construction, and a parser trained on one learns that convention. A relation name alone is therefore insufficient: compare heads, labels, POS tags and the relevant construction before pooling treebanks or comparing scores across them.

Postposition attachment. As in Section 8.1, the nominal heads the phrase and the postposition attaches to it with `case`. The nominal itself is attached as `obl` when the phrase modifies a predicate and as `nmod` when it modifies a nominal. The nominal carries the case selected by the postposition, which gives partly redundant information for semantic interpretation.

Pro-drop representation. When the subject is omitted, no subject arc appears in the parse graph. The verb's FEATS column records `Number=Sing|Person=3` (or whatever the morphological agreement is), providing information about the absent subject. Basic UD does not introduce empty words for omitted subject pronouns, regardless of language. Enhanced dependencies and empty nodes for ellipsis are a separate representation, not a Turkic-specific exception.

Possessive-existential clauses. One common Turkic possession construction combines a genitive possessor, a possessive suffix and an existential predicate. In the recurring Kyrgyz sentence *Менин машинам бар* 'I have a car' (Kyrgyz-KTMU `001`, Table 1.4), the possessor *Менин* attaches to *машинам* with `nmod`, *машинам* is the `nsubj` of *бар*, and *бар* is the root:

```
бар  --nsubj-->  машинам  --nmod-->  Менин
```

Other analyses of the same release are defensible; this is one of them. Treebanks differ in the part of speech they give the existential (*бар*, Turkish *var*), in whether the possessor is `nmod` or `nmod:poss`, and in what they treat as the head of a predicate without a verb; consult the documentation of the treebank you train on. The same release also tags *машинам* `Case=Gen` and *бар* `VERB` with `Aspect=Perf|Tense=Pres`, although *машинам* has no genitive suffix and *бар* carries no tense or aspect morphology (Section 5.4.4).

These feature values look like annotation errors rather than a convention. Check features as well as relations before training on, or teaching from, a treebank.

Converb (gerund) clauses. Turkish and related languages use forms such as *-arak*, *-ıp*, *-ken* and *-ınca* in adverbial clauses. In this use, the converbal verb attaches to the main predicate with `advcl`, and its own arguments attach to it. Converbs also occur in clause chains and auxiliary constructions (Kazakh *оқып жатыр* 'is reading', where the treebank may treat *жатыр* as `aux` of the converb), so the form alone does not determine the dependency relation.

Coordinated verb phrases. In Turkic, coordinated clauses often share the subject and other arguments, with only the final verb fully inflected, and non-final verbs in a converbal or other non-finite form. UD coordination normally attaches later conjuncts to the *first* conjunct with `conj`, not to the final fully inflected verb. Whether a non-final converb is analysed as a conjunct at all, or as an `advcl` of the final verb, is a treebank-level decision that differs between Turkic treebanks; check the guidelines of the treebank you train on.

8.2.3 Descriptive gold-arc diagnostics

The official UD 2.17 test files support a reproducible descriptive check without training a parser. Table 8.4 and Figure 8.2 use basic heads over integer-ID syntactic words. They exclude root arcs and arcs whose dependent is `PUNCT`, while retaining punctuation positions when measuring distance. Multiword-token headers and empty nodes are excluded. The generator checks every input against the inventory's SHA-256 manifest.

Table 8.4: Basic dependency distance in selected official UD 2.17 test sets, measured in integer-ID word positions. Root and punctuation-dependent arcs are excluded; intervening punctuation still occupies a position. P90 uses nearest rank. All 25 treebanks and source hashes are in the accompanying CSV. These are corpus diagnostics, not parser scores or matched language comparisons.

Treebank	Sentences	Arcs	Mean	Median	P90
Azerbaijani-TueCL	148	582	2.04	1	4
Kazakh-KTB	1,047	6,978	2.61	1	6
Kyrgyz-KTMU	1,222	8,965	2.30	1	6
Tatar-NMCTT	148	1,764	3.07	1	8
Turkish-BOUN	979	9,205	2.76	1	6
Turkish-IMST	1,100	6,999	2.53	1	5
Uyghur-UDT	900	7,386	2.89	1	7
Uzbek-UT	500	4,570	2.66	1	6

The complete CSV also records counts and distance sums for `acl` and its subtypes. That label alone does not identify all and only participial relative clauses: Exercise 2 selects participial modifiers as `acl` arcs whose dependent has `VerbForm=Part`. These counts are not parsing accuracy.

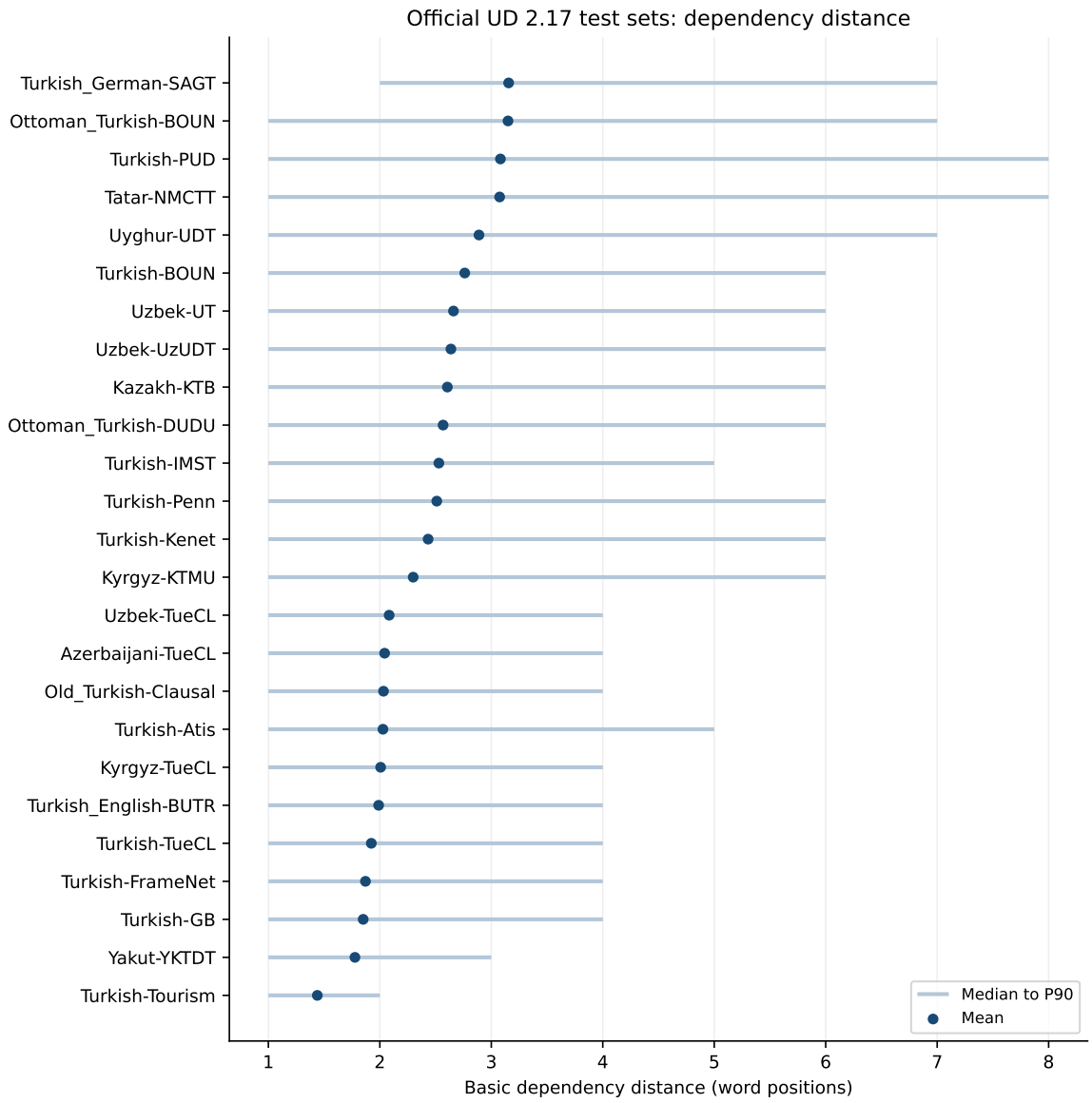


Figure 8.2: Mean, median and 90th-percentile basic dependency distances for the 25 official UD 2.17 test sets in the book inventory. Genre, sentence length, translation and annotation choices differ across treebanks; these distributions do not isolate a language effect.

8.3 Turkic Treebanks

8.3.1 Turkish in UD 2.17

This inventory is pinned to **UD 2.17**, rather than mixing live statistics with older paper counts. It was regenerated from the official release-tag CoNLL-U files. The local training directory also contains derived data; those additions are excluded. The source URLs and SHA-256 hashes are saved under `tables/tbl-ud217-inventory/`.

Table 8.5: UD 2.17 inventory, generated from the release CoNLL-U files by `tables/tbl-ud217-inventory/generate.py`; file hashes are recorded alongside the CSV. A zero means no sentences in that split. Word and token totals sum all splits; multiword-token ranges count once in the token total, and empty nodes count in neither. Historical and code-switched treebanks retain their UD language names.

Treebank	Train sentences	Dev sentences	Test sentences	Syntactic words	Orthographic tokens
Turkish-Atis	4,274	572	586	45,907	45,907
Turkish-BOUN	7,803	979	979	125,212	121,835
Turkish-FrameNet	2,288	205	205	19,223	19,223
Turkish-GB	0	0	2,880	17,177	16,803
Turkish-IMST	3,435	1,100	1,100	58,096	56,422
Turkish-Kenet	15,398	1,646	1,643	178,658	178,658
Turkish-PUD	0	0	1,000	16,881	16,535
Turkish-Penn	14,850	622	924	183,555	183,555
Turkish-Tourism	15,473	2,166	2,191	91,152	91,152
Turkish-TueCL	0	0	148	904	883

Related resources include Ottoman Turkish and Turkish–German and Turkish–English code-switched treebanks, listed separately below.

For general edited and web text, **TR-BOUN** offers several genres and carefully documented Turkish annotation decisions (Türk et al. 2022). Size alone favours other treebanks: TR-Kenet and TR-Penn contain more words, and TR-Tourism more sentences (Table 8.5). Choose according to the evaluation domain; tourism reviews or flight queries may call for a matching domain-specific treebank.

TR-IMST matters for the comparisons below because it supplies the Stanza checkpoint's training data. **TR-PUD** belongs to the Parallel UD (PUD) collection, which annotates the same 1,000 sentences in many languages.

Annotation differs between Turkish treebanks in the treatment of light verb constructions (Table 8.3), copulas and complex predicates. Researchers combining multiple treebanks should verify that these constructions are handled consistently before pooling training data.

8.3.2 Other Turkic Treebanks

Table 8.6: UD 2.17 inventory, generated from the release CoNLL-U files by `tables/tbl-ud217-inventory/generate.py`; file hashes are recorded alongside the CSV. A zero means no sentences in that split. Word and token totals sum all splits; multiword-token ranges count once in the token total, and empty nodes count in neither. Historical and code-switched treebanks retain their UD language names.

Treebank	Train sentences	Dev sentences	Test sentences	Syntactic words	Orthographic tokens
Azerbaijani-TueCL	0	0	148	912	902
Kazakh-KTB	31	0	1,047	10,536	10,495
Kyrgyz-KTMU	1,308	0	1,222	24,128	24,128
Kyrgyz-TueCL	0	0	173	1,250	1,214
Old Turkish-Clausal	0	0	20	158	158
Ottoman Turkish-BOUN	114	0	400	8,851	8,834
Ottoman Turkish-DUDU	20	0	1,762	17,125	16,728
Tatar-NMCTT	0	0	148	2,280	2,280
Turkish English-BUTR	0	0	51	393	393
Turkish German-SAGT	578	801	805	37,227	36,934
Uyghur-UDT	1,656	900	900	40,236	40,236
Uzbek-TueCL	0	0	148	939	939

Treebank	Train sentences	Dev sentences	Test sentences	Syntactic words	Orthographic tokens
Uzbek-UT	0	0	500	5,930	5,930
Uzbek-UzUDT	483	0	198	7,542	7,542
Yakut-YKTDT	0	0	299	1,460	1,459

A treebank that ships only a test split can be used for evaluation. Training on that split is ruled out. Sizes and split availability differ substantially, motivating multilingual training (Section 8.4) and cross-lingual approaches (Section 8.5). The release uses the name **Old Turkish–Clausal**; historical resource names in older publications should not be treated as additional released treebanks.

8.3.3 Languages Without Treebanks

In the pinned UD 2.17 inventory (check newer releases before starting a project), many Turkic languages have no UD treebank, among them Turkmen, Bashkir, Chuvash, Crimean Tatar, Gagauz, Nogai, Karachay-Balkar, Tuvan, Khakas, Shor, and Altai. Karakalpak and Kumyk have sentences in the in-progress UD-Turkic parallel collection but no released treebank yet. For the treebank-less languages, the routes to dependency parsing are multilingual or cross-lingual transfer, silver treebanks, and annotation projection (Section 8.5). In the long run, the route is to build a treebank (Section 8.6).

8.3.4 The UD-Turkic Parallel Treebanks

The UD-Turkic initiative addresses cross-lingual comparison directly. Akhundjanova et al. (2025) annotated the same 148 sentences, grammar-style examples of core constructions, in Azerbaijani, Kyrgyz, Turkish, and Uzbek, released as the TueCL treebanks in UD 2.16. In UD 2.17 the Kyrgyz file has grown to 173 sentences (Table 8.6), so compare the languages on shared sentence IDs. Files for Kazakh, Uyghur, Tatar, Karakalpak, Kumyk, and Sakha are in progress in the initiative's repository (`github.com/ud-turkic/parallel`).

These treebanks chiefly support **comparison and evaluation**. Their parallel content lets researchers compare how annotators handle participial clauses, light verbs, copulas and coordination, and develop shared Turkic annotation guidelines. They also supply manually annotated test sets. TurkicNLP evaluates parsers on ten languages using the released sets and the in-progress files (Section 8.4).

8.4 Dependency Parsing for Turkic

8.4.1 Transition-Based Parsing

Transition-based dependency parsing (Jurafsky and Martin 2024, chapter "Dependency Parsing") processes the input sentence left to right, maintaining a *stack* of partially built structure and a *buffer* of unprocessed words. In the *arc-standard* system, each step applies one of three actions, where s_0 is the top of the stack and s_1 the item beneath it: *shift* (move the next word from the buffer onto the stack); *left-arc* (make s_0 the head of s_1, a leftward arc $s_0 \rightarrow s_1$, and pop s_1); or *right-arc* (make s_1 the head of s_0, a rightward arc $s_1 \rightarrow s_0$, and pop s_0). In a head-final sentence most arcs are built by *left-arc*, after the head has been shifted.

Early data-driven Turkish parsers included MaltParser, which Eryiğit, Nivre, and Oflazer (2008) applied to inflectional groups: the subword units between derivational boundaries. Arc-standard decoding takes a linear number of transitions, with encoder and search costs counted separately. In an SOV sentence, the parser holds the subject and object on its stack until the main verb arrives. Long participial clauses can deepen that stack and increase opportunities for early errors to propagate.

Modern transition-based parsers can score transitions with a bidirectional LSTM or transformer that has already read the whole sentence, including the final verb. Greedy decoding can still propagate errors; beam search keeps alternatives. With contextual encoders, transition-based and graph-based parsers show similar accuracy and error profiles (Kulmizev et al. 2019).

8.4.2 Graph-Based (Biaffine) Parsing

Graph-based parsing (Dozat and Manning 2017) takes a different approach: encode the entire sentence at once, then score all possible dependency arcs globally. Biaffine graph-based parsers are among the most widely used architectures for high-accuracy dependency parsing, including the Stanza parser (Qi et al. 2020).

The architecture proceeds in three stages:

1. Contextual encoding. A bidirectional LSTM or transformer encoder processes the full sentence and produces a contextual representation for each token. The encoder sees the entire sentence, including the verb at the end, before any attachment is scored. (Transition-based parsers can use the same kind of encoder; the difference lies in how attachments are decided, not in what the encoder sees.)

2. Biaffine arc scoring. For each ordered pair of tokens (i, j), compute an arc score:

$$s_{\text{arc}}(i, j) = h_j^{(\text{head})^\top} U h_i^{(\text{dep})} + h_j^{(\text{head})^\top} u$$

the score for word j being the head of word i, where $h_i^{(\text{dep})}$ and $h_j^{(\text{head})}$ are learned dependent and head representations derived from the encoder output (via separate MLPs), U is a bilinear weight matrix and u an affine term that captures how likely word j is to take

dependents at all (Dozat and Manning 2017). Implementations differ in the exact affine terms. A separate biaffine classifier scores the dependency label for each arc.

3. Maximum spanning tree decoding. A common non-projective decoder finds a maximum spanning tree from arc scores with Chu–Liu/Edmonds, with root constraints, then assigns labels to the chosen arcs. Other implementations may score labelled arcs jointly or use projective decoding; the biaffine scorer alone does not determine the decoder. Because the tree is chosen globally, a long arc (say, from the sentence-final verb back to a subject that contains a wide relative clause) is scored directly rather than built after a sequence of earlier greedy decisions.

Stanza (Qi et al. 2020) provides BiLSTM-based biaffine parsers for Turkic UD treebanks. Its 1.11.0 model catalogue, used for the measurements in this chapter, covers five Turkic languages: Turkish, Kazakh, Kyrgyz, Uyghur, and Ottoman Turkish (plus Turkish–German code-switching). The 1.15.0 catalogue adds an Uzbek (UzUDT) model and a second Ottoman Turkish model (DUDU). TurkicNLP uses Stanza's own models for the first four languages, adds its own Stanza models for Azerbaijani, Uzbek, Turkmen, Tatar, and Bashkir (trained on silver treebanks where no UD data existed), and provides a multilingual biaffine parser on a frozen Glot500 encoder. That parser is trained on ten languages and covers 15, five of them zero-shot (Hakimov 2026).

8.4.3 Hard Cases for Turkic Parsers

Evaluation on Turkic UD treebanks reveals consistent difficult constructions:

Long arcs around participial clauses. As described in Section 8.1, the participle-to-noun edge is usually short; the difficult arcs are those from early dependents of a wide relative clause to its participle, and from the sentence-final verb back to a subject that contains such a clause. Attachment accuracy generally falls as arc length grows; in the classic comparison, the graph-based MSTParser was more precise on long arcs than the greedy transition-based MaltParser (McDonald and Nivre 2007). The fall is steep for Turkish. Exercise 2 runs the Stanza Turkish (IMST) parser on the BOUN test set from gold words: UAS is 83.3% on arcs between adjacent words, 67.7% at distance 2, 59.2% at 3–4, 54.2% at 5–7 and 43.9% at distance 8 or more. These are cross-treebank scores for a single graph-based parser; with contextual encoders, transition-based parsers show a similar error profile (Kulmizev et al. 2019).

Light verb constructions. The difference in head direction between treebanks (Table 8.3) propagates into parser confusion: a model trained on IMST or BOUN learns that the light verb depends on its noun, while the Turkish and Azerbaijani TueCL test sets expect the opposite. BOUN and IMST share the dominant head direction. Pooling them with TueCL, or evaluating across these conventions, requires checking and documenting the mismatch. The hands-on example below shows the IMST convention in a trained model.

Coordination. Where non-final verbs appear in converbal form, the parser must learn the training treebank's decision about whether such a verb is a conjunct (with later conjuncts attached to it) or an adverbial clause. The verb form alone does not settle this distinction.

Postposition attachment ambiguity. A postpositional phrase may attach to the verb as an *obl* modifier or to a nominal as an *nmod* modifier. Disambiguation requires semantic knowledge about the postposition's typical modifier type and the context.

8.4.4 Evaluation Results: Per-Language vs. Multilingual Parsing

Table 8.7 compares per-language Stanza models with TurkicNLP's multilingual parser. It records the local manuscript snapshot of 13 October 2026 (`tables/tbl-parsing-results/source.tex`). Some cells and sample sizes differ from the cited arXiv version (Hakimov 2026), so the table should be read as that frozen diagnostic.

The first two groups use gold UD-Turkic parallel test sets. The third uses silver Turkmen and Bashkir sets produced by automatic translation and annotation. Both backends receive the same automatic tokenisation. UPOS is scored on words whose boundaries match the gold tokenisation; UAS and LAS use the subset whose gold and predicted heads also match in their word boundaries. UAS counts correct heads, and LAS correct heads and labels.

These scores are conditional on alignment. Tokenisation failures remove words from the denominator. The scores therefore differ from the official CoNLL 2018 tokenisation-aware LAS F1 and cannot be compared directly with it.

Table 8.7: POS tagging and dependency parsing from the frozen local manuscript table of 13 October 2026 (`tables/tbl-parsing-results/source.tex`), associated with Hakimov (2026). This snapshot differs from the public arXiv table in some cells and sample sizes. Both backends receive the same automatic tokenisation. These are *conditional* scores, not the official tokenisation-aware CoNLL 2018 LAS F1: UPOS is computed over the n words whose boundaries match gold, and UAS/LAS additionally require matched gold and predicted heads, so tokenisation errors drop out of the denominator. †Zero-shot with proxy embeddings: Karakalpak → Uzbek, Kumyk → Tatar, Sakha → Bashkir. ‡Silver test sets (translated and automatically annotated); these mainly measure agreement with the process that produced them. Tatar (29 words), Sakha (16), Kumyk (144) and Karakalpak (235) are qualitative examples, not grounds for ranking. Bold: higher score of the two backends. The source's aggregate is omitted because it mixes gold and silver rows.

Language	Matched words (n)	Stanza UPOS	Stanza UAS	Stanza LAS	Multilingual UPOS	Multilingual UAS	Multilingual LAS
Gold UD-Turkic parallel test sets — supervised							
Turkish	877	85.1	75.3	63.5	**86.9**	**84.5**	**70.1**
Azerbaijani	869	84.9	72.3	60.8	**87.0**	**76.2**	**64.5**

Language	Matched words (n)	Stanza UPOS	Stanza UAS	Stanza LAS	Multilingual UPOS	Multilingual UAS	Multilingual LAS
Kazakh	851	87.9	67.4	57.4	**88.8**	**71.9**	**59.9**
Kyrgyz	1,166	71.1	50.2	29.4	**79.0**	**63.9**	**41.0**
Tatar	29	**86.2**	69.2	57.7	79.3	**77.8**	**70.4**
Uyghur	758	76.9	77.1	62.3	**79.7**	**81.5**	**66.7**
Uzbek	905	71.8	67.2	46.3	**87.6**	**81.5**	**68.3**
Gold UD-Turkic parallel test sets							
— *zero-shot*[†]							
Karakalpak[†]	235	—	—	—	77.5	61.7	45.8
Kumyk[†]	144	—	—	—	78.5	60.4	45.5
Sakha[†]	16	—	—	—	93.8	60.0	46.7
Silver test sets[‡]							
Turkmen[‡]	825	**86.1**	76.8	67.8	85.8	**77.9**	**68.4**
Bashkir[‡]	814	87.0	72.8	62.5	**89.6**	**75.8**	**63.8**

The multilingual parser has higher conditional LAS in every language covered by both systems; UPOS results are mixed. Architecture, encoder and training data also differ, so a controlled experiment would be needed to attribute this advantage to parameter sharing.

The zero-shot Karakalpak, Kumyk and Sakha rows reach conditional LAS in the mid-40s. They demonstrate transfer on these small samples; representative, manually checked data would be needed to assess practical usefulness. Treat the Sakha (16 words) and Tatar (29 words) rows as qualitative examples. A single word shifts their percentages by several points.

Scores are modest even for Turkish on these short, construction-dense parallel sentences. Scores from a language's own treebank test set, such as BOUN, describe a different evaluation setting.

8.4.5 Measured official-treebank baselines

A separate evaluation runs cached Stanza checkpoints on the complete official **UD 2.17 test files** for Kazakh-KTB, Kyrgyz-KTMU, Turkish-IMST and Uyghur-UDT. These are different evaluation collections from the parallel-treebank comparison above. The first condition supplies gold sentence and syntactic-word boundaries, but only word forms enter the model: POS, features, lemmas and dependencies are predicted.

Table 8.8: Measured pretrained Stanza scores (%) on official UD 2.17 test files, using gold sentence and syntactic-word boundaries and predicted tags/features. The UD scorer includes punctuation in UAS/LAS; all denominators are retained. † Kazakh is a replay diagnostic with split-exposure risk, not held-out accuracy. These are not raw-text or newly trained results. Source: `run_parsing.py`.

Test treebank	Words	UPOS	UFeats	UAS	LAS
Kyrgyz-KTMU	11771	90.40	71.78	82.98	71.17
Turkish-IMST	10032	95.18	92.47	78.37	70.23
Uyghur-UDT	10330	89.69	88.38	76.72	65.29
Kazakh-KTB† (replay, not held-out)	10007	98.74	98.23	96.83	95.57

The official UD scorer includes punctuation and uses coarse dependency relations. Task checkpoint MD5s were checked against the local Stanza resource manifest.

The Kazakh row is a replay diagnostic, not held-out accuracy. Stanza 1.11.0's `process_partial_ud_treebank` routine swaps training and test inputs for treebanks without development data when training contains at most 1,000 words and test more than 5,000. Kazakh-KTB meets this rule. The checked checkpoint configuration names the resulting `kk_ktb.train.in.conllu` and development files, though their historical sentence membership is unavailable. This specific exposure risk prevents interpreting 95.57% as held-out accuracy or comparing it with the other three rows. The audit script, source hash and rule are saved in `stanza_split_audit.json`.

The other three current treebanks do not trigger the swap rule; their historical checkpoint membership has not been independently reconstructed either. Differences in treebanks, annotation distributions and checkpoints also prevent using the table to rank languages by difficulty.

A second pass supplies each gold sentence as text and predicts tokenisation, multiword-token expansion where supported, and all downstream annotations. The official scorer aligns predicted and gold words, retaining segmentation errors in the denominator.

Table 8.9: Measured Stanza scores (%) on the same official UD 2.17 tests. Both conditions retain gold sentence boundaries; the automatic condition predicts tokenization/MWT and uses the official alignment-aware scorer. † Kazakh has split-exposure risk and is not a held-out score. Gold-word LAS is repeated from the preceding table. Source: run_parsing.py and run_parsing_auto.py.

Test treebank	Gold-word LAS	Auto word F1	Auto UAS	Auto LAS
Kyrgyz-KTMU	71.17	99.12	82.30	70.62
Turkish-IMST	70.23	97.26	73.57	65.95
Uyghur-UDT	65.29	99.48	76.17	64.80
Kazakh-KTB† (replay, not held-out)	95.57	99.80	96.57	95.34

Both conditions still supply gold **sentence boundaries**, so the automatic condition does not measure document-level sentence segmentation. No test sentence is dropped or

rewritten to force alignment. The Kyrgyz file contains ten `# text =` lines without a space after the equals sign; the loader accepts both metadata spellings and preserves the text. Comparing the two columns measures the effect of the full tokenisation pipeline in these checkpoints, not just isolated token-boundary accuracy.

8.5 Cross-Lingual Parsing

8.5.1 Zero-Shot and Proxy-Language Transfer

In zero-shot parsing, a model trained on one or more Turkic treebanks is applied to a language absent from that training. A multilingual encoder supports this setup. The ten-language parser in Table 8.7 reaches conditional LAS in the mid-40s on small Karakalpak, Kumyk and Sakha samples, using a related language's proxy embedding (Hakimov 2026). Exercise 4 tests a single-source baseline on gold words: the Stanza Turkish (IMST) parser scores LAS 43.2 on Azerbaijani TueCL and 64.3 on the Turkish versions of those sentences. Both experiments show transfer, but their different test sets and conditions prevent a direct comparison of the approaches.

The principal failure modes are:

Label conventions. Even when the parser finds the right head, it often assigns a label according to the source treebanks' conventions. Language-specific annotation decisions (light verbs, copulas, compound nouns) differ between Turkic treebanks, and the parser cannot infer the target language's conventions without target data.

Unseen function words and constructions. Postpositions, particles and auxiliaries can differ between related languages. Check their attachment errors separately rather than assuming that familiar suffix patterns are enough for transfer.

Script mismatch. An unfamiliar script can reduce the lexical information available to an encoder. Shared syntactic patterns may still transfer, but script coverage and parsing accuracy each need their own check.

8.5.2 Script Normalisation

A Turkish-only encoder may represent Cyrillic Kazakh or Arabic-script Uyghur poorly. A multilingual encoder can already know both scripts even if its parsing fine-tuning used only Turkish. Two options are worth comparing: convert input to the model's script and map predictions back, or train across scripts with script-specific adapters. TurkicNLP supports input-to-model script bridging within a language (Section 3.4); applying a Turkish model to another language still needs an explicit mapping and evaluation.

Shared-script conversion can increase cognate overlap (Section 3.6). Exercise 4 tests its effect on the Stanza Turkish parser with gold words. On Kyrgyz TueCL, LAS rises from 6.2 with Cyrillic input to 22.4 with word-by-word CTS conversion. On Kazakh KTB, it rises from 9.1 to 18.1 with the 2021 Kazakh Latin alphabet and to 27.2 with CTS. Conversion helps in these runs, though the in-language Kyrgyz KTMU model still scores 29.1 on the same Kyrgyz sentences.

8.5.3 Silver Treebanks and Annotation Projection

When a language has no treebank, training data can be created automatically.

Silver treebanks by translation. Translate sentences from an existing treebank (e.g., a Turkish training split) into the target language with an MT system or an LLM, and annotate the translations automatically with the best available analysers and parsers, followed by automatic validation. TurkicNLP's Turkmen and Bashkir models were trained this way (Hakimov 2026). Silver data is noisy and inherits the source treebank's sentence types and annotation decisions. A test set built the same way (the ‡ rows in Table 8.7) mainly measures agreement with the process that produced it, so it gives only indicative scores.

Two rules prevent leakage. Split the source material by sentence or document *before* translating, projecting or producing silver labels, so that no translation of a test sentence ends up in training data. And never train on translations of the sentences used for evaluation, including published parallel test sets such as UD-Turkic or PUD. FLORES-200 is an evaluation resource: use only its designated development split, if any, for tuning, and keep its test split untouched.

Annotation projection uses an existing parallel corpus instead:

1. Parse the source side (e.g., Turkish) with a high-quality parser.
2. Align source and target words with a word aligner (e.g., awesome-align or fast_align).
3. Project dependency arcs through the alignment: if source words i and j are aligned to target words i' and j', and there is an arc (i, j) in the source parse, propose the arc (i', j') in the target, then repair the result into a valid tree.

The quality of projection depends heavily on alignment precision and coverage, which must be measured for the pair and domain; related vocabulary and similar constituent order can help but do not ensure one-to-one alignment. Exercise 5 projects the Turkish TueCL gold trees onto the Azerbaijani versions of the same 148 sentences with a simple string-similarity aligner. Only 39.4% of the Azerbaijani words receive the right head and label overall, but 83.5% do among the words whose own link and whose head's link both exist; with Stanza parses instead of gold trees the figures fall to 30.9% and 65.6%. In this experiment, restricting evaluation to aligned words hides much of the projection failure. Report the all-word score alongside the conditional one. Parallel training text is available, for example, from the TIL corpus (Mirzakhalov, Babu, Ataman, et al. 2021) and OPUS; FLORES-200 should be reserved for evaluation.

8.6 Building a Treebank for a New Turkic Language

A manually checked treebank provides a reference for evaluating transfer and, if it includes training data, for adaptation. Plan its purpose, genres and annotation conventions before collecting sentences. One possible workflow is:

1. **Choose the sentences.** Start with the UD-Turkic parallel sentences (Akhundjanova et al. 2025): they come with annotated counterparts in related languages, so every decision can be compared, and they immediately yield a parallel test set. Then add

natural text (news, Wikipedia, fiction) with a clear licence, ideally a few hundred sentences from at least two genres.
2. **Document the guidelines.** Write down the decisions for the constructions where Turkic treebanks diverge: light verb constructions (`compound:lvc` or not), copular clauses and the copula clitics, multi-word token splitting of clitics and suffixed particles, converb chains and coordination, and the treatment of possessive compounds (*dil reform-u*). Start from the UD guidelines and the target language's grammar. Compare nearby treebanks, document the chosen conventions and explain departures.
3. **Pre-annotate automatically.** Run the best available pipeline (the multilingual parser with a suitable proxy language, a related language's model after transliteration, or projected annotation) and correct its output rather than annotating from scratch. Record which tool produced the pre-annotation.
4. **Annotate with a UD tool** such as Arborator-Grew or ConlluEditor, which validate the format as you go.
5. **Double-annotate a sample.** Have two annotators annotate the same 50–100 sentences independently and report agreement (UAS/LAS between annotators and agreement on UPOS and features); discuss disagreements and update the guidelines.
6. **Validate and release.** Run the official UD validation script, write the README and metadata (sources, licence, annotators, pre-annotation tools), and submit to Universal Dependencies so the treebank appears in the regular releases.

Case study: a Turkmen treebank. Turkmen has a mature Latin orthography, an Apertium transducer and a silver Stanza model, but no released treebank in the UD 2.17 inventory. One approach would translate Turkish TueCL and IMST sentences into Turkmen, using NLLB drafts corrected by native speakers. The Apertium morphological analyser alone cannot translate Turkish into Turkmen.

Next, pre-annotate the translations with the silver model and multilingual parser, and project the Turkish gold annotation for comparison. Trained speakers would correct the results using documented Turkmen guidelines aligned with UD. Fix the split by source sentence before starting: TueCL and IMST test translations belong in test data, and IMST training translations in training data. This would yield a manually checked parallel test set alongside corrected training material.

8.7 Hands-On: Parsing Turkish Text with TurkicNLP

Question. Which dependencies does a pretrained parser recover, and how does it transfer?

Inputs. The example sentences, Turkish PUD and Azerbaijani TueCL, with the declared parser resources.

Compute. CPU inference; download the relevant Stanza/toolkit models.

Output. CoNLL-U predictions and attachment scores under a stated alignment policy.

Interpretation. Inspect roots, relative clauses and cross-language lexical errors alongside aggregate scores.

This tutorial parses Turkish with the Stanza biaffine parser (Qi et al. 2020) through TurkicNLP, reads and writes CoNLL-U, and evaluates on the Turkish PUD test set. It then applies the Turkish parser to Azerbaijani text as a zero-shot experiment and compares it with the Azerbaijani and multilingual parsers in TurkicNLP.

8.7.1 Parsing a Turkish Sentence

```
# pip install "turkicnlp[stanza]==0.5.0"   # Stanza backend; models download from the
 ↪  Hugging Face Hub
import turkicnlp

turkicnlp.download("tur", processors=["tokenize", "pos", "lemma", "depparse"])
nlp_tr = turkicnlp.Pipeline("tur", processors=["tokenize", "pos", "lemma", "depparse"],
                            pos_backend="stanza", lemma_backend="stanza",
 ↪  depparse_backend="stanza")

sentence = "Atatürk'ün 1928'de başlattığı dil reformu Türkiye'yi kökten değiştirdi."
doc = nlp_tr(sentence)

print(f"{'ID':<4} {'FORM':<14} {'LEMMA':<10} {'UPOS':<6} {'HEAD':<5} DEPREL")
for word in doc.words:
    print(f"{word.id:<4} {word.text:<14} {word.lemma:<10} {word.upos:<6} {word.head:<5}
     ↪  {word.deprel}")
```

Output (TurkicNLP 0.5.0 with the Stanza IMST models):

```
ID   FORM           LEMMA      UPOS   HEAD  DEPREL
1    Atatürk'ün     Atatürk    PROPN  3     nsubj
2    1928'de        1928       NUM    3     obl
3    başlattığı     başla      VERB   5     acl
4    dil            dil        NOUN   5     nmod:poss
5    reformu        reform     NOUN   7     nsubj
6    Türkiye'yi     Türkiye    PROPN  7     obj
7    kökten         kök        NOUN   0     root
8    değiştirdi     değiş      VERB   7     compound
9    .              .          PUNCT  7     punct
```

The relative-clause attachment matches the intended analysis. The participle *başlattığı* (word 3) takes *Atatürk'ün* and *1928'de* as dependents. It attaches as `acl` to *reformu* (word 5), with *dil* between them.

The main-clause analysis is wrong. The parser treats *kökten değiştirdi* 'changed radically' as a noun-plus-verb predicate: it makes *kökten* the root and attaches the verb with `compound`. The intended root is *değiştirdi*, with *kökten* as an adverbial modifier. This resembles IMST's nominal-headed complex-predicate convention (Table 8.3), although a single example is too little to identify the cause.

The output was reproduced exactly on 19 October 2026 with Stanza 1.11.0 and the same IMST checkpoints. Its lemmas also reduce the causative stems *başlat-* and *değiştir-* to *başla-* and *değiş-*. Check the treebank's lemma convention before counting those reductions as errors.

8.7.2 Reading and Writing CoNLL-U

Every TurkicNLP document can be written as CoNLL-U, and CoNLL-U files can be read back into documents:

```
from turkicnlp.utils.conllu import read_conllu, write_conllu

write_conllu(doc, "output.conllu")            # or: print(doc.to_conllu())

for sent in read_conllu("output.conllu")[0].sentences:
    roots = [w for w in sent.words if w.head == 0]
    if len(roots) != 1:
        raise ValueError("expected exactly one root")
    root = roots[0]
    by_id = {w.id: w for w in sent.words if isinstance(w.id, int)}
    print(f"Root: {root.text!r}  ({root.deprel})")
    print(f"Sentence length: {len(sent.words)} words")
    # ordinary words only: integer IDs with a non-root head
    # (multiword-token ranges and empty nodes are not Word objects with integer IDs)
    arcs = [(abs(w.id - w.head), w) for w in sent.words
            if isinstance(w.id, int) and isinstance(w.head, int) and w.head > 0]
    max_arc = max((span for span, _ in arcs), default=0)
    longest = [f"{by_id[w.head].text}->{w.text}" for span, w in arcs if span == max_arc]
    print(f"Longest non-root arc span: {max_arc} ({', '.join(longest)})")
    for head, rel, dep in sent.dependencies:     # (head word, relation, dependent)
        print(f"  {head.text if head else 'ROOT':<12} --{rel}--> {dep.text}")
```

Computed from the printed tree (nine words including the final punctuation), the longest non-root arc spans two words, and four arcs tie: *başlattığı* → *Atatürk'ün* (3→1), *reformu* → *başlattığı* (5→3), *kökten* → *reformu* (7→5) and *kökten* → *.* (7→9). It is that short partly because the parser wrongly made *kökten* the root and attached the subject *reformu* to it. With the intended analysis, the arc *değiştirdi* → *reformu* (8→5) would span three words. Arc lengths are a quick check on where a parser has placed the long dependencies, but in short sentences like this one they are small whatever the analysis.

8.7.3 Evaluating on the Turkish PUD Test Set

The pinned Turkish PUD file contains 1,000 gold-annotated sentences (Table 8.5). Record the parser package/model hashes, UD release and scorer revision with the output. Parse the raw text of the test set, write CoNLL-U, and score it with the official CoNLL 2018 shared task evaluation script (`conll18_ud_eval.py`), which handles tokenisation mismatches between gold and predicted files:

```
import subprocess
import sys
from turkicnlp.utils.conllu import write_conllu

# raw text of tr_pud-ud-test.conllu (the "# text =" lines)
from pathlib import Path
blocks = Path("tr_pud-ud-test.conllu").read_text(encoding="utf-8").strip().split("\n\n")
texts = [[line[9:] for line in block.splitlines() if line.startswith("# text = ")]
         for block in blocks]
if not texts or any(len(lines) != 1 for lines in texts):
```

```
    raise ValueError("expected one original text per gold sentence")
raw = "\n".join(lines[0] for lines in texts)
write_conllu(nlp_tr(raw), "tr_pud_predicted.conllu")

try:
    result = subprocess.run(
        [sys.executable, "conll18_ud_eval.py", "-v",      # same Python as this session
         "tr_pud-ud-test.conllu", "tr_pud_predicted.conllu"],
        capture_output=True, text=True, check=True,
    )
except subprocess.CalledProcessError as err:
    print(err.stderr)                                     # e.g. text mismatch between files
    raise
print(result.stdout)
# Reports tokenisation F1, UPOS, UFeats, AllTags, Lemmas, UAS, LAS, MLAS, BLEX
```

8.7.4 Zero-Shot Application to Azerbaijani

Azerbaijani uses the Latin alphabet, the same script as Turkish, so a Turkish parser can be applied directly. Compare it with the Azerbaijani-specific and multilingual parsers in TurkicNLP:

```
az_sentence = "əhməd dünən aldığı kitabı bu gün oxudu."
# "Today Ahmad read the book he bought yesterday."

parsers = {
    "Turkish Stanza (zero-shot)": nlp_tr,
    "Azerbaijani (custom Stanza)": turkicnlp.Pipeline(
        "aze", processors=["tokenize", "pos", "depparse"], pos_backend="stanza",
        depparse_backend="stanza"),
    "Multilingual (Glot500)": turkicnlp.Pipeline(
        "aze", processors=["tokenize", "pos", "depparse"],
        pos_backend="multilingual_glot500_model",
        depparse_backend="multilingual_glot500_model"),
}
for name, nlp in parsers.items():
    print(name)
    for w in nlp(az_sentence).words:
        print(f"  {w.text:<14} {w.upos:<6} {w.head} {w.deprel}")
```

For the Turkish parser, the Stanza IMST models (Stanza 1.11.0, run directly rather than through TurkicNLP) give:

```
Turkish Stanza (zero-shot)
  əhməd          PROPN  3 nsubj
  dünən          NOUN   3 obj
  aldığı         VERB   4 acl
  kitabı         NOUN   0 root
  bu             DET    6 det
  gün            NOUN   7 obl
  oxudu          NOUN   4 conj
  .              PUNCT  7 punct
```

The participial clause *dünən aldığı kitabı* is found, but the Azerbaijani words that differ from Turkish fail: *dünən* ‘yesterday’ (Turkish *dün*) becomes an object, and the main verb

oxudu ‘read’ (Turkish *okudu*) is tagged as a noun and attached as a conjunct of *kitabı*, which becomes the root. Running the same loop prints the Azerbaijani and multilingual parsers’ analyses for comparison.

Scoring the three parsers on the Azerbaijani TueCL test set (Akhundjanova et al. 2025) with `conll18_ud_eval.py` turns this comparison into an experiment (Exercise 4).

> **Extending to Cyrillic Turkic Languages**
>
> To parse Kazakh or Kyrgyz text with the Turkish model, first convert the input to a Latin-based script with the toolkit’s transliteration mapping (Kazakh has several published Latin designs, so record which mapping is used; or use CTS for both languages; Chapter 3), apply the Turkish pipeline, and map the analyses back to the original word forms. Whether transliteration helps for a given language pair is an empirical question: compare the native-script multilingual parser, the transliterated Turkish parser and, where available, an in-language model on the same test set (Exercise 4). For Kazakh, `turkicnlp.Pipeline("kaz", processors=["tokenize", "pos", "lemma", "depparse"])` uses the Stanza model trained on the Kazakh KTB treebank, and the multilingual Glot500 parser covers Kazakh, Kyrgyz and the zero-shot languages Karakalpak, Kumyk, Sakha, Karachay-Balkar and Nogai.

Status: demonstration. The sentence analyses above are recorded output. Exercise 4 records the separate scored transfer comparison.

What to Take Forward

Dependency parsing recovers sentence structure from morphology, position and context, and UD makes the labels comparable while leaving language-specific choices visible. The measured patterns are clear: among BOUN training sentences with an overt subject and object, 87.1% are SOV, and the Stanza Turkish parser’s attachment score falls from 83.3% for adjacent words to 43.9% at distances of eight or more. Choose training and test treebanks from the named release inventory, and report alignment and scoring conventions with every score. Chapter 9 applies the same care to word-level labels and entity spans.

Further Reading

For the linguistic foundations of Turkic syntax, Johanson (2021) (Chapters 51–56, from p. 788; especially §54.7, §55 and §§56.3–56.4) discusses pro-drop, scrambling, participial clauses, converbs, postpositions, and coordination. The computational consequences of SOV order and participial subordination for parsing are discussed in the context of Turkish NLP by Oflazer and Saraçlar (2018); Eryiğit, Nivre, and Oflazer (2008) studies Turkish dependency parsing over inflectional groups, and Hoffman (1994) relates Turkish word orders to topic and focus.

The Universal Dependencies framework is defined in Marneffe et al. (2021), which covers the annotation philosophy, the design of UD relation types, and cross-linguistic consistency. The original multilingual treebank collection is described in Nivre et al. (2016). The Turkish BOUN treebank, a multi-genre treebank widely used for Turkish parser training, is described in detail in Türk et al. (2022). The Kazakh UD treebank and its annotation principles are described in Tyers and Washington (2015). The UD-Turkic parallel treebanks enabling cross-language annotation comparison are described in Akhundjanova et al. (2025). Per-language and multilingual parsing on twelve Turkic languages, including three zero-shot languages and two silver test sets, is evaluated in Hakimov (2026).

The biaffine dependency parser, one of the most widely used graph-based architectures, is introduced in Dozat and Manning (2017). McDonald and Nivre (2007) compare the error profiles of graph-based and transition-based parsers, and Kulmizev et al. (2019) revisit the comparison with contextual embeddings. Stanza, which provides accessible biaffine parsers for multiple Turkic languages, is described in Qi et al. (2020). TurkishDelightNLP (Alecakir, Bölücü, and Can 2022) uses LSTM-based joint morphology and syntax modelling for Turkish; check its reported scores against the UD release and metric you use.

Chapter 16 of this book extends the cross-lingual parsing discussion to a full treatment of transfer strategies by resource level.

Exercises

1. **Scrambling and case.** Write a Python function that takes a Turkish sentence in CoNLL-U format, identifies the subject (nsubj), object (obj), and root verb, and reports their relative positions (subject-before-verb? object-before-verb?). Apply it to the first 100 sentences of the TR-BOUN training split that contain an overt subject and an overt object of the root (report how many sentences you had to skip). What percentage of sentences have canonical SOV order? What order variants appear, and how often?

2. **Participial arc length.** First inspect how the treebank you choose labels relative clauses: some Turkish treebanks use plain `acl` for participial modifiers, so an empty `acl:relcl` query does not mean there are no relative clauses. Using TR-BOUN gold data read with `read_conllu` (and, separately, the TurkicNLP Turkish pipeline), compute for each participial modifier the three quantities of Section 8.1: clause width, the longest dependent-to-participle distance, and the participle-to-noun distance. Compare the participle-to-noun distance with the distance of `acl:relcl` arcs in English UD (EN-EWT). Plot the distributions and report the median and 90th percentile of each. Which of the three quantities differs most between the languages?

3. **Pro-drop counting.** Read the Turkish IMST training set with `turkicnlp.utils.conllu.read_conllu` (gold annotation). Select eligible predicates: finite verbs (`VerbForm=Fin` or no VerbForm with Person/Number agreement) that are not imperatives and not passive. A missing `nsubj` is not proof of pro-drop: also count `nsubj:pass`, `csubj` and other subject relations, and treat nonfinite clauses separately. For eligible predicates, compute the proportion with an overt subject per tense-mood combination (e.g., `Tense=Past|Mood=Ind`). Manually adjudicate a random sample of

50 “subjectless” cases and report how many are genuine pro-drop versus annotation errors or other constructions. Then parse the same text with the TurkicNLP Turkish pipeline: does the parser reproduce the gold proportions?

4. **Cross-lingual parser evaluation.** Apply the Turkish parser (`turkicnlp.Pipeline("tur", processors=["tokenize", "pos", "depparse"])`) to the Kyrgyz TueCL and Kazakh KTB test sets and compute UAS and LAS. Repeat after conversion to the 2021 Kazakh Latin alphabet or CTS for Kazakh, and CTS for Kyrgyz, which has no toolkit Latin mapping.

   Keep a verified word-by-word map to the original. Restore FORM values only for one-to-one alignments before scoring: `conll18_ud_eval.py` aligns text characters, so converted strings cannot be scored against the original gold file directly. If tokenisation changes, reconstruct the original character spans explicitly; list position alone is insufficient.

   Compare with Kyrgyz KTMU on Kyrgyz TueCL and Kazakh KTB on KTB. Treat the latter as an exposure-risk diagnostic because historical training membership remains unknown (Section 8.4). Also test the multilingual parser (`depparse_backend="multilingual_glot500_model"`) on original Cyrillic input and the three hands-on parsers on Azerbaijani TueCL. How does LAS change, and which dependency relations cause the most errors?

5. **Annotation projection experiment.** Projected arcs can only be scored against gold annotation of *the same* target sentences. The Turkish and Azerbaijani TueCL treebanks (Akhundjanova et al. 2025) annotate the same sentences, so they provide such pairs. Parse the Turkish side with the TurkicNLP Turkish pipeline (and, as a separate comparison, use the Turkish gold trees), align words with fast_align or awesome-align, project the arcs to Azerbaijani, repair the result into a tree, and score it against the Azerbaijani gold. What fraction of arcs project correctly? Which dependency types project most reliably? How much of the error comes from alignment rather than from the Turkish parse?

Part V

Applications

9 Sequence Labelling: POS Tagging, NER, and Chunking

Learning Objectives

After reading this chapter, you will be able to:

- Explain how POS tagging and morphological disambiguation interact in Turkic and when joint modelling is worth testing
- Describe the case suffix problem in Turkic NER, compare word-level and morpheme-level BIO tagging, and separate mention detection from name normalisation and entity linking
- Characterise the existing Turkic NER datasets (KazNERD, KyrgyzNER, Turkish datasets, WikiANN and others) as a dated inventory
- Prepare a reproducible NER fine-tuning procedure with XLM-R and the Hugging Face Trainer API, and name micro and macro F1 correctly when reporting results
- Design cross-lingual NER experiments that separate script, entity-distribution, domain and label-scheme effects
- Describe Turkish PropBank and distinguish overt-argument SRL from implicit-argument recovery in pro-drop languages

Sequence labelling assigns labels to tokens or spans. This chapter focuses on named entity recognition (NER): finding mentions such as people, organisations and places. It also covers POS tagging, phrase chunking and semantic role labelling.

These tasks use different labels. POS tagging assigns grammatical categories to words. NER marks mention boundaries and types. Chunking groups words into shallow phrases. Semantic role labelling identifies a predicate's arguments and their roles.

Morphology matters in each task. Names can carry case suffixes, POS tags depend on the chosen word analysis, and omitted arguments require a separate treatment in semantic analysis. The examples below distinguish these linguistic issues from dataset-specific annotation rules.

9.1 POS Tagging and Morphological Analysis

9.1.1 POS Tagging and Morphological Disambiguation

POS tags capture some grammatical distinctions; morphological features supply others. This division applies to English as well as Turkic languages, and its details depend on the annotation scheme.

Choosing a full analysis also chooses a POS tag: selecting *yüz+Num* assigns UPOS=NUM. The reverse does not hold. Turkish *evi* can mean 'his/her house' or 'the house (ACC)'; both readings are nouns. Joint models such as TurkishDelightNLP (Alecakir, Bölücü, and Can 2022) share information between POS and morphological prediction.

Compare joint prediction with a POS-only baseline on the same data. The neural morphology tutorial provides one joint tagging design (Section 7.6).

9.1.2 The UPOS Set for Turkic

UD uses 17 universal POS categories. The following distinctions need attention in Turkic annotation.

VERB vs. AUX. Turkish light verb constructions include *yardım etmek* "to help", literally "to do help". The status of verbs such as *et-*, *yap-* and *ol-* depends on the construction and the treebank's annotation policy. Inspect those guidelines before combining datasets (Section 8.2). A disagreement between VERB and AUX can reflect an annotation mismatch; it can also be a model error within a consistently annotated dataset.

ADP. Postpositions are common in Turkic languages; both prepositions and postpositions are tagged ADP in UPOS. The postposition follows its complement nominal, so a strictly left-to-right model sees the postposition only after the nominal it relates to, the reverse of English order. Bidirectional encoders see both, so this matters mainly for incremental (left-to-right) taggers.

NOUN vs. PROPN. Proper names can take the same suffixes as common nouns, and a common noun can also be used as a name. Context helps distinguish them. Capitalisation and gazetteers provide additional evidence, but neither is required to be correct or complete.

PART. "Particle" is a linguistic description, not a guarantee of UPOS=PART. Turkish question *mı/mi/mu/mü* and additive *da/de* are written separately in standard orthography. Treebanks may assign different categories to them, including AUX for the question marker; inspect the selected treebank's UPOS and dependency conventions rather than inferring them from a suffix-like pronunciation.

9.1.3 Evaluation Results Across Turkic Languages

Table 8.7 compares per-language Stanza models with a multilingual tagger. Its frozen manuscript snapshot reports higher UPOS for the multilingual system on Uzbek (71.8 → 87.6) and Kyrgyz (71.1 → 79.0). The scores cover matched words only, and the table includes gold and silver reference sets. Differences beyond multilingual training may contribute to the results; Section 8.4 gives the source and scoring conditions.

9.1.4 Cross-Lingual POS Tagging

Zero-shot POS tagging applies a model to a language without task-specific training on that language. The small Karakalpak and Kumyk samples in Table 8.7 illustrate this setup with proxy-language embeddings. Report per-tag errors as well as the overall score; shared suffix patterns cannot predict which POS categories will transfer best.

9.2 Named Entity Recognition in Agglutinative Languages

9.2.1 The Case Suffix Problem

Inflected names raise two separate questions: can a system recognise an unfamiliar surface form, and where should the annotated mention end? The first concerns generalisation; the second depends on the dataset's boundary policy.

Capitalisation helps, but entities need not be capitalised. Informal Latin- and Cyrillic-script text may omit capitals. Perso-Arabic writing, including Ottoman Turkish, has no upper/lowercase distinction. NER therefore also needs contextual and lexical evidence.

Entities inflect. Names take suffixes, but not every mention has an overt case marker. An unmarked form is not necessarily a subject. The Kazakh city name *Алматы* (Almaty) illustrates the surface variation:

Table 9.1: The Kazakh city name *Алматы* in six cases (the instrumental *Алматымен* 'with Almaty' is a seventh). A recogniser needs to detect mentions in these inflected forms; resolving their identity is a separate task. In Turkish, the apostrophe (*Almatı'ya*, *Almatı'dan*) separates the name from its suffix in formal writing, but is often omitted in informal text (*Almatıya*, *Almatıdan*).

Case	Kazakh form	Gloss
Nominative	*Алматы*	Almaty (unmarked)
Genitive	*Алматының*	of Almaty
Dative	*Алматыға*	to Almaty
Locative	*Алматыда*	in Almaty
Ablative	*Алматыдан*	from Almaty
Accusative	*Алматыны*	Almaty (object)

Case	Kazakh form	Gloss

A dictionary that matches only *Алматы* misses *Алматыдан*. A learned model may also struggle with inflected forms that are rare in its training data. Exercise 1 tests this by measuring the distribution of case forms and recall for each case.

Mention boundaries depend on annotation. Turkish *Ankara'da* contains the name *Ankara* and a locative suffix. A dataset may label the whole token or only the name. The apostrophe helps locate the suffix in formal Turkish, but informal *Ankarada* lacks it. Kazakh *Алматыдан* likewise has no separator. Follow the chosen annotation policy rather than assuming that every NER span must stop at the stem. The recurring Uzbek sentence (Table 1.4) has the same choice to make for *Toshkentda* 'in Tashkent', while Kazakh *Қазақстан* appears unmarked.

9.2.2 BIO Tagging Strategies

The standard BIO (Begin, Inside, Outside) tagging scheme labels each token as either the beginning of an entity span (B-TYPE), the continuation of a span (I-TYPE), or outside any span (O). Two possible labelling units are words and morphological segments:

Word-level BIO. Tag each word token with a BIO label, including the case suffix. *Алматыдан* → B-LOC. The entity boundary is the word boundary. This matches datasets whose mention spans include the suffix.

Morpheme-level BIO. Segment words into morphemes before tagging, then tag each morpheme. *Алматы-дан* → B-LOC, O. This example assumes a stem-only boundary policy. Possessive and derivational forms (*Алматылық* 'from Almaty, Almaty-an') need explicit rules. The cost is a morphological segmenter as a required upstream component, and the segmenter's errors can affect mention boundaries (Chapter 6 and Chapter 7).

Neither strategy by itself produces a canonical name or an entity identity. Three outputs should be kept apart, as in this example:

Table 9.2: Three different outputs for one Kazakh mention. A stem-only span is not necessarily the canonical name (multiword names, aliases and inflected modifiers inside names need their own rules), and a canonical name is not yet an identity: ambiguous names need linking.

Output	Example for *Алматыдан келдім* 'I came from Almaty'	Task
Mention span in the original text	zero-based span `[0, 9)`, *Алматыдан*, LOC	NER (what datasets annotate)

Output	Example for *Алматыдан келдім* ‘I came from Almaty’	Task
Normalised name	*Алматы*	name normalisation (lemmatisation, rules for multiword and possessive names)
Linked entity	the city of Almaty, represented by an identifier in the chosen knowledge base, or *unresolved*	entity linking

Score mention spans against each dataset’s own boundary policy, and evaluate normalisation and linking as separate tasks.

KazNERD (Yeshpanov, Khassanov, and Varol 2022) and WikiANN (Pan et al. 2017) supply token-level labels, so the tutorial uses word-level BIO. Yeniterzi’s Turkish CRF study represents a word as a root state followed by morphological-feature states, while constraining those states to preserve the word’s entity label. This is different from excluding suffixes from mention spans. Its morpheme-level and word-level scores were 88.94 and 88.71 F1; the difference was not significant (Yeniterzi 2011, 107–9).

9.2.3 Turkic NER Datasets

Table 9.3 is a dated inventory of the resources identified in a survey for this chapter (October 2026). It separates human annotation from weak supervision and automatic projection, because they support different conclusions. Where the table says a field is “not stated”, the source we checked did not give it.

Table 9.3: Turkic NER resources identified in this survey (searched 2026-10-13; not exhaustive). Human annotation, weak supervision and automatic projection are kept apart; fields the source does not state are marked as such. WikiANN sizes are those of the commonly used Hugging Face splits.

Language(s)	Resource	Annotation	Labels	Domain	Size	Access / licence
Kazakh	KazNERD (Yeshpanov, Khassanov, and Varol 2022)	human	25 classes (IOB2)	TV news transcripts	90,228 / 11,167 / 11,307 sentences (train/dev/test)	CC BY 4.0; github.com/IS2AI/KazNERD; HF is sai/kaznerd

Lan-guage(s)	Resource	Annotation	Labels	Domain	Size	Access / licence
Kyrgyz	KyrgyzNER (Turatali et al. 2025)	human	27 classes (25 occur in the release)	news (24.kg, 2017–2022)	10,900 sentences: 7,033 train / 3,867 test, no dev; 16,819 entity spans (the paper's 39,075 "mentions" count entity-tagged tokens)	Dataset CC BY-NC-SA 4.0; code and models MIT (README); github.com/Akyl-AI/KyrgyzNER
Uzbek	Uzbek Legal NER (Saidov et al. 2026)	semi-automatic candidates + staged human review; synthetic records for rare labels	12 classes	legal	14,036 records; splits not reported	CC BY 4.0; Zenodo 10.5281/zenodo.19682709
Azerbaijani	LocalDoc azerbaijani-ner-dataset (LocalDoc, n.d.)	not stated on the dataset card	includes PERSON, LOCATION, ORGANISATION (full list not checked)	not stated on the dataset card	99,545 examples, single split	CC BY-NC 4.0; HF LocalDoc/azerbaijani-ner-dataset
Uyghur	Azmat et al. (Azmat et al. 2020)	automatic (tags projected from Chinese through MT)	not stated in the abstract	not stated in the abstract	not stated in the abstract	release not stated
Turkish	several datasets (Safaya et al. 2022; Çöltekin, Doğruöz, and Çetinoğlu 2023)	human	mostly PER, ORG, LOC (+MISC/DATE)	news and other	see sources	see sources

Language(s)	Resource	Annotation	Labels	Domain	Size	Access / licence
Turkish, Azerbaijani, Kazakh, Uzbek, Tatar, Kyrgyz, Uyghur, Turkmen, Sakha, Bashkir, Chuvash, Crimean Tatar	WikiANN (Pan et al. 2017)	automatic (Wikipedia links)	PER, ORG, LOC	Wikipedia	tr 20,000 / 10,000 / 10,000; az 10,000 / 1,000 / 1,000; kk, uz, tt 1,000 / 1,000 / 1,000; ky, ug, tk, sah, ba, cv, crh 100 / 100 / 100 sentences	see the HF dataset card (unimelb-nlp/wikiann)

KazNERD (Yeshpanov, Khassanov, and Varol 2022) is a human-annotated Kazakh NER dataset. Built from transcripts of Kazakh television news, it contains 112,702 sentences with 136,333 entity annotations in 25 classes (the pinned release has 136,337 `B-` tags). The best model in the original paper, a fine-tuned XLM-R-large, reached an exact-match F1 of 97.22% on the test set, ahead of BERT (96.24), a BiLSTM-CNN-CRF (93.51) and a CRF (92.41). Table 9.4 lists the classes with mention counts computed from the authors' release.

Table 9.4: The 25 KazNERD entity classes (Yeshpanov, Khassanov, and Varol 2022), grouped for readability, with mention counts computed from the authors' release (IOB2; 90,228 / 11,167 / 11,307 training / validation / test sentences; 109,346 / 13,483 / 13,508 mentions). Rare classes such as NON_HUMAN have too few test mentions for a meaningful per-class F1.

Category	Classes (training / test mentions)
People and groups	PERSON (10,893 / 1,332), NORP (2,972 / 372), POSITION (4,937 / 603)
Organisations and places	ORGANISATION (6,093 / 735), GPE (14,108 / 1,742), LOCATION (1,759 / 212), FACILITY (1,752 / 198)
Time	DATE (20,226 / 2,611), TIME (1,874 / 230)
Numbers and measures	CARDINAL (23,551 / 2,824), ORDINAL (3,105 / 386), QUANTITY (3,094 / 407), MONEY (3,678 / 441), PERCENTAGE (3,384 / 456)
Other	ADAGE (159 / 19), ART (1,953 / 229), CONTACT (161 / 20), DISEASE (1,032 / 123), EVENT (1,352 / 156), LANGUAGE (352 / 46), LAW (425 / 55), NON_HUMAN (6 / 1), PRODUCT (592 / 73), PROJECT (1,694 / 211), MISCELLANEOUS (194 / 26)

TurkicNLP ships a Stanza NER model trained on KazNERD for Kazakh and one trained on

the Starlang NER corpus for Turkish; these are the two pretrained NER models described for toolkit version 0.5.0 (Hakimov 2026).

Turkish NER. Several Turkish NER datasets exist, among them MilliyetNER and the Turkish part of WikiANN, which Mukayese uses for its NER benchmark (Safaya et al. 2022), and those surveyed in Çöltekin, Doğruöz, and Çetinoğlu (2023). Check each dataset's label inventory and boundary rules before combining it with another.

A practical challenge in Turkish NER is the apostrophe: formal Turkish writing separates proper names from their suffixes with an apostrophe (*İstanbul'a* "to Istanbul"), but informal text often omits it (*İstanbula*). A model trained on edited text may have lower recall on the apostrophe-less form; to check this, evaluate separately on mentions with and without the apostrophe.

Other languages. KyrgyzNER (Turatali et al. 2025) is a human-annotated news dataset with a 27-class scheme; 25 classes occur in the release. It provides training and test splits, with no development split. The paper's count of 39,075 "entity mentions" counts entity-tagged tokens (`B-` plus `I-`); the release contains 16,819 spans. PERSON spans can include titles and positions, as in a whole *… орун басары Тожибек Норов* 'deputy head … Tozhibek Norov'. KazNERD instead gives POSITION its own type.

An Uzbek legal dataset combines semi-automatic candidates, staged human review and synthetic records for rare labels. Its authors call it human-reviewed, with further work needed for a final gold standard (Saidov et al. 2026). A large Azerbaijani dataset on the Hugging Face Hub has a non-commercial licence, but its card leaves the annotation method unspecified (LocalDoc, n.d.). For Uyghur, Azmat et al. (2020) projected training labels from Chinese through machine translation. Comparisons across these resources need a declared label mapping and an account of their different domains, boundaries and annotation methods.

WikiANN / PAN-X (Pan et al. 2017) derives NER labels from Wikipedia links in 282 languages, using PER, ORG and LOC. Its Turkic splits vary greatly in size (Table 9.3). Turkish has 20,000 training sentences; Kyrgyz, Uyghur, Turkmen, Sakha, Bashkir, Chuvash and Crimean Tatar have 100 sentences per split. Link-derived labels can omit mentions or include noisy boundaries, and their coverage reflects Wikipedia's entities. Scores describe performance on this automatically labelled material; evaluate on the intended domain before drawing conclusions about practical NER quality.

The gap. The October 2026 survey found human-annotated fine-grained datasets for Turkish, Kazakh and Kyrgyz, and human-reviewed legal data for Uzbek. It found no human-annotated NER resource for the other languages in the inventory, including Tatar, Bashkir, Turkmen and the Siberian Turkic languages. WikiANN supplies automatic labels for some of them. Check for newer or overlooked resources before starting an annotation project.

9.2.4 Neural NER Architectures

Character embedding + BiLSTM-CRF (Lample et al. 2016; Jurafsky and Martin 2024, chapter "Sequence Labelling for Parts of Speech and Named Entities") was widely used before pretrained transformers. The character-level BiLSTM is useful for Turkic: it

encodes the surface form of each word, including its suffix pattern, which can help with unseen inflections of known entity stems: a model that has seen *Алматы* in training shares the character prefix with *Алматыдан* at test time.

Transformer token classification is the architecture used in the tutorial. A pretrained encoder produces a contextual embedding for each subword; a linear head predicts the BIO label. The encoder may be monolingual (BERTurk for Turkish) or multilingual (XLM-R, which was pretrained on text in many languages including several Turkic ones). Self-attention uses context within the model's input window; the tutorial checks that each sentence fits.

Span-based NER predicts entity boundaries and types directly. It can represent nested mentions, such as *Алматы* inside *Алматы облысының әкімдігі* 'the Almaty Region administration'. A single BIO sequence can encode only one non-overlapping layer. The datasets in Table 9.3 use flat annotation, so they do not test this ability. Computation depends on the number of candidate spans and how the model selects them.

For practical work on Turkic languages, **XLM-R + token classification** is a reasonable starting point: it starts from a multilingual encoder and is the architecture behind the best KazNERD result (Yeshpanov, Khassanov, and Varol 2022). How many annotated sentences a useful system needs depends on the label set, domain and acceptance criteria; measure it with a learning curve on a fixed test set rather than assuming a number. Exercise 3 draws one for Kyrgyz.

9.3 Cross-Lingual NER Transfer

9.3.1 Encoder Coverage, Task Language and Script

Separate **pretraining coverage**, **NER training data** and **input script**. XLM-R was pretrained on Kazakh and Kyrgyz Cyrillic text (Conneau et al. 2020), so Turkish-only NER fine-tuning does not make Cyrillic input unknown to its encoder. But pretraining coverage does not guarantee accurate entity recognition: local names, context and label conventions still differ.

A fair comparison uses the same annotated target sentences under three conditions:

1. **Native script:** a multilingual model fine-tuned on source-language NER applied to the target text as written.
2. **Transliterated:** the same model applied to text converted to the source script (or to CTS), with predicted spans mapped back to the original string through a character-offset map. One-to-many letter changes shift offsets, so spans cannot be copied by position.
3. **Target-language fine-tuning:** the model fine-tuned on a small amount of annotated target-language data.

Script conversion may help or hurt, and recognising the script is not the same as recognising the entities.

9.3.2 Entity Distribution Shift

The **entity distribution** can also differ: people, organisations and places in one corpus need not appear in another. Measure this overlap instead of inferring it from language relatedness.

The recorded Exercise 4 compares PERSON name-token types in KazNERD training data and Starlang Turkish NER data. After the documented stem extraction, CTS conversion and lowercasing, their Jaccard overlap is 0.016; character-trigram overlap is 0.315. The figures describe two corpora under one normalisation method. Kazakh and Turkish names in general may behave differently. Analyse errors separately for names seen and unseen during training.

9.3.3 Label Schemes and Domains

KazNERD, WikiANN and KyrgyzNER use different label inventories and mention boundaries. Define a shared mapping before scoring. For example, collapsing GPE and LOCATION to LOC removes a distinction; mapping unsupported types to `O` excludes them from the task. Apply the same rules to gold and predictions, retain all tokens, and report the resulting label inventory and support. A label mapping does not fix boundary differences, such as whether a person's title belongs inside the mention. Use one target test set for all systems and record its domain.

9.3.4 Zero-Shot vs. Few-Shot NER

The recorded Exercise 3 compares a pretrained Stanza KazNERD model with CRFs trained on KyrgyzNER, after mapping both datasets to PER, LOC and ORG. With gold words, zero-shot Stanza reaches 39.9 micro F1. The CRF averages 38.6 at 200 training sentences and 48.0 at 500 over three sampling seeds; one 200-sentence run reaches 42.1, above the zero-shot score. Training on all 7,033 sentences gives 62.6 in one run.

The mean first exceeds the zero-shot score at the tested 500-sentence setting. No general annotation budget follows from it, since the systems, training data and domains differ. KyrgyzNER's inclusion of titles in PERSON spans also affects exact-match scoring. The comparison does not isolate the cause of the gap. The results describe the released test split. For a new tuned experiment, reserve development data from the training split, fix the sampling sizes and model settings there, and evaluate the fixed learning curve on test without further adjustment.

9.4 Other Sequence Labelling Tasks

9.4.1 Chunking

Noun phrase chunking labels the beginning, continuation and outside of a phrase, for example B-NP, I-NP and O. Define the chunk first: a base noun phrase and a full phrase

containing a relative clause are different targets.

Turkish constituent chunking has been studied with CRFs (Aslan, Gunal, and Dincer 2018). That study proposes an OE scheme that marks the final token of a chunk. Its reported advantage concerns the tested constituent-chunking setup; it would be a stretch to conclude that all Turkish phrase boundaries are easy to find. Prenominal modifiers and flexible word order still require explicit annotation rules (Section 8.1).

9.4.2 Semantic Role Labelling

Semantic role labelling (SRL) assigns predicate-argument roles to the participants of each event: who did what to whom, where, when, and how. In PropBank-style SRL, numbered roles are defined per predicate sense. ARG0 is typically the proto-agent and ARG1 the proto-patient; higher numbers depend on the frame. Roles do not map directly to grammatical relations: a passive subject need not be ARG0. Modifier roles such as ARGM-LOC (location) and ARGM-TMP (time) are shared across predicates.

Turkish SRL resources follow the PropBank convention: each verb sense has a frame with numbered argument roles. A first Turkish PropBank annotated sentences of a Turkish dependency treebank, using framesets of root verbs (Şahin and Adalı 2018); **TRopBank** extended the frame inventory to Turkish verb senses linked to the Turkish WordNet: its released frame file has 17,531 framesets (Kara et al. 2020).

Two issues from Chapter 2 matter for the task definition and evaluation:

Pro-drop. Subjects are often unexpressed (Section 8.1). Most SRL tasks annotate overt arguments only, so an unexpressed ARG0 should remain unlabelled under that policy. Recovering it requires a separate task definition and evaluation. Agreement can constrain person and number; identifying the referent also needs discourse evidence. In either task, follow the selected predicate frame's role definitions.

Flexible word order. In a canonical English active transitive clause, the subject precedes the verb and the object follows it; their numbered roles still depend on the predicate frame. Turkic arguments can be reordered for discourse reasons. An SRL system therefore needs case, agreement and frame information as well as position. This is the same combination of signals as in dependency parsing (Section 8.1).

For languages other than Turkish, this survey identified no PropBank-style resource (searched October 2026; check for newer releases). Transfer from Turkish is one option, alongside multilingual models and target-language annotation. Cognate verbs may have similar frames, but light verbs and causative chains differ between languages, so transferred frames need language-specific checking.

9.5 Pretrained Taggers and Recognisers in TurkicNLP

TurkicNLP 0.5.0 provides Stanza and multilingual Glot500 POS backends, plus pretrained NER models for Turkish and Kazakh. Check the available processors and backends for each language rather than treating support as uniform across the toolkit.

```
# pip install "turkicnlp[stanza]==0.5.0"   # Stanza POS/NER backend
import turkicnlp

for lang, text in [("tur", "Ali dün Ankara'ya gitti."),
                   ("kaz", "Қасым-Жомарт Тоқаев кеше Астанада инвесторлармен кездесті.")]:
    nlp = turkicnlp.Pipeline(lang, processors=["tokenize", "pos", "ner"])
    doc = nlp(text)
    print([(w.text, w.upos, w.ner) for w in doc.words])
    print([(e.text, e.type) for e in doc.entities])
```

Output (TurkicNLP 0.5.0, Stanza backend):

```
[('Ali', 'PROPN', 'O'), ('dün', 'NOUN', 'O'), ("Ankara'ya", 'PROPN', 'B-LOCATION'),
 ↪  ('gitti', 'VERB', 'O'), ('.', 'PUNCT', 'O')]
[("Ankara'ya", 'LOCATION')]
[('Қасым-Жомарт', 'PROPN', 'B-PERSON'), ('Тоқаев', 'PROPN', 'I-PERSON'), ('кеше', 'ADV',
 ↪  'B-DATE'), ('Астанада', 'PROPN', 'B-GPE'), ('инвесторлармен', 'NOUN', 'O'),
 ↪  ('кездесті', 'VERB', 'O'), ('.', 'PUNCT', 'O')]
[('Қасым-Жомарт Тоқаев', 'PERSON'), ('кеше', 'DATE'), ('Астанада', 'GPE')]
```

The outputs illustrate both annotation conventions and errors. Turkish uses Starlang labels such as `LOCATION`; Kazakh uses KazNERD's finer inventory, including `GPE` and `DATE`. Both outputs include case suffixes in the mention. The Turkish model misses the person name *Ali*; its NOUN tag for *dün* 'yesterday' reflects a treebank choice for temporal words rather than a clear error, since treebanks differ on NOUN or ADV for such forms. The Kazakh model includes *кеше* 'yesterday' as DATE, following KazNERD's inventory. Keep the printed predictions distinct from a manually checked reference.

Lemmas may help normalise a name, but they do not solve multiword-name normalisation or entity linking (Table 9.2). The example does not request the `lemma` processor; add it if the application needs lemmas.

For other languages, toolkit version 0.5.0 does not ship a pretrained NER model; this is not a claim that no model exists elsewhere. The toolkit can instead ask a large language model, through an OpenAI-compatible server (vLLM, Ollama), OpenAI, Claude, or a local Hugging Face model. The model labels the toolkit's own tokens, and the toolkit checks its returned structure before storing it. A well-formed response can still carry wrong entity labels:

```
nlp = turkicnlp.Pipeline(
    "kir", processors=["tokenize", "ner"], ner_backend="llm",
    ner_labels=["PER", "LOC", "ORG"],
    llm_model="qwen2.5:7b", llm_base_url="http://localhost:11434/v1", llm_api_key="ollama",
)
doc = nlp("Бишкекте Алмазбек Атамбаев менен жолугушуу өттү.")
print([(e.text, e.type) for e in doc.entities], doc._processor_log)
```

Measure exact span-and-type performance on an annotated sample before relying on the LLM backend (Exercise 6).

9.5.1 Measured pretrained KazNERD baseline

The cached Stanza KazNERD checkpoint was evaluated on all **11,307 sentences and 129,824 words** in the pinned KazNERD test split (Yeshpanov, Khassanov, and Varol 2022). Gold sentence and word boundaries are supplied. An entity is correct only when its complete span and type match; all 25 types contribute to micro-averaged precision, recall and F1. Stanza BIOES predictions are decoded to strict spans, and an independent rescore with Stanza's own entity decoder reproduces the totals.

Table 9.5: Measured cached Stanza KazNERD checkpoint on the complete pinned test split, with gold word/sentence boundaries and exact entity spans and types. No fine-tuning was performed. Source: `run_ner.py`; Yeshpanov, Khassanov, and Varol (2022).

Test sentences	Gold entities	Predicted	Correct	P (%)	R (%)	F1 (%)
11,307	13508	13501	12821	94.96	94.91	94.94

The pretrained Stanza checkpoint reaches **94.94% F1** with gold tokenisation on this test set. This baseline measures exact entity spans and types under those conditions; raw-text performance and cross-language transfer require separate evaluations. No independent contamination audit was performed. Predictions, per-type counts and model/input hashes accompany `run_ner.py`. The exercises use this checkpoint to measure case-stratified recall (Exercise 1) and zero-shot transfer to Kyrgyz (Exercise 3).

9.6 Hands-On: A Fine-Tuning Procedure for Kazakh NER with XLM-R

Question. How can a reproducible NER recipe align subwords and entity labels?

Inputs. KazNERD splits, XLM-R and the training script linked below.

Compute. A large checkpoint and substantial training memory; choose hardware and batch settings before the full run.

Output. A saved model, predictions and entity-level micro and macro F1.

Interpretation. Inspect suffix boundaries and label mappings, then compare a target-language transfer condition.

This tutorial fine-tunes `xlm-roberta-large` for named entity recognition on KazNERD (Yeshpanov, Khassanov, and Varol 2022) and then applies the model to Kyrgyz text as a zero-shot experiment. The code provides a training recipe; the book reports no fine-tuned score from it, and the measured pretrained Stanza baseline above is the reference point.

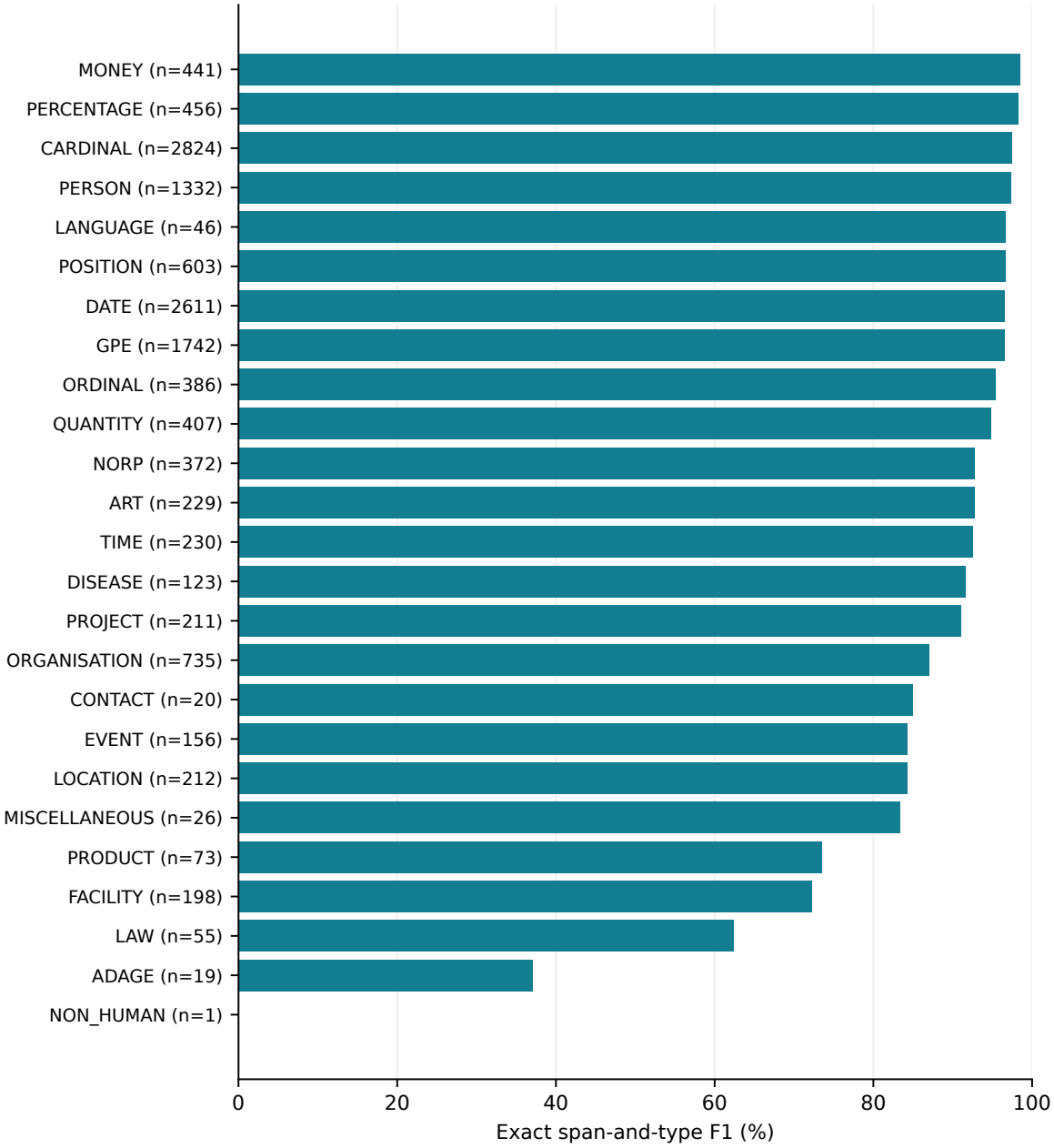


Figure 9.1: Measured exact span-and-type F1 for the pretrained Stanza KazNERD checkpoint. Each label shows the number of gold test entities for that type; small-support categories should not be interpreted as equally precise estimates.

9.6.1 Setup

```
# Tested mechanics: transformers==5.3.0 datasets==4.6.1 seqeval==1.2.2
# torch==2.10.0 accelerate==1.12.0; install sentencepiece for the tokenizer.
# Run from the book root, with the pinned KazNERD clone at KazNERD/.
# Record the versions you used: they are saved with the model below.
import json, random, sys, hashlib
from importlib.metadata import version
from pathlib import Path
sys.path.insert(0, str(Path("code/ch09-ner").resolve()))
from ner_utils import read_iob2, valid_iob2, repair_training_iob2, align_batch,
 ↪  tag_sequences

import numpy as np
import torch
import transformers
from datasets import Dataset, DatasetDict
from transformers import (
    AutoTokenizer,
    AutoModelForTokenClassification,
    TrainingArguments,
    Trainer,
    DataCollatorForTokenClassification,
)
from seqeval.metrics import classification_report, f1_score
from seqeval.scheme import IOB2

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
```

9.6.2 Loading KazNERD

The authoritative release is the authors' repository (`github.com/IS2AI/KazNERD`, CC BY 4.0), which stores the three splits as two-column CoNLL files in IOB2. A Hugging Face copy also exists (`issai/kaznerd`, with the same 90,228 / 11,167 / 11,307 split sizes); if you use it, pin a revision with `load_dataset("issai/kaznerd", revision=...)` and check that the label list and counts below are reproduced. Here we read the release directly, validate it, and build the label map from the training split:

```
ROOT = Path("KazNERD/KazNERD")      # git clone https://github.com/IS2AI/KazNERD then git
 ↪  checkout bd4333d0f5952b9fafb2ef2ac2fefa0ad3c0333f
raw = {split: read_iob2(ROOT / f"IOB2_{name}.txt")
       for split, name in [("train", "train"), ("validation", "valid"), ("test", "test")]}

label_names = ["O"] + sorted({t for _, tags in raw["train"] for t in tags} - {"O"})
label2id = {label: i for i, label in enumerate(label_names)}
id2label = {i: label for label, i in label2id.items()}

for split, sents in raw.items():
    unseen = {t for _, tags in sents for t in tags} - set(label_names)
    invalid = sum(not valid_iob2(tags) for _, tags in sents)
    assert not unseen, (split, unseen)          # every tag must be known from training
    print(f"{split:<10} {len(sents):>6} sentences  longest {max(len(t) for t, _ in sents)}
     ↪  words"
```

```
          f"  invalid IOB2: {invalid}")
print(len(label_names), "labels:", label_names[:5], "...")

if any(not valid_iob2(tags) for split in ("validation", "test") for _, tags in raw[split]):
    raise ValueError("malformed evaluation gold; resolve the source before scoring")
raw["train"], training_repairs = repair_training_iob2(raw["train"])

dataset = DatasetDict({
    split: Dataset.from_dict({"tokens": [t for t, _ in sents],
                              "ner_tags": [[label2id[x] for x in g] for _, g in sents]})
    for split, sents in raw.items()
})
```

Output (release commit `bd4333d`):

```
train       90228 sentences  longest 71 words  invalid IOB2: 14
validation  11167 sentences  longest 51 words  invalid IOB2: 0
test        11307 sentences  longest 70 words  invalid IOB2: 0
51 labels: ['O', 'B-ADAGE', 'B-ART', 'B-CARDINAL', 'B-CONTACT'] ...
```

The 51 labels are B- and I- tags for the 25 types of Table 9.4 plus `O`. The release audit found 16 orphan `I-` tags across 14 training sentences. No malformed IOB2 sequences were found in the evaluation splits; this does not certify every annotation. This tutorial repairs orphan training tags (`I-X` → `B-X`) and saves each changed position; evaluation labels remain unchanged. The class-count table counts the original release's B-tags, so its training total precedes this repair.

9.6.3 Tokenising with Word Alignment

The same word-alignment strategy from Chapter 7 applies: only the first subword token of each word receives the NER label; subsequent tokens receive -100 (ignored by the loss). A word count does not bound subword length. The code tokenises every sentence without truncation and rejects any that exceed the model window, because truncation would drop trailing entities from both training and scoring. For longer documents, split them into overlapping windows, keep the original word index of every subword, and merge the per-word predictions before scoring.

```
MODEL_NAME = "FacebookAI/xlm-roberta-large"
MODEL_REVISION = "c23d21b0620b635a76227c604d44e43a9f0ee389"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, revision=MODEL_REVISION)
MAX_LEN = 512


def tokenise_and_align(examples):
    return align_batch(examples, tokenizer, MAX_LEN)


tokenised_dataset = dataset.map(tokenise_and_align, batched=True,
                                remove_columns=dataset["train"].column_names)
```

The complete tokeniser audit retained all 1,302,352 words. Maximum encoded lengths, including special tokens, were 139 / 91 / 108 for train / validation / test. These are

measured values from `code/ch09-ner/audit_release.py`; they do not follow from the longest sentence's word count.

9.6.4 Defining the Evaluation Metric

NER scoring requires both the predicted span and its type to match the gold annotation. With `seqeval`, specify the averaging method and treatment of malformed tag sequences. Its `f1_score` defaults to **micro** averaging, which gives every mention equal weight and lets frequent types dominate. **Macro** F1 averages the per-type scores. Report both and state which selects the checkpoint. For strict IOB2 scoring, set `mode="strict", scheme=IOB2`; the default mode accepts some ill-formed sequences.

```
def to_tag_sequences(predictions, labels):
    return tag_sequences(predictions, labels, id2label)


def compute_metrics(eval_pred):
    logits, labels = eval_pred
    true_tags, pred_tags = to_tag_sequences(np.argmax(logits, axis=-1), labels)
    return {
        "micro_f1": float(f1_score(true_tags, pred_tags, average="micro", mode="strict",
        ↪  scheme=IOB2, zero_division=0)),
        "macro_f1": float(f1_score(true_tags, pred_tags, average="macro", mode="strict",
        ↪  scheme=IOB2, zero_division=0)),
    }
```

9.6.5 Fine-Tuning

```
model = AutoModelForTokenClassification.from_pretrained(
    MODEL_NAME, revision=MODEL_REVISION, num_labels=len(label_names), id2label=id2label,
 ↪  label2id=label2id)

use_bf16 = torch.cuda.is_available() and torch.cuda.is_bf16_supported()
use_fp16 = torch.cuda.is_available() and not use_bf16     # mixed precision only where
 ↪  supported

training_args = TrainingArguments(
    output_dir="xlmr-kaznerd",
    num_train_epochs=5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16,
    learning_rate=1e-5,
    warmup_ratio=0.1,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="micro_f1",       # the named development metric
    bf16=use_bf16,
    fp16=use_fp16,
    seed=SEED,
    report_to="none",
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenised_dataset["train"],
    eval_dataset=tokenised_dataset["validation"],
    processing_class=tokenizer,             # `tokenizer=` in transformers < 4.46
    data_collator=DataCollatorForTokenClassification(tokenizer),
    compute_metrics=compute_metrics,
)
trainer.train()

# Save everything needed to reproduce the predictions
out = Path("xlmr-kaznerd/best")
trainer.save_model(out)
tokenizer.save_pretrained(out)
(out / "run_info.json").write_text(json.dumps({
    "labels": label_names, "seed": SEED, "kaznerd_commit":
     ↪  "bd4333d0f5952b9fafb2ef2ac2fefa0ad3c0333f",
    "model_revision": MODEL_REVISION, "training_repairs": training_repairs,
    "data_sha256": {p.name: hashlib.sha256(p.read_bytes()).hexdigest() for p in
     ↪  ROOT.glob("IOB2_*.txt")},
    "scorer": {"mode": "strict", "scheme": "IOB2", "seqeval": version("seqeval")},
    "dependencies": {name: version(name) for name in ("datasets", "accelerate", "numpy",
     ↪  "sentencepiece")},
    "versions": {"transformers": transformers.__version__, "torch": torch.__version__},
}, indent=2))
```

Record the development scores and wall time from this experiment, together with the GPU type; no target score is guaranteed.

9.6.6 Evaluating on the Test Set

Evaluate the selected checkpoint once on the test set, with the same strict IOB2 scoring. Predicted orphan I-tags are left unchanged and scored under strict IOB2; report that policy rather than silently repairing predictions. Macro F1 averages entity types present in the gold/predicted span union, not a fixed inventory of 25 types. The report lists precision, recall, F1 and support (number of gold mentions) for each type:

```
pred = trainer.predict(tokenised_dataset["test"])
true_tags, pred_tags = to_tag_sequences(np.argmax(pred.predictions, axis=-1),
 ↪  pred.label_ids)
print(compute_metrics((pred.predictions, pred.label_ids)))
print(classification_report(true_tags, pred_tags, mode="strict", scheme=IOB2, digits=3))
```

Every word of every test sentence is scored, because no sentence was truncated. Per-type scores for types with few test mentions are unstable: NON_HUMAN has a single test mention, and ADAGE and CONTACT have about twenty each (Table 9.4). The original paper's 97.22 is an exact-match F1 (Yeshpanov, Khassanov, and Varol 2022); compare with it only after confirming that its scorer and averaging match yours, and with the pretrained Stanza baseline (94.94, micro F1, strict spans) on the same test split.

9.6.7 Zero-Shot Application to Kyrgyz

The multilingual tokeniser accepts Kyrgyz text directly. The standard Kyrgyz and Kazakh scripts are both Cyrillic, but a shared script does not guarantee useful NER predictions:

```
# pip install "turkicnlp[stanza]==0.5.0"
import turkicnlp

kir_tokenizer = turkicnlp.Pipeline("kir", processors=["tokenize"])   # Kyrgyz tokenizer
model.eval()


def ner_predict(text: str) -> list[tuple[str, str]]:
    """Run NER on a string; return one (word, label) pair per word."""
    words = [w.text for w in kir_tokenizer(text).words]
    if not words:
        return []
    inputs = tokenizer(words, is_split_into_words=True, return_tensors="pt")
    if inputs["input_ids"].shape[1] > MAX_LEN:
        raise ValueError("text too long for one window; split it into sentences first")
    word_ids = inputs.word_ids()
    inputs = {k: v.to(model.device) for k, v in inputs.items()}
    with torch.inference_mode():
        preds = model(**inputs).logits.argmax(-1)[0].tolist()

    results, seen = [], set()
    for wid, lid in zip(word_ids, preds):
        if wid is not None and wid not in seen:       # first subword of each word
            results.append((words[wid], id2label[lid]))
            seen.add(wid)
    assert len(results) == len(words)
    return results


# "A meeting with Almazbek Atambayev took place in Bishkek."
for token, label in ner_predict("Бишкекте Алмазбек Атамбаев менен жолугушуу өттү."):
    print(f"{token:<25} {label}")
```

For comparison, the pretrained Stanza KazNERD model recognises the person and place in this sentence when given the same seven words (Stanza 1.11.0). The output below comes from Stanza; the XLM-R model trained by the recipe would give its own:

```
Бишкекте                  B-GPE
Алмазбек                  B-PERSON
Атамбаев                  I-PERSON
менен                     O
жолугушуу                 O
өттү                      O
.                         O
```

(Stanza's BIOES tags `S-GPE` and `E-PERSON` are shown as their IOB2 equivalents.) One sentence does not estimate transfer accuracy; Exercise 3 evaluates the same model on the KyrgyzNER test set.

For a controlled script-conversion experiment, use the three conditions in Section 9.3 and preserve offsets when mapping predictions back to the original text. The single-sentence illustration in Chapter 16 does not measure a transfer gain.

Status: procedure only. The XLM-R fine-tuning run is not reported here. The pretrained baseline and the exercise experiments are separate results.

What to Take Forward

Sequence labelling starts with a label definition and an alignment policy. In Exercise 3, the CRF trained on 500 Kyrgyz sentences outperforms zero-shot transfer from the Kazakh model on average across three sampling seeds. This result suggests measuring a small learning curve before choosing between transfer and annotation; it does not set a general annotation budget. Preserve the label mappings and compare systems on the same evaluation items. Chapter 10 turns from labelling words to translating sentences between Turkic languages.

Further Reading

The Kazakh NER dataset KazNERD, with its 25-entity-type scheme, is described in detail in Yeshpanov, Khassanov, and Varol (2022). For Turkish NER and the wider body of Turkish NLP work, including POS tagging, chunking, and semantic role labelling, Oflazer and Saraçlar (2018) is the standard reference, and Çöltekin, Doğruöz, and Çetinoğlu (2023) provides a critical survey of available resources through 2022. Mukayese (Safaya et al. 2022) is the primary multi-task Turkish NLP benchmark, covering NER among its eight tasks.

WikiANN, which provides automatically derived NER data for several Turkic languages, is described in Pan et al. (2017); KyrgyzNER is described in Turatali et al. (2025). Yeniterzi (2011) compares word- and morpheme-level NER for Turkish, and Aslan, Gunal, and Dincer (2018) studies Turkish chunking schemes. Multilingual POS tagging across the Turkic family, including zero-shot languages, is evaluated in Hakimov (2026).

For the algorithmic foundations of sequence labelling (CRF, BiLSTM-CRF and BERT-based token classification), Jurafsky and Martin (2024) (chapter "Sequence Labelling for Parts of Speech and Named Entities") provides an accessible treatment, and Lample et al. (2016) introduced the BiLSTM-CRF tagger for NER. The XLM-R encoder used in this chapter's hands-on is described in Conneau et al. (2020); TurkishDelightNLP (Alecakir, Bölücü, and Can 2022) provides a unified Turkish POS tagging, NER, and dependency parsing system. The `seqeval` documentation explains its averaging options and strict IOB2 mode.

Exercises

1. **Case suffix analysis in NER.** Take the KazNERD test set. For every GPE, LOCATION and PERSON mention, analyse the final word with the Apertium analyser in TurkicNLP and record its case. Keep three outcomes apart: a non-nominative case, nominative, and *unknown* (no analysis, or several readings with different cases). Do not count analyser failures as nominative. Manually check a random sample of 50

mentions and report the analyser's error rate on it. What proportion of mentions of each type is inflected? Is a trained model's recall (for example that of the pretrained Stanza KazNERD model) lower for some cases?

2. **Vocabulary overlap and NER.** Fine-tune two models on KazNERD: one using `xlm-roberta-base` and one using `dbmdz/bert-base-turkish-cased` (BERTurk). Compare their micro and macro F1 (strict IOB2) on the KazNERD test set. Then measure BERTurk's tokeniser coverage on a declared Cyrillic Kazakh word list. Report both the fraction of words containing an unknown token and mean subwords per word; a fragmented word is not an unknown word. Is the vocabulary gap a plausible explanation for the F1 difference, and what experiment would test it beyond this single pair of models?

3. **Few-shot NER for Kyrgyz.** Use KyrgyzNER (Turatali et al. 2025), after checking its splits and licence; WikiANN Kyrgyz has only 100 training sentences. Reserve a fixed development subset before drawing nested training samples, such as 50, 200, 500 and 2,000 sentences where the release permits. Train XLM-R or a CPU-based CRF with three seeds per size. Choose hyperparameters and checkpoints on development data. Once the recipe and sizes are fixed, plot test F1 without using the curve to retune.

   Compare with zero-shot transfer from the pretrained Stanza KazNERD model or your fine-tuned XLM-R. Before examining scores, map the 25 KazNERD and 27 KyrgyzNER types to a shared inventory. Map unsupported types to `O` in both gold and predictions, retain all tokens and score both systems on the same mapped test set.

   Document unresolved mention-boundary differences and the domain difference between TV news and a news portal. At what sampled training size does the Kyrgyz model overtake zero-shot transfer?

4. **Entity distribution shift measurement.** Extract all PERSON entity spans from the KazNERD training set and a Turkish NER training set (for example the Starlang Turkish NER data or MilliyetNER). Normalise both sets (lemmatise or strip case suffixes, and transliterate to a common script with a documented mapping), then compute the Jaccard overlap of the name tokens and of their character trigrams. Which normalised strings are shared and which are not? Do not infer a name's etymology from string overlap alone. Stratify Exercise 3's PERSON mentions by whether their normalised name strings occur in the source training data: are unseen names the main source of cross-lingual PERSON errors?

5. **PropBank argument detection.** Use sense-annotated Turkish PropBank sentences (Şahin and Adalı 2018) and read the selected predicates' role definitions. Measure the position of overt ARG0 spans relative to the predicate. For examples without an overt ARG0, inspect the annotation policy and context before treating the argument as omitted. What does agreement tell you, and what discourse evidence is needed to recover a referent? If you use TRopBank frames with a UD treebank instead, treat subjects as syntactic observations: `nsubj` is not a gold ARG0 label. Manually establish predicate senses and roles before making an SRL claim. Evaluate overt-argument labelling and implicit-argument recovery separately.

6. **An LLM as a recogniser.** Annotate 50 Kyrgyz (or Tatar) news sentences with `PER`, `LOC` and `ORG` by hand. Run TurkicNLP with `ner_backend="llm"` and two models of different sizes, and compute entity-level F1 with `seqeval`. Then run the same LLM backend on 50 sentences of the KazNERD test set, where you can compare it with the trained Stanza model. Declare a mapping from KazNERD types to PER/LOC/ORG before scoring, use the same mention boundaries, and apply that mapping to both systems and gold. How large is the gap, and which entity types does the LLM miss?

10 Machine Translation: Within and Beyond the Family

Learning Objectives

After reading this chapter, you will be able to:

- Explain how morphology, constituent order and data availability affect translation into Turkic languages
- Describe the Apertium rule-based MT architecture and distinguish released language pairs from measured translation quality
- Characterise the TIL corpus and NLLB-200's coverage of Turkic languages, including data sizes and quality variation
- Compare the signals and limitations of BLEU, chrF and COMET for morphologically rich targets
- Describe intra-family translation strategies (pivot routing, multilingual NMT and zero-shot transfer) and reason about when each is appropriate
- Choose and compare low-resource strategies (back-translation, NLLB-200 fine-tuning, transfer from Turkish) for a new or under-resourced Turkic language pair

Machine translation brings together morphological generation, syntactic reordering, lexical choice and discourse interpretation. English→Turkish translation must interpret the English source and generate Turkish with appropriate inflection and constituent order. Translation between Turkic languages can exploit related vocabulary and constructions, but must still handle language-specific meanings, scripts and unequal resources.

The chapter covers Apertium's rule-based pipeline (Section 10.2), multilingual neural models and evaluation (Section 10.3), direct and pivot translation (Section 10.4), and low-resource adaptation (Section 10.5). The hands-on provides a Kazakh→Kyrgyz fine-tuning recipe.

10.1 The Challenges of Turkic MT

10.1.1 Target-Side Morphological Richness

Translation maps between different grammatical distinctions. English "she came" can be translated as Turkish *geldi*: the verb marks past tense and third-person singular agreement, while the subject's gender is not encoded. Related forms such as *gelmedi*

“did not come”, *gelmiş* (an indirect/evidential reading in context), *gelecek* “will come”, and *geldiler* “they came” illustrate choices a translator must make. Features need not each have an independent overt suffix.

Russian also has rich inflection. Translation must map between grammatical systems with different distinctions and forms. An incorrect case or agreement marker can change the meaning even when most characters match the reference.

10.1.2 SOV–SVO Reordering

English has a relatively fixed SVO order, while Russian permits substantial reordering. Turkic languages commonly have SOV order, prenominal modifiers and postpositions, with variation across constructions and languages (Johanson 2021, secs. 56.3–56.4). English→Turkish translation may therefore require moving whole constituents.

Relative clauses illustrate the reordering. English commonly places them after the noun, sometimes with an overt marker: *the book that I read yesterday*. Turkish places a participial clause before it: *dün okuduğum kitap*, literally “yesterday-read-my book”. Translation must produce the correct participial form and make the clause modify the intended noun. Both deserve checking even when the output reads fluently.

10.1.3 Data Scarcity

The TIL corpus (Mirzakhalov, Babu, Ataman, et al. 2021) contains sharply unequal amounts of training data across pairs. The following table is generated from a frozen release README; its counts are training sentence pairs, not the aggregate language totals in the paper.

Table 10.1: Training-set sizes for selected language pairs in the TIL corpus release (Mirzakhalov, Babu, Ataman, et al. 2021); counts from the release README (til-mt, commit 7d988cc), not the paper’s appendix table, which gives different totals for some pairs. English–Turkish has about 370 times more training data than English–Uyghur. Karachay-Balkar, Khakas and Tuvan are covered, but with small pair sizes; Nogai is not covered.

Language pair	Training sentence pairs
English–Turkish	35.9 M
Russian–Turkish	15.1 M
Kazakh–Russian	4.4 M
English–Kazakh	564.8 K
English–Azerbaijani	548.9 K
English–Uzbek	529.6 K
English–Kyrgyz	312.6 K
English–Uyghur	96.9 K
Turkish–Azerbaijani	410.1 K
Kazakh–Kyrgyz	12.2 K
Russian–Khakas	60.3 K
English–Tuvan	11.5 K

Language pair	Training sentence pairs
English–Karachay-Balkar	8.2 K

Karachay-Balkar, Khakas and Tuvan are included in this release. Nogai is outside its coverage; other parallel resources may still exist. Corpus size alone cannot define a universal threshold for usable MT: domain, duplication, alignment quality, pretraining and the evaluation criterion all matter. The TIL paper reports merging the original Tatoeba training, development and test portions during corpus construction (Mirzakhalov, Babu, Ataman, et al. 2021, sec. 3.1). Audit overlap with any intended evaluation set before using the corpus for training.

10.1.4 Script Mismatch Within the Family

Intra-family MT must identify the scripts actually present in each corpus. Uzbek occurs in both Latin and Cyrillic; Kazakh script reform does not imply that a corpus has already switched scripts. Uyghur also has more than one writing tradition (Chapter 3). Compare native-script training with a documented conversion if useful, retain the original text, and audit lossy mappings. Translation does not require script conversion. Whether to convert is an experimental choice.

10.2 Rule-Based MT: Apertium

10.2.1 Architecture

Apertium is an open-source rule-based machine translation platform originally designed for related language pairs (Forcada et al. 2011; Khanna et al. 2021). Its pipeline for a Turkic source–target pair can be understood through five stages (the precise modules depend on the pair):

1. **Morphological analysis**: the source-side text is processed by the source language's FST analyser (Chapter 6), producing candidate lemma and morphological-feature analyses for each recognised word.
2. **Disambiguation and selection**: choose among competing source morphological readings in context; pair-specific lexical-selection modules may also select among translation candidates.
3. **Lexical transfer**: the source lemma is mapped to the corresponding target lemma using a bilingual dictionary. Morphological features are passed through unchanged where they apply; category-specific transfer rules handle feature mismatches between languages.
4. **Structural transfer**: chunk-level reordering rules handle systematic constituent order differences. For a Turkish–English system, relative-clause placement and verb order are examples of differences that transfer rules would need to address; coverage depends on the implemented pair.

5. **Morphological generation**: the target lemma and morphological feature bundle are passed to the target language's FST generator, which produces a surface form licensed by the grammar.

The generator restricts output to forms licensed by its transducer. Errors can still enter through an incomplete lexicon or grammar, a wrong source analysis or a mistaken target choice. Define how the system handles unknown words and failed generation.

10.2.2 Turkic Pairs in Apertium

Apertium has released Turkic pairs as well as development repositories (Khanna et al. 2021). The wiki groups pairs by development stage; this is not a quality ranking:

Table 10.2: Apertium MT pairs with a Turkic language, by the release stage in Apertium's list of language pairs (checked October 2026). The page was generated in April 2020, and its arrows describe planned directions. Check each repository's release and supported directions before installation; a trunk listing is no guarantee of current support in both directions.

Pair	Relationship	Wiki stage
Kazakh ↔ Tatar	Both Kipchak	Trunk
Crimean Tatar ↔ Turkish	Kipchak ↔ Oghuz	Trunk
Kazakh ↔ Russian, English ↔ Kazakh	Cross-family	Trunk
Kazakh ↔ Kyrgyz, Tatar ↔ Bashkir, Turkish → Azerbaijani, Turkish ↔ Kyrgyz, Turkish ↔ Tatar, Turkish ↔ Uzbek, Kazakh → Kumyk	Within the family	In development (nursery)
Further pairs with Chuvash, Kyrgyz, Karakalpak, Sakha, Tuvan, Uyghur, Uzbek and others	—	Early development (incubator)

Shared vocabulary and morphology can reduce dictionary and transfer-rule work, while leaving ambiguity to resolve. Evaluate translation in the intended domain for both within-branch and cross-branch pairs; genealogy is no guarantee of quality.

10.2.3 Strengths and Limitations

Apertium provides inspectable intermediate analyses and can be built without a sentence-aligned training corpus, though it requires dictionaries, rules and linguistic work. Errors

may arise in analysis, disambiguation, lexical selection, transfer or generation. Trace the entire pipeline before deciding which component to repair.

Missing lexical entries require unknown-word handling: depending on the pipeline, a word may be copied or marked rather than translated. Even a well-formed generated word can have the wrong meaning if the chosen analysis or target lemma is wrong. Domain-specific accuracy claims require a held-out comparison.

Apertium output can also supply synthetic parallel data for a neural system; Section 10.5 discusses how to test its contribution.

10.3 Neural MT for Turkic Languages

10.3.1 The TIL Corpus

The Turkic Interlingua (TIL) corpus (Mirzakhalov, Babu, Ataman, et al. 2021) is a broad parallel resource for the Turkic family. It covers 22 Turkic languages and more than 400 translation directions, with English and Russian included as hub languages (Table 10.1). The corpus was assembled from publicly available parallel data and cleaned and deduplicated for each language pair.

The TIL release includes the **X-WMT evaluation sets**: human-post-edited test sets of at least 300 sentences for 88 translation directions among eight Turkic languages (Azerbaijani, Bashkir, Karakalpak, Kazakh, Kyrgyz, Sakha, Turkish and Uzbek) plus English and Russian. X-WMT supports comparison on common sentence content. The corpus paper reports bilingual baselines (Mirzakhalov, Babu, Ataman, et al. 2021); the separate multilingual study evaluates multiway systems (Mirzakhalov, Babu, Kunafin, et al. 2021). Do not attribute the multilingual experiment to the corpus paper or treat every release split as independent of every training collection.

10.3.2 NLLB-200

Meta's No Language Left Behind (NLLB-200) model (NLLB Team et al. 2024) is a multilingual sequence-to-sequence model covering 200 languages and built with both human-translated and automatically constructed parallel data. It covers eleven Turkic language/script varieties, including separate North and South Azerbaijani entries. FLORES-200 codes combine an ISO 639-3 code with a script subtag:

Table 10.3: Turkic varieties in NLLB-200 and their language/script codes. Note `uzn_Latn` and `azj_Latn`, not `uzb_Latn` or `aze_Latn`.

Language	NLLB code	Language	NLLB code
Turkish	`tur_Latn`	Kyrgyz	`kir_Cyrl`
North Azerbaijani	`azj_Latn`	Tatar	`tat_Cyrl`
South Azerbaijani	`azb_Arab`	Bashkir	`bak_Cyrl`
Northern Uzbek	`uzn_Latn`	Crimean Tatar	`crh_Latn`

Language	NLLB code	Language	NLLB code
Kazakh	`kaz_Cyrl`	Turkmen	`tuk_Latn`
Uyghur	`uig_Arab`		

NLLB's language list excludes Karakalpak, Gagauz, Chuvash and Sakha. For Karakalpak, Mamasaidov and Shopulatov (2024) created a FLORES+ devtest translation and 100,000-pair parallel corpora with Uzbek, Russian and English for the WMT 2024 Open Language Data Initiative, then fine-tuned NLLB-200.

For supported varieties, specify the full language code, including its script subtag. The target-language token controls what the model generates: a wrong valid code requests a different language, while an unsupported code can fail during tokenisation or generation. Test quality for each direction and domain. TurkicNLP's `translate` and `embeddings` processors wrap the distilled 600M-parameter model (Hakimov 2026), whose model card specifies a CC BY-NC 4.0 licence.

10.3.3 Evaluation Metrics

Use complementary metrics, state their configurations and inspect linguistic errors. **BLEU** (Papineni et al. 2002) measures word n-gram overlap with references and is sensitive to tokenisation and alternative word forms. A correct paraphrase can receive less credit, but a wrong case or tense can also change meaning: it should not automatically be excused as a morphological variant.

chrF (Popović 2015) measures character n-gram overlap. It gives partial credit to shared stems and suffixes, but can also reward a fluent or closely spelled mistranslation. **chrF++** (Popović 2017) adds word n-grams; the tutorial uses `CHRF(word_order=2)`. Its score is not numerically comparable with BLEU even though both are conventionally reported on a 0–100 scale.

COMET (Rei et al. 2020) is a learned metric; reference-based checkpoints use the source, hypothesis and reference. Reliability depends on the checkpoint, training distribution, language and domain. That a language is covered by the encoder tells us nothing about how well the metric correlates with human judgements in it. Record the exact checkpoint and validate it against human judgements when using it to support quality claims.

Table 10.4: Complementary MT evaluation signals; no universal language-specific correlation ranking is assumed.

Metric	Signal	Limitation to inspect
BLEU	Word n-gram overlap	Tokenisation and legitimate alternative wording
chrF / chrF++	Character overlap / character plus word overlap	High overlap despite wrong inflection or meaning
COMET	Learned quality estimate	Language/domain shift and checkpoint-specific behaviour

Report corpus BLEU and chrF++ with SacreBLEU signatures (Post 2018), the test-set revision and sentence count, and human error analysis where possible. Compare systems using the same metric and references.

10.3.4 Measured greedy translation baseline

The October suite runs the pinned **NLLB-200 distilled 600M** checkpoint in both Turkish–Kazakh directions on all 1,012 original FLORES-200 devtest rows (NLLB Team et al. 2024). Source and target language tokens are explicit. Decoding is greedy (one beam, no sampling), with a 256-token generation ceiling and no source truncation. This fixed pretrained baseline does not reproduce the published NLLB beam-search configuration.

Table 10.5: Measured NLLB-200 distilled 600M greedy translation on original FLORES-200 devtest. No fine-tuning or input truncation; explicit target language token, one beam and a 256-token generation ceiling. Exact SacreBLEU signatures and output audits are retained. Source: `run_translation.py`; NLLB Team et al. (2024).

Direction	Sentences	BLEU-13a	chrF	chrF++
Turkish → Kazakh	1012	9.48	41.97	37.55
Kazakh → Turkish	1012	12.02	45.25	41.04

Corpus BLEU uses SacreBLEU's `13a` tokeniser. chrF uses character order six and $\beta = 2$; chrF++ additionally uses word order two. The suite saves exact SacreBLEU signatures, every translation, input/model hashes and end-of-sequence audits. **One output in each direction reaches the generation ceiling without a terminating EOS token; none is empty.** Those outputs remain in the corpus scores and are not silently excluded. These automatic scores do not substitute for a human adequacy and morphology review. There is no fine-tuning or independent pretraining-overlap audit.

10.3.5 ISSAI Tilmash

The ISSAI Tilmash model card describes a system fine-tuned from NLLB-200 (distilled 1.3B) for Kazakh, Russian, English and Turkish, using Kazakh-centred parallel data including KazParC (Yeshpanov, Polonskaya, and Varol 2024). The model is gated: access requires agreeing to share contact information. The page lists FLORES in its dataset metadata and reports FLORES test scores, but does not identify the training split there. This leaves overlap unresolved; verify the training records before treating those scores as an independent comparison with NLLB. The existence of a release is not a benchmark result for every direction.

10.3.6 General-Purpose LLMs as Translators

General-purpose LLMs can also be tested as translators. Evaluate the intended language and domain, and check whether the output is in the requested language. Unrestricted language-ID predictions (Section 3.7) can flag possible mistakes, but need manual checking. A classifier restricted to Turkic labels cannot reject an English answer. Higher token fertility increases token usage for a fixed text; runtime and cost also depend on the model and output length (Section 7.1).

TurkicNLP puts both kinds of system behind the same `translate` processor: NLLB-200 for the varieties above, and an LLM backend that accepts all 24 toolkit language labels. A label that the model accepts may still produce poor translations. The following example compares outputs and language-ID predictions:

```
import turkicnlp

lid = turkicnlp.LanguageDetection()
text = "Алматы — Қазақстанның ең ірі қаласы."

nllb = turkicnlp.Pipeline("kaz", processors=["translate"], translate_tgt_lang="kir")
llm = turkicnlp.Pipeline("kaz", processors=["tokenize", "translate"],
 ↪  translate_backend="llm",
                          translate_tgt_lang="kir",
                          llm_model="qwen2.5:7b", llm_base_url="http://localhost:11434/v1",
                          llm_api_key="ollama")
for name, nlp in [("NLLB", nllb), ("LLM", llm)]:
    out = nlp(text).translation
    print(name, out, lid.model.predict(out.replace("\n", " "), k=3))  # unrestricted labels
    # Expected language is kir_Cyrl; these scores are diagnostics, not proof of correctness.
```

`translate_tgt_lang` accepts an ISO 639-3 code (`kir`), a FLORES-200 code (`kir_Cyrl`), or, for the LLM backend, a language name; with `translate_tgt_script` the LLM backend can also be asked for a specific script (e.g. Uzbek in Cyrillic).

10.4 Intra-Family Translation

10.4.1 Closely Related Pairs

Related languages share vocabulary and constructions that a model may exploit. Whether this improves translation over an equally resourced cross-family pair is a controlled empirical question. False cognates, different loanwords, language-specific auxiliary constructions and suffix allomorphy can still prevent correct copying or transfer.

An exploratory all-pairs comparison is available in the recorded Exercise 6. NLLB-200 600M with greedy decoding was applied to the first 15 X-WMT sentences per direction for six languages. chrF ranges from 31.3 (Turkish→Bashkir) to 56.6 (Bashkir→Kyrgyz). The mean is 44.3 for eight same-branch, same-script directions, 43.5 for four cross-branch, same-script directions, and 40.3 for eighteen cross-branch, cross-script directions. There are no same-branch, cross-script directions in this sample. Directions also use different

sentence content. The groups are small and unbalanced. They cannot isolate script or genealogical effects, let alone rank language pairs.

10.4.2 Pivot Strategies

When direct parallel data is limited, **pivot translation** routes the translation through an intermediate language. How limited depends on the pair: the TIL release has 12,216 Kazakh–Kyrgyz training pairs but 119,946 Uzbek–Kyrgyz ones, and NLLB-200 was trained on its own collection. Two possible routes for Uzbek→Kyrgyz are illustrated below.

Turkish pivot: Uzbek→Turkish→Kyrgyz. A larger English–Turkish corpus is no evidence that both legs of this route beat direct Uzbek→Kyrgyz translation. Inspect pair-specific data and compare outputs on the same held-out sentences.

Russian pivot: Uzbek→Russian→Kyrgyz provides another route. Shared terminology may help in some domains, but each decoding step can introduce errors. Exercise 4 compares both pivots with direct NLLB-200 translation on the same 50 X-WMT Uzbek–Kyrgyz sentences. Direct translation scores chrF++ 41.2, the Russian pivot 40.3 and the Turkish pivot 36.5. The Turkish route takes about twice as long in that run. In one sentence, its first leg enters a repetition loop and the second translates the repetitions. Record such error propagation alongside scores and latency, using fixed decoding settings.

10.4.3 Multilingual NMT

A multilingual model shares parameters across directions and uses language tags to specify the requested output. The multiway TIL study evaluates this approach (Mirzakhalov, Babu, Kunafin, et al. 2021). Results depend on the directions, sampling strategy, model capacity and domains; parameter sharing does not guarantee an improvement for every pair.

A useful experiment compares bilingual and multilingual systems with matched architectures, evaluation data and reported training budgets. Target-language tags must follow the model's own convention: an illustrative `<2kaz>` prefix is not interchangeable with NLLB's `kaz_Cyrl` token.

10.4.4 Zero-Shot Intra-Family Transfer

Zero-shot translation means evaluating a direction without its task-specific parallel training data. A multilingual model that represents both languages is a different starting point from a Turkish–Kazakh-only model fed Azerbaijani input. Related vocabulary may permit some transfer, but neither genealogy nor transliteration guarantees it. Compare native-script and converted input on the same held-out set, and check output language as well as adequacy.

10.5 Low-Resource Neural MT Strategies

10.5.1 Back-Translation

Back-translation is a widely used data augmentation strategy for low-resource MT (Sennrich, Haddow, and Birch 2016a). Given a monolingual corpus in the target language (e.g., Kyrgyz news articles), an existing MT system translates this corpus in the reverse direction (Kyrgyz→Turkish) to produce synthetic source sentences. The resulting synthetic parallel corpus (synthetic Turkish, real Kyrgyz) is then combined with real parallel data to train the final system.

Back-translation depends on the backward system's output. A Kyrgyz→Turkish Apertium system or small TIL baseline may generate noisy synthetic source sentences; test whether adding them helps. Source-side perplexity can flag some disfluency but can miss fluent mistranslations. Also inspect a sample for language, alignment, duplication and adequacy.

For Uyghur and other very low-resource Turkic languages, back-translation from a monolingual corpus offers a way to bootstrap: a reverse-direction system, where one exists, can generate candidate training pairs whose usefulness must be evaluated.

10.5.2 Fine-Tuning NLLB-200

NLLB-200 (NLLB Team et al. 2024) provides a pretrained starting point for the varieties in Table 10.3. Compare fine-tuning on a language pair or domain with the unadapted model and a bilingual system trained from scratch. Adaptation may improve the target task, but can also overfit or reduce performance elsewhere.

The tutorial uses `facebook/nllb-200-distilled-600M`. Memory requirements depend on sequence length, batch size, optimiser and precision; a single GPU is not guaranteed to suffice. The model card describes research use and a CC BY-NC 4.0 licence. Check suitability and licence terms before deployment; a large model is not automatically ready for production.

10.5.3 Transfer Learning from Turkish

Turkish-trained weights are one possible initialisation for an under-resourced language. Their value relative to a multilingual checkpoint must be tested. Kyrgyz and Tatar already have NLLB language codes, so they are not examples of missing NLLB coverage.

Adapting NLLB to a language outside its inventory requires training data and a policy for language tokens and vocabulary. For Karakalpak, Mamasaidov and Shopulatov (2024) trained a SentencePiece model, added 13,195 tokens to the distilled 600M model and initialised them from their subword embeddings. They used 100,000 sentence pairs each with Uzbek, Russian and English. Their baselines assigned Karakalpak the Kazakh or Uzbek code; that code substitution alone leaves the model without language-specific adaptation.

10.5.4 Synthetic Data from Apertium

For a supported translation direction, an Apertium system can generate synthetic parallel sentences from monolingual data (Section 10.2). A neural system can then train on a mixture of these sentence pairs and genuine parallel data. Evaluate whether the synthetic pairs help on held-out data from the intended domain.

10.6 Hands-On: A Fine-Tuning Procedure for Kazakh–Kyrgyz Translation with NLLB-200

Question. Does adaptation improve Kazakh–Kyrgyz translation over the original checkpoint?

Inputs. Prepared disjoint parallel splits and the pinned NLLB checkpoint.

Compute. Model downloads and fine-tuning resources; record the actual hardware and elapsed time.

Output. Baseline and adapted predictions, BLEU/chrF++ signatures and split manifests.

Interpretation. Compare both systems on the same reserved sentences and inspect incomplete outputs.

The executable tutorial is `code/ch10-mt/train.py`, with its input contract in `code/ch10-mt/README.md`. It adapts `facebook/nllb-200-distilled-600M` from Kazakh (`kaz_Cyrl`) to Kyrgyz (`kir_Cyrl`). The code mechanics have been checked with a tiny random model. The book reports no fine-tuned score; the Turkish–Kazakh greedy scores above illustrate baseline reporting but are not a Kazakh–Kyrgyz baseline. The runner evaluates the unadapted model on the prepared Kazakh–Kyrgyz data under the same four-beam decoding policy as the adapted model.

10.6.1 Prepare independent splits

Prepare `train.tsv`, `validation.tsv` and `test.tsv`, each with the header `document_id`, `src`, `tgt` (tab-separated). Keep original source-document groups together across all corpora and synthetic translations. Use validation for checkpoint selection and reserve the test set for the final fixed comparison. The loader rejects malformed or empty records, duplicate pairs, and cross-split document or normalised source/target text overlap. Unknown source-document provenance is a limitation to resolve, not a reason to invent independent IDs.

The TIL release lists 12,216 Kazakh–Kyrgyz training pairs (Table 10.1); an arbitrary tail of that file is not necessarily an independent validation set. Record upstream corpus revisions, licences, grouping and split rules alongside the prepared files. The runner saves their hashes and actual sizes.

10.6.2 Set and verify both language tokens

The runner uses the explicit `NllbTokenizer` class and pins the model and tokeniser revision. This matters because an automatic tokeniser loader may not preserve language-specific behaviour across library versions. The source encoding must begin with the Kazakh token and target labels with the Kyrgyz token; both must terminate in EOS.

```
from transformers import NllbTokenizer

MODEL_NAME = "facebook/nllb-200-distilled-600M"
REVISION = "f8d333a098d19b4fd9a8b18f94170487ad3f821d"
tokenizer = NllbTokenizer.from_pretrained(
    MODEL_NAME, revision=REVISION,
    src_lang="kaz_Cyrl", tgt_lang="kir_Cyrl", legacy_behaviour=False,
)
source_ids = tokenizer("Алматы")["input_ids"]
target_ids = tokenizer(text_target="Бишкек")["input_ids"]
assert source_ids[0] == tokenizer.convert_tokens_to_ids("kaz_Cyrl")
assert target_ids[0] == tokenizer.convert_tokens_to_ids("kir_Cyrl")
assert source_ids[-1] == target_ids[-1] == tokenizer.eos_token_id
```

All training and evaluation sentences are encoded without truncation. Examples above the declared token budgets stop the run so that segmentation and reconstruction can be addressed explicitly. Dynamic padding masks target padding with `-100` for the loss.

10.6.3 Use the same decoding policy throughout

The target token must be set **before development evaluation**, not only during later interactive inference. The runner applies this configuration to both the original and adapted model:

```
model.generation_config.forced_bos_token_id = tokenizer.convert_tokens_to_ids("kir_Cyrl")
model.generation_config.max_length = 256
model.generation_config.max_new_tokens = None
model.generation_config.num_beams = 4
model.generation_config.do_sample = False
```

The corresponding `Seq2SeqTrainingArguments` use `predict_with_generate=True`, `generation_max_length=256` and `generation_num_beams=4`. They were chosen for the tutorial and have not been tuned. The model is selected by validation **chrF++** (`metric_for_best_model="chrfpp"`), with epoch evaluation and checkpoint saving. Batch size and precision must be chosen for the available hardware.

10.6.4 Score original references and retain outputs

The metric callback decodes generated token IDs but compares them with the **original reference strings**, avoiding changes caused by decoding tokenised labels. It checks equal sentence counts and reports corpus BLEU (`tokenize="13a"`) and chrF++ (`word_o`

rder=2). The 13a tokeniser gives a reproducible baseline; it knows nothing about Turkic morphology.

Run from the book root in an environment with the versions listed in the companion README:

```
python code/ch10-mt/train.py path/to/prepared-splits --output nllb-kaz-kir
```

The script first measures the original model's development performance. It then trains, restores and saves the checkpoint with the best development score. Once training choices are fixed, it evaluates both checkpoints on the same test data with identical decoding.

Saved outputs include predictions, SacreBLEU signatures, versions, language-token IDs, split hashes and counts of outputs that reach the generation limit. Inspect those outputs for incomplete translations. Report development and test BLEU and chrF++ for both models, hardware, wall time and a paired analysis of their errors. The adapted model may improve or worsen the baseline.

Status: procedure only. The full adaptation run is not reported here. The exercise comparisons use their own declared model configurations.

What to Take Forward

Translation experiments need supported language directions, documented parallel data and a reserved test set. Performance varies across the tested directions, but the small all-pairs comparison cannot attribute those differences to relatedness or script. In a separate comparison on 50 Uzbek–Kyrgyz sentences, direct translation outperforms both tested pivots (Exercise 4). Establish the pretrained direct baseline first, then test pivoting, back-translation or adaptation under the same decoding and scoring settings. Chapter 11 extends the pipeline from written text to speech.

Further Reading

The TIL corpus and bilingual baselines are described in Mirzakhalov, Babu, Ataman, et al. (2021); multiway multilingual experiments are reported separately in Mirzakhalov, Babu, Kunafin, et al. (2021). NLLB Team et al. (2024) describes the NLLB-200 model architecture, training data, and evaluation methodology; its supplementary materials include coverage statistics for all 200 languages, including the eleven Turkic varieties.

For Apertium's rule-based approach, Forcada et al. (2011) provides the architectural overview, while Khanna et al. (2021) documents Apertium as of that publication. J. Washington, Salimzyanov, and Tyers (2014) describes the finite-state morphological transducers for three Kipchak languages (Kazakh, Tatar and Kumyk) on which Apertium's analysis and generation stages for these languages are built.

MT evaluation metrics are introduced in the "Machine Translation" chapter of Jurafsky and Martin (2024). Sennrich, Haddow, and Birch (2016a) describes back-translation

for neural MT. For using NLLB-200 inside a Turkic pipeline (translation and sentence embeddings for the covered Turkic varieties), see Hakimov (2026).

Exercises

1. **Data scarcity analysis.** Using Table 10.1 as a reference, compute the ratio between English–Turkish and English–Uyghur sentence pair counts. Choose two differently resourced pairs and design a learning-curve experiment. Specify domain, metric and acceptance criteria; do not assume a universal sentence-count threshold.
2. **Metric comparison.** Take 20 English sentences and translate them into Turkish using `facebook/nllb-200-distilled-600M`, with two decoding settings (for example greedy and beam search) as competing systems. Use references written or checked by a competent speaker, such as the human-post-edited X-WMT test sets. Compute corpus BLEU and chrF++, retaining metric signatures. For sentence-level diagnostics, state the BLEU smoothing and effective-order settings. Inspect examples where the two metrics rank competing outputs differently; their raw score magnitudes are not directly comparable. Distinguish legitimate variants from meaning-changing suffix errors.
3. **Apertium pipeline trace.** Install Apertium and the Kazakh–Tatar language pair (or, without an Apertium installation, run its stages by hand with the Kazakh analyser, the pair's bilingual dictionary and the Tatar generator). Translate the Kazakh sentence *Бүгін ауа райы өте жақсы.* ("The weather is very nice today.") and trace each stage of the pipeline: morphological analysis output, lexical transfer output, structural transfer output, and morphological generation output. What does the trace reveal about how each stage handles this sentence?
4. **Pivot translation comparison.** For the language pair Uzbek→Kyrgyz, implement two translation strategies using NLLB-200: (a) direct translation with codes `uzn_Latn → kir_Cyrl`; (b) pivot through Turkish (`uzn_Latn → tur_Latn`, then `tur_Latn → kir_Cyrl`). With TurkicNLP, (a) is `Pipeline("uzb", processors=["translate"], translate_tgt_lang="kir")` and (b) chains a `uzb→tur` and a `tur→kir` pipeline. Translate the same 50 held-out sentences (for example from the X-WMT Uzbek–Kyrgyz test set) under matched decoding settings; evaluate each strategy with chrF++, record added latency and inspect error propagation. Which approach scores higher on this sample? Use paired examples to investigate possible causes rather than treating the score difference as an explanation.
5. **Back-translation experiment.** Configure a separate Kyrgyz→Kazakh NLLB model (`src_lang="kir_Cyrl"`, `tgt_lang="kaz_Cyrl"`, forced target token `kaz_Cyrl`) to back-translate a small training-only monolingual Kyrgyz corpus (1,000 sentences from Wikipedia or news, disjoint from development/test source documents) into Kazakh, producing synthetic Kazakh. Combine this synthetic data with the available real training pairs and retrain the model. Tune the synthetic-data amount on validation, then compare fixed systems on the same untouched test set. Does adding back-translated data improve chrF++? How sensitive is the gain to the size of the back-translated corpus?

6. **An intra-Turkic quality matrix.** Load the FLORES-200 devtest sentences for the eleven NLLB Turkic varieties (Table 10.3), or the X-WMT test sets for the six languages that X-WMT and NLLB share. Align by documented sentence IDs or verify correspondence from the release; do not assume that independently filtered files remain row-aligned. For an exploratory subset, select and record the sentence IDs (for example 200) and translate between distinct-language pairs with `facebook/nllb-200-distilled-600M` (one TurkicNLP `translate` pipeline per source language, with `translate_tgt_lang` set to each target) and compute chrF against the references. Plot the matrix as a heatmap. Label the matrix as a subset experiment, exclude identity directions, and tune choices only on development data. Do within-branch pairs score higher than cross-branch pairs? Do same-script pairs score higher than cross-script pairs with similar genealogical distance?

7. **Beyond NLLB's languages.** NLLB-200 does not cover Nogai, Kumyk or Sakha, but TurkicNLP's LLM backend accepts all 24 toolkit language labels. Take 20 Nogai (or Sakha) sentences with reference translations into Russian, written or checked by a native speaker. Translate them with `Pipeline("nog", processors=["tokenize", "translate"], translate_backend="llm", translate_tgt_lang="rus", ...)` using two models of different sizes, and compute chrF against the references. Then reverse the task with a short prompt to the same models (Russian → Nogai) and use the unrestricted `lid.model.predict(...)` method of `turkicnlp.LanguageDetection()` on the outputs. How often does it assign another language label (Section 3.7)? Have a competent speaker check those cases: an LID label is not a verified language annotation.

11 Speech Technologies

Learning Objectives

After reading this chapter, you will be able to:

- Describe the phonological characteristics of Turkic languages (vowel systems, consonant inventories and prosodic patterns) and explain how they affect speech technology design
- Explain the three main end-to-end ASR architectures (CTC, attention encoder-decoder, RNN-T) at the level needed to evaluate system choices for Turkic languages
- Characterise the Turkic ASR ecosystem: Whisper coverage, TurkicASR, Söyle, massively multilingual models, and the open datasets Common Voice and FLEURS
- Explain how written vowel distinctions, pronunciation and prosody affect text-to-speech synthesis
- Audit released TTS coverage separately from evaluated quality and identify evidence gaps
- Design a first ASR experiment using cross-lingual acoustic transfer and data augmentation for a low-resource Turkic language

Speech systems connect recordings with written text. They must handle the target language's sounds, spelling and pronunciation. Related languages may offer useful training data, but performance also depends on speakers, recording conditions and domain.

This chapter covers the phonological foundations relevant to speech processing (Section 11.1), automatic speech recognition (Section 11.2), text-to-speech synthesis (Section 11.3), and the specific low-resource strategies applicable to Turkic speech (Section 11.4). The hands-on provides a Whisper fine-tuning recipe for Kazakh ASR using the Hugging Face ecosystem.

11.1 Phonological Sketch of Turkic Languages

11.1.1 Vowel Systems and Orthography

The grammar-based inventory in Table 2.4 distinguishes phoneme analyses from alphabet letters. Use that inventory and the harmony discussion in Chapter 5 when designing a pronunciation lexicon; a single family-wide vowel count is inappropriate. Johanson

treats sound types and vowel systems in Chapters 12–18 and word accents in Chapter 50 (Johanson 2021). Standard Uzbek and harmony-preserving Uzbek varieties must be distinguished.

A speech system maps between acoustic realisations and a particular transcription convention. Surface suffix vowels are normally already written: the abstract morphophoneme *A* in a morphological rule is not an unspecified vowel in ordinary Turkish input. Pronunciation modelling must still handle allophony, loans, abbreviations and variety-specific spelling–sound correspondences.

11.1.2 Consonants and Transfer

Transliteration can obscure sound differences. Standard Turkish *ğ* usually lengthens the preceding vowel or surfaces as a glide; Kazakh *ғ* is a voiced back fricative. An acoustic encoder may nevertheless transfer useful features to sounds absent from its source language. Evaluate that transfer on held-out target speech and inspect the confusions, rather than predicting accuracy from shared letters.

11.1.3 Prosody

Stress, phrase boundaries, duration and intonation depend on the language, variety and construction. A blanket final-stress rule is insufficient. A morphological analyser can supply candidate features, but selecting the reading and predicting prosody require contextual modelling. Evaluate an analyser-assisted front end against the same model without those features, with fluent listeners and matched recordings.

A Turkmen example. Turkmen retains primary long vowels (Gökçür 2016). Its spelling usually leaves length unmarked. The exception is ‹üý› for /yː/ (Johanson's /üː/), as in *süýt* 'milk' and *güýç* 'strength' (Johanson 2021, 257–58). Thus *at* can represent /at/ 'horse' or /aːt/ 'name', and *ot* /ot/ 'grass' or /oːt/ 'fire' (Johanson 2021, 248, 258). Before a vowel-initial suffix, the consonant following the long vowel becomes voiced: compare *ady* 'his/her name' with *aty* 'his/her horse'.

The Apertium analyser gives both *at* nouns the same `at<n>` analysis, alongside a verb reading, and both *ot* nouns the same `ot<n>`. A TTS front end therefore needs lexical and contextual information beyond these analyses to choose a pronunciation.

Turkmen also differs from Turkish in its consonants. In native words, ‹s› and ‹z› represent /θ/ and /ð/: *süýt* /θyːt/ 'milk', *duz* /duːð/ 'salt'. Plain /s/ and /z/ occur in loans (Johanson 2021, 258, 262). Turkish letter-to-sound rules would miss this distinction. These transcriptions use IPA; Johanson writes the same sounds as /üː/, /θ/ and /δ/ in Turkological notation.

11.2 Automatic Speech Recognition

11.2.1 End-to-End Architectures

Three widely used end-to-end ASR architectures provide the background for the Turkic systems discussed here. For algorithmic details see the chapter on automatic speech recognition in Jurafsky and Martin (2024).

CTC (Connectionist Temporal Classification) (Graves et al. 2006) trains an encoder to predict a probability distribution over output units (such as characters or subwords) and a blank symbol at each encoder time step, then uses the CTC loss to align the predicted sequence with the target text without requiring explicit frame-to-phoneme alignment. Its frame-level predictions are conditionally independent given the encoded audio. Greedy decoding needs no autoregressive decoder, while beam search may incorporate an external language model; MMS and the Omnilingual CTC models below use this design.

Attention encoder-decoder uses an encoder (typically a convolutional front-end followed by transformer layers) to produce a sequence of audio representations, and a decoder that attends over these representations to generate output tokens autoregressively. This architecture is the basis of Whisper (Radford et al. 2023). The decoder uses preceding output tokens as well as the audio.

RNN-T (Recurrent Neural Network Transducer) (Graves 2012) combines an acoustic encoder with a prediction network that encodes the preceding output labels. A joint network uses both representations to predict the next label or blank. RNN-T supports streaming when the acoustic encoder and decoding policy are causal. Compare latency and recognition error on the intended workload; architecture by itself decides little about suitability for deployment.

11.2.2 Whisper

The original Whisper models (Radford et al. 2023) were trained by OpenAI on 680,000 hours of weakly supervised web audio; 117,000 of those hours cover 96 languages other than English. Their tokeniser has 99 language tokens. The later `large-v3` model added Cantonese, bringing the count to 100 in its generation configuration. Whisper uses an attention encoder-decoder architecture with a log-mel spectrogram front end and a transformer encoder and decoder. The original release had five sizes (tiny, base, small, medium, large), with the large model having 1,550 million parameters.

Whisper has dedicated tokens for seven Turkic languages: Turkish (`tr`), Azerbaijani (`az`), Uzbek (`uz`), Kazakh (`kk`), Tatar (`tt`), Turkmen (`tk`) and Bashkir (`ba`). **Kyrgyz and Uyghur have no dedicated Whisper language tokens.** Training audio varies widely by language; the paper notes that most languages have less than 1,000 hours. Its FLEURS and Common Voice word error rates provide per-language reference baselines.

Exercise 4 tests an int8 ONNX export of Whisper small with greedy decoding on Piper-synthesised speech: 20 Turkish items from one voice and 30 Kazakh items from six voices. Normalised CER is 5.9% and 54.4%, respectively. Different texts and voices prevent attributing this gap to language alone. Within the Kazakh sample, the Kazakh token

outperforms each tested alternative language token and automatic language detection. These synthetic-audio diagnostics leave performance on real recordings unmeasured.

Evaluate Whisper on the intended domain for every language, including Turkish; for Kyrgyz and Uyghur, start from a model that covers them (TurkicASR, Söyle, MMS, or Omnilingual ASR) instead. The `whisper-small` (244 million parameters) and `whisper-medium` (769 million) checkpoints are common starting points for fine-tuning; measure memory and runtime on the hardware at hand.

11.2.3 TurkicASR

TurkicASR (Mussakhojayeva et al. 2023) is a multilingual ASR system developed at ISSAI that covers 10 Turkic languages: Azerbaijani, Bashkir, Chuvash, Kazakh, Kyrgyz, Sakha, Turkish, Tatar, Uzbek, and Uyghur. A single model is trained jointly on data from all ten languages; the work also released a 218.2-hour Turkish speech corpus.

Kazakh Speech Corpus 2 (KSC2) contains over a thousand hours of transcribed Kazakh speech, combining the earlier KSC and KazakhTTS2 recordings with television and radio, parliamentary speeches and podcasts, and including Kazakh–Russian code-switching (Mussakhojayeva, Khassanov, and Varol 2022a).

TurkicASR's best jointly trained model reduced character and word error rates by 56.7% and 54.3% relative, on average, compared with monolingual models. Adding related Turkic language data helped more than adding English or Russian (Mussakhojayeva et al. 2023). These results support the tested multilingual recipe. A controlled comparison would be needed to separate the contributions of shared phonology, vocabulary and other factors, or to assess the benefit for another language.

Söyle (Mussakhojayeva et al. 2024) is a Whisper-based system from ISSAI covering 11 Turkic languages and the official languages of the United Nations, released together with the Tatar speech corpus TatSC. It is designed for noisy recording conditions and fine-tunes Whisper on noise-augmented speech, with improvements reported on the paper's evaluation conditions. Deployment on a new microphone, domain or speaker population still requires a separate test.

11.2.4 Open Speech Datasets: Common Voice and FLEURS

Mozilla Common Voice (Ardila et al. 2020) is a crowd-sourced speech dataset collected through volunteer recording. Several Turkic languages have Common Voice datasets, and their sizes do not follow the general resource hierarchy. In the release statistics for Common Voice 27.0 (11 September 2026), Uyghur has 462 validated hours and Bashkir 259, against 130 for Turkish, 102 for Uzbek, 39 for Kyrgyz, 33 for Tatar, 25 for Chuvash, 16 for Sakha, 3 each for Kazakh and Turkmen and 1 for Azerbaijani (`datasets/scripted-speech/cv-corpus-27.0-2026-09-11.json` in the `common-voice/cv-dataset` repository, commit `65ca246`). These counts describe validated audio, not all submitted recordings. Check the current release statistics for each language before planning an experiment. Mozilla's distribution notice directs downloads to Mozilla Data Collective from October 2025, so older loading code may no longer work.

FLEURS (Conneau et al. 2023) provides about 12 hours of read speech per language for 102 languages, recorded for FLORES sentences, with fixed train/dev/test splits; its Turkic languages are Turkish, Azerbaijani, Kazakh, Kyrgyz and Uzbek. Its shared source-sentence design supports comparisons, but recordings, retained items and speakers differ across languages. Check prompt IDs and split contents when constructing a matched comparison; the approximate hours describe the whole dataset per language, not the training split alone.

11.3 Text-to-Speech

11.3.1 Text and Pronunciation in Turkic TTS

A grapheme-based TTS system receives the written surface form, including the selected suffix vowels. A phoneme-based system needs a pronunciation front end that respects the target variety. Suffix vowels use the language's vowel inventory; each grammatical suffix does not need its own phoneme symbol. Morphological features may help disambiguation or prosody, but their benefit is an experimental question.

Keep text normalisation (numbers, abbreviations and names), grapheme-to-phoneme conversion and prosodic prediction separate in error analysis. Test both frequent forms and productive unseen inflections. For synthesis, report intelligibility separately from naturalness and speaker similarity.

11.3.2 KazakhTTS2 and TurkicTTS

KazakhTTS2 (Mussakhojayeva, Khassanov, and Varol 2022b) contains 271 hours of studio-recorded Kazakh speech from five speakers (three female, two male), up from 93 hours and two speakers in the first release, with subjective mean opinion scores of 3.6 to 4.2 across the speakers. Its repository releases Tacotron 2 models with Parallel WaveGAN vocoders for each speaker. Emotional speech is covered by the separate **KazEmoTTS** dataset: 74.85 hours from three narrators in six emotions (Abilbekov et al. 2024). The open Piper voice `kk_KZ-issai-high`, a VITS model trained on the ISSAI data with six speakers, is used in this chapter's exercises.

TurkicTTS (Yeshpanov, Mussakhojayeva, and Khassanov 2023) tests transliteration-based synthesis for ten languages, including Kazakh, its training language. The other nine are zero-shot targets. Letters are mapped through IPA to Kazakh letters before a Kazakh-trained Tacotron 2 model synthesises the text. The authors report subjective evaluations of this procedure. Mapping target sounds to Kazakh letters can approximate or merge distinctions, so a shared representation does not guarantee faithful target-language pronunciation.

11.3.3 Neural TTS Architectures

The released systems discussed here use Tacotron 2 and VITS. Codec language modelling provides a further architectural comparison:

Tacotron 2 (Shen et al. 2018) uses a sequence-to-sequence architecture with attention to convert grapheme or phoneme sequences to mel spectrogram frames, followed by a separate neural vocoder (WaveNet in the original paper, Parallel WaveGAN in the KazakhTTS2 release) to convert spectrograms to audio. KazakhTTS2 and TurkicTTS both use it.

VITS (Variational Inference with adversarial learning for end-to-end TTS) (Kim, Kong, and Son 2021) is an end-to-end architecture that combines the acoustic model and vocoder in a single network trained with a generative adversarial loss. Its inference procedure can sample durations and latent variables. Repeated output therefore depends on the random seed and noise settings (Exercise 3 measures this for the selected Piper implementation). The MMS and Piper Turkic voices are VITS models. Its relative quality depends on data, training and the listener evaluation.

VALL-E style codec language modelling (S. Chen et al. 2025) treats TTS as a language modelling problem over discrete speech codec tokens. The cited work studies voice-conditioned synthesis; its quality on Turkic languages is untested. Data and compute requirements depend on the checkpoint and adaptation protocol. Voice-cloning experiments additionally require explicit permission for the intended voice use.

11.3.4 Availability and Evaluated Quality

Public VITS checkpoints exist for MMS Chuvash TTS and MMS Crimean Tatar TTS, both under CC BY-NC 4.0. The pinned entries in Table B.6 document their availability. Assess naturalness, dialect coverage and suitability for the intended use separately.

For each candidate, check its tokeniser's expected script, training domain, speaker coverage and licence, then run a shared prompt set. Use blinded, counterbalanced listener judgements with the same prompts, listening conditions and evaluation scale for competing systems. Do not compare a new MOS directly with a historical score collected from different listeners.

11.4 Speech for Low-Resource Languages

11.4.1 Cross-Lingual Acoustic Model Transfer

Start with a checkpoint whose input/output conventions and language coverage fit the task. Compare its unadapted output with full fine-tuning and a declared partial-freezing policy. Build learning curves by increasing **training** hours while keeping validation speakers and transcripts fixed; select the recipe on validation, then evaluate once on the reserved test set. No universal number of hours guarantees useful ASR. Shared

phonetic structure motivates transfer but does not identify its cause without controlled comparisons (Mussakhojayeva et al. 2023).

11.4.2 Data Augmentation

Apply augmentation to training recordings only, recording its algorithm, probability, seed and parameter distribution. Resampling-based speed perturbation changes pitch and duration together (Ko et al. 2015). Pitch-preserving time stretching changes duration separately. Pitch shifting varies a recording's pitch while retaining the same speaker.

Exercise 2 checks these operations on synthetic Kazakh speech. Duration and median fundamental-frequency ratios use six items; ASR errors use 30. A speed factor of 0.9 produces about 1.11 times the duration and 0.90 times the fundamental frequency. The other operations approximately preserve the quantity they are designed to leave fixed.

On this sample, time stretching and pitch shifting increase CER more than the tested speed perturbations or 20 dB noise. This tests recogniser sensitivity at inference. An experiment that trains with augmentation is needed to measure its benefit.

SpecAugment (Park et al. 2019) masks time and frequency regions of the acoustic representation. Additive noise is mixed at a declared signal-to-noise ratio; reverberation is produced by **convolving** audio with a room impulse response. These are separate interventions. Preserve intelligibility and transcript alignment, and compare each intervention at the same update budget before combining them.

11.4.3 Massively Multilingual Speech Models

Meta's **Massively Multilingual Speech (MMS)** project covers recognition and synthesis in over 1,100 languages, and language identification in about 4,000 (Pratap et al. 2024). It uses separate systems: wav2vec 2.0-based models for recognition and identification, and VITS for synthesis. Recordings of religious texts supply much of the data for small languages, limiting domain coverage; recording conditions also need checking.

The later **Omnilingual ASR** system covers more than 1,600 languages with CTC models and models using an LLM-inspired decoder, ranging from about 300 million to 7 billion parameters (Omnilingual ASR Team et al. 2025). TurkicNLP accepts 20 Turkic languages for this backend. Tuvan, Khalaj, Ottoman Turkish and Old Turkic lack an Omnilingual language code (TurkicNLP 0.5.0 README).

MMS and Omnilingual ASR offer further candidates where the earlier systems lack coverage. Compare an unadapted model with fine-tuning on target-domain speech. Evaluate on FLEURS where available, or on held-out recordings from the intended domain.

11.4.4 Speech Recognition with TurkicNLP

TurkicNLP wraps Omnilingual ASR as `SpeechRecognizer`. The wrapper takes TurkicNLP language codes, checks that the model supports them, splits recordings longer than the model's 40-second limit at pauses, and returns transcripts in the language's TurkicNLP script. Where the model writes another script, the toolkit transliterates: Karakalpak and Crimean Tatar come out of the model in Cyrillic and are returned in Latin. Speech recognition needs its own Python environment (`pip install "turkicnlp[all,asr]"` in a fresh environment, because the ASR dependencies pin older versions of PyTorch and NumPy).

```
import turkicnlp

print(turkicnlp.list_asr_languages(check_model=True))   # e.g. {'kaz': {'Cyrl': 'kaz_Cyrl'},
 ↪  ...}

asr = turkicnlp.SpeechRecognizer(model_card="omniASR_LLM_1B", device="cpu")
texts = asr.transcribe(["kaz_sample.wav", "tat_sample.wav"], lang=["kaz", "tat"],
 ↪  batch_size=2)

res = asr.transcribe("kaz_sample.wav", lang="kaz", return_details=True)
for seg in res.segments:                          # time-aligned pieces of long recordings
    print(f"[{seg.start:.2f}-{seg.end:.2f}] {seg.text}")
print(asr.transcribe("kaz_sample.wav", lang="kaz", script="Latn"))   # 2021 Latin alphabet
```

The transcript can go straight into the text pipeline:

```
nlp = turkicnlp.Pipeline("kaz", processors=["asr", "tokenize", "pos", "lemma", "depparse"],
                         asr_model_card="omniASR_LLM_1B", asr_device="cpu")
doc = nlp.from_audio("kaz_sample.wav")
print(doc.text, doc.audio_segments)
for w in doc.words:
    print(w.text, w.upos, w.lemma, w.head, w.deprel)
```

The wrapper's documented Omnilingual output is lower-case and unpunctuated. This differs from edited training text and can affect tagging and parsing; measure downstream accuracy on checked transcripts rather than assuming the size or direction of the change. Memory use depends on the checkpoint, precision, loader and recording length; measure it in the intended environment.

11.5 Hands-On: A Fine-Tuning Procedure for Kazakh ASR with Whisper

Question. Does Whisper adaptation improve recognition on held-out Kazakh recordings?

Inputs. The pinned FLEURS Kazakh release and Whisper-small checkpoint below.

Compute. Audio decoding, checkpoint downloads and training memory; a GPU is useful for fine-tuning.

Output. Saved predictions, raw and normalised WER/CER, and an experiment manifest.

Interpretation. Compare real-recording baseline and adapted output under the same scoring policy.

The executable implementation is speech.py. It uses `google/fleurs`, configuration `kk_kz`, at revision `70bb2e84b976b7e960aa89f1c648e09c59f894dd`, and the pinned `openai/whisper-small` checkpoint in Table B.6. The current dataset release has Parquet files; the loader requests undecoded audio and uses SoundFile explicitly rather than assuming an older `datasets.Audio` array interface.

```
python code/ch11-speech/speech.py runs/kazakh-asr --epochs 3 --seed 42
```

The script keeps official train, validation and test splits separate. It rejects duplicate recording paths and cross-split identical transcripts, reports retained duration per split, and relies on the dataset card's speaker separation because speaker IDs are not exposed in this schema. If these checks fail, investigate the release rather than silently redistributing benchmark items.

Only finite mono 16 kHz recordings of at most 30 seconds enter this short-utterance recipe. Longer audio must be segmented with aligned transcripts in a separately documented experiment. Truncating a recording while retaining its whole transcript teaches the wrong alignment. Overlength decoder targets are also rejected.

The collator removes the leading **decoder-start** token exactly once; Whisper inserts it during label shifting. It retains language/task tokens, supplies the acoustic attention mask, and masks label padding with `-100`. Checkpoint selection uses validation WER. After selection, the baseline and adapted checkpoint are decoded on the **same complete test items** with the same language, task and generation limit.

The scorer compares predictions with the original reference strings. It reports corpus-level raw and conservatively normalised WER/CER, empty-reference counts, recording-ID hashes and per-item transcripts. Its normalisation preserves alphabet distinctions and apostrophes; state that policy when comparing scores. Reference words or characters form the denominator, while insertions against empty references still contribute to the numerator.

The implementation checks exercise the collator, audio guards, metric arithmetic and a tiny randomly initialised Whisper forward/backward pass. The recorded Exercise 1 uses the same scoring functions on synthetic speech, but a different int8 ONNX export and runtime. Its error rates are not the unadapted FLEURS baseline for this training recipe.

For long-recording inference, use a separately validated chunking/stitching procedure. For short recordings, call `model.eval()` and `torch.inference_mode()` and enforce the same sample-rate and duration checks.

Status: procedure only. The fine-tuning run on real recordings is not reported here.

What to Take Forward

Speech systems connect written conventions with sounds, timing and speakers. The synthetic-audio probes illustrate recogniser behaviour on specific texts and voices; ranking languages by quality on this basis would overreach. Evaluate ASR on real recordings under a stated normalisation policy and TTS with listeners. Compare cross-language training and augmentation on held-out speech. Chapter 12 returns to text and pretrained models.

Further Reading

The TurkicASR system is described in Mussakhojayeva et al. (2023), the primary reference for ISSAI's multilingual Turkic ASR. Mussakhojayeva, Khassanov, and Varol (2022b) describes KazakhTTS2, and Yeshpanov, Mussakhojayeva, and Khassanov (2023) the zero-shot TurkicTTS approach.

The Whisper paper (Radford et al. 2023) describes the training procedure, data filtering methodology, and language coverage of the Whisper model series; its supplementary materials include per-language WER breakdowns that are the primary source for Turkic ASR baselines. For speech processing fundamentals including CTC, attention-based encoder-decoders, and the RNN-T architecture, the speech chapters of Jurafsky and Martin (2024) provide a thorough treatment.

For Turkic phonology, Johanson (2021) (chapters 12–18 and 50) covers vowel systems, consonant inventories, and prosodic patterns with systematic comparison across the family. Use these descriptions when checking pronunciation assumptions in a speech system.

Exercises

1. Measure the unadapted Whisper baseline on a frozen Kazakh test set: raw and normalised WER and CER, total duration, results per speaker where identifiers are available, and error examples. FLEURS does not expose speaker IDs in the schema used here, so do not invent a speaker breakdown from recording paths. With the Hands-On script, compare it with the adapted checkpoint on the same items, choosing all decoding settings on validation.
2. Apply resampling speed perturbation, pitch-preserving time stretching, pitch shifting and additive noise separately. Verify what each operation changes (duration, fundamental frequency, signal-to-noise ratio) and measure the recogniser's sensitivity to each before using any of them in training, where they are compared at matched updates and seeds and kept in their source split.
3. Compare TTS voices with an ASR-based transcription-error diagnostic, measure how much repeated sampling of the same text changes it, and distinguish recogniser errors from synthesis errors. This diagnostic alone cannot rank human intelligibility

or naturalness. Use the same prompts for every voice and design a blinded, counter-balanced listening test for fluent, compensated listeners. Obtain permission for any cloned voice.

4. Decode Kazakh and Turkish speech with the Whisper tokens of related languages and with automatic language detection. Check which Turkic language tokens the model accepts; for Kyrgyz or Uyghur, compare related-language and multilingual initialisations by fine-tuning instead of forcing an unsupported token.

Part VI

Language Models

12 Pretrained Language Models for Turkic

Learning Objectives

After reading this chapter, you will be able to:

- Survey the encoder-only (BERT-class) models for Turkic languages and compare monolingual and multilingual models for a task
- Distinguish changes in tokeniser fertility from measured changes in task performance
- Identify the major generative language models for Turkish and Kazakh and how they were built
- Describe the strategies for adapting a pretrained model to a new Turkic language (continued pretraining, vocabulary extension, LoRA) and the steps of adapting a decoder LLM, from tokeniser extension to instruction tuning and evaluation
- Explain the vocabulary overlap problem and its consequence for cross-lingual transfer across script boundaries within the Turkic family
- Reason about the implications of high tokeniser fertility for context window consumption and deployment cost in Turkic-language applications

Chapter 7 introduced neural morphology and tokenisation. This chapter builds on that foundation: selecting pretrained encoders, adapting generative models and evaluating the results. It separates language coverage, tokenisation and task accuracy.

The chapter covers encoders (Section 12.1), generative models (Section 12.2), adaptation (Section 12.3) and context budgeting (Section 12.4). The hands-on provides an XLM-R classification recipe with LoRA. The exercises report separate experiments with Glot500 on SIB-200.

12.1 Encoder-Only Models for Turkic Languages

12.1.1 The BERT Architecture as Infrastructure

BERT-style encoders (Devlin et al. 2019) learn contextual token representations through masked language modelling: predict selected tokens from their surrounding text. A task-specific head then maps those representations to labels or answer spans.

For POS tagging and NER, a classification head predicts labels from token representations. Sequence classifiers use a pooled representation, often the first special token. Extractive question-answering heads predict answer boundaries in the supplied context. Token alignment and label conventions still need task-specific handling, as in Section 9.6.

The training pattern is reusable across languages, but changing the model name and dataset is not enough. Check the tokeniser, input length, annotation scheme and evaluation procedure for each model and task.

12.1.2 BERTurk

BERTurk has four main BERT variants: cased and uncased models, each with a 32K or 128K vocabulary (Schweter 2020). The release README reports a 35 GB, 4.4-billion-token training corpus drawn from Turkish OSCAR, Wikipedia, OPUS and material supplied by Kemal Oflazer. Cased models preserve capitalisation; uncased models lowercase the input. Check how each normaliser handles Turkish dotted and dotless letters (Chapter 3).

A larger vocabulary increases the embedding matrix and may shorten token sequences (Section 7.2). Exercise 1 compares published BERTurk task scores. Its recorded run did not measure the BERTurk tokenisers, which were unavailable locally.

Table 12.1: The main BERTurk variants. The family also includes DistilBERTurk, ConvBERTurk, ELECTRA and T5 models, some trained on the Turkish part of mC4 (242 GB).

Model identifier	Vocabulary	Cased
`dbmdz/bert-base-turkish-cased`	32K	yes
`dbmdz/bert-base-turkish-uncased`	32K	no
`dbmdz/bert-base-turkish-128k-cased`	128K	yes
`dbmdz/bert-base-turkish-128k-uncased`	128K	no

The 32K and 128K vocabularies require different-sized embedding matrices, so the models also differ in parameter count. To compare monolingual and multilingual encoders, use the same splits, preprocessing and search budget.

The BERTurk repository's five-run evaluations show that the preferred model depends on the task. On WikiANN NER, cased BERTurk 128K has the best BERTurk test F1: 93.90, compared with 92.96 for XLM-R base. On BOUN POS tagging, XLM-R base scores highest at 91.86% accuracy, followed by cased BERTurk 32K at 91.49 and cased 128K at 91.02 (`0LD_EVALUATIONS.md` in `stefan-it/turkish-bert`, commit `0ab49e2`).

12.1.3 Multilingual Models: mBERT and XLM-R

Multilingual BERT (mBERT) (Devlin et al. 2019) uses a shared WordPiece vocabulary and multilingual masked-language pretraining. Current Wikipedia article counts cannot reconstruct the training distribution of a historical checkpoint. Record the checkpoint

and its documented training coverage, then measure tokenisation on the actual target corpus.

XLM-R (Conneau et al. 2020) is trained on 2.5 terabytes of filtered CommonCrawl data in 100 languages and reported large gains over mBERT on cross-lingual benchmarks such as XNLI, which includes Turkish. The model is available in two sizes: `xlm-roberta-base` and `xlm-roberta-large` (approximately 270M and 550M parameters in the original paper; exact counts depend on the loaded task head).

XLM-R's CC-100 training corpus includes Turkish, Azerbaijani, Uzbek, Kazakh, Kyrgyz and Uyghur. It excludes Tatar, Bashkir, Turkmen, Chuvash, Sakha and the smaller Turkic languages. Related languages and shared scripts may still support transfer to them; measure the resulting representation quality.

Glot500 extends XLM-R to 500+ languages and expands its vocabulary from 250,002 to 401,145 entries (Imani et al. 2023). All but one of XLM-R's pieces retain their IDs (Exercise 1). It covers many more Turkic languages and supplies the frozen encoder for TurkicNLP's multilingual POS tagger and dependency parser (Chapter 8).

For cross-lingual work, include a multilingual encoder as a baseline. Kardeş-NLU reports gains from Turkish intermediate training and fine-tuning on its tested NLU tasks in Azerbaijani, Kazakh, Kyrgyz, Uzbek and Uyghur (Senel et al. 2024). The parsing results in Chapter 8 use different tasks and reference sets; they should be interpreted separately.

12.1.4 Language-Specific Models and Model Selection

BERTbek (Kuriyozov, Vilares, and Gómez-Rodríguez 2024) is a set of three Uzbek BERT models trained on different text sources and corpus sizes; the authors report that they outperform mBERT on sentiment analysis, multi-label topic classification and NER. Script, casing, training domain and released weights must be checked for each model separately; "Uzbek" alone does not specify its input conventions. The catalogue is selective; many more models have been released.

For within-language work, compare a suitable monolingual encoder with a multilingual baseline. For transfer, include a multilingual model whose language coverage and tokeniser fit the target. Keep mBERT where it is relevant to a historical comparison or deployment constraint. Neither monolingual specialisation nor a larger corpus guarantees superiority on every task. Table B.6 records exact configurations used in the book's experiments.

12.2 Generative Language Models for Turkic

12.2.1 Turkish Generative Models

TURNA is a Turkish encoder-decoder model pretrained with UL2 on a corpus curated for the project (Uludoğan et al. 2024). The authors evaluate three generation and five understanding tasks. They report gains over several multilingual models and competitive

understanding results against monolingual Turkish models. Those comparisons support conclusions about the tested datasets and baselines; they leave broader comparisons with decoder-only models open.

For any additional academic, community or commercial checkpoint, record its exact model card, base revision, tokeniser, adaptation data and licence before adding it to an experiment. A name without a verified release is not a reproducible baseline.

12.2.2 Kazakh: KAZ-LLM and Sherkala

KAZ-LLM (Institute of Smart Systems and Artificial Intelligence (ISSAI) 2024), released by ISSAI in December 2024, comes in 8-billion and 70-billion-parameter versions built on Llama 3.1. ISSAI reports training on more than 150 billion tokens of Kazakh, Russian and English text, with additional support for Turkish; the weights are licensed CC BY-NC 4.0. Its evaluation code (the `IS2AI/KazLLM_Benchmark` repository) runs MMLU, Winogrande, HellaSwag, ARC, GSM8K and DROP in Kazakh, Russian and English.

Sherkala-Chat (Koto et al. 2025) is an 8-billion-parameter Kazakh chat model built by adapting Llama 3.1 8B in a "moderately resourced" setting: continued pretraining on 45.3 billion tokens of Kazakh, English, Russian and Turkish text, instruction tuning on translated data plus an automatically built, manually checked Kazakhstan-specific instruction set, and Kazakh safety data. Its paper is a documented recipe for adapting an existing LLM to a Turkic language.

12.2.3 Other Generative Models and the Multilingual Fallback

Other languages. This catalogue is selective. A missing entry means that a release has not been verified here, not that no dedicated model exists.

Multilingual models. Where no dedicated model fits the task, evaluate a multilingual model. TUMLU (Isbarov et al. 2025) tests multiple-choice performance on native questions in the listed Turkic languages (Chapter 15). It does not directly measure grammaticality, generation quality or cultural adequacy.

Exercise 4 gives a narrower comparison: Qwen2.5-1.5B-Instruct (Qwen Team et al. 2024) scores 84.2% on 500 English COPA test items and 52.2% on their Kazakh translations from Kardeş-NLU (Ponti et al. 2020; Senel et al. 2024). The Kazakh score's 95% interval, 47.8–56.5%, includes the 50% random-choice baseline. This result concerns one model and scoring protocol; it does not identify the cause of the language gap.

Combining an LLM with toolkit analyses. TurkicNLP offers LLM backends for morphology, NER, translation, sentiment and spelling correction (Section 9.5, Section 10.3). With `morph_backend="apertium_llm"`, the LLM selects among the analyser's candidate readings for ambiguous words. This constrains the selection, but the candidates may be incomplete and the chosen reading may be wrong. A free-form explanation from `llm.ask` is a separate output and needs checking:

```
import turkicnlp

llm = dict(llm_model="qwen2.5:7b", llm_base_url="http://localhost:11434/v1",
 ↪  llm_api_key="ollama")

nlp = turkicnlp.Pipeline("kaz", processors=["tokenize", "morph"],
 ↪  morph_backend="apertium_llm", **llm)
doc = nlp("Ол кітабымды оқып жатыр.")
print([(w.text, w.lemma, w.upos, w.feats) for w in doc.words])

print(turkicnlp.llm.ask("Explain the verb form in this sentence.", doc=doc, **llm))
```

The same `llm_*` options work with an OpenAI-compatible server (vLLM, Ollama), OpenAI, Claude (`llm_provider="anthropic"`) or an in-process Hugging Face model (`llm_provider="hf"`). `doc._processor_log` records the model and prompt version used, which you need when reporting results.

12.3 Building Turkic Language Models

12.3.1 Continued Pretraining

Continued pretraining updates a pretrained model on additional text. For BERT-style encoders, it continues masked language modelling. Decoder LLMs instead continue their next-token prediction objective. Compare the adapted checkpoint with the original on the intended tasks.

Continued pretraining can adapt a model even when the target language was absent from its documented training data. Glot500 is an example of extending multilingual coverage. Gains, forgetting and preservation of cross-lingual alignment must be measured; compare downstream validation performance before and after adaptation, including a retained source-language task.

Use audited, deduplicated target-language text and document the effective token count. Section 4.1 provides dated corpus measurements with their units; raw web totals are not a guarantee of clean usable training data.

12.3.2 Vocabulary Extension

Measure full-text fertility, unknown-token behaviour and downstream performance before adding tokens. Adding an already representable word can shorten sequences; byte fallback means missing whole words need not be unrepresentable. Tokeniser `add_tokens` adds matching rules and entries; it does not train new BPE merges.

Keep existing token IDs when adding vocabulary entries. Resize the input and output embeddings as needed, check weight tying and initialise each new row from its old-token decomposition or another documented method.

Make sure the new rows receive gradients. If every base parameter is frozen, attention LoRA alone leaves them untrained. Either train and save the full embedding module, including its parameters in the count, or use a tested method that updates selected rows. Save the tokeniser, resized embeddings/output head and adapter together. After reloading, compare logits and token IDs. Renumbering old IDs to merge independently trained vocabularies breaks their association with the learned embeddings.

Exercise 5 carries out these checks for 300 Kazakh words added to Glot500: the old IDs survive, only the new rows receive gradients when PEFT's `trainable_token_indices` is used, and the checked logits match after reload. In that recorded PEFT version and configuration, saving also writes the whole resized embedding matrix, producing a 1.2 GB adapter file. The extension shortened the Kazakh test text by 2.5% and gave a lower mean classification score (mean macro-F1 0.818 against 0.837 without it, over three seeds).

12.3.3 Training from Scratch

Decide between training from scratch and adaptation using the cleaned data volume, compute budget and deployment requirements. Compare adaptation with a smaller model trained from scratch on the same held-out tasks. Keep benchmark text out of tokeniser training and pretraining as well as supervised fine-tuning.

12.3.4 Adapting a Decoder LLM to a Turkic Language

Adapting an existing open-weight LLM is one route; KAZ-LLM and Sherkala are documented examples. The following workflow separates the main decisions:

1. **Tokeniser extension.** Measure the base tokeniser's fertility on your language (Table 7.1). If extension is justified, follow the ID, embedding and reload checks above and measure the resulting full-text token counts.
2. **Continued pretraining.** Continue training on target-language text mixed with a share of the base model's original languages (often English, and for Central Asian languages Russian) and test whether the mixture reduces forgetting. Turkish is another candidate to compare, although Kardeş-NLU's NLU results say little about the best mixture for decoder pretraining.
3. **Instruction tuning.** Audit the available native, translated and generated instruction data. Check meaning, language and relevance to the intended users. Compare data sources at matched training budgets; the Sherkala recipe above provides one documented example.
4. **Preference tuning and safety.** Check what preference data and evaluations exist for the selected language. Test both appropriate assistance and refusal behaviour with target-language prompts; behaviour in English may not carry over.
5. **Evaluation.** Use native benchmarks (TUMLU, KazMMLU, and others in Chapter 15), measure retention on any source languages the application needs, and use unrestricted language identification to flag outputs for manual review (Section 3.7).

12.3.5 LoRA: Parameter-Efficient Fine-tuning

For an adapted projection $W \in \mathbb{R}^{d_{out} \times d_{in}}$, standard LoRA uses

$$W' = W + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d_{in}}, \quad B \in \mathbb{R}^{d_{out} \times r}.$$

The added parameter count is $r(d_{in} + d_{out})$ **per projection**, not per transformer layer. Count every adapted projection plus any trainable classifier, embedding or bias parameters. Target module names depend on the architecture: XLM-R uses `query` and `value`, unlike models whose projections are named `q_proj` and `v_proj` (Hu et al. 2022). For an XLM-R-base-sized model with rank 8 on `query` and `value`, that is 12 × 2 × 8 × (768 + 768) = 294,912 adapter weights; the seven-class classification head adds 595,975.

In Exercise 2, LoRA with ranks 2, 8 and 32 on Glot500 gave Kazakh SIB-200 test macro-F1 between 0.807 and 0.861; a comparator with frozen word embeddings gave 0.792. Although the saved output calls it `full`, only the other encoder weights and classifier were updated: 86,047,495 parameters. It is partial fine-tuning. Only rank 8 has three seeds, so the runs are too few to rank training methods or ranks reliably.

QLoRA quantises frozen base weights. Ideal four-bit weight storage is one quarter of sixteen-bit storage, but scales, unquantised modules, gradients, optimiser states and activations also occupy memory. Measure peak memory and throughput for the actual batch and sequence lengths; a nominal GPU capacity does not guarantee a configuration will fit.

12.4 The Tokeniser Problem for Large Language Models

12.4.1 Estimating Context Use

Section 7.1 defines fertility. For context budgeting, the relevant quantity is the token count of the complete prompt, including formatting and special tokens. This applies to encoders as well as decoder LLMs.

A model's context limit is measured in tokens. How much text fits depends on the language, tokeniser and text itself. A larger limit also does not guarantee that the model uses all parts of a long input effectively.

The table below estimates word counts by dividing a token budget by the isolated-word fertility values in Table 7.1. This arithmetic describes token consumption; evaluating long-context reasoning requires a separate task.

Table 12.2: Approximate number of words that fit into a context window, computed as window size divided by the lowest and highest fertility measured among the four tokenisers in Table 7.1.

Language	Fertility range	Words in 4,096 tokens	Words in 32,768 tokens
English (reference)	1.08–1.09	3,757–3,792	30,062–30,340
Turkish	1.95–2.33	1,757–2,100	14,063–16,804
Kazakh	2.37–4.05	1,011–1,728	8,090–13,826
Kyrgyz	2.52–3.80	1,077–1,625	8,623–13,003
Tatar	2.51–4.46	918–1,631	7,347–13,054
Uyghur	3.38–6.85	597–1,211	4,783–9,694

Autoregressive models generate tokens sequentially. For a fixed text, higher fertility can increase decoding work, though elapsed time also depends on architecture, batching, hardware, prompt processing and output length. The estimates above use isolated words, omit chat-template and special tokens, and leave no response space. Treat them as rough capacity calculations.

Running-text counts can differ. In Exercise 1's SIB-200 sentence sample, including punctuation, the Llama 3 tokeniser uses 4.77 tokens per Kazakh whitespace word and 2.15 per Turkish one. The isolated-word figures in Table 7.1 are 3.51 and 2.03, measured on a different sample.

You can measure fertility for your own text and tokeniser in a few lines, in the native script and after conversion to the toolkit's Common Turkic Script (CTS):

```
from transformers import AutoTokenizer
import turkicnlp
from turkicnlp.scripts import Script
from turkicnlp.scripts.transliterator import Transliterator

tok = AutoTokenizer.from_pretrained("FacebookAI/xlm-roberta-base",
    revision="e73636d4f797dec63c3081bb6ed5c7b0bb3f2089")  # public encoder tokenizer
 ↪  diagnostic
words = [w.text for w in turkicnlp.Pipeline("kaz", processors=["tokenize"],
 ↪  tokenize_backend="rule")(
            "Мен мектептерімізде оқыдым. Алматы — Қазақстанның ең үлкен қаласы.").words
         if w.text.isalpha()]
to_cts = Transliterator("kaz", Script.CYRILLIC, Script.COMMON_TURKIC)

def fertility(ws):
    if not ws:
        raise ValueError("no words to measure")
    return sum(len(tok.encode(w, add_special_tokens=False)) for w in ws) / len(ws)

print(f"native {fertility(words):.2f}   CTS {fertility([to_cts.transliterate(w) for w in
 ↪  words]):.2f}")
```

Use `add_special_tokens=False` for this isolated-word diagnostic. Context budgeting requires encoding the complete prompt with the actual model tokeniser and template, including any required access to gated checkpoints.

12.4.2 The Vocabulary Overlap Problem

A shared vocabulary reuses character sequences across languages, but context determines their meaning. Models can also learn relationships between forms with different token IDs.

Different scripts reduce literal overlap, but do not prevent a multilingual encoder from learning aligned representations. The overlap study in Figure 3.3 is a diagnostic on its stated sample, not proof that all cross-script trigrams are absent or that Turkish knowledge cannot transfer to Kazakh.

Script conversion can increase literal overlap while changing the input distribution (Section 3.6, Section 16.2). Exercise 3 illustrates why the task result must be measured: Glot500 fine-tuned on Turkish SIB-200 topic classification (Adelani et al. 2024) reaches a Kazakh macro-F1 of 0.838 on Cyrillic test sentences but 0.694 on the same sentences in CTS (means over three seeds), although conversion raises the share of Kazakh subword pieces seen in the Turkish training text from 9% to 52%.

12.4.3 LoRA for Vocabulary and Script Adaptation

Treat script conversion, vocabulary extension and LoRA as separate factors. Compare the native-script baseline, conversion alone, extension alone and their combinations under a matched update budget. Follow the embedding-saving contract above whenever the vocabulary changes. Evaluate task accuracy alongside fertility (Section 15.1).

12.5 Hands-On: A LoRA Fine-Tuning Procedure for Kazakh Text Classification with XLM-R

Question. Does LoRA improve Kazakh text classification on source-disjoint data?

Inputs. Licensed JSONL train/dev/test files in the documented schema and XLM-R.

Compute. Checkpoint download and training memory; record the selected trainable modules and hardware.

Output. Saved adapters and task head, label mappings, predictions and scores.

Interpretation. Select the checkpoint on development data and compare the reserved test predictions with a baseline.

The complete runner is classify.py. Supply `train.jsonl`, `dev.jsonl` and `test.jsonl` with `id`, `group_id`, `text` and a string `label`; `group_id` identifies the original article or source group across excerpts and variants. The runner rejects cross-split IDs, source groups and exact normalised texts, unknown evaluation labels and overlength examples. Define document chunking before using long articles.

```
python code/ch12-classify/classify.py data/kazakh-news runs/kazakh-lora --lora
```

The tokeniser and base model share a pinned revision. LoRA adapts XLM-R's `query` and `value` projections with rank 8 and scaling 16/8. The randomly initialised classifier is explicitly trainable and saved using `modules_to_save=["classifier"]`. The run manifest counts actual trainable and total parameters rather than printing a fixed percentage.

Development macro-F1 selects the checkpoint. The runner saves the model/adapter and tokeniser, reloads onto a fresh base model, and checks evaluation logits before scoring the reserved test split. It saves per-item predictions, label mappings, input hashes and seed. The inference checkpoint includes the classifier; loading the base encoder alone creates an untrained task head.

The local mechanics checks use a tiny randomly initialised encoder and balanced synthetic labels to test loss, gradients and adapter reload. They provide no evidence of classification accuracy. The exercises instead use Glot500 on SIB-200, selecting checkpoints on development data. Their separate runner truncates at 256 tokens, while this hands-on rejects overlength inputs. Record affected items when interpreting those exercise scores.

Status: procedure only. No licensed news corpus with source-disjoint splits was prepared for this run, so no Kazakh news-classification score is reported.

What to Take Forward

Choose a pretrained model for the task, then test whether adaptation improves task performance or meets a justified cost, latency or deployment requirement. Exercise 4 shows why evaluation matters: Qwen2.5-1.5B-Instruct scores 52.2% on the Kazakh COPA translations, with a 95% interval of 47.8–56.5% that includes the 50% chance baseline. Use token counts to plan context and cost, and task scores to assess an adaptation. Chapter 13 applies these models to representations and search.

Further Reading

The BERTurk model family is documented in its release (Schweter 2020), whose repository describes the training corpora and compares the BERTurk variants with mBERT and XLM-R on Turkish POS tagging, NER and question answering; no variant wins every task. For multilingual encoders more generally, Conneau et al. (2020) describes the training data, language coverage and cross-lingual transfer properties of XLM-R, including its XNLI results for Turkish.

Hu et al. (2022) introduces LoRA and compares it with full fine-tuning on the paper's tasks. Use those results as motivation for a matched comparison on the target task. For Turkic-specific LLM evaluation, Isbarov et al. (2025) is a primary reference for native multiple-choice evaluation, with results by language, subject and script. See Chapter 15 for the release-count caveat.

The fertility analysis underlying the context window calculations in Section 12.4 is from Hakimov (2026), which provides measurements across 20 Turkic languages and four LLM tokenisers. The same paper shows the effect of a common script on fertility and lexical

overlap (Section 12.4). Senel et al. (2024) shows how a high-resource relative improves sentence-level transfer within the family; Koto et al. (2025) documents the adaptation of an open LLM to Kazakh.

Exercises

1. Compare vocabulary variants of one model family on the same documents. For BERTurk's 32K and 128K models, use the scores published in its repository; measure full-text tokens per word and embedding parameters for XLM-R and Glot500 (whose vocabulary extends XLM-R's), and compare them with LLM tokenisers.
2. Compare classifier-only training, full fine-tuning and several LoRA ranks; report any frozen modules explicitly. Use the same real train/dev/test splits and training budget. Report macro-F1, class support, trainable parameters, peak memory and measured time.
3. Test native script and CTS as separate treatments for zero-shot transfer from Turkish to Kazakh and Kyrgyz. Keep original IDs and labels, and freeze the protocol on development data.
4. Evaluate Kazakh multiple-choice items with a conditional scorer, as in Section 15.6, and compare it with an encoder whose multiple-choice head is randomly initialised. The latter is not a zero-shot reasoning model.
5. Extend a tokeniser without changing old IDs. Verify new embedding gradients and save/reload equivalence, then test whether reduced fertility improves downstream performance.

13 Meaning: Embeddings, Semantic Resources, and Retrieval

Learning Objectives

After reading this chapter, you will be able to:

- Survey the lexical-semantic resources for Turkic languages (WordNets, PropBanks, discourse and coreference corpora) and identify the gaps
- Explain how agglutination and the use of several scripts affect word and sentence embeddings
- Evaluate multilingual sentence encoders on Turkic languages with parallel-sentence retrieval and semantic similarity
- Choose how to reduce word forms (surface form, prefix, stem, lemma, subword) for keyword search in an agglutinative language
- Design a retrieval-augmented generation system for a multilingual collection, such as Turkic-language questions over Russian documents
- Account for script-dependent tokenisation when splitting documents into chunks

Chapter 12 introduced pretrained models and adaptation. This chapter turns to semantic resources and retrieval: representing text, comparing meanings and finding relevant passages. It then connects retrieval to generation, which Chapter 14 examines in more detail.

Inflection and script variation both affect matching. Turkish *ev* 'house', *evler* and *evimiz* are related forms, but exact matching treats them as different strings. Script differences can also hide shared forms (Section 3.6). The chapter compares lexical and dense retrieval, including queries and documents in different languages.

13.1 Lexical-Semantic Resources

Lexical-semantic resources describe what words mean and how they relate: synonymy and hypernymy in a WordNet, predicate–argument structure in a PropBank, relations between clauses in a discourse treebank. The selected inventory below includes several Turkish resources, WordNets for Tatar and Uzbek, and a Kyrgyz word-similarity set (Table 13.1):

- **WordNet.** The Turkish WordNet KeNet (Bakay et al. 2021), TatWordNet for Tatar (Kirillovich et al. 2021), and UZWORDNET for Uzbek (Agostini et al. 2021).

- **Semantic roles.** The Turkish PropBank (Şahin and Adalı 2018) and TRopBank, published as Turkish PropBank v2.0 (Kara et al. 2020).
- **Discourse and coreference.** The Turkish Discourse Bank (Zeyrek, Demirşahin, and Çallı 2013) and the Marmara Turkish Coreference Corpus (Schüller et al. 2017).
- **Inference and similarity.** NLI-TR (Budur et al. 2020) and STSb-TR (Beken Fikri, Oflazer, and Yanıkoğlu 2021), both machine-translated from English datasets.

Other lexical sources include Wiktionary, the Apertium bilingual dictionaries used by the translation pairs in Chapter 10, and lemma/POS outputs from Chapter 6. These supply lexical features. Sense relations, coreference and semantic roles require their own annotation.

Table 13.1: Selected lexical-semantic resources, checked October 2026. Sizes refer to the cited release or publication; this is not an exhaustive family inventory.

Language	Type	Resource	Size
Turkish	WordNet	KeNet (Bakay et al. 2021)	76,757 synsets; linked to Princeton WordNet
Turkish	Semantic roles	Turkish PropBank (v1) (Şahin and Adalı 2018)	Semantic roles of the arguments in a Turkish dependency treebank, using framesets of root verbs; SRL baseline 79.10 F1
Turkish	Semantic roles	TRopBank (Turkish PropBank v2.0) (Kara et al. 2020)	17,673 verbs
Turkish	Discourse relations	Turkish Discourse Bank (Zeyrek, Demirşahin, and Çallı 2013)	Approximately 400,000 words; explicit connectives and arguments
Turkish	Coreference	Marmara Turkish Coreference Corpus (Schüller et al. 2017)	33 documents; 53,925 tokens; 5,170 mentions; 944 chains
Turkish	Natural language inference	NLI-TR (SNLI-TR and MultiNLI-TR) (Budur et al. 2020)	550,152 + 392,702 training pairs (machine-translated; sample checked by experts)
Turkish	Semantic textual similarity	STSb-TR (Beken Fikri, Oflazer, and Yanıkoğlu 2021)	5,749 train / 1,500 validation / 1,379 test pairs (8,628 total)
Tatar	WordNet	TatWordNet (Kirillovich et al. 2021)	18,583 synsets; 36,540 lexical entries; 49,525 senses; CC BY-SA 4.0
Uzbek	WordNet	UZWORDNET (Agostini et al. 2021)	28,140 synsets; 64,389 senses; 20,683 words; est. accuracy 75.98%

Language	Type	Resource	Size
Kyrgyz	Word similarity	HJ-Ky-0.1 (silver-standard word-similarity set) (Alekseev and Kabaeva 2024)	361 word pairs; inherited Russian similarity scores

TRopBank's paper reports 17,673 verbs; Chapter 9 counts 17,531 framesets in the inspected release. The two figures come from different source descriptions and cannot be swapped for each other. A frame inventory also does not substitute for sense-annotated sentences.

13.2 Word Embeddings and Morphology

Whole-word static embeddings assign a vector to each word form. Rare inflected forms may have little training evidence. Character n-gram embeddings share information across forms and can construct vectors for unseen words; useful task performance still needs to be tested. Lemma-based embeddings instead collapse forms to an analyser's selected lemma, losing explicit inflectional distinctions and inheriting analysis errors.

Static embeddings for several Turkic languages can be aligned into a shared space with a bilingual dictionary. Kuriyozov, Doval, and Gómez-Rodríguez (2020) built new monolingual embeddings and bilingual dictionaries for Turkish, Uzbek, Azerbaijani, Kazakh and Kyrgyz and aligned the embedding spaces with established, weakly supervised mapping methods. In bilingual dictionary induction, the authors report that the embeddings of the lower-resourced languages benefit from alignment with a closely related, better-resourced language; in Uzbek sentiment analysis, aligned embeddings gave a slight improvement over monolingual ones.

13.3 Sentence Encoders

13.3.1 Models

Sentence encoders map text to vectors for comparison. Whether vector similarity reflects the intended meaning or relevance must be tested. Four multilingual model families are:

- **LaBSE** (Feng et al. 2022), trained on translation pairs;
- **LASER3** (Heffernan, Çelebi, and Schwenk 2022), with encoders for low-resource languages;
- **multilingual E5** (Wang et al. 2024);
- **BGE-M3** (J. Chen et al. 2024).

TurkicNLP adds a fifth option: sentence vectors from the encoder of NLLB-200, which was trained on translation between 200 languages, including the eleven Turkic varieties of FLORES-200 (Table 10.3):

```
import numpy as np
import turkicnlp

def embed(lang, text):
    return np.array(turkicnlp.Pipeline(lang, processors=["embeddings"])(text).embedding)

tr = embed("tur", "Bugün hava çok güzel.")
kk = embed("kaz", "Бүгін ауа райы өте жақсы.")
print(tr @ kk / (np.linalg.norm(tr) * np.linalg.norm(kk)))   # cosine similarity
```

In real use, build each language's pipeline once and reuse it.

Training coverage differs across encoders. LaBSE's 109 languages include Turkish, Azerbaijani, Kazakh, Kyrgyz, Uzbek, Turkmen, Tatar and Uyghur, but exclude Bashkir, Chuvash and Sakha (Feng et al. 2022). The LASER repository lists a separate LASER3 encoder for each of the eleven Turkic varieties in FLORES-200.

Multilingual E5 starts from XLM-R and lists support for its 100 languages. Its Turkic members are Turkish, Azerbaijani, Uzbek, Kazakh, Kyrgyz and Uyghur (Chapter 12). BGE-M3 extends XLM-R-large to 8,192-token inputs and reports "more than 100 working languages", without a separate Turkic list (J. Chen et al. 2024). Models may accept languages outside their documented coverage, but their vectors need direct evaluation. Dedicated Turkish encoders are also evaluated on TR-MTEB, the Turkish version of the MTEB embedding benchmark (Baysan and Gungor 2025).

13.3.2 Evaluation

Three kinds of evaluation are available for Turkic languages:

- **Semantic textual similarity.** Kardeş-NLU includes an STS task for Azerbaijani, Kazakh, Kyrgyz, Uzbek, and Uyghur (Senel et al. 2024): 800 test pairs per language, translated from English STS pairs that keep their English gold scores. In Exercise 3, the NLLB encoder has the highest point estimates among the three tested systems for Kazakh, Kyrgyz, Uzbek and Uyghur (Spearman's ρ 0.58–0.75) and the XLM-R paraphrase encoder for Azerbaijani, while the two paraphrase encoders lead on the English originals (0.83–0.84). TR-MTEB includes Turkish similarity and retrieval tasks (Baysan and Gungor 2025).
- **Natural language inference.** Kardeş-NLU and the Turkish part of XNLI test entailment, contradiction and neutrality (Chapter 15). Evaluating an encoder on this task needs a declared classifier or scoring method; cosine similarity alone does not distinguish those relations.
- **Parallel-sentence retrieval.** Given a sentence in one language, find its translation among all sentences in another. FLORES-200 provides the same 1,012 test sentences in eleven Turkic varieties (Table 10.3), so it supports a full matrix of language pairs within the family. What it tests is translation matching. General search relevance is a different question.

13.3.3 Measured native-script sentence retrieval

The October empirical suite evaluates two released paraphrase encoders, `paraphrase-multilingual-MiniLM-L12-v2` and `paraphrase-xlm-r-multilingual-v1`, on original FLORES-200 devtest (NLLB Team et al. 2024). Each of thirteen varieties supplies 1,012 queries against all 1,012 sentences in each other variety: **156 directed comparisons per encoder, 312 in total**. The release's aligned row order is retained and its files are checksummed; this is separate from the FLORES+ ID-based loader in the hands-on section.

Both pinned checkpoints use attention-mask mean pooling, L2 normalisation, cosine similarity and their configured **128-token ceiling**. Per-language truncation counts are saved, so this is a comparison of the complete encoder configurations rather than unlimited-context representations. Exact duplicate target strings count as additional positives; top-score ties within 10^{-6} receive fractional credit instead of favouring an earlier row.

Table 13.2: Native-script bitext top-1 accuracy, averaged equally over the other 12 target varieties. Each direction uses all 1,012 original FLORES-200 devtest rows as queries and candidates. Models are the pinned multilingual paraphrase MiniLM and XLM-R encoders, with their 128-token limit and recorded truncation. Source: `run_bitext.py`; NLLB Team et al. (2024).

Query variety	MiniLM (%)	XLM-R (%)
English	54.7	77.3
Russian	54.0	76.7
Turkish	54.1	77.1
N. Azerbaijani	39.4	73.5
S. Azerbaijani	31.4	40.3
Turkmen	23.8	33.6
Kazakh	31.1	75.1
Kyrgyz	35.3	71.0
Tatar	30.2	40.2
Bashkir	27.5	31.7
Crimean Tatar	44.8	65.2
N. Uzbek	32.8	50.7
Uyghur	18.1	60.9

The equally weighted mean over directed pairs is **36.70% for MiniLM and 59.49% for XLM-R** (`run_bitext.py`). These descriptive scores do not isolate script, genealogy or training exposure. The original release also contains spacing anomalies in South Azerbaijani, documented in Chapter 4; all rows are retained. There is no independent pretraining-overlap audit, and semantically equivalent nonidentical alternatives have not been manually labelled. Exercise 1 adds CTS conditions and the NLLB encoder on a different FLORES-200 sample; independent evaluation material is still lacking.

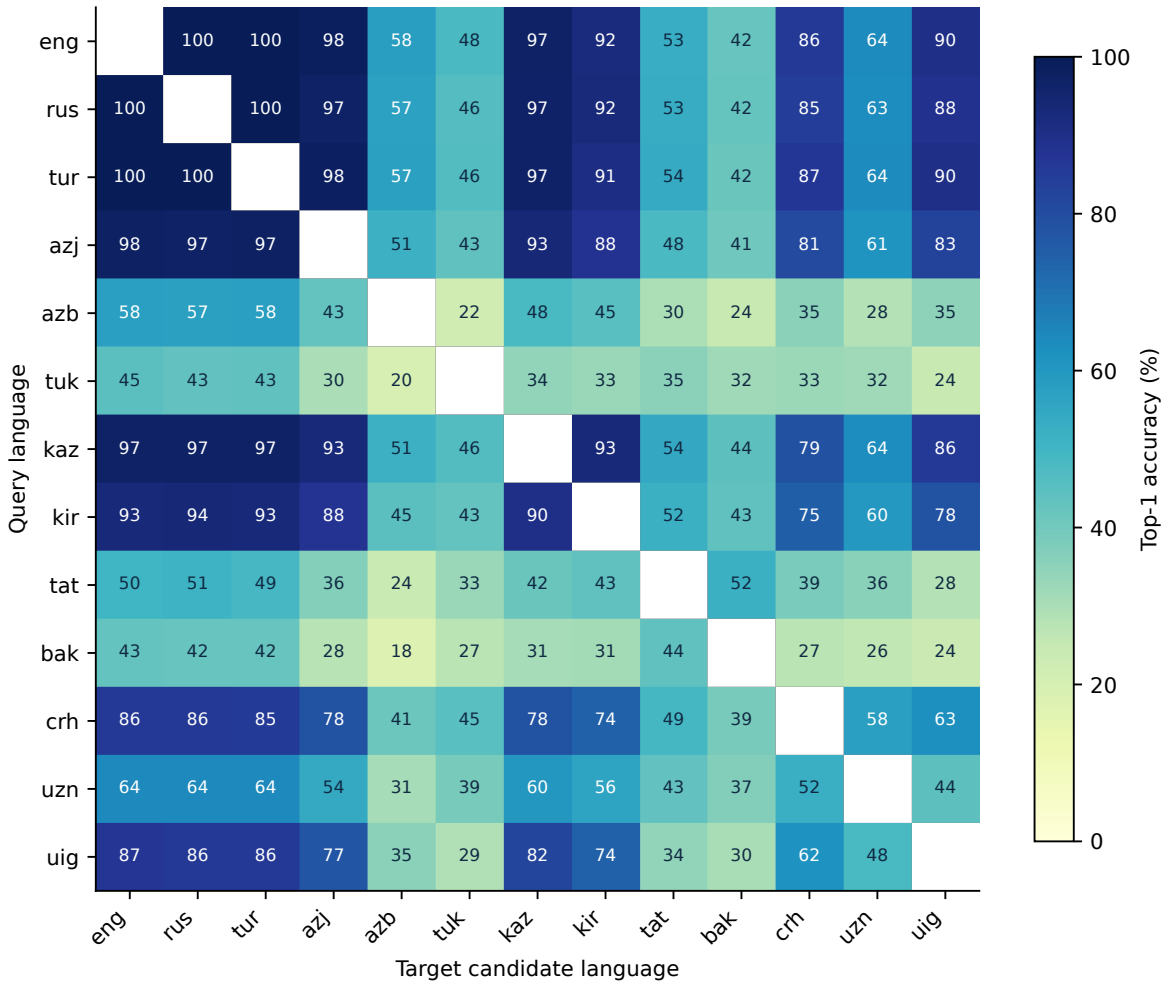


Figure 13.1: Measured XLM-R paraphrase-encoder top-1 retrieval accuracy on original FLORES-200 devtest. Rows are query varieties and columns are target candidates; self-retrieval is omitted. Codes follow the dataset's language identifiers. Each cell uses 1,012 queries.

13.3.4 Script and morphology in embedding space

Compare native text with the toolkit's common-script representation (CTS; Section 3.6) on the same retrieval task. Conversion changes tokenisation and the model's input distribution. The direction of any benefit cannot be inferred from shared letters alone.

Exercise 1 uses 1,004 SIB-200 sentences derived from FLORES-200 (Adelani et al. 2024). For Kazakh→Kyrgyz, converting both sides to CTS reduces top-1 accuracy from 93.7% to 46.1% with the XLM-R paraphrase encoder and from 99.6% to 46.0% with NLLB. MiniLM rises from 41.7% to 45.4%. These outcomes do not identify the mechanism behind the changes.

The exercise uses 128-token limits for the paraphrase encoders and 512 for NLLB, retains the original NLLB language token after conversion, and resolves score ties by the first candidate. It sorts each language's rows by ID but does not perform the hands-on's explicit alignment checks. These protocol differences matter when comparing it with the full FLORES baseline. NLLB's 98.9–100% native-script scores motivate an independent evaluation, yet high scores and FLORES's origin together still fall short of proving contamination. LaBSE, LASER3, E5 and BGE-M3 were not run here.

13.4 Keyword Search in Agglutinative Languages

13.4.1 The problem

Keyword search ranks documents by the terms they share with the query, weighted by how rare each term is (BM25 is the standard formula). A query *ev fiyatları* 'house prices' should match a document containing *evlerin fiyatı* 'the price of the houses'. With exact surface-word matching, neither query word matches either document word. Five representations of word forms before indexing are available:

Table 13.3: Ways to reduce Turkic word forms for keyword search.

Method	Example: *evlerimizden*	Needs
Surface form (lower-cased)	*evlerimizden*	nothing
Fixed-length prefix (first k letters)	*evler* ($k = 5$)	nothing
Stemmer	*ev*	a stemmer for the language
Lemma from the analyser	*ev*	a morphological analyser
Subword units	*ev ##ler ##imiz ##den* (depends on the model)	a subword vocabulary

Lower-casing itself needs care: Turkish and Azerbaijani have dotted and dotless *i* (*İ/i*, *I/ı*), and locale-independent lowercasing may not preserve the intended distinctions (Section 3.3).

Prefix truncation is a cheap baseline motivated by Turkish retrieval work (Can et al. 2008). Choose k on development relevance judgements; truncation can merge unrelated roots as well as recover inflections. The result for one collection is not a universal adequacy guarantee.

Exercise 2 searches 825,309 Kazakh Wikipedia passages for 1,929 KazQAD test questions, using BM25 with $k_1 = 1.2$ and $b = 0.75$ (Yeshpanov et al. 2024). Surface forms score 0.140 nDCG@10. Five-letter prefixes score 0.198, context-free Apertium lemmas 0.200 and Glot500 subwords 0.204. Prefix length was selected on 548 validation questions before testing.

Paired sign-flip tests compare each alternative with surface forms over the 1,929 test questions. None of 10,000 random draws reaches the observed absolute gain for any alternative. The two-sided Monte Carlo estimate is therefore $p \leq 1/10{,}001 \approx 10^{-4}$, rather than exactly zero. The comparisons remain clear after Bonferroni correction for six alternatives: four prefix lengths, lemmas and subwords. Questions are the sampling unit, so these tests support conclusions about this question set and collection.

The tests do not distinguish the three leading alternatives from one another: lemma vs. prefix, subword vs. prefix and subword vs. lemma give $p = 0.63$, 0.18 and 0.38, respectively. Their relative performance stays unresolved, which is different from showing that they are equivalent.

Apertium returns analyses for 82.1% of the collection's word tokens. Some queries losing retrieval accuracy contain merged or ambiguous lemma forms: *жүзін* and *жүзіп*, for example, both map to *жүз*. They are candidates for manual error analysis; on their own they do not explain the rank change. Dense and hybrid retrieval were not run on this collection.

```
import turkicnlp

# Rule-based tokenizer and Apertium lemmas: fast, no neural model. For Kazakh,
# processors=["tokenize", "lemma"] with the default backends gives Stanza lemmas instead.
nlp = turkicnlp.Pipeline("kaz", processors=["tokenize", "morph"], tokenize_backend="rule",
 ↪  morph_backend="apertium")

def index_terms(text: str, mode: str = "lemma", k: int = 5) -> list[str]:
    if mode not in {"surface", "prefix", "lemma"} or k < 1:
        raise ValueError("invalid mode or prefix length")
    words = [w for w in nlp(text).words if w.upos != "PUNCT"]
    if mode == "surface":
        return [w.text.lower() for w in words]
    if mode == "prefix":
        return [w.text.lower()[:k] for w in words]
    return [(w.lemma if w.lemma and w.lemma != "_" else w.text).lower() for w in words]   #
     ↪  unknown words keep their surface form

query = "Мектептеріміздеn кітаптар алдық."     # 'We took books from our schools.'
for mode in ["surface", "prefix", "lemma"]:
    print(mode, index_terms(query, mode))
# surface ['мектептеріміздеn', 'кітаптар', 'алдық']
```

```
# prefix ['мекте', 'кітап', 'алдық']
# lemma ['мектеп', 'кітап', 'ал']
```

The example shows the trade-off in miniature. The five-letter prefix happens to give the right stem for *кітаптар* 'books', cuts *мектеп* 'school' short, and leaves the short verb *алдық* 'we took' unchanged; the analyser gets all three lemmas right. For fair baselines, cache the same token boundaries once; surface/prefix indexing need not run a lemmatiser. Apply the identical normalisation to queries and documents. For Turkish and Azerbaijani, lower-case with `turkicnlp.utils.unicode_utils.normalize_turkish_i(text).lower()` so that *İ* and *I* are handled correctly.

13.4.2 Measured passage-recovery baseline

Belebele supplies questions and translated source passages (Bandarkar et al. 2024). Here it is repurposed as **source-passage retrieval**, not the benchmark's multiple-choice task: each of seven languages contributes 900 questions and 488 candidate passages. The gold candidate is the passage attached to that question. Passage links and question numbers are checked for uniqueness and joined across languages. Other potentially relevant passages are not exhaustively judged.

Table 13.4: Measured nDCG@10 for source-passage retrieval derived from Belebele: 900 questions and 488 candidate passages per language. This is not Belebele multiple-choice accuracy. BM25 uses fixed k1=1.2 and b=0.75; dense encoders use their 128-token limit. Gold-score ties receive expected rank credit. Source: `run_retrieval.py`; Bandarkar et al. (2024).

Language	Surface	Prefix 5	MiniLM	XLM-R
N. Azerbaijani	0.638	0.827	0.347	0.639
English	0.881	0.904	0.858	0.882
Kazakh	0.653	0.818	0.310	0.697
Kyrgyz	0.668	0.846	0.257	0.620
Russian	0.731	0.888	0.797	0.844
Turkish	0.744	0.855	0.789	0.850
N. Uzbek	0.703	0.839	0.270	0.355

BM25 uses fixed $k_1 = 1.2$, $b = 0.75$, NFC normalisation and Unicode alphanumeric tokens, with Turkish/Azerbaijani case handling. The fixed prefix-five condition clips each token after five code points; it is not a morphological analyser. The dense systems use the same pinned encoders and 128-token ceiling as the sentence experiment. All systems receive the same candidate collection. Metrics average expected rank credit over gold-score ties, including queries with no lexical overlap.

The fixed five-character prefix baseline scores higher nDCG@10 than either dense model in all seven collections. Both dense encoders truncate 1,570 of the 3,416 passage inputs at 128 tokens. Their scores therefore reflect this length limit as well as the learned representations; statistical significance has not been established.

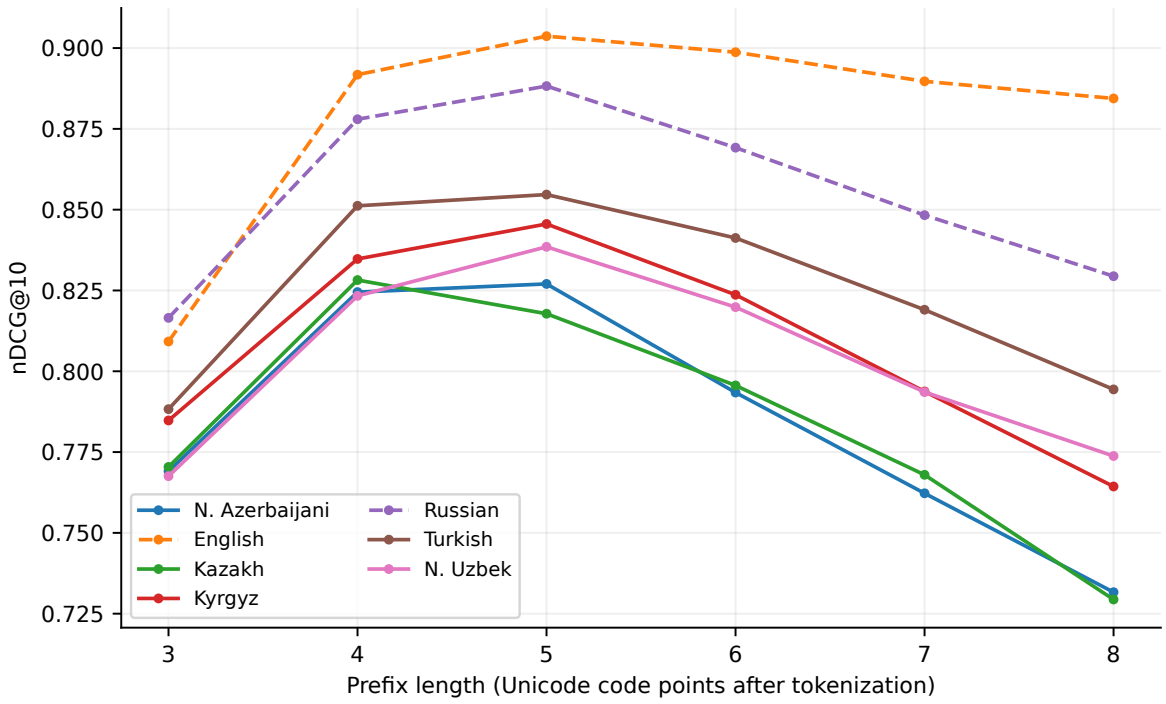


Figure 13.2: Exploratory sensitivity of BM25 source-passage retrieval to prefix length on the seven Belebele language collections. Each point is nDCG@10 over 900 questions. The test grid is descriptive; no best prefix is selected as a tuned result.

Figure 13.2 tests lengths three through eight on the same test questions. Selecting a length for deployment would require separate development data. These lexical conditions, dense baselines and the script/language probes below make up the 77-condition run. Exercise 2 compares lemmas and subwords on KazQAD; hybrid systems remain untested.

13.5 Retrieval across Scripts and Languages

13.5.1 Across scripts

A Kazakh user may type a query in Cyrillic while some documents are in Latin; an Uzbek user may search in Cyrillic over a collection in Latin. Literal keyword overlap may fall sharply, though names, numbers, shared symbols and mixed-script text can still match. Dense retrieval may partly bridge the gap if the encoder has seen both scripts. Query transliteration is one condition to test (Section 3.4); preserve the original and check ambiguous mappings.

13.5.2 Across languages: Turkic queries and Russian documents

A collection may contain Russian documents that answer questions asked in Kazakh, Kyrgyz or Uzbek. Two candidate designs are:

- **Multilingual dense retrieval.** Encode the question and the Russian documents with the same multilingual encoder.
- **Query translation.** Translate the question into Russian (for example with NLLB, Section 10.3) and use Russian keyword or dense search.

Exercise 1 offers a translation-matching diagnostic: the XLM-R paraphrase encoder retrieves the aligned Russian sentence for 97.8% of Kazakh SIB-200 queries, compared with 93.7% for Kyrgyz candidates. MiniLM scores 43.9% on Kazakh→Russian. Matching translations is different from finding an answer passage in an independently collected Russian corpus. Query translation and hybrid retrieval remain untested here.

13.5.3 Measured script and language probes

The Uzbek script probe keeps the 488 Latin candidate passages fixed and transforms the same 900 questions with the recorded toolkit Latin→Cyrillic mapping. A third condition converts those generated questions back to Latin. This is a **synthetic query transformation**, with source hashes retained; it has not been validated by native speakers and does not represent a naturally occurring Cyrillic query collection.

Table 13.5: Measured nDCG@10 on the same 900 Uzbek questions and 488 Latin passages. Cyrillic queries are generated by the pinned toolkit mapping, then optionally converted back to Latin. No native-speaker validation is claimed. Source: `run_retrieval.py` and `prepare_script_probe.py`; Bandarkar et al. (2024).

Query condition	BM25	MiniLM	XLM-R
Original Latin	0.703	0.270	0.355
Synthetic Cyrillic	0.028	0.110	0.172
Round-trip Latin	0.702	0.269	0.354

BM25 nDCG@10 falls from **0.703 to 0.028** with Cyrillic queries and returns to **0.702** after round-trip conversion. Both dense models also lose accuracy. This result measures sensitivity to the chosen mapping; meaning preservation needs a separate review.

Table 13.6: Cross-language passage-recovery nDCG@10: 900 questions and 488 aligned target-language passages per direction. Source: `run_retrieval.py`; Bandarkar et al. (2024).

Question → passages	MiniLM	XLM-R
Kazakh → Russian	0.402	0.681
Russian → Kazakh	0.607	0.763
Turkish → English	0.772	0.842
N. Uzbek → Russian	0.286	0.444

The cross-language probe joins the same question/source-passage identities across translations and searches the target-language collection directly with each dense encoder. It includes Kazakh→Russian, Russian→Kazakh, Turkish→English and Uzbek→Russian. No query-translation or hybrid retriever is run here, and performance on translated parallel passages need not transfer to independent Russian documents. Human mapping checks and comparisons with query translation and hybrid retrieval remain to be completed.

13.6 Retrieval-Augmented Generation for Turkic Languages

Retrieval-augmented generation (RAG) gives a language model retrieved passages as context and asks it to answer from them. Three design decisions connect the earlier sections.

A token budget holds different amounts of text. Language variety, writing system, tokeniser and the text itself affect capacity. Isolated-word fertility (Table 7.1) and corpus characters per token (Figure 4.2) provide useful diagnostics. To estimate document capacity, measure running text directly; separating script effects requires a controlled comparison.

Table 13.7 and Figure 13.3 report the sentence-stream packing experiment on the **original FLORES-200 devtest release**, using 1,012 aligned sentence rows for each of 11 Turkic varieties plus English (NLLB Team et al. 2024). This release is distinct from the subsequently revised FLORES+ dataset. The tokenisers are pinned snapshots of Qwen3.5-35B-A3B-FP8, Meta-Llama-3-8B and Olmo-3-1025-7B, plus `o200k_base`; the experiment records their hashes and library versions. No model weights, chat templates or special tokens are used.

The packing procedure takes sentences in release order, trims outer whitespace and joins them with one space. It re-tokenises each proposed chunk as a whole. If the next sentence would push the chunk beyond **512 tokens**, it closes the chunk and starts another. Every individual sentence fits; the largest takes 388 tokens.

Means cover closed chunks and exclude each language/tokeniser's final tail. The saved audit still accounts for all 1,012 rows, including the tail. Here, "words" means nonempty whitespace-delimited units and "sentences" means aligned dataset rows.

Table 13.7: Measured content of sentence-packed chunks with a 512-token ceiling on original FLORES-200 devtest (1,012 aligned sentence rows per variety). Ranges are the minimum and maximum means across four pinned tokenisers; extrema in different columns need not come from the same tokeniser. Words are whitespace-delimited units. The final tail is excluded from means, with all rows retained in the chunk audit. Fill is the mean percentage of the 512-token budget used. Source: experiment N2-4; NLLB Team et al. (2024).

Language	Words/chunk	Sentences/chunk	Fill (%)
English	397–405	18.3–18.7	96.8–97.4
Turkish	164–229	9.4–13.2	94.8–96.5
North Azerbaijani	126–202	6.7–10.7	92.2–94.8
South Azerbaijani	90–172	5.2–9.9	90.0–94.7
Turkmen	131–196	7.4–11.1	93.0–95.6
Kazakh	77–201	4.5–11.7	88.7–96.0
Kyrgyz	83–190	4.8–11.0	88.8–95.3
Tatar	81–188	4.5–10.5	88.6–95.1
Bashkir	69–186	3.9–10.5	86.7–94.7
Crimean Tatar	151–191	8.5–10.7	93.2–95.5
Northern Uzbek	149–187	8.3–10.4	93.8–95.3
Uyghur	54–124	3.1–7.1	84.4–92.8

On this sample, English holds approximately **397–405 words** per closed chunk across tokenisers, Turkish **164–229**, Kazakh **77–201**, and Uyghur **54–124**. Whole-sentence packing leaves unused space: mean utilisation ranges from **84.4% to 97.4%** across the 48 language/tokeniser combinations. Thus, the observed capacity reflects both tokenisation and the packing rule. The figure also reports aligned sentence counts so that differences in translated word counts are visible alongside differences in shared sentence coverage.

This experiment measures **sentence-stream packing**. Chunks may cross the source articles from which FLORES sampled its sentences, so retrieval relevance, chunk coherence and answer quality need separate evaluation (Section 15.1). Translations differ in length, whitespace conventions affect word counts, and the design does not vary language

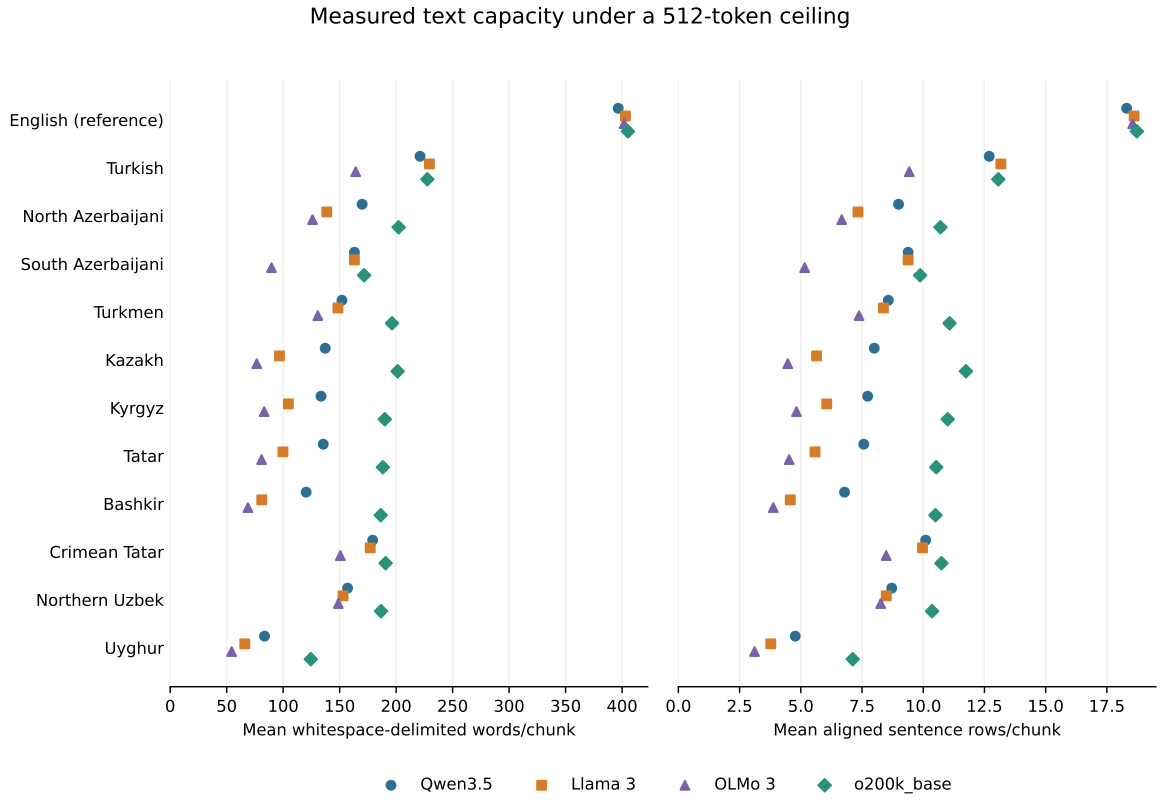


Figure 13.3: Measured mean words and aligned sentence rows per closed chunk under a 512-token ceiling, using four pinned tokenisers on original FLORES-200 devtest. Counts exclude the final tail. Exact inputs and per-chunk counts accompany the sentence-stream packing experiment.

and script independently. In deployment, budget indexing chunks with the embedding tokeniser and final context with the generator's tokeniser, allowing space for prompts, citations and output.

A separate probe uses the ten longest Kazakh Wikipedia articles of KazQAD (13,700–43,000 words each). A 512-token chunk holds on average 95 words with the Llama 3 tokeniser, 76 with OLMo 3 and 126 with Qwen3.6, against 203 with Glot500's tokeniser, an encoder-side vocabulary that splits Kazakh far less (Exercise 4). A system that retrieves with one model and generates with another must therefore budget chunks for each tokeniser separately. Whether token-based or sentence-based chunking retrieves better on KazQAD has not been measured.

The answer must stay in the user's language. Check whether Russian context changes the language of an answer requested in a Turkic language. Evaluate this separately from correctness, using the language-fidelity checks in Section 14.1.

Evaluate each stage. Measure retrieval relevance, answer correctness, citation support, language fidelity and abstention on unanswerable questions separately. Retain document provenance and permissions, remove duplicate candidates and record all judged relevant passage IDs for each query. A translated parallel sentence may have other relevant alternatives.

Tune prefix length, BM25 parameters and fusion weights on development data. Report collection size, MRR cutoff, nDCG gains and the treatment of queries without relevant items. Set evaluation sample size according to the precision the study needs.

13.7 Hands-On: A Procedure for Finding Translations Across the Family

Question. Can an encoder retrieve the aligned translation among candidate sentences?

Inputs. An authorised FLORES+ export with stable IDs and the chosen encoder.

Compute. CPU inference is possible; model size and candidate count determine memory needs.

Output. Ranked candidates, retrieval scores and input/encoder metadata.

Interpretation. Compare native and CTS conditions with the same candidate set and explicit tie handling.

FLORES+ requires acceptance of its access conditions. Use an authorised local export and save its revision, selected language/script/variety, split and file hashes; keep it evaluation-only. The loader must preserve `id`, `split` and `text` rather than discarding metadata and assuming row-order alignment.

The tested helpers in retrieval.py require unique `(id, split)` keys, equal coverage and nonempty text, then join and sort the two varieties by those keys. They validate embedding dimensions, finite values and nonzero norms. Exact duplicate target strings count as additional positives; tied highest scores receive fractional credit across all

ties. Semantically equivalent but nonidentical alternatives need separate relevance judgements.

```
from retrieval import align, top1  # code/ch13-retrieval on PYTHONPATH
# source_rows and target_rows are authorised exports retaining IDs and split.
source_rows, target_rows = align(source_rows, target_rows)
a = encoder.encode([r["text"] for r in source_rows], normalize_embeddings=True)
b = encoder.encode([r["text"] for r in target_rows], normalize_embeddings=True)
result = top1(source_rows, target_rows, a, b)
```

Build and cache each embedding matrix once per model revision, variety, condition and preprocessing hash. For larger candidate sets, score in blocks. Record truncation rate and maximum length; do not silently compare different retained subsets.

A model comparison needs its own interface specification. LaBSE uses its sentence embedding interface; multilingual E5 uses the documented `query:` and `passage:` prefixes for retrieval, including non-English text. Select a LASER language encoder explicitly. BGE-M3 dense, sparse and multi-vector modes are different systems: start with a declared dense mode. Record pooling, normalisation, similarity and training-language evidence for each checkpoint.

Test native/native, CTS/CTS and native/CTS separately. Every included variety needs an intentional supported mapping, including native Latin and Arabic inputs; do not return Latin unchanged while labelling it CTS. Unsupported mappings are missing conditions. Preserve originals, converter versions and checked examples.

Status: procedure only. The full FLORES+ protocol is not run here. Exercise 1 records a separate SIB-200 retrieval diagnostic with different alignment, tie and length policies.

What to Take Forward

Representation quality depends on the retrieval task, candidate set and input conventions. In Exercise 1, converting both sides of the Kazakh–Kyrgyz task to CTS sharply reduces retrieval accuracy for the XLM-R paraphrase and NLLB encoders, while MiniLM improves slightly in this sample. Test input changes, including conversion and chunking, with the same queries and candidate set. For a retrieval-augmented system, check retrieval before judging the generated answer. Chapter 14 turns to models that write Turkic text.

Further Reading

- Feng et al. (2022), Heffernan, Çelebi, and Schwenk (2022), Wang et al. (2024), and J. Chen et al. (2024) describe the multilingual encoders surveyed in this chapter, and Artetxe and Schwenk (2019) the margin criterion for mining parallel sentences.
- Baysan and Gungor (2025) describes TR-MTEB, the Turkish embedding benchmark.
- Kuriyozov, Doval, and Gómez-Rodríguez (2020) builds cross-lingual word embeddings for Turkic languages.

- Yeshpanov et al. (2024) describes KazQAD, the Kazakh open-domain QA dataset with a retrieval component.
- Can et al. (2008) is the classic study of stemming for Turkish information retrieval.

Exercises

1. **Russian documents and scripts.** Extend the hands-on to SIB-200: for Kazakh, Kyrgyz and Tatar queries, retrieve the Russian translation of each sentence and compare with Turkic neighbours. Then test native/native, CTS/CTS and native→CTS conditions for four pairs.
2. **Lemmatisation errors.** On KazQAD, compare BM25 over surface forms, prefixes (choose k on validation), analyser lemmas and subwords. Inspect the test queries where lemmas lose most: how many losses come from wrong or merging lemmas?
3. **STS.** Compare encoders on the Kardeş-NLU STS test sets with Spearman's ρ and bootstrap intervals. Then design a 50-pair STS set for a Turkic language without one: two native raters, a 0–5 scale, and quadratic-weighted κ for their agreement.
4. **Chunking.** Take the ten longest Kazakh articles in KazQAD and split them into 512-token chunks with three LLM tokenisers and one embedding tokeniser. How many words does a chunk hold on average with each? What does this mean for a RAG system's context budget?

14 Generation, Dialogue, and Interactive Evaluation

Learning Objectives

After reading this chapter, you will be able to:

- Describe the ways in which language models fail when they write Turkic text: drift into another language, script switching, malformed words, and pragmatic errors
- Measure language and script fidelity automatically, and use morphological-analyser coverage as a diagnostic requiring human validation
- Design a validation of an LLM judge against human judgements in the target language
- Survey the data for generation and dialogue in Turkic languages, and its gaps
- Adapt dialogue games such as Taboo and Wordle to Turkic languages, including rule checking that respects morphology
- Measure how language fidelity changes over the turns of an interaction

Chapter 13 examined how to find supporting text. This chapter asks how to evaluate the answers and conversations a model produces. Multiple-choice scores do not directly test whether an answer stays in the requested language and script, uses appropriate word forms or suits the social context.

The chapter moves from individual answers to extended interaction. Its final section uses dialogue games: structured conversations between models with explicit rules (Chalamalasetti et al. 2023). Those rules make comparisons repeatable, once the language-specific word lists and instructions have been validated.

14.1 What Goes Wrong When Models Write Turkic Text

The following seven categories organise the evaluation. They can occur in other languages too, and their order says nothing about frequency.

1. **Drift into a neighbouring language.** Kazakh output with Kyrgyz or Tatar words; Turkish words in Azerbaijani, Turkmen, or Gagauz output. Turkish has the largest token total among the listed Turkic subsets of HPLT v3 (Table 4.1); this does not reveal any particular model's training mixture or prove the cause of drift. In Exercise

2, GlotLID (Kargaran et al. 2023) labels 20 of the 66 sentences that Qwen2.5-1.5B-Instruct wrote in answer to Kazakh questions as Kyrgyz.
2. **Drift into the contact language.** Russian words and phrases in Kazakh, Kyrgyz, Tatar, or Bashkir output; sometimes a switch to Russian after a few sentences. Language confusion of this kind has been documented for many languages (Marchisio et al. 2024).
3. **Script switching.** An Uzbek answer in Cyrillic to a question asked in Latin, or the reverse; Kazakh in a mixture of Cyrillic and one of the Latin alphabets.
4. **Malformed words.** Vowel-harmony violations (a back-vowel suffix on a front-vowel stem), suffixes in the wrong order, or invented suffix combinations. An annotator must distinguish genuine errors from loanword exceptions, dialectal forms and legitimate allomorphy.
5. **Wrong level of address.** Languages such as Turkish, Uzbek and Kazakh distinguish a familiar 'you' (Turkish and Uzbek *sen*, Kazakh *сен*) from a polite or plural one (*siz*, *сіз*). The choice depends on age, relationship, and setting, and it shows up in verb agreement and possessive suffixes as well as in the pronoun.
6. **Evidential meaning.** Some forms can convey indirect evidence or reported information: Turkish *-mIş*, Kazakh *-ған екен*. Evaluate the choice in its discourse context; a past-tense form is not universally a claim of eyewitness knowledge, and a summary need not mechanically copy every source suffix.
7. **Cultural grounding.** Names, holidays, units, place names or idioms that do not fit the requested context. A foreign name or borrowed expression is not itself an error.

Exercise 2 collects examples of several types from one small open model, Qwen2.5-1.5B-Instruct (Qwen Team et al. 2024), answering everyday cultural questions:

- *Neighbouring language.* Asked in Kazakh when and how Nauryz is celebrated, the model begins *Наурыз - бул Кыргызстандын …*, with Kyrgyz *бул* 'this' (Kazakh *бұл*) and the Kyrgyz genitive of Kyrgyzstan (Kazakh *Қырғызстанның*), and goes on to talk about Kyrgyzstan.
- *Contact language and script.* Asked in Latin-script Uzbek about Navroʻz, it answers *Navroʻz bayrami 15-16 декабря*: a Russian month name in Cyrillic inside a Latin-script sentence. Asked about Alisher Navoiy, it writes *literaturniy tili*, a transliterated Russian adjective, instead of Uzbek *adabiy til* 'literary language'.
- *Cultural grounding.* The Kazakh answer about Abai Qunanbaiuly calls him one of Kazakhstan's cities.

The model did not produce clear cases of the address and evidentiality types in these answers; finding them needs prompts that require them, such as the role plays described below. Repetition loops, which affected several answers, are not specific to Turkic languages.

14.2 Measuring Fidelity Automatically

Three automatic signals help triage output; none establishes correctness alone:

- **Language, per sentence.** Search the full LID label space, including Russian and English. A Turkic-restricted classifier must select a Turkic label even for Russian.

Retain the top label, score and text; choose an abstention threshold on development examples from the target and competing languages, then validate it on held-out data. Audit short, mixed and ambiguous spans manually (Section 3.7). Shared words and quotations are not automatically drift.

- **Script, per word or letter.** Declare the requested script and the denominator. Report mixed and unknown cases separately; names, quotations and code can legitimately use another script.
- **Raw analyser recognition.** Count alphabetic word tokens with at least one finite, non-failure FST reading. Report numerator, denominator and transducer hash. A pipeline's guessed lemma or `UPOS != "X"` can include heuristic fallback and is not raw transducer acceptance.

An FST can accept a grammatical word used in the wrong context, including a word shared with another language. It can also reject a valid name, loan or dialectal form missing from its lexicon. Use coverage as a diagnostic. Compare matched human-written text and validate against independent error annotations before setting a threshold.

```
# Run from the book root; provide a tokenizer and a pinned HFST transducer.
import sys
sys.path.insert(0, "code/ch04-corpus")
from corpus import fst_coverage

def analyser_coverage(text, tokenize, transducer):
    return fst_coverage(tokenize(text), transducer.lookup)
# Returns accepted_words, eligible_words and coverage (None if no eligible words).
```

A handful of illustrative sentences says nothing reliable about the languages as a whole. To validate this diagnostic, measure false alarms and missed errors on held-out examples, including names, shared vocabulary and code-switching. Keep raw recognition and any normalised-lookup variant as separate measurements.

Exercise 3 compares 1,000 KazNERD news sentences (Yeshpanov, Khassanov, and Varol 2022) with 668 news-style sentences generated by Qwen2.5-1.5B-Instruct. Apertium accepts 94.5% of the human text's word tokens and 84.1% of the model's. Restricting the model sample to sentences labelled Kazakh by GlotLID raises coverage to 84.7%, though such sentences can still contain mixed-language words.

Frequent rejected model tokens include the Kyrgyz forms *Берилген*, *өзгөрүшү* and *Бул* (Kazakh *берілген*, *өзгеруі*, *бұл*). Rejections in the human text include different cases: *i* and *ii* are Roman numerals typed with Cyrillic *i*; *нші* is an ordinal suffix separated from its numeral (*2-нші*); and *көпбалалы* 'having many children' is a valid word absent from the lexicon. These observations need systematic annotation. The samples differ in content and length, and repeated model phrases affect token counts. Independent judgements are needed to measure false alarms and missed errors.

14.3 Generation Tasks and Data

Summarisation. MLSUM includes Turkish news summaries (Scialom et al. 2020). XL-Sum's published current repository statistics list Azerbaijani (6,478 train / 809 validation

/ 809 test), Kyrgyz (2,266 / 500 / 500), Turkish (27,176 / 3,397 / 3,397) and Uzbek (4,728 / 590 / 590) (Hasan et al. 2021). These are article–summary pairs from BBC; the totals differ from the repository's legacy v1.0 release. See the maintainers' split table. The four entries show what data exist; other Turkic summarisation resources may be missing from the list.

Instruction and chat data. Distinguish native-authored, translated, synthetic and mixed collections. Translation can introduce unsuitable names, units or contexts; native authorship alone does not guarantee factual or linguistic quality. For every selected release record provenance, revision, split sizes, language/script, licence and independent quality checks. Sherkala-Chat's Kazakh instruction data, for example, combine translated sets with an automatically built, manually checked Kazakhstan-specific set (Chapter 12). This chapter does not inventory instruction releases, and no family-wide absence is claimed.

Dialogue data. Separate single-turn instruction pairs, open conversation, task-oriented exchanges and spoken dialogue. Their supervision and evaluation units differ. For each release, check native versus translated provenance and redistribution terms, and record what the documentation does not say. A search that finds no resource is not proof that none exists.

14.4 Evaluating Generated Text

14.4.1 Metrics against references

For summaries and translations with references, chrF adds character-level overlap to the word-level evidence from BLEU. A wrong suffix affects the whole word match in BLEU and a smaller set of character matches in chrF (Chapter 10). Neither metric directly measures morphological correctness, and their relative usefulness depends on the task. Open-ended answers, dialogue and pragmatic choices need evaluation beyond reference overlap.

14.4.2 LLMs as judges

Using a strong LLM to rate generated text is now common (Zheng et al. 2023). Validate each judge on the target language and error types. Include language confusion and morphological errors rather than assuming a general rating captures them.

Exercise 4 pairs 100 Kazakh SIB-200 sentences (Adelani et al. 2024) with automatic changes: 25 each involving vowel harmony, a Russian conjunction, a Kyrgyz cognate or a word in Latin script. Only the harmony change (*қайықтар* → **қайықтер*) reliably creates an ungrammatical standard form. The other changes may produce acceptable variants. Russian *но* occurs in informal Kazakh–Russian mixing, Latin spelling need not be a grammatical error, and replacing *мен* with *менен* yields an existing Kazakh word: the ablative of *мен* 'I', meaning 'from me'.

The pairs carry labels for the applied changes, without speaker validation of whether each result is wrong. Qwen2.5-1.5B-Instruct chooses "A" in 91 pairs with an English prompt and 88 with a Kazakh prompt. It chooses the original sentence in 34 and 32 pairs; 5 and 12 replies are unparsable. Randomisation leaves the original first in only 37 pairs. These results reveal a strong position preference in this setup, while leaving error-detection ability unresolved.

Before comparing judges, have speakers label errors, acceptable variants and register differences. Drop or relabel substitutions that produce valid words, and present every pair in both orders.

Until such checks have been done for a language, LLM judgements of Turkic text should be validated against at least a small sample of human judgements (Section 15.4).

14.5 Interactive Evaluation with Dialogue Games

14.5.1 Why games

A dialogue game is a short, rule-governed interaction between two players. Here the players are both models, or a model and a program. clembench (Chalamalasetti et al. 2023) evaluates chat-optimised language models by having them play such games: word guessing, reference games, and others. The program that runs the game checks every move against the rules and scores the outcome. Games have three advantages for comparing Turkic languages:

- **The same game can be run in every language**, but translation alone does not equate difficulty. Match word frequency, ambiguity, clue informativeness, alphabet rules and cultural familiarity, and report remaining differences.
- **Rule violations are measured, not just success.** A model that wins Taboo by using a forbidden word has broken a rule.
- **Long interactions show drift.** Language and script fidelity can be measured turn by turn.

14.5.2 Taboo needs morphology

In Taboo, one player must describe a target word without using it or a few related words, and the other player must guess the target. Exact matching can miss inflected forbidden words, in English as well as Turkic languages. If the target is *ev* 'house', the describer can write *evlerimizden* 'from our houses' and a surface-string check will not notice. Morphological analysis can identify additional candidates, but ambiguous readings and unknown words need an explicit review policy.

Use the rule checker (`code/ch14-generation/turkic_taboo_checker.py`) with a pinned transducer. It reports exact surface hits, lemma candidates and unresolved tokens separately. It considers all analyser readings, so an ambiguous reading can produce a candidate that needs contextual adjudication. There is no prefix fallback: a shared prefix

is not evidence of a forbidden lemma, and a retrieval experiment's best truncation length is not a validated game-rule threshold.

```
import sys
sys.path.insert(0, "code/ch14-generation")
from turkic_taboo_checker import check_tokens

# tokens: output of the declared tokenizer; lookup: pinned HFST lookup callback.
result = check_tokens(tokens, forbidden=["үй", "пәтер", "тұр"],
                      lookup=transducer.lookup, lang="kaz")
# result contains surface_hits, lemma_candidates and unresolved.
```

The forbidden list must use the selected analyser's lemma convention. Record any infinitive/stem conversion explicitly; do not strip a universal suffix across languages. Freeze whether ambiguous candidates invalidate a turn or trigger human review. Tokenisation and casing must be identical for the surface and morphological comparisons.

14.5.3 Wordle and other games

Wordle asks a player to find a five-letter word from feedback on each letter. It depends on the alphabet and on what counts as a word. Turkic alphabets differ in size and letter frequencies, and the allowed word list may contain lemmas, inflected forms or both. Specify that choice and whether length counts orthographic letters, grapheme clusters or code points; normalise Unicode and apply language-aware casing consistently. Reference games, where one player describes an item so the other can pick it out, can be localised with culturally familiar content.

14.5.4 Results

Exercise 1 uses Qwen2.5-1.5B-Instruct as both describer and guesser for twelve Kazakh targets, with at most three rounds. No game is won. Three opening clues contain the target itself (*Кітап*, *Жылқы*, *Балық*), which the surface check catches; the other nine games reach the turn limit. The lemma checker detects no additional violations and leaves the outcomes unchanged. Missed violations remain possible: 72 of the 246 clue words have no analyser reading, and the exercise allows unresolved tokens to pass. GlotLID labels four of the 30 clues Kyrgyz.

The target lists still need speaker review. All twelve targets are frequent concrete nouns, so they cover no range of frequency bands. *Күн* means both 'sun' and 'day', but its forbidden words address only 'sun'. Other forbidden lemmas have homonyms: *іш-* 'drink' and *іш* 'inside, belly'; *жаз-* 'write' and *жаз* 'summer'; *ат* 'horse' and 'name'; *бет* 'page' and 'face'; *мін-* 'ride' and *мін* 'fault'. A lemma-only checker can flag an allowed sense. Record part of speech as well as lemma, and adjudicate remaining ambiguity in context.

The exercise also differs from the Hands-On protocol: unresolved words pass without adjudication, and the guess matcher checks only the first word. It could therefore accept a multiword response despite the one-word instruction. Fix these policies and validate the word list before comparing systems.

14.5.5 Address and politeness

Test address with role plays written by native speakers. Each should specify a relationship and setting, such as a student speaking to a teacher or two friends talking. Ask speakers which forms are acceptable for that situation and variety. Pronouns and agreement offer clues, but pro-drop may leave the choice unexpressed. Retain ambiguous and omitted cases in the report instead of forcing them into familiar or polite categories.

14.6 Language Drift in Ordinary Chat

Ask models ordinary questions in a Turkic language and check the language and script of each answer. Short and mixed-language answers need manual review.

Exercise 2 does this for 25 culturally grounded questions each in Kazakh and Uzbek with Qwen2.5-1.5B-Instruct. Only 13 of the 25 Kazakh answers have every sentence labelled Kazakh by GlotLID; 20 of their 66 sentences are labelled Kyrgyz, and Apertium accepts only 58.7% of their words. GlotLID labels 83 of the 84 Uzbek-answer sentences Uzbek, with every sentence so labelled in 24 of the 25 answers. Yet one answer contains Cyrillic Russian month names. The labels help select passages for review; an answer that passes may still mix languages or get the facts wrong.

14.7 Spoken Dialogue

Voice assistants combine speech recognition (Chapter 11), a dialogue model, and speech synthesis. Published MMS checkpoints include Chuvash and Crimean Tatar TTS (Table B.6); checkpoint availability is not evidence of conversational quality. Audit each component's language, licence, input/output script, latency and held-out quality, then test the combined system with speakers.

14.8 Hands-On: A Procedure for Kazakh Taboo with a Morphological Rule Checker

Question. How well can a Kazakh Taboo game enforce morphological rules?

Inputs. Twenty speaker-reviewed targets, forbidden lemmas, two declared model roles and the bounded controller.

Compute. Inference for the selected models, plus fluent-speaker adjudication; local controller tests use scripted replies.

Output. Turn logs, rule-check outcomes, guesses and language/script diagnostics.

Interpretation. Count failed and unresolved games and compare automated checks with human decisions.

This is a protocol for a simplified Kazakh Taboo experiment. Exercise 1 illustrates a simpler version with one small model and twelve targets; the differences are described above.

1. Ask a speaker to select 20 target lemmas across frequency bands, with three related forbidden lemmas each. Record the variety, script and adjudication policy.
2. Use the bounded controller in `code/ch14-generation/game.py`. Supply separate describer and guesser callbacks, a clue checker and an explicit guess-matching policy. The target and forbidden list belong only in the describer's context.
3. Allow at most three rounds. Stop on a rejected clue, a correct guess, an unresolved rule check or the turn limit. Save each turn and its outcome; do not drop failed games from denominators.
4. Record unrestricted language labels, script signals and raw FST recognition for each utterance. Have a speaker adjudicate ambiguous lemma candidates and unresolved tokens before comparing checkers.

The local scripted-reply test verifies controller flow and target secrecy; the model game requires the protocol above.

Questions to answer:

1. How many violations does the surface checker miss?
2. Do the models drift into Russian or another Turkic language during the game? In which turns?
3. Are the guesser's answers in the base form, or inflected? How should the game score an inflected correct guess?

Status: procedure only. The speaker-reviewed twenty-target game is not run here. Exercise 1 records a separate twelve-target diagnostic whose target list remains unchecked.

What to Take Forward

Useful generated text needs more than fluent-looking words. The Kazakh generation exercises produce sentences labelled Kyrgyz by GlotLID and lower analyser recognition than the human news sample. These diagnostics flag passages for review; speaker judgements establish whether they contain language or grammatical errors. Validate automatic judges against human decisions before relying on their scores. Chapter 15 combines these checks with task scores, baselines and uncertainty.

Further Reading

- Chalamalasetti et al. (2023) introduces clembench and the dialogue-game approach to evaluating chat models.
- Marchisio et al. (2024) studies language confusion: models answering in the wrong language or mixing languages.
- Zheng et al. (2023) studies strong LLMs as judges and discusses their biases.

- Scialom et al. (2020) and Hasan et al. (2021) describe the multilingual summarisation datasets that include Turkic languages.

Exercises

1. **Port a game.** Play Kazakh Taboo with a model as describer and guesser, using the controller and rule checker of the Hands-On. Compare the surface and lemma checkers, track the language of each clue by round, and decide how to score inflected guesses. Then document every language-specific decision needed to port one more clembench game: word lists, morphology in the rule checks, script and cultural content.
2. **Error collection.** Ask a model 25 culturally grounded questions each in two Turkic languages (holidays, food, place names, proverbs). Measure language, script and analyser coverage automatically, then classify the errors in the answers with the seven types of Section 14.1.
3. **Coverage baseline.** Compute analyser coverage on 1,000 sentences of human-written news text and on model-written news in the same language. How large is the difference, and what kinds of words account for it?
4. **Judging a judge.** Create pairs in which the second sentence contains one Turkic-specific error from Section 14.1. Ask an LLM judge which sentence is correct, with the order counterbalanced. Which error types does it miss, and does it prefer one position?

15 Evaluation, Benchmarks, and Responsible Deployment

Learning Objectives

After reading this chapter, you will be able to:

- Explain how task definitions and scoring conventions affect the interpretation of accuracy, BLEU and F1
- Compare selected Turkic NLP benchmarks by task, language, script and data provenance
- Combine task scores with human evaluation, language-fidelity checks and interactive tests
- Identify gaps between benchmark coverage and an application's needs
- Plan human evaluation for the intended variety and domain, and report annotator agreement
- Assess deployment risks from uneven performance, domain shift and inadequate community involvement
- Set deployment criteria based on the consequences of errors

The preceding chapters report task scores and describe their limits. Chapter 14 also examines generation and dialogue, where a single score can hide different kinds of failure. This chapter brings the evaluation methods together: choosing metrics, checking benchmark exposure, estimating uncertainty and assessing errors in the intended use.

A score on exam questions, news text or translated sentences answers a specific question. Deployment may involve other domains, varieties and error costs. Human evaluation and task-specific checks connect those scores to the application; Chapter 17 develops the community and governance questions.

15.1 Principles for Reading and Reporting Results

Start with the decision an experiment should support. A useful report makes five things clear:

1. **Task and data.** Name the language, variety, script, domain, release and split. Keep evaluation items and their source groups out of training and model selection.
2. **Comparison.** Include a simple baseline and use the same evaluation items. Record differences in training size, tuning budget, input length and inference settings.

3. **Measurement.** Define the unit, denominator, label mapping and exclusions. Coverage measures how often a system supplies an output; accuracy requires an appropriate reference. Human review supplies evidence about meaning and acceptability.
4. **Uncertainty.** Report sample size, variation across runs where measured, and paired uncertainty when comparing systems. A difference observed in one configuration motivates replication on other data.
5. **Scope and provenance.** Link the claim to its predictions, table or worked exercise. Identify model and dataset revisions, known training exposure and unresolved exposure. A diagnostic illustrates behaviour under its stated conditions; a deployment claim needs evidence from the intended use.

These principles apply throughout the book. The following sections develop the task-specific metrics, contamination checks and human-evaluation choices. Community authority, consent and maintenance have their main treatment in Section 17.4.

15.2 Evaluation Challenges for Agglutinative Languages

15.2.1 Features, Bundles and Sentences

Different metrics answer different questions. For gold `Case=Acc|Number=Sing` and predicted `Case=Nom|Number=Sing`, one of the two values is correct: feature accuracy is 50%, while exact-bundle accuracy is 0%. Predicting nominative **plural** instead gives 0/2, not 50%. Specify how absent and extra features count. Lemma accuracy and lemma-plus-bundle accuracy are separate measures.

If word analyses were independent and each had correctness probability 0.9, a ten-word sentence would be entirely correct with probability $0.9^{10} \approx 0.349$. Real errors need not be independent or equally likely. Measure sentence-exact accuracy directly.

Exercise 1 compares these measures using gold word boundaries, including punctuation. Stanza's Turkish IMST predictions achieve 91.3% exact-bundle accuracy and 50.7% sentence-exact bundle accuracy. The sentence measure requires every feature bundle to match, leaving lemmas and syntax untested. Applying the same per-word probability to each sentence's actual length gives a mean estimate of 50.3%. Its closeness to the observed rate cannot validate the independence assumption. The notebook's length-bin estimates are rougher because they substitute one representative length per bin.

Kazakh's observed and estimated rates are 89.0% and 84.0%. Treat both as replay diagnostics: checkpoint preparation creates a concrete test-exposure risk, and historical training membership has not been reconstructed (Chapter 8).

15.2.2 Surface and Lemma Novelty

Surface-form OOV includes unseen inflections of familiar lemmas. Lemma OOV tracks unfamiliar lemmas themselves. Define the training vocabulary and use gold lemmas

consistent with the annotation scheme. Then separate four cases: seen surface/seen lemma, unseen surface/seen lemma, seen surface/unseen lemma (possible with ambiguity), and unseen surface/unseen lemma. Report counts and error rates for each; leave empty cells undefined. Subword unknown-token rates measure a different property.

On the Turkish IMST test set (Exercise 2), 33.2% of the non-punctuation words are surface-unseen in training but only 10.2% lemma-unseen; the feature-bundle error rate is 6.3% for seen forms with seen lemmas and 20.5% for unseen forms with seen lemmas. These cells show where errors occur; they do not isolate the cause of the difference.

15.2.3 Translation Metrics

BLEU compares **token** n-grams under a specified tokenisation; chrF compares character n-grams. A suffix error can break a word match while preserving some character overlap. Neither metric decides whether the changed suffix preserves meaning. Exercise 3 adds negation to one verb in each of 100 Turkish X-WMT references, for example *söyledi* 'said' → *söylemedi* 'did not say'. Mean sentence chrF remains 97.7. Word deletion gives 92.3 on a separately selected set. Both edits can change meaning; these scores measure overlap, not the severity of the change. The generated edits still need contextual review.

Report the metric signatures and reference policy from Chapter 10, and validate learned metrics such as COMET against judgements in the target language/domain rather than assuming training-language coverage guarantees reliability.

Choose human-evaluation sample size for the desired interval width, error prevalence and subgroup analysis. A fixed sample of 100–200 outputs is not a universal adequacy threshold. Preserve paired outputs when comparing systems.

15.2.4 Benchmark Contamination

Native exam questions, translations and public games can appear in pretraining or adaptation data, including for languages with a small web presence. Save benchmark release and model cutoff information where available, search for overlap in accessible training sources, and distinguish known exposure from unknown exposure. A performance drop on rewritten questions can reflect changed difficulty as well as memorisation. Use independently authored held-out material as a complementary test (Isbarov et al. 2025).

15.3 Existing Benchmarks

15.3.1 TUMLU: Native Multiple-Choice Evaluation

TUMLU collects native middle- and high-school multiple-choice questions in 11 subjects (Isbarov et al. 2025). Its abstract and tables list nine languages: Azerbaijani, Crimean Tatar, Karakalpak, Kazakh, Kyrgyz, Tatar, Turkish, Uyghur and Uzbek. Other passages describe eight languages and a total of 38,139 questions. Because these descriptions

conflict, base experiments on audited counts from a pinned release. The Hands-On uses only Kazakh TUMLU-mini.

The TUMLU paper compares proprietary and open models by language and subject. Use the model-specific tables; differences in question content, subject coverage and training exposure prevent a language-level score from isolating a resource effect.

15.3.2 Kardeş-NLU: Cross-Lingual Transfer Evaluation

Kardeş-NLU (Senel et al. 2024) covers Azerbaijani, Kazakh, Kyrgyz, Uzbek and Uyghur. Its tasks are natural language inference from XNLI, semantic textual similarity from STS-Benchmark and causal reasoning from XCOPA. English items were machine-translated and post-edited by one native speaker per target language. The Uyghur set uses Cyrillic, which matters when choosing a model or comparing it with Arabic-script data.

The paper tests Turkish as a source for intermediate training and downstream fine-tuning, reporting gains in its evaluated settings. These results motivate the source-language comparisons in Chapter 12 and Chapter 16.

15.3.3 Turkish Benchmark Suites

Mukayese (Safaya et al. 2022) groups Turkish datasets and baselines for language modelling, translation, named entity recognition, sentence segmentation, spell checking and correction, summarisation, and text classification. Each task has its own metric, and adding them up would not yield a measure of Turkish proficiency. Reading comprehension is also covered by Turkish XQuAD and Kazakh KazQAD, discussed in Chapter 13.

15.3.4 Kazakh and Multilingual Benchmarks

KAZ-LLM reports evaluation in Kazakh, Russian and English (Institute of Smart Systems and Artificial Intelligence (ISSAI) 2024). KazMMLU (Togmanov et al. 2025) provides 23,000 questions (10,969 in Kazakh and 12,031 in Russian) from authentic educational material, validated by native speakers and educators, on general and Kazakhstan-specific knowledge. These are not matched translations: KazMMLU's university-level section is Russian-only. Language-level scores therefore also reflect differences in subject and education level.

Broader multilingual datasets include FLORES-200 for MT (11 Turkic varieties), SIB-200 for topic classification (11 Turkic varieties), and Belebele for reading comprehension (Bandarkar et al. 2024) (Turkish, North Azerbaijani, Kazakh, Kyrgyz, and Northern Uzbek).

15.3.5 X-WMT: Turkic Machine Translation Evaluation

X-WMT, produced as part of the Turkic Interlingua (TIL) project, provides machine translation test sets of at least 300 sentences for 88 translation directions involving eight Turkic languages (Azerbaijani, Bashkir, Karakalpak, Kazakh, Kyrgyz, Sakha, Turkish and Uzbek) plus English and Russian (Mirzakhalov, Babu, Ataman, et al. 2021). The release covers intra-Turkic pairs (Kazakh–Uzbek, Turkish–Azerbaijani, and others) as well as Turkic-to-Russian and Turkic-to-English directions.

FLORES-200 complements X-WMT with aligned evaluation sentences for the eleven Turkic varieties in NLLB-200. Use the split and overlap checks in Chapter 10.

15.3.6 Beyond Multiple Choice: Generation and Interaction

Multiple-choice benchmarks measure answer selection under a specified protocol. Complement them with the methods in Chapter 14:

- **Human review of generation:** assess meaning, fluency and use of the requested language on answers, summaries or rewrites.
- **Automatic language checks:** use unrestricted language identification to select outputs for review. Keep predicted labels separate from confirmed language drift.
- **Interactive tests:** use dialogue games with explicit rules, validated local word lists and recorded failures. Fresh instances help test beyond public examples, but do not by themselves rule out training exposure.

Exercise 5 illustrates the limits of automatic checks. It selects an abstention threshold from Kazakh-labelled news sentences, then applies it to whole model answers. The threshold is provisional, and it has not been validated as a multilingual detector: the sample lacks labelled non-Kazakh and mixed-language controls. One example per predicted category is a starting point for annotation, not enough to estimate error rates.

15.3.7 The Benchmark Gap

The catalogue verifies selected releases and leaves gaps in language coverage. For languages without a suitable public benchmark, work with the community on a domain-specific evaluation and document its sampling frame and limits. Check for overlooked or newer resources before concluding that a benchmark is unavailable.

Table 15.1: Selected benchmarks covering Turkic languages.

Benchmark	Languages	Task	Venue
TUMLU	9 listed; see release caveat above	Native multiple-choice QA	ACL 2025
Kardeş-NLU	5 Turkic (az, kk, ky, uz, ug)	Cross-lingual NLU	EACL 2024

Benchmark	Languages	Task	Venue
Mukayese	Turkish	Multiple NLP tasks	Findings of ACL 2022
X-WMT	8 Turkic + en, ru	MT (88 directions)	TIL (Mirzakhalov, Babu, Ataman, et al. 2021)
KazMMLU	Kazakh, Russian	Native multiple-choice QA	ACL 2025
Belebele	5 Turkic	Reading comprehension	ACL 2024
FLORES-200 / SIB-200	11 Turkic varieties	MT / topic classification	NLLB; EACL 2024
UD-Turkic parallel treebanks	Release-specific inventory in Chapter 8	Tagging and parsing	UDW 2025; UD 2.17

15.4 Human Evaluation for Turkic NLP

15.4.1 Evaluator Competence and Agreement

Recruit evaluators who know the target variety, register and domain. Native-speaker status alone may leave gaps in that expertise. Work with community partners, pay for expertise and annotation time, and pilot the guidelines before scaling. Seek representation beyond a single institution, and verify the competence of evaluators who speak a neighbouring language.

Report agreement before adjudication, along with category frequencies and a table of disagreements. State how the agreement statistic accounts for chance. There is no universal kappa cutoff or inherent agglutinative-language agreement ceiling. Feature-level agreement, exact bundles and span/attachment agreement expose different disagreements. Revise unclear guidelines on development material, keeping the final evaluation independent.

15.4.2 Scale and Uncertainty

Specify the population from which items and evaluators were sampled. Use paired uncertainty estimates for model differences and account for clustering by document, speaker or annotator. Small sets can support a pilot. They are too small to measure performance for every variety or rare error type. Plan sample size and annotation effort from a timed pilot rather than asserting a fixed limit on community capacity.

15.5 Responsible Deployment

15.5.1 Resource Imbalance and Model Quality

Training volume, sampling, tokenisation, domain and adaptation can all affect quality. Temperature sampling, for example, changes language exposure relative to raw corpus size (Section 16.4). Report results by task and subgroup. A single language comparison cannot separate these factors or justify predicting quality from speaker population or web volume.

15.5.2 Precision, Recall and Consequences

An entity F1 of 0.70 does **not** mean that 30% of entities were missed. For example, 7 true positives, 0 false positives and 6 false negatives give precision 1, recall 7/13 and F1 0.70: 6/13 of gold entities were missed. Swapping the false-positive and false-negative counts gives the same F1 with recall 1 and no missed entities. These arithmetic examples are checked by `code/common/check.py`.

Set acceptance criteria for the intended task: which false positives or omissions matter, when the system should abstain, and which outputs need review. Test those criteria on representative data, record who handles failures, and reassess after changes to the model or input domain. Aggregate F1 alone cannot make that decision.

15.5.3 Domain, Script and Variety

Good results on news say little about private messages, legal records, medical text or speech transcripts. Evaluate script variants and mixed-language inputs explicitly; a Cyrillic-trained model may degrade on informal Latin input, but failure is not guaranteed merely by the script difference. Preserve community names and variety distinctions instead of reducing language boundaries to a technical convenience.

15.5.4 Environmental and Community Costs

Record hardware, runtime and measured energy where available, stating the assumptions of any emissions estimate. Fewer trainable parameters do not automatically imply proportionally less compute. Discuss who sets the priorities, who bears annotation and maintenance work, and who can access the result; do not infer the geographic distribution of costs and benefits without project evidence.

15.6 Hands-On: Evaluating a Multilingual Model on TUMLU

Question. How does a small model compare with simple answer-choice baselines?

Inputs. The pinned Kazakh TUMLU-mini release and schema audit. The procedure uses mT5; the measured comparison below uses Qwen.

Compute. Model download and inference memory. The mT5 procedure is not run in full; the two Qwen probes use different precision/runtime settings.

Output. Per-item scores, exclusions, accuracy, uncertainty and answer-label distributions.

Interpretation. Compare the development-selected baseline and inspect option-order sensitivity.

Use the released TUMLU-mini, pinned at `3523de73e1d1d784118d328f40b49d53afd6f334`. The verified Kazakh configuration contains **40 development and 799 test rows**; the audit found **two test rows with empty choices**, leaving 797 schema-valid test questions under an explicit exclusion policy. Only one language subset of the mini release is used here; the full benchmark is larger.

```
python code/ch15-tumlu/tumlu.py --config kazakh --audit-only --output runs/tumlu-audit
python code/ch15-tumlu/tumlu.py --config kazakh --split dev --output runs/tumlu-dev
# Freeze the prompt and scoring rule before this final evaluation:
python code/ch15-tumlu/tumlu.py --config kazakh --split test --exclude-invalid --output
 ↪  runs/tumlu-test
```

The scorer reads `question`, `choices`, `answer` and `subject`. It checks the number of choices and the letter-to-choice mapping, creates stable hashes and rejects exact question-and-ordered-choice duplicates within or across development and test. Paraphrases and reordered duplicates require further checks. The prompt excludes the released `CoT` explanation.

`audit.json` identifies invalid rows by original index and content hash. Scoring stops when invalid rows are present unless `--exclude-invalid` is supplied. That option selects the audited 797-item subset and records its denominator; the score covers that subset of the 799-row release.

The prompt contains the question and all labelled options. For each possible answer letter, mT5 computes the **sum** of conditional token log probabilities, including EOS and excluding padded positions. This differs from the negative mean loss returned by the model. The predicted letter maximises that sum; exact score ties use the first option. Test robustness to option permutation on development data and freeze the policy. Inputs are not silently truncated.

Base mT5 was pretrained for span corruption, not instruction following. The result diagnoses one fixed conditional-likelihood protocol. It is not a validated measure of how instruction-tuned models reason. Report the exact checkpoint, prompt, length rule and scoring method when comparing it with another model.

The output includes every item's scores and prediction, overall and per-subject support, accuracy, a Wilson interval and the analytical uniform-random expectation $\frac{1}{N}\sum_i 1/K_i$. The interval is descriptive under an independent-item assumption; source-clustered uncertainty requires source metadata. Never substitute a random simulation for the analytical baseline or describe below-expectation accuracy as significant without uncertainty analysis.

The book reports no mT5 score: the release schema and all Kazakh rows were audited and tiny-model tests verify the scoring arithmetic, but the full mT5 run and an independently authored contamination check are left to the reader.

15.6.1 A measured pretrained TUMLU-mini baseline

A separate zero-shot run uses **Qwen2.5-1.5B-Instruct** on the pinned Kazakh TUMLU-mini test split (Isbarov et al. 2025). Of 799 rows, two have empty choice lists (zero-based indices 362 and 390); the explicitly defined valid subset therefore contains **797 questions**. The user message contains the native question and four choices, preceded by "Select the correct option. Reply with only its letter." The model's chat template is applied without a gold answer or explanation.

The score is the next-token log probability of each single-token answer label A–D, selecting the highest. This constrained-letter protocol differs from free generation, full-choice likelihood and the mT5 example above. Inputs are not truncated. The pinned weights, prompt, per-item label scores and exclusions are recorded with the run.

Table 15.2: Measured zero-shot constrained-letter Qwen score on 797 valid TUMLU-mini Kazakh test items; two malformed items of 799 are excluded and recorded. The letter-D baseline is selected from the 40 development items, not the test answers. Uniform accuracy is an expectation, not a model run. Source: `run_tumlu.py`; Isbarov et al. (2025).

System	Correct / 797	Accuracy (%)
Qwen2.5-1.5B-Instruct	226	28.36
Development-majority letter D	304	38.14
Uniform choice (expectation)	Expected	25.00

Qwen answers **226/797 correctly: 28.36%, with a Wilson 95% interval of 25.34–31.58%**. The most frequent answer in the 40 development items is D. Always choosing D scores **304/797 (38.14%)** on test. That beats the model, even though the model itself clears the 25% uniform-choice expectation. Report this answer-label imbalance: uniform guessing is a weaker baseline than the simple strategy selected on development data.

The Wilson interval assumes independent items and excludes variation from prompts, option order and subjects. These results describe one small checkpoint on the valid Kazakh mini subset. They leave full TUMLU performance and independent contamination assessment open.

Exercise 4 uses an 8-bit version of the same model with a different inference implementation (230/797 correct, 28.9%) and measures option-order sensitivity on development data: under four cyclic rotations of the options, the model picks the same option content for only 4 of the 40 development questions, and it answers B in half of the 160 rotated prompts.

Status: completed for the Qwen probe; the mT5 scoring procedure is not run. The table above records the 226/797 main probe. Exercise 4 records a separate 8-bit probe (230/797) that also tests option order.

What to Take Forward

An evaluation starts from the decision the score must support. The unit matters: all morphological feature bundles match in 50.7% of the Turkish sentences; this says nothing about whether their lemmas or syntax are correct. Across 100 automatically negated references, mean sentence chrF remains 97.7, illustrating why overlap alone cannot assess meaning. Define the task, denominator, baseline and held-out data before scoring, report uncertainty, and inspect consequential errors. Chapter 16 applies these principles to choosing source languages and transfer methods.

Further Reading

Isbarov et al. (2025) describes TUMLU's construction and evaluation; Senel et al. (2024) studies transfer through Turkish. Safaya et al. (2022) provides task-specific datasets and baselines for Turkish. For resource availability and limitations, see Çöltekin, Doğruöz, and Çetinoğlu (2023).

Marneffe et al. (2021) describes the Universal Dependencies annotation framework. Shared feature names support comparisons, but treebank conventions and reference quality still need checking (Chapter 8). For generation and dialogue evaluation, return to Chapter 14.

Exercises

1. Compare feature, exact-bundle and observed sentence-exact accuracy on the same morphological predictions, for Kazakh and Turkish UD test sets. Explain the independence assumption behind a product estimate and where it fails.
2. Partition tagging errors by surface and gold-lemma novelty, reporting counts and undefined empty cells.
3. Measure how BLEU and chrF react to single-suffix changes that alter meaning (negation, case, person) in correct English→Turkish references, then compare two decoding settings of an MT model and prepare a blinded sheet for human judges. Document metric signatures.

4. Test TUMLU option-order sensitivity on development data, then freeze the scoring protocol before running the audited test subset.
5. Audit generated language with unrestricted LID, using human text to choose a provisional abstention threshold, and prepare a sample for fluent reviewers. Include Russian, other languages, code-switching and indeterminate short answers.

Part VII

Cross-Lingual Methods and the Resource Frontier

16 Cross-Lingual Transfer within the Turkic Family

Learning Objectives

After reading this chapter, you will be able to:

- Form testable hypotheses about within-family transfer using morphological and lexical evidence
- Design comparisons that distinguish script, relatedness and resource effects
- Evaluate proxy-language configurations where target task training data are unavailable
- Choose transfer methods to compare using the available target-language evidence
- Explain pooled training and test how temperature sampling changes language exposure
- Compare language-specific adapters with separate models and pooled training
- Distinguish pivot translation, back-translation and synthetic-data filtering
- Recognise the challenges of code-switched Turkic text

Earlier chapters tested transfer in morphology, parsing, NER, translation and speech. Chapter 12 and Chapter 13 also compared native and converted text for classification and retrieval. This chapter brings those methods together: choosing source languages, controlling script conversion, pooling training data and evaluating target-language results.

16.1 Why Related Languages Can Help

16.1.1 Structural and Lexical Hypotheses

Transfer reuses a model trained on one language to make predictions in another. Shared suffix patterns can help a tagger recognise similar grammatical functions; shared words can give an encoder familiar inputs. Multilingual pretraining offers another route: it can place texts from different languages in a shared representation space. A source language is therefore a candidate to test, using the target task and data to judge its usefulness.

The morphology in Chapter 5 suggests which constructions to compare. Written overlap may come from inherited cognates or shared loans, such as *kitap/kitab* ‘book’. Script

conversion changes that overlap. Compare sources on the same target sentences and record script, training size and domain alongside the score (Section 15.1).

16.1.2 Evidence from Tasks

Kardeş-NLU reports benefits from Turkish intermediate training in its evaluated settings (Senel et al. 2024). The parser comparison in Chapter 8 and TurkicASR in Chapter 11 provide further task-specific evidence. The parser table uses matched words and includes very small target samples, especially Sakha; interpret its scores under that evaluation convention.

"Zero-shot" here means applying a model without target task-labelled training. The model may have encountered the target language during multilingual pretraining. A proxy language is a chosen model input or routing configuration. Evaluate that choice on independent target annotations.

16.2 Script Conversion as an Experimental Condition

Script conversion supplies an alternative written input to the same task. The conversion conventions and their losses are explained in Section 3.4 and Section 3.6. Here the question is whether conversion helps transfer: compare it with native text under the model's documented input conventions. Muller et al. (2021) motivates testing this for unseen scripts.

1. Identify the language and script on the original text.
2. Record the converter, version and supported direction. Native Latin is not automatically CTS.
3. Preserve original strings, whitespace, item IDs and an explicit character-boundary map.
4. Run the model, recording any additional automatic script bridging.
5. Map predictions back only where the alignment is unambiguous; report failures and coverage.

Digraphs, deletions and context-sensitive mappings can shift character offsets even when word counts stay the same. Rebuilding text with `split()`/`join()` can also change its spacing. Preserve a map to the original boundaries; a lossy conversion may have no exact inverse.

16.3 Transfer Strategy by Available Evidence

Available annotation, domain match, model coverage and cost jointly determine which comparisons are feasible. No fixed sentence count marks the point at which a larger model or training from scratch becomes preferable.

Table 16.1: Candidate comparisons, not measured resource thresholds.

Available evidence	Initial comparisons	What to reserve
No target task labels	Native multilingual/proxy baseline; optional conversion; existing FST where relevant	Obtain independent target evaluation before claiming quality
Small labelled collection	Zero-shot, head/adapter tuning, pooled training	Source-disjoint development and test items
Labelled data plus unlabelled text	Continued pretraining vs. task tuning alone	Exclude all evaluation texts from adaptation
Larger task data and compute budget	Monolingual vs. multilingual; model-size and scratch-training controls	Same test set and comparable tuning budgets

Check existing resources before adapting a neighbouring language's system. Nogai, for example, already has an Apertium module whose lexicon and rules can be extended. For translation, verify NLLB's supported codes: adding a pivot cannot supply a missing language interface. Use development learning curves to estimate the value of further annotation or parallel data, and reserve separate gold sentences for final evaluation.

16.4 Multilingual Turkic Training

16.4.1 Pooling Data Across Languages

Pool only after checking label conventions, tokenisation and split provenance. Language tags, language embeddings and separate adapters are alternative design choices; none is required by multilingual training. Compare a language-conditioned model with an otherwise matched shared model. A dominant corpus can influence training, but raw language percentages alone do not identify which linguistic mechanism produced a gain.

16.4.2 Evidence from Speech and Parsing

Speech and parsing provide two task-specific comparisons. In speech, the TurkicASR system (Mussakhojayeva et al. 2023) trains a single model jointly on transcribed speech in ten Turkic languages and reports improvements for languages with little data; these results do not isolate the contribution of shared phonetics. In parsing, TurkicNLP's multilingual parser, trained on twenty UD-format treebanks of ten Turkic languages, is evaluated in the frozen matched-word comparison in Chapter 8. Its gold/silver reference mixture and small samples limit interpretation (Hakimov 2026).

Transfer failures should be analysed by language, domain and task. Report branch and script alongside the results, without treating them as explanations by themselves.

16.4.3 Language-Specific Adapters

State each training stage. One recipe begins with a shared pretrained encoder, optionally updates it on pooled text, then freezes it while training a target adapter **and task head**. Another trains shared weights and adapters jointly. Compare these as separate experiments. Distinguish bottleneck adapters from LoRA and report the modules trained, parameter counts and measured inference overhead.

Freezing shared weights prevents those weights from being overwritten during that stage, but does not guarantee preservation of end-to-end predictions: adapters and heads change outputs. Save all trained components and check reload equivalence.

16.4.4 Addressing Data Imbalance

Temperature sampling chooses language i with probability

$$p_i = \frac{n_i^{1/T}}{\sum_j n_j^{1/T}}.$$

With a synthetic 9:1 corpus-size ratio, $T = 1$ samples 90%/10% and $T = 2$ samples 75%/25%. The figures are illustrative arithmetic. In an actual run, track examples and updates per language. Choosing one language per batch controls exposure across steps; mixing all languages into every batch needs a compatible batch design. Select temperature on development data and check both smaller and larger languages.

16.5 Pivot Translation as Data Augmentation

A pivot translates source→pivot→target when both component directions are supported. Verify the actual released pair/model and script for each leg; a development repository is evidence neither of released coverage nor of translation quality. Compare direct translation, pivoting and any synthetic-data adaptation on the same independent target test set (Chapter 10).

Evaluate the complete source→pivot→target output. Scores for the two legs cannot simply be combined: the second model may preserve, introduce or repair errors.

Back-translation uses a target→source system to pair real target sentences with synthetic source text. Document filtering and keep source-document groups intact across splits. Check the real text too. Round-trip consistency can reward copying or repeatable mistakes, so combine it with language checks, alignment diagnostics and independent human review.

Compare filtered data with random subsets of equal size. Exercise 4 measures reference overlap on such subsets without training on them. To test their value for adaptation, generate separate training data and reserve the benchmark test set for final evaluation.

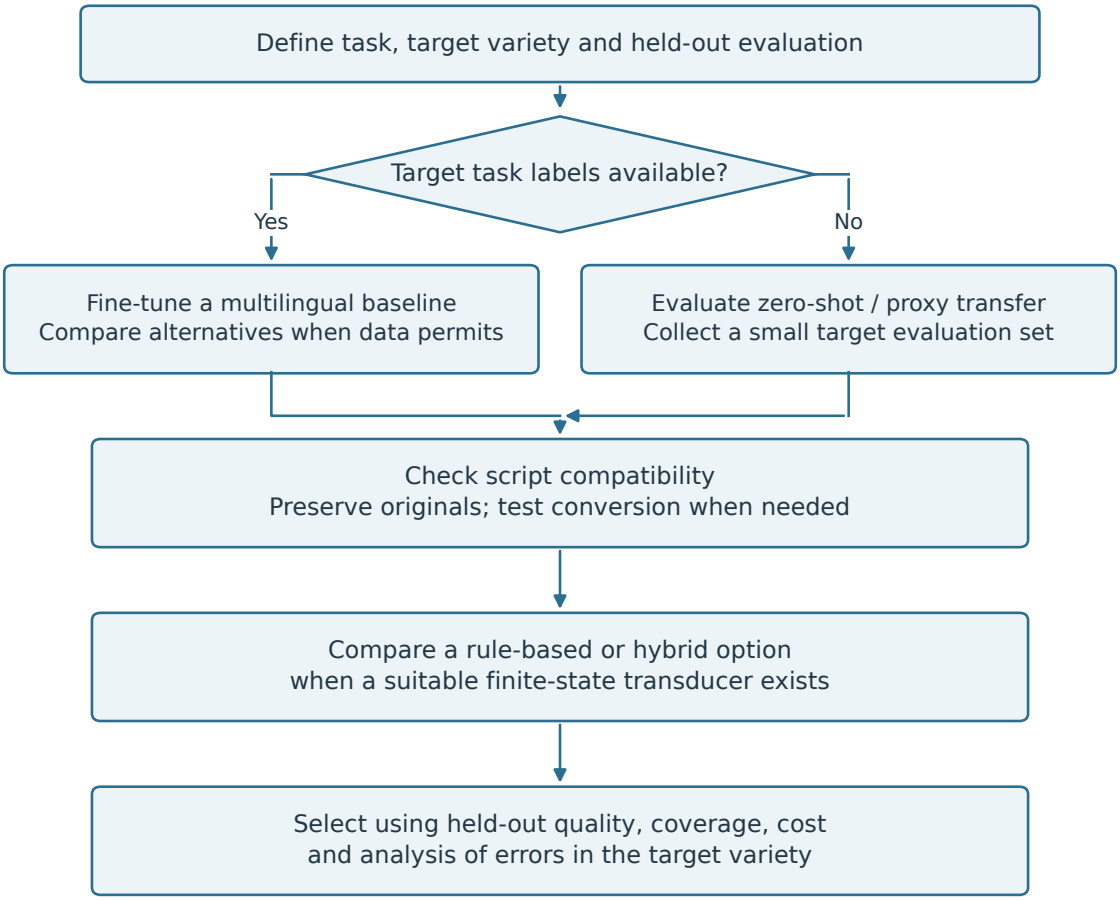


Figure 16.1: An evaluation workflow for choosing a cross-lingual transfer strategy. Target task labels determine the initial comparison. Script conversion and an available finite-state transducer are options to test; final selection depends on held-out target performance, coverage, cost and error analysis.

16.6 Code-Switched and Mixed Text

Distinguish an established borrowing from an active switch between languages, and state the annotation convention for ambiguous cases. A shared script cannot identify language boundaries. Preserve mixed-language segments and evaluate token-level language labels, task labels and segment boundaries separately. Turkish–German SAGT and Turkish–English BUTR resources must be named according to the exact released treebank; consult the official inventory in Chapter 8 rather than treating every Turkish treebank as monolingual.

Inserting words synthetically perturbs the input. Natural code-switching may look quite different. Have proficient speakers check naturalness and preservation of the task label, and report real mixed-language evaluation separately.

16.7 Hands-On: Comparing Native Text and Script Conversion

Question. Does consistent script conversion change Kazakh-to-Kyrgyz NER?

Inputs. Source training and independent target annotations, converter metadata and a declared entity schema.

Compute. The Exercise 1 CRF runs on CPU; the full protocol also needs original character offsets.

Output. Span scores by script condition and a mapping-failure report.

Interpretation. Compare matched conditions and keep token-level results distinct from original-offset validation.

Use a source-trained model and independently annotated target sentences with immutable IDs and original character spans. Keep source training data and target development/test groups disjoint. Harmonise entity schemas **before** scoring: map PERSON/PER to PER, ORGANIZATION/ORG to ORG, and GPE/LOCATION/LOC to LOC; excluded types become O under this explicitly reduced task.

The tested helpers in transfer.py reconstruct BIO spans from original offsets, reject orphan I tags, and map converted spans through unique stable character boundaries. A span cutting a digraph or ambiguous deleted unit is rejected and counted, not silently moved. The conservative prefix-probing converter is intended for short diagnostics and is quadratic in text length.

```
from transfer import conversion_map, restore_span  # code/ch16-transfer on PYTHONPATH
from turkicnlp.scripts import Script
from turkicnlp.scripts.transliterator import Transliterator
converter = Transliterator("kir", Script.CYRILLIC, Script.COMMON_TURKIC)
original = "Бишкекте жыйын өттү."
converted, boundaries = conversion_map(original, converter.transliterate)
# For each model-predicted (start, end, type) span in converted text:
# original_span = restore_span(predicted_span, boundaries)
```

Log the checkpoint, converter and processor trace. If the pipeline automatically bridges the input to a different model script, the nominal native/CTS labels do not describe the actual model inputs; inspect those inputs before comparing results. Score strict entity spans on the same original gold annotations, with mapping failures included in the coverage report.

To investigate genealogy, add related and unrelated source languages under comparable training sizes and tuning budgets. A descriptive regression across a few dependent language pairs cannot isolate a causal script effect. Use paired item-level comparisons, multiple seeds and an independently collected set in addition to potentially exposed parallel benchmarks.

Exercise 1 compares token-aligned versions of Kazakh→Kyrgyz NER. A CRF with the Chapter 9 features is trained on 20,000 KazNERD training sentences and evaluated on 3,867 KyrgyzNER test sentences. The reduced schema also maps KyrgyzNER's INSTITUTION and BUSINESS types to ORG, as in Chapter 9. Native Cyrillic gives span F1 31.9; converting both sides to the toolkit's Common Turkic Script (CTS) gives 34.6. Training on native text and testing on CTS gives 6.9. This single-sample comparison describes one feature set; uncertainty across training samples remains unmeasured (Section 15.1).

Word-by-word conversion preserves token indices, so the exercise scores token-level entity boundaries. It does not restore predictions to the source's original character offsets. Its span parser ignores orphan `I` tags.

A separate diagnostic uses the stricter `bio_spans` helper, which rejects one sentence. It restores 3,090 gold spans and fails on one. However, it reconstructs sentences with spaces, so those offsets belong to reconstructed text. The full Hands-On requires original offsets and an explicit policy for malformed BIO sequences.

Status: completed. Exercise 1 records the token-aligned comparison: strict entity-span F1 is 31.9 native/native and 34.6 CTS/CTS. The original-character-offset protocol remains a procedure; its mapping diagnostic is described above.

What to Take Forward

Transfer reuses supervision, representations or tools across languages, and each method changes a different part of the system. The Kazakh–Kyrgyz NER comparison shows how much the input convention matters: matching scripts on both sides gave span F1 31.9 (native) and 34.6 (CTS), while a native-trained model tested on converted text fell to 6.9. Choose candidate sources from linguistic and resource evidence, keep the input conditions explicit, and decide on target-language scores with a budget for annotation and review. Chapter 17 asks what to build when target data are scarce and the community decides.

Further Reading

Kardeş-NLU (Senel et al. 2024) studies sentence-level transfer to Turkic languages with Turkish as a high-resource relative. Hakimov (2026) measures transfer of morphology and parsing through proxy languages, and the effect of a common script on tokenisation and lexical overlap. For choosing transfer languages in general, see Lin et al. (2019).

For multilingual training methodology, the TurkicASR paper (Mussakhojayeva et al. 2023) provides a case study of Turkic multilingual pooling, with experiments comparing combinations of training languages. For text, Hakimov (2026) evaluates multilingual Turkic morphology and parsing with proxy languages and temperature sampling, and Muller et al. (2021) studies transliteration as a way to help multilingual models with unseen scripts.

Conneau et al. (2020) introduces XLM-R and reports multilingual transfer experiments. For translation, NLLB Team et al. (2024) discusses low-resource training, back-translation and filtering. The TIL corpus (Mirzakhalov, Babu, Ataman, et al. 2021) supplies parallel data; a separate multiway study reports multilingual experiments (Mirzakhalov, Babu, Kunafin, et al. 2021).

Forcada et al. (2011) and Khanna et al. (2021) describe Apertium. The Kazakh and Kyrgyz FST papers (J. Washington, Salimzyanov, and Tyers 2014; J. Washington, Ipasov, and Tyers 2012) give examples of rule-based morphology that can be extended or combined with other methods.

Exercises

1. Compare native/native, converted/converted and native/converted conditions using declared converter directions. Record failed mappings and actual model input scripts.
2. Compare related and unrelated source training at matched data and compute budgets. Keep target test data out of proxy and hyperparameter selection.
3. Test temperature sampling on an imbalanced multilingual training set and report effective exposure and per-language development scores.
4. Compare unfiltered, quality-filtered and randomly reduced synthetic parallel data at the same retained size, scoring each retained set against real target references and checking what the filter selects for.
5. Evaluate real code-switched text separately from synthetic perturbations, with borrowing conventions and label-preservation checks.

17 Endangered Languages, Digital Preservation, and Ethics

Learning Objectives

After reading this chapter, you will be able to:

- Identify task-specific resource gaps and distinguish them from language vitality and community priorities
- Compare digitisation, annotation, text creation and speech collection for a community-defined purpose
- Plan a useful first tool and estimate effort from a pilot
- Apply the CARE principles to evaluate the ethical dimensions of an endangered-language NLP project
- Prepare alternative research and development priorities for discussion with community partners
- Build a dated resource inventory that distinguishes observations, missing information and failed queries

Turkic languages differ sharply in available resources, and coverage also varies by task. These inventories tell us little about grammatical complexity, language vitality or community priorities. The previous chapters explain evaluation and adaptation (Chapter 15; Chapter 16). Here, the questions are what to build, with whom and under whose control. Resource audits can inform those decisions, but usefulness depends on the people using a tool and evidence from its intended setting.

17.1 What a Resource Gap Means

Define “zero-resource” by task, release and search scope. A language might have an FST and parallel text while lacking an independently annotated NER test set. TIL includes Tuvan, Khakas, Karachay-Balkar and Shor (Mirzakhalov, Babu, Ataman, et al. 2021), so each has at least some computational resources. Record model input support, released weights and measured task quality separately.

Use the official UD 2.17 inventory in Chapter 8, the corpus manifests in Section 4.1, the toolkit registry and the verified model entries in Appendix B. Record a missing search result as “not found in this audit”, an unchecked field as “not checked”, and a failed API call as an error. Do not convert all three to zero.

Endangerment concerns intergenerational transmission and community use, not the number of Hub models. Use dated evidence about transmission and language use when discussing endangerment. Appendix C explains the requirements for population estimates. Community documentation may be valuable even where it produces no machine-learning benchmark.

17.2 Digital Preservation Strategies

17.2.1 Digitising Print Materials

Preserve page images, stable page/document IDs and provenance alongside OCR. Separate diplomatic transcription (what the source writes) from editorial correction, transliteration and modernisation. Store uncertain readings explicitly and make transformations reversible through the retained original.

Evaluate OCR on document-disjoint pages manually transcribed under a stated policy. Report character and word error rates with the same normalisation, plus error categories for special letters, segmentation and layout. A CER/WER gap alone cannot prove that morphology caused the errors. Train and evaluate layout extraction separately where columns, footnotes or marginalia alter reading order.

17.2.2 Historical and Heritage Text

Ottoman, Chagatai, Old Turkic runiform and Old Uyghur materials differ in script, period, genre and editorial practice. Old Turkic extends beyond the Orkhon inscriptions. Check access terms for each collection. One released annotation resource is the **Old Turkish–Clausal** UD treebank (Chapter 8).

A historical corpus should link every token or segment to its source, editorial reading and transformation history. Keep original spelling and normalised search fields separately. Morphological annotation needs period-specific guidelines and uncertainty labels; a modern-language analyser can propose readings but cannot certify them.

This chapter reports no OCR accuracy: a historical OCR case study needs rights-cleared page images, independent transcriptions and philological review.

17.2.3 Field Linguistics Tools

FLEx supports dictionaries and interlinear analysis; ELAN aligns annotation with audio and video. To use exports for training, map their tiers, speaker/session IDs, time spans and gloss conventions explicitly. Preserve alignment, follow the archive's access rules and exclude restricted fields from public releases. The archive, dictionary or teaching material may itself be the project's main outcome.

17.2.4 Community Text and Speech Collection

Wikipedia editing, local publishing and speech recording can create useful materials when participants choose those activities. Article counts do not measure clean running text or prove that articles are machine-generated. Measure quality and genre coverage rather than treating volume as the objective.

For speech, agree recording conditions, transcript conventions, speaker permissions and intended uses before collection. A release's total, validated and training hours are different quantities. Common Voice access and release details should be checked through the distribution source described in Chapter 11; keep a dated local manifest. Consent to archiving a voice does not automatically include voice cloning.

17.3 Building a Useful First System

17.3.1 Take Stock and Agree the Use Case

Begin with a community-defined task and the resource audit below. Check existing target-language modules, compatible multilingual models and documented evaluations. Select a small, independently reviewed sample for a pilot. Protect a final test set from all adaptation, including tokeniser training and synthetic-data generation.

17.3.2 Extend Existing Morphology Where It Helps

An FST can support many tasks, though neural tagging and ASR can also be developed without one. Nogai already has an Apertium module. Extend its lexicon and continuation classes with help from grammars and speakers; a Kazakh module can offer engineering patterns, but its linguistic rules require checking before reuse.

Keep regression tests for accepted and rejected forms, ambiguity and generation. Measure analysis accuracy against gold readings alongside coverage (Section 15.1). Record the transducer revision and dictionary rights.

17.3.3 Projection and Few-Shot Training

Annotation projection uses aligned source and target sentences. Preserve many-to-many alignments and handle unaligned words, conflicting heads, cycles and roots explicitly (Chapter 8). The output is silver annotation. Evaluate a held-out sample against gold labels on **those same target sentences**. Tag agreement measures the projected annotation; it does not measure word-alignment accuracy, since an incorrect link can connect two words with the same tag.

Combine suitable silver data with separately identified gold training data and tune their weights on development data. Keep document/source families disjoint. A small hand-corrected collection cannot simultaneously be the adaptation set and an independent

final test set. FST features and related-language initialisation are optional conditions to compare.

17.3.4 Speech and Writing Tools

Use the learning-curve protocol in Chapter 11 to evaluate speech adaptation on a fixed validation population. Success on news recordings does not carry over automatically to meetings, spontaneous conversation or accessibility captioning: each has its own speaker, acoustic and error-cost requirements.

For spelling assistance, distinguish an analyser-licensed form from a contextually correct form. Test valid rare words and names alongside misspellings, and measure harmful changes. Suggestions with an undo path may be preferable to automatic replacement, depending on the community's use case.

17.3.5 Budget and Maintenance

Use a timed pilot to estimate recruitment, permissions, annotation, adjudication, development, device testing and maintenance. Partners might choose an archive, keyboard, corpus, evaluation set, morphological tool or transcription assistant. Agree on deliverables and budget together, using staffing and cost evidence to set the schedule.

17.4 Community, Consent, and Data Sovereignty

17.4.1 Roles and Permissions

Separate individual participant consent, collective governance, source copyright and software/model licences. Record these separately for the people, data and software involved. Community representation must be agreed rather than assumed from one institution or consultant, and individuals retain their own choices about participation.

Before collection, record:

- who can approve new uses and resolve disputes;
- which recordings, transcripts, annotations and models may be public, restricted or withheld;
- whether commercial use, redistribution, model training or voice synthesis is permitted;
- where data is stored, who can access it, and how access is reviewed;
- how withdrawal changes future collection, releases and training, and what cannot be recalled from already distributed copies;
- who maintains the tools and how contributors receive payment, credit and usable outputs.

Check software, model and data licences separately, alongside participant permissions. Permission to release software does not by itself authorise release or reuse of recordings.

17.4.2 CARE in Context

The CARE principles (Collective benefit, Authority to control, Responsibility and Ethics) were developed in Indigenous data governance (Carroll et al. 2020). They offer questions for collaboration without assuming that all Turkic communities have the same identities, institutions or preferences. Partners should determine how the framework fits their setting.

Collective benefit requires an outcome people want and can use. Authority to control means an agreed process for making decisions before release and afterwards. Responsibility includes maintenance and training. Ethics calls for attention to changing risks and both individual and collective interests. When reviewing a paper, mark unreported consent as **not documented**; silence leaves the question unresolved.

17.4.3 Risks to Evaluate

Potential harms include unwanted standardisation of a variety, confident incorrect output, unequal speech recognition, and reuse of recordings beyond agreed purposes. Treat these as risks to investigate; do not label an illustrative scenario a documented case or infer hallucination rates from corpus size alone.

Script choice affects who can use the result. Preserve originals, identify conversion schemes and provide alternatives when partners request them. CTS is an experimental model input in this book; decisions about community writing belong to its users. Include diaspora and regional varieties in evaluation when they are part of the intended audience.

Language technology can also support monitoring. Assess likely users, collection sensitivity and release granularity with partners. Offline access can reduce network dependence and exposure of personal text, but it does not by itself resolve governance or misuse concerns. Avoid publishing sensitive location or speaker information merely to make a dataset easier to reproduce.

17.5 Open Problems and Research Priorities

17.5.1 Evaluation and Corpora

The official release inventory identifies specific treebank gaps; local silver or experimental files must not be counted as official UD releases. Independent target gold data makes transfer measurable even when training relies on a multilingual model. Parallel annotation provides one comparison domain, while natural target text is needed for broader claims.

TUMLU's published coverage leaves many languages outside that particular benchmark. Expansion can use community-authored questions, with provenance and independent review. Native and translated questions differ in content and construction; neither is automatically harder. Public games are also potentially exposed in training (Chapter 15).

17.5.2 Tokenisation and Model Adaptation

The fertility measurements in Section 7.1 suggest model-input experiments. Compare native text, conversion, vocabulary extension and morphology-aware tokenisation using downstream scores as well as token counts. Preserve original text and test loss from ambiguous mappings. Corpus quality and licensing remain separate requirements.

17.5.3 Community-Defined Roadmaps

For languages with an existing FST, extending coverage may be useful; for another community, searchable recordings or teaching materials may be the priority. For Shor or any other language, establish partners and their aims before assigning documentation, ASR or benchmarking as the “most impactful” intervention. Budget pilot work, compensation, access and maintenance, with explicit points where the plan can change.

SIGTURK, UD-Turkic, Apertium, TIL and toolkit maintainers provide possible coordination channels (Akhundjanova et al. 2025; Mirzakhalov, Babu, Ataman, et al. 2021; Khanna et al. 2021). Check which language/resource already exists before proposing an extension. A shared registry should record source, version, licence, task and evaluation status. A single maturity score would hide these distinctions.

17.6 Hands-On: Building a Turkic Language Resource Audit

Question. Which resources support a specified task, and what remains unknown?

Inputs. Named release inventories, repository queries and explicit language-tag aliases.

Compute. CPU and network access; no model training is required for the inventory.

Output. A dated inventory with returned IDs, provenance, unknown fields and query errors.

Interpretation. Use the inventory to choose a next investigation and verify resource quality separately.

The resource-audit script records the query, timestamp, result status, exact returned IDs and any network error. For model searches it uses the supported `filter` argument; dataset searches use `language`. Language-tag aliases are supplied explicitly and results are deduplicated by repository ID.

```
python code/ch17-resources/resources.py --iso3 kaz --hub-alias kk --hub-alias kaz \
  --output runs-kazakh-resource-audit.json
```

Hub tags are self-reported. Searches can miss resources or return adapters, copies and unusable releases, so inspect results before drawing conclusions about task coverage. A zero records the outcome of **this query**. Failed requests retain `value: null, status:`

`error`; unchecked UD, Wikipedia and FST-quality fields retain `not_checked` until a dated source verifies them.

For UD counts, use `tables/tbl-ud217-inventory/` and name the release. For Wikipedia, verify the actual edition code before using MediaWiki statistics; ISO 639-3 is not a reliable URL substitution. For FST quality, record the evaluation task and gold denominator instead of converting feature-coverage percentages into "high/medium/low" tiers. Link tested model observations to Table B.6.

Exercise 1 combines these helpers with the UD 2.17 inventory, Apertium repository checks, the TIL language-pair table, SIB-200 files and model language lists. The dated snapshot covers Kazakh, Kyrgyz, Nogai, Tuvan and Shor.

On 20 October 2026, the audit found an Apertium repository and a GlotLID label for Nogai, but no Nogai entry in the checked UD, TIL, SIB-200 or NLLB-200 sources. The pinned TIL table lists ten Tuvan pairs, the largest with 12,531 training sentences (Tuvan–Bashkir), and one Shor pair (Shor–Russian, 2,294 sentences). The Apertium lookup for Shor returned no repository. Hub queries failed and remain errors, not zeros. A repository entry and a classifier's language label prove neither working coverage nor quality.

A separate fresh-environment query on 20 October 2026 succeeded. The counts below are deduplicated repository IDs returned by the supplied aliases. They describe Hub metadata at that time; the earlier failed queries remain in the historical exercise output. The follow-up audit records the exact IDs and UTC timestamps.

Table 17.1: Follow-up Hub tag queries from an isolated environment; repository counts do not measure usable task coverage.

Language	Query aliases	Model repositories	Dataset repositories
Kazakh	`kk`, `kaz`	2,870	743
Kyrgyz	`ky`, `kir`	1,496	428
Nogai	`nog`	120	28
Tuvan	`tyv`	149	106
Shor	`cjs`	20	18

The remaining exercises connect the inventory to governance review, analyser extension, projected tags and a project plan. Use each output for its stated purpose: regression checks track analyser behaviour, gold tags assess projected labels, and the plan prepares decisions for discussion with community partners.

Status: completed. Exercise 1 records the dated five-language inventory above, including the failed Hub queries. A later audit must be stored separately.

What to Take Forward

A resource inventory supports a community-defined decision about what to build. The Nogai audit identifies an existing Apertium module, but the checked data inventories do not supply the annotated task data needed for supervised training. Extending the

analyser is one option to discuss with partners, alongside further resource discovery or data collection. Record observations, unknowns and failed queries separately, estimate effort from a pilot, and include benefit, authority, responsibility and ethics in the technical plan. The earlier chapters supply methods for the data handling, models and evaluation that partners choose.

Further Reading

Bird (2020) and Carroll et al. (2020) discuss community governance and research priorities. Johanson (2021) provides the linguistic reference, while Çöltekin, Doğruöz, and Çetinoğlu (2023) surveys computational resources for Turkish. Chapter 8, Chapter 11 and Chapter 16 give the evaluation and adaptation protocols needed to turn a resource inventory into evidence about a system.

Exercises

1. Audit five languages with explicit language-tag aliases and a dated UD release. Show unknown and error values distinctly. Explain why repository-count ratios cannot be converted into person-hours of development.
2. Review a published project against CARE. Separate documented decisions, missing reporting and your own recommendations; do not infer absent consent from silence in a paper.
3. Time a small extension to an existing FST. Include linguistic review, ambiguity tests and maintenance in the effort estimate, not only dictionary entry counts.
4. Project tags onto aligned target sentences and compare with independent gold on those exact sentences. Report unaligned cases and keep the evaluation sentences out of training.
5. Draft a community-led preservation project with alternative deliverables, budget, individual and collective permissions, withdrawal limits and maintenance. This is a desk exercise; contacting or recording a vulnerable community is not required.

References

Abilbekov, Adal, Saida Mussakhojayeva, Rustem Yeshpanov, and Huseyin Atakan Varol. 2024. “KazEmoTTS: A Dataset for Kazakh Emotional Text-to-Speech Synthesis.” In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 9626–32. Torino, Italia: ELRA; ICCL. https://aclanthology.org/2024.lrec-main.841/.

Adelani, David Ifeoluwa, Hannah Liu, Xiaoyu Shen, Nikita Vassilyev, Jesujoba O. Alabi, Yanke Mao, Haonan Gao, and En-Shiun Annie Lee. 2024. “SIB-200: A Simple, Inclusive, and Big Evaluation Dataset for Topic Classification in 200+ Languages and Dialects.” In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 226–45. St. Julian’s, Malta: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.eacl-long.14.

Agostini, Alessandro, Timur Usmanov, Ulugbek Khamdamov, Nilufar Abdurakhmonova, and Mukhammadsaid Mamasaidov. 2021. “UZWORDNET: A Lexical-Semantic Database for the Uzbek Language.” In *Proceedings of the 11th Global Wordnet Conference*, 8–19. University of South Africa (UNISA): Global Wordnet Association. https://doi.org/10.18653/v1/2021.gwc-1.2.

Akhundjanova, Arofat, Furkan Akkurt, Bermet Chontaeva, Soudabeh Eslami, and Çağrı Çöltekin. 2025. “Parallel Universal Dependencies Treebanks for Turkic Languages.” In *Proceedings of the Eighth Workshop on Universal Dependencies (UDW, SyntaxFest 2025)*, 129–36. Ljubljana, Slovenia: Association for Computational Linguistics. https://aclanthology.org/2025.udw-1.14/.

AKIpress. 2024. “Turkic Academy Publishes Declaration on Common Turkic Alphabet.” AKIpress News Agency, 17 September 2024. https://akipress.com/news:796275:Turkic_Academy_publishes_Declaration_on_common_Turkic_alphabet/.

Akın, Ahmet Afşin, and Mehmet Dündar Akın. 2007. “Zemberek, an Open Source NLP Framework for Turkic Languages.” Project white paper.

Aksan, Yeşim, Mustafa Aksan, Ahmet Koltuksuz, Taner Sezer, Ümit Mersinli, Umut Ufuk Demirhan, Hakan Yılmazer, et al. 2012. “Construction of the Turkish National Corpus (TNC).” In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC’12)*, 3223–27. Istanbul, Turkey: European Language Resources Association (ELRA). https://aclanthology.org/L12-1590/.

Alecakir, Huseyin, Necva Bölücü, and Burcu Can. 2022. “TurkishDelightNLP: A Neural Turkish NLP Toolkit.” In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: System Demonstrations*, 17–26. Hybrid: Seattle, Washington + Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.naacl-demo.3.

Alekseev, Anton, and Gulnara Kabaeva. 2024. “HJ-Ky-0.1: An Evaluation Dataset for Kyrgyz Word Embeddings.” arXiv preprint arXiv:2411.10724. https://arxiv.org/abs/2411.10724.

Ardila, Rosana, Megan Branson, Kelly Davis, Michael Kohler, Josh Meyer, Michael Henretty, Reuben Morais, Lindsay Saunders, Francis Tyers, and Gregor Weber. 2020. "Common Voice: A Massively-Multilingual Speech Corpus." In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 4218–22. Marseille, France: European Language Resources Association. https://aclanthology.org/2020.lrec-1.520/.

Artetxe, Mikel, and Holger Schwenk. 2019. "Margin-Based Parallel Corpus Mining with Multilingual Sentence Embeddings." In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3197–3203. Florence, Italy: Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-1309.

Aslan, Ozkan, Serkan Gunal, and Bekir Taner Dincer. 2018. "On Constituent Chunking for Turkish." *Information Processing & Management* 54 (6): 1262–76. https://doi.org/10.1016/j.ipm.2018.05.004.

Azmat, Anwar, Li Xiao, Yang Yating, Dong Rui, and Osman Turghun. 2020. "Constructing Uyghur Name Entity Recognition System Using Neural Machine Translation Tag Projection." In *Proceedings of the 19th Chinese National Conference on Computational Linguistics*, 1006–16. Haikou, China: Chinese Information Processing Society of China. https://aclanthology.org/2020.ccl-1.93/.

Bakay, Özge, Özlem Ergelen, Elif Sarmış, Selin Yıldırım, Bilge Nas Arıcan, Atilla Kocabalcıoğlu, Merve Özçelik, et al. 2021. "Turkish WordNet KeNet." In *Proceedings of the 11th Global Wordnet Conference*, 166–74. University of South Africa (UNISA): Global Wordnet Association. https://doi.org/10.18653/v1/2021.gwc-1.19.

Bandarkar, Lucas, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Donald Husa, Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabsa. 2024. "The Belebele Benchmark: A Parallel Reading Comprehension Dataset in 122 Language Variants." In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 749–75. Bangkok, Thailand: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.acl-long.44.

Baysan, Mehmet Selman, and Tunga Gungor. 2025. "TR-MTEB: A Comprehensive Benchmark and Embedding Model Suite for Turkish Sentence Representations." In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 8867–87. Suzhou, China: Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-emnlp.471.

Beken Fikri, Figen, Kemal Oflazer, and Berrin Yanıkoğlu. 2021. "Turkish Dataset for Semantic Textual Similarity [Anlamsal Metin Benzerliği için Türkçe Veri Kümesi]." In *29th Signal Processing and Communications Applications Conference (SIU)*, 1–4. IEEE. https://doi.org/10.1109/SIU53274.2021.9477982.

Bird, Steven. 2020. "Decolonising Speech and Language Technology." In *Proceedings of the 28th International Conference on Computational Linguistics*, 3504–19. Barcelona, Spain (Online): International Committee on Computational Linguistics. https://doi.org/10.18653/v1/2020.coling-main.313.

Budur, Emrah, Rıza Özçelik, Tunga Gungor, and Christopher Potts. 2020. "Data and Representation for Turkish Natural Language Inference." In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 8253–67. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.662.

Burchell, Laurie, Alexandra Birch, Nikolay Bogoychev, and Kenneth Heafield. 2023. "An Open Dataset and Model for Language Identification." In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2:*

Short Papers), 865–79. Toronto, Canada: Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.acl-short.75.

Burchell, Laurie, Ona de Gibert, Nikolay Arefyev, Mikko Aulamo, Marta Bañón, Pinzhen Chen, Mariia Fedorova, et al. 2025. "An Expanded Massive Multilingual Dataset for High-Performance Language Technologies (HPLT)." In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 17452–85. Vienna, Austria: Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.854.

Can, Fazli, Seyit Kocberber, Erman Balcik, Cihan Kaynak, H. Cagdas Ocalan, and Onur M. Vursavas. 2008. "Information Retrieval on Turkish Texts." *Journal of the American Society for Information Science and Technology* 59 (3): 407–21. https://doi.org/10.1002/asi.20750.

Carroll, Stephanie Russo, Ibrahim Garba, Oscar L. Figueroa-Rodríguez, Jarita Holbrook, Raymond Lovett, Simeon Materechera, Mark Parsons, et al. 2020. "The CARE Principles for Indigenous Data Governance." *Data Science Journal* 19: 43. https://doi.org/10.5334/dsj-2020-043.

Chalamalasetti, Kranti, Jana Götze, Sherzod Hakimov, Brielen Madureira, Philipp Sadler, and David Schlangen. 2023. "Clembench: Using Game Play to Evaluate Chat-Optimized Language Models as Conversational Agents." In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 11174–219. Singapore: Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.emnlp-main.689.

Chen, Jianlyu, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. "M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation." In *Findings of the Association for Computational Linguistics: ACL 2024*, 2318–35. Bangkok, Thailand: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.137.

Chen, Sanyuan, Chengyi Wang, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, et al. 2025. "Neural Codec Language Models Are Zero-Shot Text to Speech Synthesizers." *IEEE Transactions on Audio, Speech and Language Processing* 33: 705–18. https://doi.org/10.1109/TASLPRO.2025.3530270.

Çöltekin, Çağrı. 2010. "A Freely Available Morphological Analyzer for Turkish." In *Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC'10)*. Valletta, Malta: European Language Resources Association (ELRA). https://aclanthology.org/L10-1068/.

Çöltekin, Çağrı, A. Seza Doğruöz, and Özlem Çetinoğlu. 2023. "Resources for Turkish Natural Language Processing: A Critical Survey." *Language Resources and Evaluation* 57 (1): 449–88. https://doi.org/10.1007/s10579-022-09605-4.

Conneau, Alexis, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. "Unsupervised Cross-Lingual Representation Learning at Scale." In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 8440–51. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.747.

Conneau, Alexis, Min Ma, Simran Khanuja, Yu Zhang, Vera Axelrod, Siddharth Dalmia, Jason Riesa, Clara Rivera, and Ankur Bapna. 2023. "FLEURS: Few-Shot Learning Evaluation of Universal Representations of Speech." In *2022 IEEE Spoken Language Technology Workshop (SLT)*, 798–805. IEEE. https://doi.org/10.1109/SLT54892.2023.10023141.

Devlet, Nadir, ed. 1992. *Milletlerarası Çağdaş Türk Alfabeleri Sempozyumu, 18–20*

Kasım 1991. Marmara Üniversitesi Yayın No. 509; Türkiyat Araştırmaları Enstitüsü Yayın No. 1. Istanbul: Marmara Üniversitesi Türkiyat Araştırmaları Enstitüsü. https://mupress-source.marmara.edu.tr/1366-file-pdf-milletlerarasi-cagdas-turk-alfabeleri-sempozyumu-18-20-kasim-1991_1783668182.pdf.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding." In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–86. Minneapolis, Minnesota: Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423.

Dozat, Timothy, and Christopher D. Manning. 2017. "Deep Biaffine Attention for Neural Dependency Parsing." In *5th International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=Hk95PK9le.

Eryiğit, Gülşen, Joakim Nivre, and Kemal Oflazer. 2008. "Dependency Parsing of Turkish." *Computational Linguistics* 34 (3): 357–89. https://doi.org/10.1162/coli.2008.07-017-R1-06-83.

Feng, Fangxiaoyu, Yinfei Yang, Daniel Cer, Naveen Arivazhagan, and Wei Wang. 2022. "Language-Agnostic BERT Sentence Embedding." In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 878–91. Dublin, Ireland: Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.acl-long.62.

Forcada, Mikel L., Mireia Ginestí-Rosell, Jacob Nordfalk, Jim O'Regan, Sergio Ortiz-Rojas, Juan Antonio Pérez-Ortiz, Felipe Sánchez-Martínez, Gema Ramírez-Sánchez, and Francis M. Tyers. 2011. "Apertium: A Free/Open-Source Platform for Rule-Based Machine Translation." *Machine Translation* 25 (2): 127–44. https://doi.org/10.1007/s10590-011-9090-0.

Gibert, Ona de, Graeme Nail, Nikolay Arefyev, Marta Bañón, Jelmer van der Linde, Shaoxiong Ji, Jaume Zaragoza-Bernabeu, et al. 2024. "A New Massive Multilingual Dataset for High-Performance Language Technologies." In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 1116–28. Torino, Italia: ELRA; ICCL. https://aclanthology.org/2024.lrec-main.100/.

Gökçür, Engin. 2016. "Primary Long Vowels in Turkmen Turkish [Türkmen Türkçesinde Birincil Uzun Ünlüler]." *The Journal of Academic Social Science Studies* 6 (47): 235–53. https://doi.org/10.9761/JASSS3460.

Graves, Alex. 2012. "Sequence Transduction with Recurrent Neural Networks." arXiv preprint arXiv:1211.3711. https://arxiv.org/abs/1211.3711.

Graves, Alex, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. 2006. "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks." In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 369–76. ACM. https://doi.org/10.1145/1143844.1143891.

Hakimov, Sherzod. 2026. "TurkicNLP: An NLP Toolkit for Turkic Languages." arXiv preprint arXiv:2602.19174. https://arxiv.org/abs/2602.19174.

Hakkani-Tür, Dilek Z., Kemal Oflazer, and Gökhan Tür. 2000. "Statistical Morphological Disambiguation for Agglutinative Languages." In *COLING 2000 Volume 1: The 18th International Conference on Computational Linguistics*. https://aclanthology.org/C00-1042/.

Hasan, Tahmid, Abhik Bhattacharjee, Md. Saiful Islam, Kazi Mubasshir, Yuan-Fang Li, Yong-Bin Kang, M. Sohel Rahman, and Rifat Shahriyar. 2021. "XL-Sum: Large-

Scale Multilingual Abstractive Summarization for 44 Languages." In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 4693–703. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.findings-acl.413.

Heffernan, Kevin, Onur Çelebi, and Holger Schwenk. 2022. "Bitext Mining Using Distilled Sentence Representations for Low-Resource Languages." In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2101–12. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.findings-emnlp.154.

Hoffman, Beryl. 1994. "Generating Context Appropriate Word Orders in Turkish." In *Proceedings of the Seventh International Workshop on Natural Language Generation*. https://aclanthology.org/W94-0314/.

HPLT Consortium. 2025. "HPLT Monolingual Datasets v3.0." https://hplt-project.org/datasets/v3.0.

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. "LoRA: Low-Rank Adaptation of Large Language Models." In *The Tenth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=nZeVKeeFYf9.

Imani, Ayyoob, Peiqin Lin, Amir Hossein Kargaran, Silvia Severini, Masoud Jalili Sabet, Nora Kassner, Chunlan Ma, et al. 2023. "Glot500: Scaling Multilingual Corpora and Language Models to 500 Languages." In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1082–1117. Toronto, Canada: Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.acl-long.61.

Institute of Smart Systems and Artificial Intelligence (ISSAI). 2024. "KAZ-LLM: Large Language Models for Kazakh." Model release on the Hugging Face Hub. https://huggingface.co/issai/LLama-3.1-KazLLM-1.0-8B.

Isbarov, Jafar, Arofat Akhundjanova, Mammad Hajili, Kavsar Huseynova, Dmitry Gaynullin, Anar Rzayev, Osman Tursun, et al. 2025. "TUMLU: A Unified and Native Language Understanding Benchmark for Turkic Languages." In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 22816–38. Vienna, Austria: Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.1112.

Johanson, Lars. 2021. *Turkic*. Cambridge Language Surveys. Cambridge University Press. https://doi.org/10.1017/9781139016704.

Johanson, Lars, and Éva Á. Csató, eds. 1998. *The Turkic Languages*. Routledge Language Family Descriptions. Routledge.

Jurafsky, Daniel, and James H. Martin. 2024. *Speech and Language Processing*. 3rd ed.

Kara, Neslihan, Deniz Baran Aslan, Büşra Marşan, Özge Bakay, Koray Ak, and Olcay Taner Yıldız. 2020. "TRopBank: Turkish PropBank V2.0." In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 2763–72. Marseille, France: European Language Resources Association. https://aclanthology.org/2020.lrec-1.336/.

Kargaran, Amir Hossein, Ayyoob Imani, François Yvon, and Hinrich Schuetze. 2023. "GlotLID: Language Identification for Low-Resource Languages." In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 6155–6218. Singapore: Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.findings-emnlp.410.

Khanna, Tanmai, Jonathan N. Washington, Francis M. Tyers, Sevilay Bayatlı, Daniel G. Swanson, Tommi A. Pirinen, Irene Tang, and Hèctor Alòs i Font. 2021. "Recent

Advances in Apertium, a Free/Open-Source Rule-Based Machine Translation Platform for Low-Resource Languages." *Machine Translation* 35 (4): 475–502. https://doi.org/10.1007/s10590-021-09260-6.

Kim, Jaehyeon, Jungil Kong, and Juhee Son. 2021. "Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech." In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 139:5530–40. Proceedings of Machine Learning Research. PMLR.

Kirillovich, Alexander, Marat Shaekhov, Alfiya Galieva, Olga Nevzorova, Dmitry Ilvovsky, and Natalia Loukachevitch. 2021. "TatWordNet: A Linguistic Linked Open Data-Integrated WordNet Resource for Tatar." In *3rd Conference on Language, Data and Knowledge (LDK 2021)*, 93:16:1–12. Open Access Series in Informatics (OASIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik. https://doi.org/10.4230/OASIcs.LDK.2021.16.

Ko, Tom, Vijayaditya Peddinti, Daniel Povey, and Sanjeev Khudanpur. 2015. "Audio Augmentation for Speech Recognition." In *Proceedings of Interspeech 2015*, 3586–89. ISCA. https://doi.org/10.21437/Interspeech.2015-711.

Koskenniemi, Kimmo. 1983. "Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production." Publication No. 11, University of Helsinki, Department of General Linguistics.

Koto, Fajri, Rituraj Joshi, Nurdaulet Mukhituly, Yuxia Wang, Zhuohan Xie, Rahul Pal, Daniil Orel, et al. 2025. "Sherkala-Chat: Building a State-of-the-Art LLM for Kazakh in a Moderately Resourced Setting." In *Second Conference on Language Modeling (COLM)*. https://arxiv.org/abs/2503.01493.

Kreutzer, Julia, Isaac Caswell, Lisa Wang, Ahsan Wahab, Daan van Esch, Nasanbayar Ulzii-Orshikh, Allahsera Tapo, et al. 2022. "Quality at a Glance: An Audit of Web-Crawled Multilingual Datasets." *Transactions of the Association for Computational Linguistics* 10: 50–72. https://doi.org/10.1162/tacl_a_00447.

Kudo, Taku. 2018. "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates." In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 66–75. Melbourne, Australia: Association for Computational Linguistics. https://doi.org/10.18653/v1/P18-1007.

Kudo, Taku, and John Richardson. 2018. "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing." In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 66–71. Brussels, Belgium: Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-2012.

Kudugunta, Sneha, Isaac Caswell, Biao Zhang, Xavier Garcia, Derrick Xin, Aditya Kusupati, Romi Stella, Ankur Bapna, and Orhan Firat. 2023. "MADLAD-400: A Multilingual and Document-Level Large Audited Dataset." In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*.

Kulmizev, Artur, Miryam de Lhoneux, Johannes Gontrum, Elena Fano, and Joakim Nivre. 2019. "Deep Contextualized Word Embeddings in Transition-Based and Graph-Based Dependency Parsing – a Tale of Two Parsers Revisited." In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2755–68. Hong Kong, China: Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1277.

Kuriyozov, Elmurod, Yerai Doval, and Carlos Gómez-Rodríguez. 2020. "Cross-Lingual

Word Embeddings for Turkic Languages." In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 4054–62. Marseille, France: European Language Resources Association. https://aclanthology.org/2020.lrec-1.499/.

Kuriyozov, Elmurod, David Vilares, and Carlos Gómez-Rodríguez. 2024. "BERTbek: A Pretrained Language Model for Uzbek." In *Proceedings of the 3rd Annual Meeting of the Special Interest Group on Under-Resourced Languages @ LREC-COLING 2024*, 33–44. Torino, Italia: ELRA; ICCL. https://aclanthology.org/2024.sigul-1.5/.

Lample, Guillaume, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. 2016. "Neural Architectures for Named Entity Recognition." In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 260–70. San Diego, California: Association for Computational Linguistics. https://doi.org/10.18653/v1/N16-1030.

Lin, Yu-Hsiang, Chian-Yu Chen, Jean Lee, Zirui Li, Yuyan Zhang, Mengzhou Xia, Shruti Rijhwani, et al. 2019. "Choosing Transfer Languages for Cross-Lingual Learning." In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3125–35. Florence, Italy: Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-1301.

Lindén, Krister, Erik Axelson, Sam Hardwick, Tommi A. Pirinen, and Miikka Silfverberg. 2011. "HFST—Framework for Compiling and Applying Morphologies." In *Systems and Frameworks for Computational Morphology (SFCM 2011)*, 100:67–85. Communications in Computer and Information Science. Springer. https://doi.org/10.1007/978-3-642-23138-4_5.

LocalDoc. n.d. "azerbaijani-ner-dataset." Hugging Face dataset `LocalDoc/azerbaijani-ner-dataset`, CC BY-NC 4.0. https://huggingface.co/datasets/LocalDoc/azerbaijani-ner-dataset.

Mamasaidov, Mukhammadsaid, and Abror Shopulatov. 2024. "Open Language Data Initiative: Advancing Low-Resource Machine Translation for Karakalpak." In *Proceedings of the Ninth Conference on Machine Translation*, 606–13. Miami, Florida, USA: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.wmt-1.48.

Marchisio, Kelly, Wei-Yin Ko, Alexandre Berard, Théo Dehaze, and Sebastian Ruder. 2024. "Understanding and Mitigating Language Confusion in LLMs." In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 6653–77. Miami, Florida, USA: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-main.380.

Marneffe, Marie-Catherine de, Christopher D. Manning, Joakim Nivre, and Daniel Zeman. 2021. "Universal Dependencies." *Computational Linguistics* 47 (2): 255–308. https://doi.org/10.1162/coli_a_00402.

McDonald, Ryan, and Joakim Nivre. 2007. "Characterizing the Errors of Data-Driven Dependency Parsing Models." In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 122–31. Prague, Czech Republic: Association for Computational Linguistics. https://aclanthology.org/D07-1013/.

Mirzakhalov, Jamshidbek, Anoop Babu, Duygu Ataman, Sherzod Kariev, Francis Tyers, Otabek Abduraufov, Mammad Hajili, et al. 2021. "A Large-Scale Study of Machine Translation in Turkic Languages." In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 5876–90. Online; Punta Cana, Dominican Republic: Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.emnlp-main.475.

Mirzakhalov, Jamshidbek, Anoop Babu, Aigiz Kunafin, Ahsan Wahab, Bekhzodbek Moydin-

boyev, Sardana Ivanova, Mokhiyakhon Uzokova, et al. 2021. “Evaluating Multiway Multilingual NMT in the Turkic Languages.” In *Proceedings of the Sixth Conference on Machine Translation*, 518–30. Online: Association for Computational Linguistics. https://aclanthology.org/2021.wmt-1.60/.

Muller, Benjamin, Antonios Anastasopoulos, Benoît Sagot, and Djamé Seddah. 2021. “When Being Unseen from mBERT Is Just the Beginning: Handling New Languages with Multilingual Language Models.” In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 448–62. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.naacl-main.38.

Mussakhojayeva, Saida, Kaisar Dauletbek, Rustem Yeshpanov, and Huseyin Atakan Varol. 2023. “Multilingual Speech Recognition for Turkic Languages.” *Information* 14 (2): 74. https://doi.org/10.3390/info14020074.

Mussakhojayeva, Saida, Rinat Gilmullin, Bulat Khakimov, Mansur Galimov, Daniil Orel, Adal Adilbekov, and Huseyin Atakan Varol. 2024. “Noise-Robust Multilingual Speech Recognition and the Tatar Speech Corpus.” In *2024 International Conference on Artificial Intelligence in Information and Communication (ICAIIC)*, 732–37. IEEE. https://doi.org/10.1109/ICAIIC60209.2024.10463419.

Mussakhojayeva, Saida, Yerbolat Khassanov, and Huseyin Atakan Varol. 2022a. “KSC2: An Industrial-Scale Open-Source Kazakh Speech Corpus.” In *Proceedings of Interspeech 2022*, 1367–71. ISCA. https://doi.org/10.21437/Interspeech.2022-421.

———. 2022b. “KazakhTTS2: Extending the Open-Source Kazakh TTS Corpus with More Data, Speakers, and Topics.” In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 5404–11. Marseille, France: European Language Resources Association. https://aclanthology.org/2022.lrec-1.578/.

Nguyen, Thuat, Chien Van Nguyen, Viet Dac Lai, Hieu Man, Nghia Trung Ngo, Franck Dernoncourt, Ryan A. Rossi, and Thien Huu Nguyen. 2024. “CulturaX: A Cleaned, Enormous, and Multilingual Dataset for Large Language Models in 167 Languages.” In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 4226–37. Torino, Italia: ELRA; ICCL. https://aclanthology.org/2024.lrec-main.377/.

Nivre, Joakim, Marie-Catherine de Marneffe, Filip Ginter, Yoav Goldberg, Jan Hajič, Christopher D. Manning, Ryan McDonald, et al. 2016. “Universal Dependencies V1: A Multilingual Treebank Collection.” In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, 1659–66. Portorož, Slovenia: European Language Resources Association (ELRA). https://aclanthology.org/L16-1262/.

NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, et al. 2024. “Scaling Neural Machine Translation to 200 Languages.” *Nature* 630 (8018): 841–46. https://doi.org/10.1038/s41586-024-07335-x.

Oepen, Stephan, Nikolay Arefyev, Mikko Aulamo, Marta Bañón, Maja Buljan, Laurie V. Burchell, Lucas Georges Gabriel Charpentier, et al. 2026. “HPLT 3.0: Very Large-Scale Multilingual Resources for LLMs and MT. Mono- and Bi-Lingual Data, Multilingual Evaluation, and Pre-Trained Models.” In *Proceedings of the Fifteenth Language Resources and Evaluation Conference*, 1409–34. Palma de Mallorca, Spain: ELRA Language Resource Association. https://doi.org/10.63317/25xbdofco9od.

Oflazer, Kemal. 1994. “Two-Level Description of Turkish Morphology.” *Literary and Linguistic Computing* 9 (2): 137–48. https://doi.org/10.1093/llc/9.2.137.

Oflazer, Kemal, and Murat Saraçlar, eds. 2018. *Turkish Natural Language Processing*.

Theory and Applications of Natural Language Processing. Cham: Springer. https://doi.org/10.1007/978-3-319-90165-7.

Omnilingual ASR Team, Gil Keren, Artyom Kozhevnikov, Yen Meng, Christophe Ropers, Matthew Setzler, Skyler Wang, et al. 2025. "Omnilingual ASR: Open-Source Multilingual Speech Recognition for 1600+ Languages." arXiv preprint arXiv:2511.09690. https://arxiv.org/abs/2511.09690.

Ozturel, Adnan, Tolga Kayadelen, and Isin Demirsahin. 2019. "A Syntactically Expressive Morphological Analyzer for Turkish." In *Proceedings of the 14th International Conference on Finite-State Methods and Natural Language Processing*, 65–75. Dresden, Germany: Association for Computational Linguistics. https://doi.org/10.18653/v1/W19-3110.

Pan, Xiaoman, Boliang Zhang, Jonathan May, Joel Nothman, Kevin Knight, and Heng Ji. 2017. "Cross-Lingual Name Tagging and Linking for 282 Languages." In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1946–58. Vancouver, Canada: Association for Computational Linguistics. https://doi.org/10.18653/v1/P17-1178.

Papineni, Kishore, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. "Bleu: A Method for Automatic Evaluation of Machine Translation." In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311–18. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics. https://doi.org/10.3115/1073083.1073135.

Park, Daniel S., William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. 2019. "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition." In *Proceedings of Interspeech 2019*, 2613–17. ISCA. https://doi.org/10.21437/Interspeech.2019-2680.

Penedo, Guilherme, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro von Werra, and Thomas Wolf. 2025. "FineWeb2: One Pipeline to Scale Them All – Adapting Pre-Training Data Processing to Every Language." arXiv preprint arXiv:2506.20920. https://arxiv.org/abs/2506.20920.

Petrov, Aleksandar, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. 2023. "Language Model Tokenizers Introduce Unfairness Between Languages." In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Ponti, Edoardo Maria, Goran Glavaš, Olga Majewska, Qianchu Liu, Ivan Vulić, and Anna Korhonen. 2020. "XCOPA: A Multilingual Dataset for Causal Commonsense Reasoning." In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2362–76. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.185.

Popović, Maja. 2015. "ChrF: Character n-Gram F-Score for Automatic MT Evaluation." In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, 392–95. Lisbon, Portugal: Association for Computational Linguistics. https://doi.org/10.18653/v1/W15-3049.

———. 2017. "ChrF++: Words Helping Character n-Grams." In *Proceedings of the Second Conference on Machine Translation*, 612–18. Copenhagen, Denmark: Association for Computational Linguistics. https://doi.org/10.18653/v1/W17-4770.

Post, Matt. 2018. "A Call for Clarity in Reporting BLEU Scores." In *Proceedings of the Third Conference on Machine Translation: Research Papers*, 186–91. Brussels, Belgium: Association for Computational Linguistics. https://doi.org/10.18653/v1/W18-6319.

Pratap, Vineel, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Sayani Kundu, Ali Elkahky, et al. 2024. “Scaling Speech Technology to 1,000+ Languages.” *Journal of Machine Learning Research* 25 (97): 1–52. https://jmlr.org/papers/v25/23-1318.html.

Qi, Peng, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. 2020. “Stanza: A Python Natural Language Processing Toolkit for Many Human Languages.” In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 101–8. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-demos.14.

Qwen Team, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. 2024. “Qwen2.5 Technical Report.” arXiv preprint arXiv:2412.15115. https://arxiv.org/abs/2412.15115.

Radford, Alec, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. “Robust Speech Recognition via Large-Scale Weak Supervision.” In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 202:28492–518. Proceedings of Machine Learning Research. PMLR.

Rae, Jack W., Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, et al. 2021. “Scaling Language Models: Methods, Analysis & Insights from Training Gopher.” https://arxiv.org/abs/2112.11446.

Rei, Ricardo, Craig Stewart, Ana C Farinha, and Alon Lavie. 2020. “COMET: A Neural Framework for MT Evaluation.” In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2685–2702. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.213.

Rust, Phillip, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. 2021. “How Good Is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models.” In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 3118–35. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.acl-long.243.

Safaya, Ali, Emirhan Kurtuluş, Arda Goktogan, and Deniz Yuret. 2022. “Mukayese: Turkish NLP Strikes Back.” In *Findings of the Association for Computational Linguistics: ACL 2022*, 846–63. Dublin, Ireland: Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.findings-acl.69.

Şahin, Gözde Gül, and Eşref Adalı. 2018. “Annotation of Semantic Roles for the Turkish Proposition Bank.” *Language Resources and Evaluation* 52 (3): 673–706. https://doi.org/10.1007/s10579-017-9390-y.

Saidov, Bobur, Zilolakhon Ruzmetova, Umida Bazarova, Gulnoza Narkabilova, Nasiba Azizova, Feruzakhon Rustamova, and Firuza Halimova. 2026. “Human-Reviewed Uzbek Legal Named Entity Recognition Dataset.” *F1000Research* 15: 909. https://doi.org/10.12688/f1000research.180408.2.

Schüller, Peter, Kübra Cıngıllı, Ferit Tunçer, Barış Gün Sürmeli, Ayşegül Pekel, Ayşe Hande Karatay, and Hacer Ezgi Karakaş. 2017. “Marmara Turkish Coreference Corpus and Coreference Resolution Baseline.” arXiv preprint arXiv:1706.01863. https://arxiv.org/abs/1706.01863.

Schweter, Stefan. 2020. “BERTurk – BERT Models for Turkish.” https://doi.org/10.5281/zenodo.3770924.

Scialom, Thomas, Paul-Alexis Dray, Sylvain Lamprier, Benjamin Piwowarski, and Jacopo Staiano. 2020. “MLSUM: The Multilingual Summarization Corpus.” In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 8051–67. Online: Association for Computational Linguistics. https://doi.org/10.18653

/v1/2020.emnlp-main.647.
Senel, Lütfi Kerem, Benedikt Ebing, Konul Baghirova, Hinrich Schuetze, and Goran Glavaš. 2024. "Kardeş-NLU: Transfer to Low-Resource Languages with the Help of a High-Resource Cousin – a Benchmark and Evaluation for Turkic Languages." In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1672–88. St. Julian's, Malta: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.eacl-long.100.
Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2016a. "Improving Neural Machine Translation Models with Monolingual Data." In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 86–96. Berlin, Germany: Association for Computational Linguistics. https://doi.org/10.18653/v1/P16-1009.
———. 2016b. "Neural Machine Translation of Rare Words with Subword Units." In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1715–25. Berlin, Germany: Association for Computational Linguistics. https://doi.org/10.18653/v1/P16-1162.
Shen, Jonathan, Ruoming Pang, Ron J. Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, et al. 2018. "Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions." In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 4779–83. IEEE. https://doi.org/10.1109/ICASSP.2018.8461368.
Steimel, Kenneth, Akbar Amat, Arienne Dwyer, and Sandra Kübler. 2020. "Fine-Grained Morpho-Syntactic Analysis for the Under-Resourced Language Chaghatay." In *Proceedings of the 19th International Workshop on Treebanks and Linguistic Theories*, 43–54. Düsseldorf, Germany: Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.tlt-1.4.
Tamchyna, Aleš, Marion Weller-Di Marco, and Alexander Fraser. 2017. "Modeling Target-Side Inflection in Neural Machine Translation." In *Proceedings of the Second Conference on Machine Translation*, 32–42. Copenhagen, Denmark: Association for Computational Linguistics. https://doi.org/10.18653/v1/W17-4704.
Tiedemann, Jörg. 2012. "Parallel Data, Tools and Interfaces in OPUS." In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*, 2214–18. Istanbul, Turkey: European Language Resources Association (ELRA). https://aclanthology.org/L12-1246/.
Togmanov, Mukhammed, Nurdaulet Mukhituly, Diana Turmakhan, Jonibek Mansurov, Maiya Goloburda, Akhmed Sakip, Zhuohan Xie, et al. 2025. "KazMMLU: Evaluating Language Models on Kazakh, Russian, and Regional Knowledge of Kazakhstan." In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 14403–16. Vienna, Austria: Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.701.
Tolegen, Gulmira, Alymzhan Toleu, and Rustam Mussabayev. 2020. "Voted-Perceptron Approach for Kazakh Morphological Disambiguation." In *Proceedings of the 1st Joint Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU) and Collaboration and Computing for Under-Resourced Languages (CCURL)*, 258–64. Marseille, France: European Language Resources association. https://aclanthology.org/2020.sltu-1.36/.
Toraman, Cagri, Eyup Halit Yilmaz, Furkan Şahinuç, and Oguzhan Ozcelik. 2023. "Impact of Tokenization on Language Models: An Analysis for Turkish." *ACM Transactions on Asian and Low-Resource Language Information Processing* 22 (4): 1–21. https:

//doi.org/10.1145/3578707.

Turatali, Timur, Anton Alekseev, Gulira Jumalieva, Gulnara Kabaeva, and Sergey Nikolenko. 2025. "Human-Annotated NER Dataset for the Kyrgyz Language." In *2025 10th International Conference on Computer Science and Engineering (UBMK)*, 1607–12. Istanbul, Turkey: IEEE. https://doi.org/10.1109/UBMK67458.2025.11206879.

Türk, Utku, Furkan Atmaca, Şaziye Betül Özateş, Gözde Berk, Seyyit Talha Bedir, Abdullatif Köksal, Balkız Öztürk Başaran, Tunga Güngör, and Arzucan Özgür. 2022. "Resources for Turkish Dependency Parsing: Introducing the BOUN Treebank and the BoAT Annotation Tool." *Language Resources and Evaluation* 56 (1): 259–307. https://doi.org/10.1007/s10579-021-09558-0.

Turkic World Common Alphabet Commission. 2024. "Declaration on Common Turkic Alphabet." Published by the International Turkic Academy, 16 September 2024. https://turkicacademy.org/en/smi/declaration-common-turkic-alphabet.

Tyers, Francis M., and Jonathan N. Washington. 2015. "Towards a Free/Open-Source Universal-Dependency Treebank for Kazakh." In *Proceedings of the 3rd International Conference on Turkic Languages Processing (TurkLang 2015)*, 276–89. Kazan, Russia.

Uludoğan, Gökçe, Zeynep Balal, Furkan Akkurt, Meliksah Turker, Onur Gungor, and Susan Üsküdarlı. 2024. "TURNA: A Turkish Encoder-Decoder Language Model for Enhanced Understanding and Generation." In *Findings of the Association for Computational Linguistics: ACL 2024*, 10103–17. Bangkok, Thailand: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.600.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. "Attention Is All You Need." In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 5998–6008.

Wang, Liang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2024. "Multilingual E5 Text Embeddings: A Technical Report." arXiv preprint arXiv:2402.05672. https://arxiv.org/abs/2402.05672.

Washington, Jonathan North, Francis M. Tyers, and Oğuzhan Kuyrukçu. 2020. "Multi-Script Morphological Transducers and Transcribers for Seven Turkic Languages." *Proceedings of the Workshop on Turkic and Languages in Contact with Turkic* 5 (1): 173–85. https://doi.org/10.3765/ptu.v5i1.4783.

Washington, Jonathan, Mirlan Ipasov, and Francis Tyers. 2012. "A Finite-State Morphological Transducer for Kyrgyz." In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*, 934–40. Istanbul, Turkey: European Language Resources Association (ELRA). https://aclanthology.org/L12-1642/.

Washington, Jonathan, Ilnar Salimzyanov, and Francis Tyers. 2014. "Finite-State Morphological Transducers for Three Kypchak Languages." In *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14)*, 3378–85. Reykjavik, Iceland: European Language Resources Association (ELRA). https://aclanthology.org/L14-1143/.

Wolf, Thomas, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, et al. 2020. "Transformers: State-of-the-Art Natural Language Processing." In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-demos.6.

Xue, Linting, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. "MT5: A Massively Multilingual Pre-Trained Text-to-Text Transformer." In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human*

Language Technologies, 483–98. Online: Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.naacl-main.41.

Yeniterzi, Reyyan. 2011. "Exploiting Morphology in Turkish Named Entity Recognition System." In *Proceedings of the ACL 2011 Student Session*, 105–10. Portland, OR, USA: Association for Computational Linguistics. https://aclanthology.org/P11-3019/.

Yeshpanov, Rustem, Pavel Efimov, Leonid Boytsov, Ardak Shalkarbayuli, and Pavel Braslavski. 2024. "KazQAD: Kazakh Open-Domain Question Answering Dataset." In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 9645–56. Torino, Italia: ELRA; ICCL. https://aclanthology.org/2024.lrec-main.843/.

Yeshpanov, Rustem, Yerbolat Khassanov, and Huseyin Atakan Varol. 2022. "KazNERD: Kazakh Named Entity Recognition Dataset." In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 417–26. Marseille, France: European Language Resources Association. https://aclanthology.org/2022.lrec-1.44/.

Yeshpanov, Rustem, Saida Mussakhojayeva, and Yerbolat Khassanov. 2023. "Multilingual Text-to-Speech Synthesis for Turkic Languages Using Transliteration." In *Proceedings of Interspeech 2023*, 5521–25. ISCA. https://doi.org/10.21437/Interspeech.2023-249.

Yeshpanov, Rustem, Alina Polonskaya, and Huseyin Atakan Varol. 2024. "KazParC: Kazakh Parallel Corpus for Machine Translation." In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 9633–44. Torino, Italia: ELRA; ICCL. https://aclanthology.org/2024.lrec-main.842/.

Zeyrek, Deniz, Işın Demirşahin, and Ayışığı B. Sevdik Çallı. 2013. "Turkish Discourse Bank: Porting a Discourse Annotation Style to a Morphologically Rich Language." *Dialogue & Discourse* 4 (2): 174–84. https://doi.org/10.5087/dad.2013.208.

Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, et al. 2023. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*.

A Mathematical Foundations

> **What This Appendix Covers**
>
> This appendix reviews the linear algebra, probability and machine-learning concepts used in the book. It defines the notation needed for tokenisation, morphology, parsing, retrieval and language modelling. Use it as a reference or a refresher alongside a fuller treatment of each subject.

Four areas recur: vectors and similarity (embeddings and retrieval), probability and information theory (language models and evaluation), optimisation (fine-tuning) and the building blocks of neural architectures (attention and sequence models).

A.1 Linear Algebra

A.1.1 Vectors and Matrices

A **vector** $\mathbf{v} \in \mathbb{R}^d$ is an ordered sequence of d real numbers, written as a column:

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_d \end{pmatrix}$$

In NLP, **embeddings** represent tokens, words and sentences as vectors. Each vocabulary token has a fixed-dimensional vector: d is 384 for the MiniLM sentence encoder, 768 for XLM-R base and Glot500, and 1,024 for the NLLB-200 600M encoder used here. The **embedding matrix** $\mathbf{E} \in \mathbb{R}^{|V| \times d}$ stores one vector per row for vocabulary V. Looking up token i retrieves row i of $\mathbf{E}$.

The **dot product** of two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ is the scalar:

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^{d} u_i v_i$$

The dot product measures the component-wise alignment between two vectors. In attention mechanisms (Section A.4), dot products between query and key vectors determine how much attention is paid to each position.

A **matrix** $\mathbf{M} \in \mathbb{R}^{m \times n}$ has m rows and n columns. The **transpose** $\mathbf{M}^\top \in \mathbb{R}^{n \times m}$ swaps rows and columns. **Matrix multiplication** $\mathbf{AB}$ is defined when $\mathbf{A} \in \mathbb{R}^{m \times k}$ and $\mathbf{B} \in \mathbb{R}^{k \times n}$, producing $\mathbf{C} \in \mathbb{R}^{m \times n}$ where $C_{ij} = \sum_{l=1}^{k} A_{il} B_{lj}$.

A.1.2 Norms and Distances

The **L2 norm** (Euclidean norm) of a vector measures its length:

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{d} v_i^2}$$

A vector is **normalised** when its L2 norm equals 1. The **L1 norm** sums absolute values: $\|\mathbf{v}\|_1 = \sum_i |v_i|$.

Cosine similarity measures the angle between two vectors, ignoring their magnitudes:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \, \|\mathbf{v}\|_2}$$

For nonzero vectors, cosine similarity ranges from -1 to 1 and is invariant to their positive rescaling. Zero vectors need an explicit exclusion policy. On unit vectors, squared Euclidean distance is $2 - 2\,\text{sim}(\mathbf{u}, \mathbf{v})$, so the rankings agree. Embedding norms are model-dependent; they are not universally word-frequency measures. For the analogy "Istanbul is to Turkey as Almaty is to ___", the usual offset query is $\mathbf{v}_{\text{Turkey}} - \mathbf{v}_{\text{Istanbul}} + \mathbf{v}_{\text{Almaty}}$. Whether it retrieves Kazakhstan is an empirical property of the embedding.

A.2 Probability and Information Theory

A.2.1 Probability Basics

A **probability distribution** over a discrete set $\mathcal{X}$ assigns a non-negative value $P(x)$ to each outcome $x \in \mathcal{X}$ such that $\sum_{x \in \mathcal{X}} P(x) = 1$.

Conditional probability defines the probability of event A given that B has occurred:

$$P(A \mid B) = \frac{P(A, B)}{P(B)}$$

where $P(A, B)$ is the joint probability and $P(B) > 0$. **Bayes' theorem** inverts the conditioning when $P(A) > 0$:

$$P(B \mid A) = \frac{P(A \mid B)\, P(B)}{P(A)}$$

In NLP, Bayes' theorem underlies naive Bayes classifiers for text categorisation: the prior $P(B)$ is the class prior and $P(A \mid B)$ is the likelihood of the observed features given the class. This framework appears in text classification and in the HMM formulation for morphological disambiguation (Chapter 7).

The **chain rule of probability** decomposes joint probabilities into a product of conditionals:

$$P(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1})$$

Autoregressive language models use this identity to factor a token sequence's probability into next-token probabilities. The probability of a complete sentence also includes its end-of-sequence marker; otherwise the product describes a prefix.

A.2.2 Distributions

The **categorical distribution** is a generalisation of the Bernoulli distribution to k outcomes. A random variable X follows $\text{Cat}(\pi)$ if $P(X = k) = \pi_k$ where $\sum_k \pi_k = 1$. Autoregressive language model outputs are categorical: at each position, the model produces a probability distribution over the entire vocabulary, and the next token is drawn (or selected via argmax) from this distribution.

A.2.3 Information Theory

Entropy measures the average uncertainty of a distribution:

$$H(X) = -\sum_{x \in \mathcal{X}} P(x) \log P(x)$$

Here log is the natural logarithm, so entropy is in nats; use $\log_2$ for bits. Adopt $0 \log 0 = 0$. On a fixed finite support, entropy is maximised by the uniform distribution and is zero for a point mass. Lower predictive entropy means greater concentration, not better calibration: a confidently wrong model can have very low entropy.

Cross-entropy measures how well a model distribution Q approximates a true distribution P:

$$H(P, Q) = -\sum_{x} P(x) \log Q(x)$$

Cross-entropy is the standard training objective for neural language models and classifiers: minimising $H(P, Q)$ encourages Q to concentrate probability on the outcomes that P assigns high probability. When P is the one-hot encoding of the correct class (for classification) or the empirical distribution over the training corpus (for language modelling), cross-entropy reduces to the negative log-likelihood of the correct labels.

Perplexity converts cross-entropy to a more interpretable scale:

$$\mathrm{PPL} = \exp(H_{\text{nats}}(P, Q)) = 2^{H_{\text{bits}}(P,Q)}$$

A uniform four-outcome distribution has entropy $\ln 4$ nats or 2 bits, giving perplexity 4 under either consistent convention. Held-out language-model perplexity is exponentiated average negative log likelihood per declared prediction unit. Compare it only under compatible tokenisation, text and context policies. Cross-language differences cannot be attributed to morphology alone when these conditions differ.

KL divergence (Kullback-Leibler divergence) measures the information cost of using Q to approximate P:

$$D_{\mathrm{KL}}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

If $P(x) > 0$ but $Q(x) = 0$, the divergence is infinite; zero-P terms contribute zero. Use the same log base throughout the entropy identity. KL divergence is non-negative and equals zero only when $P = Q$. It is asymmetric: $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$ in general. The cross-entropy decomposes as $H(P, Q) = H(P) + D_{\mathrm{KL}}(P\|Q)$, showing that minimising cross-entropy is equivalent to minimising KL divergence when the true distribution P is fixed. KL divergence appears in variational inference and in preference-tuning objectives such as RLHF and DPO, which keep a tuned model close to a reference model; Chapter 12 places preference tuning in the adaptation workflow.

A.3 Optimisation

A.3.1 Gradient Descent

Neural network training is an optimisation problem: find the parameter vector θ that minimises a scalar loss function $\mathcal{L}(\theta)$ defined over the training data.

The **gradient** $\nabla_\theta \mathcal{L}$ is the vector of partial derivatives of the loss with respect to each parameter. It points in the direction of steepest ascent. **Gradient descent** takes a step in the opposite direction:

$$\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$$

where $\alpha > 0$ is the **learning rate**. If α is too large, updates overshoot minima; if too small, convergence is slow.

In practice, computing the gradient over the whole training set at every update is expensive. **Stochastic Gradient Descent (SGD)** estimates it from a randomly sampled **minibatch** of B examples. Larger batches generally reduce sampling noise and use more memory per update. Their throughput depends on the model and hardware. An **epoch** is one pass through the training set.

Adam (Adaptive Moment Estimation) maintains running estimates of the first and second moments of the gradient to adaptively scale the learning rate for each parameter:

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \\
\theta_t &= \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}
\end{aligned}
$$

where g_t is the gradient at step t and the vector operations are element-wise. Common defaults are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$; the small positive ε keeps the denominator nonzero. AdamW, with decoupled weight decay, is used in many of the book's transformer examples; the equations above define Adam. Fine-tuning recipes often pair it with a learning-rate warmup followed by linear or cosine decay; record the schedule actually used, as the exercises do.

A.3.2 Regularisation

A model can fit the training data while performing poorly on new examples. Regularisation aims to reduce this overfitting. Three common techniques follow.

L2 regularisation adds $\lambda \|\theta\|_2^2$ to the loss. In plain SGD it corresponds to a shrinkage update with a matching coefficient. **AdamW** applies weight decay separately from the adaptive gradient update; adding an L2 penalty to Adam is not generally equivalent.

Dropout in PyTorch uses inverted dropout: with independent $m_i \sim \text{Bernoulli}(1-p)$, training uses $y_i = m_i x_i / (1-p)$ for $0 \le p < 1$. Evaluation uses $y_i = x_i$ with no further scaling. Call `model.eval()` for inference.

Early stopping monitors a declared development metric with a chosen patience and minimum improvement. Restore the best development checkpoint and use the reserved test set only after selection. A single noisy increase need not trigger stopping, and validation loss need not be the task's selection metric.

A.4 Neural Network Basics

A.4.1 The Feedforward Layer

The building block of neural networks is the **feedforward (fully connected) layer**:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

where $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is the input, $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the weight matrix, $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ is the bias, and σ is an element-wise nonlinear **activation function**.

Stacking affine layers without nonlinear activations is equivalent to a single affine transformation. Nonlinear activations give deeper networks the ability to represent more complex functions.

A.4.2 The Attention Mechanism

The **scaled dot-product attention** mechanism (Jurafsky and Martin 2024, ch. 9) computes a weighted average of value vectors, where weights are determined by the compatibility between query and key vectors:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Here $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$ is the query matrix (one row per query position), $\mathbf{K} \in \mathbb{R}^{m \times d_k}$ is the key matrix, $\mathbf{V} \in \mathbb{R}^{m \times d_v}$ is the value matrix, and d_k is the dimension of the query/key vectors. The division by $\sqrt{d_k}$ prevents the dot products from growing too large in magnitude as d_k increases, which would push the softmax into regions of very small gradients.

Multi-head attention projects $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ into h separate lower-dimensional subspaces, computes attention independently in each head, and concatenates the results:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\,\mathbf{W}^O$$
$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

Multiple heads provide separate learned projections. Claims that particular heads track syntax, semantics or morphological agreement need a probe or intervention on the specific trained model; the architecture alone cannot show that they do.

A.4.3 Softmax and Cross-Entropy

The **softmax** function converts a vector of raw scores (logits) $\mathbf{z} \in \mathbb{R}^k$ to a probability distribution:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

Implement softmax and log-sum-exp stably by subtracting $\max_j z_j$ before exponentiation. The attention softmax is applied row-wise over permitted keys; padding and causal masks exclude prohibited positions. Softmax is used at the output layer of classifiers (morphological taggers, NER models, dependency parsers) and at every decoding step of a language model. The **cross-entropy loss** for a single example with correct class y and predicted distribution $\hat{\mathbf{p}} = \text{softmax}(\mathbf{z})$ is:

$$\mathcal{L} = -\log \hat{p}_y = -z_y + \log \sum_j e^{z_j}$$

Minimising this loss encourages the model to assign high probability to the correct class. For sequence labelling (morphological tagging, NER), sum or average the loss over labelled, unmasked positions and state which reduction is used. Padding or ignored positions must not enter the denominator.

A.4.4 Sequence Models

Recurrent Neural Networks (RNNs) process sequences by maintaining a hidden state $\mathbf{h}_t$ that is updated at each time step:

$$\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

RNNs can represent long dependencies, but learning them can be difficult. During backpropagation through time, repeated transformations can make gradients shrink (**vanishing gradients**) or grow (**exploding gradients**). The former weakens the training signal from distant positions.

LSTMs (Long Short-Term Memory) use a cell state and gates to help preserve information. In the equations below, σ is the logistic sigmoid and $[\mathbf{h}; \mathbf{x}]$ concatenates the two vectors. Three gates control information flow:

- **Forget gate** $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f)$: decides what to erase from the cell state
- **Input gate** $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i)$: decides what new information to write
- **Output gate** $\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o)$: decides what to expose as the hidden state

The candidate state is $\tilde{c}_t = \tanh(W_c[h_{t-1}; x_t] + b_c)$; the cell update is $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ and the hidden state is $h_t = o_t \odot \tanh(c_t)$. The additive path can mitigate vanishing gradients but does not guarantee perfect long-range memory. BiLSTM-CRF remains a useful sequence-labelling baseline to compare with transformers.

Transformer training can compute representations in parallel across positions subject to attention masks; autoregressive generation remains sequential. Full self-attention has $O(n^2 d)$ arithmetic for sequence length n and total representation width d. Explicit attention-score matrices use $O(hn^2)$ storage for h heads, in addition to activations; efficient kernels can avoid materialising that matrix without eliminating the quadratic arithmetic of full attention. High fertility increases n for a given text, motivating the context-budget checks in Chapter 12.

Further Reading

For a self-contained and pedagogically careful treatment of all the material in this appendix (probability for NLP, neural networks, backpropagation, the attention mechanism and language model evaluation), Chapters 5 to 10 of Jurafsky and Martin (2024) are the recommended starting point. The notation used throughout this book follows Jurafsky and Martin (2024) wherever possible.

Stanford CS229's online lecture notes and Goodfellow, Bengio and Courville's *Deep Learning* cover linear algebra and optimisation for machine learning. Cover and Thomas's *Elements of Information Theory* develops the information-theoretic foundations. For transformers, read the original architecture in Vaswani et al. (2017) alongside the explanation in Jurafsky and Martin (2024).

Worked Checks

1. Uniform four-way prediction: $H = \ln 4$ nats, $H = 2$ bits, perplexity 4.
2. A confidently wrong binary prediction $Q = (0.99, 0.01)$ for gold class 2 has low predictive entropy but loss $-\ln 0.01$; confidence and calibration are different things.
3. Two orthogonal unit vectors have cosine 0 and squared Euclidean distance 2.
4. For dropout rate 0.25, a retained activation of 3 becomes 4 during training and remains 3 at evaluation. Its expectation across masks is 3.

These arithmetic identities, stable-softmax behaviour and loss masking are checked in `code/common/check.py`.

B Computational Resources and Tools

> **What This Appendix Covers**
>
> This appendix catalogues resources used in the book: environments and libraries (Section B.1, Section B.2), datasets (Section B.3), pretrained models (Section B.4) and community infrastructure (Section B.5). Reproduction notes follow in Section B.6. Entries refer to the sources and revisions checked; newer and unlisted resources need their own verification.

The resources include rule-based tools from Apertium and HFST, corpora from OPUS and Common Voice, models from the Hugging Face Hub, and language-specific datasets from groups such as ISSAI and SIGTURK. This appendix gives starting points; the task chapters describe the relevant releases and limitations.

B.1 Environment Setup

B.1.1 Tested Profiles and Installation

Use separate environments for HFST/toolkit work, transformer experiments and Omnilingual ASR. The book's runs used Python 3.11 to 3.13, with packages varying by chapter. Each run records Python and package versions and downloaded-file SHA-256 hashes. Reuse those records when reproducing a result, and check other platform or package combinations before relying on them.

For a fresh environment, choose a supported Python/runtime and the platform-specific PyTorch installation from its official instructions, then install the pinned direct dependencies in `code/common/requirements-tested.txt`. That file records the checked package versions; it is not a complete cross-platform lock or evidence of a clean install on every OS. Save the resolved environment for a published experiment.

Install only the toolkit extras needed by the chosen example. HFST availability and native dependencies differ by platform; a Windows environment may need WSL for those tools. The Omnilingual wrapper's environment is separate from the Whisper recipe in Chapter 11. Do not install conflicting speech stacks into an existing working text environment.

Memory and runtime depend on the exact configuration, so measure them on the planned run; each exercise README records the runtime of the book's run. The exercises run on a CPU; the procedure-only fine-tuning recipes are faster on a GPU.

B.2 Key Libraries and Frameworks

B.2.1 Hugging Face Ecosystem

The Hugging Face ecosystem is the primary software infrastructure used throughout this book. Four libraries are central.

`transformers` provides a unified API for loading, fine-tuning, and running pretrained models. Models are identified by a Hub ID string (e.g., `dbmdz/bert-base-turkish-cased`); the appropriate model class and input conventions depend on whether the task uses an encoder, a causal language model or a sequence-to-sequence model. Pin the revision of every checkpoint, as the exercises do; a newly added task head is random until trained.

`datasets` loads corpora and benchmarks from the Hugging Face Hub or from local files, with streaming for large datasets. The book's exercises instead read most datasets from commit-pinned GitHub files and check their SHA-256 hashes (for KazNERD and KyrgyzNER, see Chapter 9), so that a rerun uses exactly the same release.

`peft` implements parameter-efficient fine-tuning methods including LoRA and QLoRA, which reduce the memory needed to adapt large models. Use the complete XLM-R sequence-classification recipe in Section 12.5. Target names are architecture-specific, and a new classifier head must be trainable and saved with its adapter. Copying `q_proj/v_proj` from a decoder into a BERT example is not a valid universal setup.

`evaluate` wraps standard metrics such as `seqeval` for BIO entity spans, `sacrebleu` and `chrf` for machine translation and `rouge` for summarisation. Feature and exact-bundle accuracy for morphology are computed by the book's own code (Chapter 15); report the metric signature with every score.

The Hugging Face Hub at huggingface.co hosts models, datasets and spaces relevant to Turkic NLP. Language tags (e.g. `kk` or `kaz` for Kazakh) are self-reported metadata: a tag search can miss resources and count copies, so record the query and date, as the resource audit in Section 17.6 does.

B.2.2 TurkicNLP Toolkit

The TurkicNLP toolkit (Hakimov 2026) provides one pipeline for 24 Turkic languages, with interchangeable rule-based, finite-state (Apertium), per-language neural (Stanza), and multilingual neural (Glot500) backends, plus script detection, transliteration, Common Turkic Script (CTS) conversion, language identification (GlotLID), morpheme segmentation, and NLLB-based embeddings and translation.

- **GitHub**: `github.com/turkic-nlp/turkicnlp`
- **Installation**: `pip install turkicnlp` (extras such as `turkicnlp[hfst]` add heavier backends)
- **Languages**: 24 (coverage per component differs; see the model catalogue)
- **Processors**: `tokenize`, `mwt`, `morph`, `morph_neural`, `pos`, `feats`, `lemma`, `depparse`, `ner`, `embeddings`, `translate`, `sentiment`, `spellcheck`, `sp_tokenize`, `asr`

- **Other entry points**: `LanguageDetection`, `SpeechRecognizer`, `MorphemeTokenizer`, `Transliterator`, `turkicnlp.llm.ask`, `list_processors`, `list_asr_languages`
- **LLM backends**: `morph`, `ner`, `translate`, `sentiment` and `spellcheck` can use any LLM (vLLM, Ollama, OpenAI, Claude, or a local Hugging Face model) for all 24 languages
- **Licences**: the library is Apache-2.0; Apertium transducers (GPL-3.0) and NLLB weights (CC-BY-NC-4.0) are downloaded separately

```
# pip install "turkicnlp[stanza]==0.5.0"   # list_processors works without it; loading
 ↪  Stanza models needs it
import turkicnlp

print(sorted(turkicnlp.list_processors("kaz")))   # what the catalog offers for Kazakh
turkicnlp.download("kaz", processors=["tokenize", "morph"])
kz = turkicnlp.Pipeline("kaz", processors=["tokenize", "morph"])
doc = kz("Үйлерімізден өте алыс кеткенбіз.")
for word in doc.words:
    print(word.text, word.lemma, word.upos, word.feats)
print(doc._processor_log)                          # which backend produced what
print(doc.to_conllu())
```

Three behaviours are worth knowing. **Field ownership**: when several processors provide the same field, the last one in the pipeline owns it, and the others' values are kept in `word._provenance`. **Script bridging**: input in a script without models (Kazakh Latin, Uzbek Cyrillic) is transliterated, processed, and mapped back to the original text. **Batching**: `nlp.batch(texts, batch_size=32)` hands whole batches to the neural and LLM backends.

B.2.3 Apertium

Apertium (Forcada et al. 2011; Khanna et al. 2021) is an open-source rule-based machine translation and morphological analysis platform. For Turkic NLP, its primary value is the collection of finite-state morphological analysers for 20+ Turkic languages, whose coverage and ambiguity should be evaluated against independent gold data.

- **Website**: apertium.org
- **GitHub**: `github.com/apertium`
- **Turkic language modules**: `github.com/apertium/apertium-tur` (Turkish), `github.com/apertium/apertium-kaz` (Kazakh), and equivalents for other languages

Section 10.2 gives a dated inventory of Apertium translation pairs and their development stages. Check the selected pair's release and supported direction before use; a repository listing does not guarantee support in both directions.

B.2.4 HFST (Helsinki Finite-State Toolkit)

HFST (Lindén et al. 2011) is a toolkit for compiling and running finite-state transducers. The Apertium Turkic analysers used in this book are compiled with HFST; Apertium also supports other finite-state toolchains.

- **Website**: hfst.github.io
- **Installation**: `pip install hfst` (Python bindings) or system packages

HFST supports the standard finite-state operations (composition, intersection, minimisation); Chapter 6 shows how to load and query the Apertium transducers with its Python bindings.

B.2.5 Stanza (Stanford NLP)

Stanza (Qi et al. 2020) provides neural NLP pipelines for tokenisation, POS tagging, morphological analysis, and dependency parsing. Its 1.11.0 model catalogue has models for five Turkic languages with UD data: Turkish, Kazakh, Kyrgyz, Uyghur and Ottoman Turkish (later catalogues add Uzbek; Chapter 8); TurkicNLP adds Stanza models for Azerbaijani, Uzbek, Turkmen, Tatar, and Bashkir.

TurkicNLP exposes Stanza backends through its pipeline (Section 8.7). Check the catalogue for the requested language and processor: a parsing model does not imply that a dedicated NER model is available.

Use Stanza 1.11 (`pip install "turkicnlp[stanza]"` enforces `stanza>=1.8,<1.12`): the custom Azerbaijani, Uzbek, Turkmen, Tatar and Bashkir models were trained with it, and newer Stanza versions fail to load their parsers.

The measured Stanza baselines, including the caveat on the Kazakh replay score, are in Chapter 8. For other languages, TurkicNLP's multilingual Glot500 tagger and parser and its Apertium backend are the alternatives.

B.2.6 Zemberek-NLP (Turkish)

Zemberek (Akın and Akın 2007) is an open-source Java library for Turkish morphological analysis, disambiguation, tokenisation, spell-checking and normalisation (github.com/ahmetaa/zemberek-nlp); community Python wrappers exist. Verify the maintained implementation and evaluate it on the intended domain.

B.3 Datasets and Corpora

B.3.1 Parallel Corpora

Table B.1: Parallel corpora covering Turkic languages

Corpus	Languages	Size	URL
TIL corpus	22 Turkic languages	release-specific totals and direction counts in Chapter 10	github.com/turkic-interlingua/til-mt

Corpus	Languages	Size	URL
OPUS	Many, incl. all major Turkic	Varies by subcorpus	opus.nlpl.eu
FLORES-200	200 languages	997 dev; 1,012 devtest sentences	huggingface.co/datasets/facebook/flores

The **TIL corpus** (Mirzakhalov, Babu, Ataman, et al. 2021) provides parallel data for 22 Turkic languages; release-specific counts, directions and split-overlap cautions are discussed in Chapter 10. **FLORES-200** provides the same evaluation sentences for the eleven Turkic varieties in NLLB-200 and is one possible MT evaluation source, provided language coverage, contamination and domain fit are checked.

B.3.2 Monolingual Corpora

Table B.2: Monolingual corpora for Turkic languages

Corpus	Language	Approx. Size	URL
HPLT v3.0	many, incl. 11 Turkic subsets	per-language sizes in Section 4.1	hplt-project.org
FineWeb2	many, incl. Turkic	per-language sizes in Section 4.1	huggingface.co/datasets/HuggingFaceFW/fineweb-2
CulturaX	167 languages	per-language sizes in Section 4.1	huggingface.co/datasets/uonlp/CulturaX
OSCAR	160+ languages incl. Turkic	release-specific	huggingface.co/datasets/oscar-corpus/OSCAR-2301
CC-100	100 languages	release-specific	data.statmt.org/cc-100
Turkish National Corpus	tur	50M words (2012 description)	Corpus description

The **Turkish National Corpus (TNC)** (Aksan et al. 2012) was designed as a balanced, general-purpose corpus of 50 million words. This is the size reported in the 2012 description. **HPLT**, **FineWeb2** and **CulturaX** are automatically labelled web collections. Their per-language sizes differ substantially; Chapter 4 compares their releases and explains how to audit a subset before use.

B.3.3 Treebanks

UD 2.17 has treebanks for Turkish, Ottoman Turkish, Kazakh, Kyrgyz, Uzbek, Uyghur, Tatar, Azerbaijani, Sakha (UD: Yakut) and Old Turkic (UD: Old Turkish), plus Turkish–German and Turkish–English code-switching treebanks (Chapter 8, Table 8.5 and Table 8.6). They provide reference annotations for tokenisation, POS tagging, morphological features and dependency structure in CoNLL-U format. Annotation methods and quality vary by treebank; check the release documentation and the examples in Chapter 8.

- **Download**: universaldependencies.org (all treebanks, versioned releases)
- **Parallel UD-Turkic sentences**: `github.com/ud-turkic/parallel` (Akhundjanova et al. 2025)
- **Format**: CoNLL-U (tab-separated syntactic-word records, with additional records for multiword tokens and empty nodes)
- **Python loading**: `pip install conllu`

In the audited official UD 2.17 inventory, languages without a dedicated released treebank include Turkmen, Bashkir, Chuvash, Crimean Tatar, Gagauz, Karakalpak, Kumyk, Nogai, Karachay-Balkar, and all Siberian Turkic languages except Sakha.

B.3.4 NER Datasets

Table B.3: Named entity recognition datasets for Turkic languages

Dataset	Language	Entity Types	Source	Reference
KazNERD	Kazakh	25 classes; 112,702 sentences	github.com/IS2AI/KazNERD	Yeshpanov, Khassanov, and Varol (2022)
KyrgyzNER	Kyrgyz	27 classes; 7,033 training and 3,867 test sentences	github.com/Akyl-AI/KyrgyzNER	Turatali et al. (2025)
WikiANN	10+ Turkic	PER, ORG, LOC	Hub configuration per language	Pan et al. (2017)

KazNERD reports 112,702 sentences from Kazakh television news, with 136,333 annotations in 25 classes (Yeshpanov, Khassanov, and Varol 2022). It supplies the NER examples in Chapter 9. **WikiANN** derives silver-standard annotations from Wikipedia in many languages. Its original publication and later releases differ in coverage; use the pinned configurations in that chapter and assess label quality for the chosen language and task.

B.3.5 Benchmarks

Table B.4: Evaluation benchmarks for Turkic NLP

Benchmark	Languages	Tasks	Reference
TUMLU	9 listed; release caveat in Section 15.3	Native MCQs	Isbarov et al. (2025)
Kardeş-NLU	az, kk, ky, uz, ug	Cross-lingual NLU	Senel et al. (2024)
Mukayese	Turkish	Multiple NLP tasks	Safaya et al. (2022)
KazQAD	Kazakh	QA (reading comprehension)	Yeshpanov et al. (2024)
X-WMT	8 Turkic + en, ru	MT (88 directions)	Mirzakhalov, Babu, Ataman, et al. (2021)
KazMMLU	Kazakh, Russian	Native MCQs	Togmanov et al. (2025)
FLORES-200 / SIB-200	11 Turkic varieties	MT / topic classification	NLLB Team et al. (2024); Adelani et al. (2024)

TUMLU (Isbarov et al. 2025) lists Azerbaijani, Crimean Tatar, Karakalpak, Kazakh, Kyrgyz, Tatar, Turkish, Uyghur and Uzbek, with school questions in 11 subjects; its conflicting language counts require release-specific checks (Chapter 15). **Kardeş-NLU** (Senel et al. 2024) evaluates cross-lingual NLU for five Turkic languages with Turkish as a high-resource relative. **Mukayese** (Safaya et al. 2022) is a multi-task benchmark suite for Turkish.

B.3.6 Speech Datasets

Table B.5: Speech corpora for Turkic languages

Dataset	Language	Size	Source
KSC2	Kazakh	>1,000 hours	issai.nu.edu.kz (Mussakhojayeva, Khassanov, and Varol 2022a)
KazakhTTS2	Kazakh	~271 hours, 5 speakers	issai.nu.edu.kz
FLEURS	tr, az, kk, ky, uz, …	n-way parallel read speech	google/fleurs
KazEmoTTS	Kazakh	Emotional speech	issai.nu.edu.kz
Common Voice	Multiple Turkic	Per release; validated and training hours differ	Mozilla Data Collective; see Chapter 11

ISSAI (Nazarbayev University) (Mussakhojayeva et al. 2023; Mussakhojayeva, Khassanov, and Varol 2022b) has produced Kazakh speech resources including the KSC2 ASR corpus

and the KazakhTTS2 synthesis corpus. **Common Voice** has datasets for several Turkic languages whose sizes depend on volunteer activity; check the current release statistics.

B.4 Pretrained Models

B.4.1 Verified Configurations

Table B.6: Pinned configurations checked 15 October 2026. Counts are generated from `AutoModel` on the meta device, without pretrained weights; a task head changes the count. Availability is not a quality evaluation. Sources, revisions and licence metadata: `tables/tbl-verified-models/data.csv`.

Checkpoint	Role	Base parameters	Loaded class
dbmdz/bert-base-turkish-cased	Turkish encoder	110,617,344	`BertModel`
FacebookAI/xlm-roberta-base	Multilingual encoder	278,043,648	`XLMRobertaModel`
google/mt5-base	Multilingual encoder-decoder	390,315,264	`MT5Model`
openai/whisper-small	Speech encoder-decoder	241,734,912	`WhisperModel`
facebook/mms-tts-chv	Chuvash TTS	36,284,784	`VitsModel`
facebook/mms-tts-crh	Crimean Tatar TTS	36,285,744	`VitsModel`

The table counts base-model parameters from pinned configurations on a meta device, without downloading weights. BERTurk vocabulary variants have different embedding sizes; XLM-R base is not the 125M-parameter monolingual RoBERTa base. A task head changes the count. The CSV preserves model class, source URL, revision and licence metadata.

B.4.2 Additional Models

Glot500, NLLB, BERTbek, TURNA, KAZ-LLM and Sherkala are discussed with their publications and task results in Chapter 7, Chapter 10 and Chapter 12. Use each experiment's pinned checkpoint and verify any additional Hub release. Choose by measured performance and model purpose: NLLB is trained for translation, while base mT5 uses span-corruption pretraining. Instruction-following chat requires further adaptation or evaluation.

B.4.3 Speech Models

Table B.7: Speech models for Turkic languages

Model	Coverage	Source
Whisper large-v3	100 language tokens, including seven Turkic languages	openai/whisper-large-v3 configuration
TurkicASR	10 Turkic languages	github.com/IS2AI/TurkicASR
TurkicTTS	Kazakh-trained, zero-shot for other Turkic languages	github.com/IS2AI/TurkicTTS
Söyle	11 Turkic and other languages	github.com/IS2AI/Soyle
MMS	1,100+ languages	facebook/mms-1b-all
Omnilingual ASR	20 Turkic languages through TurkicNLP	via `turkicnlp[asr]`; see Chapter 11

Whisper (OpenAI) covers seven Turkic languages (tr, az, uz, kk, tt, tk, ba) but not Kyrgyz or Uyghur. **TurkicASR** (Mussakhojayeva et al. 2023) covers Azerbaijani, Bashkir, Chuvash, Kazakh, Kyrgyz, Sakha, Tatar, Turkish, Uyghur, and Uzbek; **TurkicTTS** (Yeshpanov, Mussakhojayeva, and Khassanov 2023) synthesises Turkic languages zero-shot from a Kazakh-trained model (Chapter 11).

B.5 Community and Infrastructure

The organisations behind most of these resources (Apertium, SIGTURK, ISSAI, Universal Dependencies, Turkic Interlingua, the UD-Turkic group, TurkLang and the Turkic Academy) are described in Section 1.5. Two practical points for contributors:

- **Universal Dependencies** publishes two releases a year; new treebanks are submitted through the UD GitHub organisation, and language-specific guidelines are in the UD documentation. For a language without a treebank, see Section 8.6.
- **Apertium** modules are separate GitHub repositories (`apertium-kaz`, `apertium-kir`, `apertium-nog`, …) documented on wiki.apertium.org; extending an existing module is the route described in Section 17.3.

B.6 Reproducing Results from This Book

B.6.1 Local Reproduction Manifest

The code is organised by chapter: `code/ch04-corpus/`, `code/ch05-ambiguity/`, `code/ch06-toy-fst/`, `code/ch07-neural/`, `code/ch09-ner/`, `code/ch10-mt/`, `code/ch11-speech/`, `code/ch12-classify/`, `code/ch13-retrieval/`, `code/ch14-generation/`, `code/`

`ch15-tumlu/`, `code/ch16-transfer/` and `code/ch17-resources/`. Shared helpers, the mechanical checks and the tested package versions are in `code/common/`, whose README documents inputs, commands, outputs and limits for classification, speech, TUMLU, corpus auditing, retrieval, transfer and resource observations. The exercise notebooks are in `exercises/`.

Run `python code/common/check.py` in the recorded test environment. This checks fixtures, tiny models and artefact reload; it does not reproduce pretrained-model task accuracy. Full data/model downloads, dataset access conditions and independent human labels are separate prerequisites. Code-block extraction alone is not an adequate end-to-end reproduction test.

B.6.2 Setting Up Chapter by Chapter

The reading routes in the Preface give the order of chapters. Install only the extras a chapter uses, and keep the speech environment separate.

C Turkic Language Reference

> **What This Appendix Covers**
>
> This appendix separates language identifiers, writing systems and computational evidence: identifiers and scope (Section C.1), the evidence needed before calling a resource available (Section C.2), a script reference (Section C.3) and the typological properties the chapters use (Section C.4).

The language table lists **23** contemporary languages. It differs from the 24 entries of TurkicNLP 0.5.0 (Chapter 1): the table adds Shor (`cjs`) and Chulym (`clc`), which the toolkit does not cover, and leaves out the toolkit's historical Ottoman Turkish (`ota`) and Old Turkic (`otk`) and its separate South Azerbaijani entry (`azb`). For toolkit coverage, use its versioned registry; for corpus or model labels, use a resource-specific crosswalk.

C.1 Language Identifiers and Scope

Table C.1: Selected contemporary languages in this appendix; 23 rows, distinct from toolkit coverage. Groupings summarise the book's linguistic discussion; script examples are not an exhaustive statement of legal status. Population estimates and unverified Glottocodes are omitted.

Language	ISO 639-3	Descriptive grouping	Script examples
Turkish	tur	Oghuz	Latin
Azerbaijani	aze	Oghuz	Latin / Cyrillic / Arabic (variety-dependent)
Turkmen	tuk	Oghuz	Latin
Gagauz	gag	Oghuz	Latin
Kazakh	kaz	Kipchak	Cyrillic; versioned Latin schemes
Kyrgyz	kir	Kipchak	Cyrillic
Tatar	tat	Kipchak	Cyrillic
Bashkir	bak	Kipchak	Cyrillic
Kumyk	kum	Kipchak	Cyrillic
Nogai	nog	Kipchak	Cyrillic
Karachay-Balkar	krc	Kipchak	Cyrillic
Crimean Tatar	crh	Kipchak	Latin / Cyrillic
Karakalpak	kaa	Kipchak	Latin
Uzbek	uzb	Karluk	Latin / Cyrillic
Uyghur	uig	Karluk	Perso-Arabic

Language	ISO 639-3	Descriptive grouping	Script examples
Sakha (Yakut)	sah	Siberian (Northern)	Cyrillic
Tuvan	tyv	Siberian (Southern)	Cyrillic
Khakas	kjh	Siberian (Southern)	Cyrillic
Southern Altai	alt	Kipchak (Eastern)	Cyrillic
Shor	cjs	Siberian (Southern)	Cyrillic
Chulym	clc	Siberian (Southern)	Cyrillic
Chuvash	chv	Oghur	Cyrillic
Khalaj	klj	Arghu	Arabic / Latin

`aze` (Azerbaijani) and `uzb` (Uzbek) are macrolanguage identifiers; resources may instead use North/South Azerbaijani (`azj`/`azb`) or Northern/Southern Uzbek (`uzn`/`uzs`). `alt` denotes Southern Altai, not every variety called Altai. Do not join corpus labels, Wikipedia editions, Hub tags and toolkit codes by string substitution. Record a many-to-one or one-to-many mapping explicitly.

Speaker estimates need a source table or page, reference year, geography and a definition such as first language, self-reported ability or total speakers. Estimates with different definitions cannot support a single ranking, so this table omits them. Glottocodes also await individual verification. Consult the ISO 639-3 registry and Glottolog for identifiers. The codes here were checked against an installed ISO dataset, rather than a fresh registry snapshot.

C.2 NLP Resource Evidence

Table C.2: Resource evidence must retain task, source, release and evaluation status. Unknown and search failure are not “No”.

Question	Versioned evidence	What it does not establish
Is a treebank officially released?	UD 2.17 inventory in Chapter 8	A local silver file is not a released gold treebank
Is an analyser registered?	Toolkit/Apertium module and revision in Chapter 6	Nonempty features are not accuracy
Are model weights released?	Pinned entries in Table B.6	Availability is not target-domain quality
Is evaluation available?	Dataset release, labels and splits in Chapter 9 through Chapter 15	Software support is not benchmark coverage

MMS, for example, releases Chuvash and Crimean Tatar TTS checkpoints (Chapter 11); their suitability for production needs separate testing. The five-language audit in Section 17.6 records availability, unknowns and failed queries separately. Verify model architecture before assigning a label such as “BERT-class”, and retain each resource’s licence and task type.

C.3 Script Reference

C.3.1 Latin Characters and Letter Marks

Turkish dotted İ/i and dotless I/ı need locale-aware casing. Azerbaijani Latin ə/Ə (U+0259/U+018F) must not be confused with Cyrillic ə/Ə (U+04D9/U+04D8). Similar appearance does not imply identical code points or script.

Uzbek *o‘/g‘* and the tutuq sign appear with several digital apostrophe-like characters. Choose a declared encoding convention for a derived copy, retaining the original; using U+0027 does not by itself prove that every tokeniser will segment incorrectly. The context-sensitive search-key helper in Section 4.3 only normalises *o/g* letter marks and leaves other punctuation alone.

Kazakh Latin schemes are versioned proposals and orthographic conventions, not interchangeable transliterations. Use the dated sources in Chapter 3 and record the chosen converter instead of combining apostrophe, acute and later diacritic inventories into one “current alphabet”.

C.3.2 Cyrillic Characters

Table C.3: Selected Cyrillic characters, checked against Unicode names and code points by the generator. These are encoding identities, not cross-language pronunciation claims.

Lowercase	Uppercase	Code points (lower / upper)	Unicode name
ҥ	Ҥ	U+04A3 / U+04A2	CYRILLIC SMALL LETTER EN WITH DESCENDER
ү	Ү	U+04AF / U+04AE	CYRILLIC SMALL LETTER STRAIGHT U
ұ	Ұ	U+04B1 / U+04B0	CYRILLIC SMALL LETTER STRAIGHT U WITH STROKE
ә	Ә	U+04D9 / U+04D8	CYRILLIC SMALL LETTER SCHWA
ғ	Ғ	U+0493 / U+0492	CYRILLIC SMALL LETTER GHE WITH STROKE
қ	Қ	U+049B / U+049A	CYRILLIC SMALL LETTER KA WITH DESCENDER
ө	Ө	U+04E9 / U+04E8	CYRILLIC SMALL LETTER BARRED O
h	Һ	U+04BB / U+04BA	CYRILLIC SMALL LETTER SHHA

Lowercase	Uppercase	Code points (lower / upper)	Unicode name
ҙ	Ҙ	U+0499 / U+0498	CYRILLIC SMALL LETTER ZE WITH DESCENDER
ҫ	Ҫ	U+04AB / U+04AA	CYRILLIC SMALL LETTER ES WITH DESCENDER
ҥ	Ҥ	U+04A5 / U+04A4	CYRILLIC SMALL LIGATURE EN GHE

The table gives no single IPA value per character: pronunciation and phonemic analysis depend on the language. Kazakh ҚҒ are not letters of the Kyrgyz alphabet. Chapter 2's source-specific vowel table and Chapter 5's harmony descriptions provide the linguistic context.

C.3.3 Perso-Arabic Uyghur

Distinguish vowel letters from sequences that include an initial hamza carrier; strings such as ئو contain more than one Unicode code point. Preserve right-to-left text logically, including joining behaviour and meaningful controls. Inspect NFC, code points and rendered output separately. Removing every invisible control as "noise" can damage text. Conversion to ULY or a Cyrillic convention needs a declared scheme and loss report (Chapter 3). The recurring Uyghur sentence (Table 1.4) shows one text in three views: the original, the treebank's ULY transliteration and the toolkit's CTS rendering (Section 3.6).

C.4 Typology and Computational Interpretation

Table C.4: Typological interpretation replaces unsupported numerical rankings.

Property	Interpretation for this book
Vowel inventories and harmony	Use source-specific analyses in Chapter 2 and Chapter 5; avoid a binary strict/partial scale
Constituent order	SOV is the usual base order; contact varieties and discourse patterns require separate treatment (Section 8.1)
Null subjects	Grammar and context may license an absent subject; agreement is not gender marking
Case and morphology	Specify the grammatical/annotation scheme instead of ranking languages by raw case counts
Morphological density	Measure a defined corpus statistic; "high/moderate" is not an observed morpheme ratio

These properties suggest what to test; they guarantee no particular NLP outcome. SOV order does not by itself imply longer dependencies, and morphological richness alone does not explain every model error. See Chapter 5 and Chapter 8 for constructions, annotation choices and controlled comparisons (Johanson 2021).

D Glossary

This glossary defines technical terms used throughout the book. Entries are listed alphabetically. *See:* names the chapter or section where a term is explained in depth; *See also:* points to related entries. Gloss abbreviations come first.

Gloss Abbreviations

Interlinear glosses follow the Leipzig Glossing Rules: hyphens separate morphs and their glosses, and a dot joins several meanings expressed by one form (*POSS.3SG*). Numbers mark person (1, 2, 3).

Table D.1: Abbreviations used in the book's interlinear glosses.

Label	Meaning	Label	Meaning
ABIL	ability	LOC	locative
ABL	ablative	NEG	negation
ACC	accusative	NMLZ	nominaliser
ATTR	attributive (Kazakh *-ГІ*, Uyghur *-diki*)	NOM	nominative
AUX	auxiliary	PASS	passive
CAUS	causative	PL	plural
COM	comitative	POSS	possessive
COP	copula	PROG	progressive
CVB	converb	PRS	present
DAT	dative	PST	past
GEN	genitive	PTCP	participle
INS	instrumental	Q	question particle
		SG	singular

A

Ablative case A grammatical case marking a noun as the source, origin, or point of departure of an action (e.g., "from the city"). In Turkic languages the ablative also encodes comparison ("bigger than the city"). It is marked with *-dan/-den/-tan/-ten* in Turkish and *-дан/-ден*, *-ман/-мен*, *-нан/-нен* in Kazakh. *See also: Case (grammatical), Dative case.*

Accusative case A grammatical case that marks the direct object of a transitive verb. Many Turkic languages have differential object marking. In Turkish, specific objects, including specific indefinites, usually take an overt accusative suffix (*-(y)ı/i/u/ü*), while non-specific objects usually appear unmarked. The conditions vary across languages (Section 5.4). *See also: Case (grammatical), Nominative case.*

Agglutination A morphological strategy in which words are formed by appending a sequence of largely segmentable morphemes, often with identifiable grammatical or derivational functions. Surface shapes can alternate, and form–function mappings are not always one-to-one. Turkic languages are strongly agglutinative: a single verb form may carry tense, aspect, mood, and person agreement through a chain of suffixes. *See also: Concatenative morphology, Productive morphology, Suffix template.*

Allomorph One of two or more phonological variants of the same morpheme. In Turkic languages, allomorphy is principally driven by vowel harmony and consonant assimilation. For example, the Turkish plural morpheme has the allomorphs *-lar* and *-ler* depending on the backness of the preceding vowel. *See also: Vowel harmony, Morpheme.*

Adapter A small set of trainable parameters inserted into a frozen pretrained model and trained for a task, language or script. Bottleneck adapters and LoRA are different parameterisations; report the trainable modules and save the adapter with its task head. *See:* Section 16.4.

Annotation projection Transferring labels from annotated source sentences to aligned target sentences through word alignments. The result is silver annotation; measure it against gold labels on the same target sentences and report unaligned words. *See:* Section 8.5.

Annotator agreement (Kappa) A statistical measure of the degree to which two or more human annotators assign the same label to the same item, corrected for chance agreement. Cohen's kappa and Fleiss's kappa are the most common variants. Interpretation depends on the task, label prevalence and chance model; report raw agreement and label counts as well. There is no universal adequacy threshold. *See also: Inter-annotator agreement.*

Attention mechanism A mechanism that combines information from permitted positions using weights computed from their compatibility with a query. Self-attention relates positions within one sequence; cross-attention draws on another sequence, such as encoder outputs. The transformer uses scaled dot-product attention to combine value vectors, with masks controlling which positions are available (Vaswani et al. 2017). *See also: Multi-head attention, Encoder-decoder.*

B

Back-translation A data augmentation technique for machine translation in which a target-language monolingual corpus is translated into the source language by a reverse MT system, generating synthetic source-target sentence pairs for training. Its value for a Turkic pair is an empirical question that depends on the reverse system's quality (Chapter 10). *See also: Transfer learning.*

BIO tagging A sequence labelling scheme used for span-level tasks such as named entity recognition. Each token is assigned a tag indicating whether it is at the Beginning

of a span (B-), Inside a span (I-), or Outside any span (O). Extended variants include BIOES (End, Single) and BILOU. *See also: Named entity recognition (NER), POS tagging.*

BLEU A machine-translation metric combining modified token n-gram precision with a brevity penalty against reference translations. It is sensitive to tokenisation and to alternative word forms, which matters for agglutinative languages; report it with its SacreBLEU signature. *See also:* chrF. *See:* Section 10.3.

Byte-Pair Encoding (BPE) A subword tokenisation algorithm that iteratively merges the most frequent adjacent pair of symbols in the training representation (characters or bytes, depending on the tokeniser), producing a vocabulary of variable-length subword units. Originally proposed for data compression and adapted for NLP by Sennrich, Haddow, and Birch (2016b). BPE is used by many Turkic language models and MT systems. *See also: Subword tokenisation, Fertility (tokeniser), SentencePiece.*

C

Calibration The correspondence between predicted confidence and observed outcome frequency under a specified evaluation distribution. Low entropy is not evidence of good calibration.

CARE principles Collective benefit, Authority to control, Responsibility and Ethics: principles for Indigenous data governance (Carroll et al. 2020). They pose questions for collaboration with language communities rather than a checklist that settles them. *See:* Section 17.4.

Case (grammatical) A grammatical category that encodes the syntactic or semantic role of a noun phrase in a clause. Turkic languages use suffixal case marking; case inventories depend on the language and grammatical analysis; see Chapter 5. *See also: Nominative case, Accusative case, Dative case, Genitive case, Ablative case.*

chrF A machine-translation metric based on character n-gram overlap with reference translations; chrF++ adds word n-grams. It is less sensitive than BLEU to suffix variation, but a meaning-changing suffix such as negation can leave it almost unchanged. *See also:* BLEU. *See:* Section 10.3 and Section 15.2.

Code-switching The alternation between two or more languages within a conversation, sentence, or word, e.g. Kazakh–Russian or Turkish–German mixing. *See:* Section 16.6.

Common Turkic Alphabet and Common Turkic Script (CTS) A 34-letter Latin alphabet proposed in 1991; a revised version was agreed in 2024 by the Turkic World Common Alphabet Commission as a recommendation to the member states. In this book, CTS names TurkicNLP's versioned common-script mapping, used as a shared representation for text processing. *See:* Section 3.6.

Complementiser A functional morpheme that introduces a complement clause. Turkic languages often express complementiser-like functions through nominalised or participial verb forms. Finite subordination also uses elements such as Turkish *ki* and *diye*.

Concatenative morphology Building forms by joining morphemes in a linear sequence. Their surface shapes may alternate. Turkic languages rely heavily on suffixation, alongside alternations, reduplication and other processes. *See also: Agglutination.*

CoNLL-U The tab-separated file format of Universal Dependencies: one word per line with ID, form, lemma, UPOS, XPOS, features, head, relation, enhanced dependencies and miscellaneous fields. Range IDs mark multiword tokens; decimal-style IDs mark empty nodes in enhanced dependencies. *See:* Section 8.2.

Contamination (benchmark) Evaluation items, or close variants of them, appearing in a model's training data, so that a score can reflect exposure rather than ability. High scores are not proof of it; an independent check is needed. *See:* Section 15.3. *See also:* Data leakage.

Contextual embedding A vector representation of a word or token that is computed in the context of the surrounding sentence, as opposed to a fixed lookup-table embedding. Contextual embeddings are the output of each layer of a transformer encoder; they capture disambiguation information that static word vectors cannot represent. *See also: Masked language model (MLM), Fine-tuning.*

Converb A non-finite verb form used in adverbial clauses, for example to express time or manner. Turkic converbs also occur in clause chains and auxiliary constructions; their syntax depends on the form and construction. *See:* Section 2.5.

Cross-entropy A loss function used to train classification and language models, measuring the expected negative log probability assigned by a model Q to outcomes distributed as P: $H(P, Q) = H(P) + D_{KL}(P \| Q)$. It is not itself a divergence. In language modelling, cross-entropy over a test set can be converted to perplexity by exponentiation with the matching log base (Section A.2).

D

Data leakage Test data influencing model training, fitted preprocessing or model selection, so that the test evaluation is no longer independent. Examples include duplicates across splits, translated test items in training, and fitting a tokeniser on the test corpus. Applying a fixed preprocessing rule to test input is ordinary inference.

Dative case A grammatical case marking the indirect object or goal of an action (e.g., "to the city"). Turkic dative suffixes take the form *-(y)a/-(y)e* in Turkish and *-ғa/-ге/-қа/-ке* in Kazakh, with allomorphs selected by vowel harmony and by the stem-final consonant. *See also: Case (grammatical), Ablative case.*

Dependency parsing A syntactic analysis task that assigns a head word to each word in a sentence and labels the directed arc between them with a grammatical relation (subject, object, etc.). The Universal Dependencies framework provides a cross-linguistically consistent annotation scheme used across the Turkic UD treebanks. *See:* Chapter 8.

Derivational morphology The branch of morphology concerned with forming new lexical items from existing ones, typically by adding derivational affixes that change the word's part of speech or core meaning (e.g., Turkish *göz* 'eye' → *göz-lük* 'glasses'). Turkic languages have extensive derivational morphology. *See also: Inflectional morphology, Agglutination.*

Disambiguation (morphological) The task of selecting the correct morphological analysis for a word form given its sentential context, when multiple analyses are possible. Agglutinative languages like Turkish and Kazakh frequently produce highly ambigu-

ous surface forms, making disambiguation a non-trivial component of NLP pipelines. *See also: Morphological analysis, FST (finite-state transducer).*

Distribution shift A mismatch between the statistical distribution of the data used to train a model and the data it is applied to at inference time. For Turkic NLP, distribution shift is common when models trained on literary or news text are applied to social media, SMS, or spoken transcripts.

Dropout A regularisation technique for neural networks in which a random subset of activations is set to zero during training. It can reduce co-adaptation between units; its effect on generalisation must be evaluated.

E

Embedding A learned vector representation of a token, word, sentence or other object. Vectors of equal dimension need not be semantically aligned across languages.

Encoder-decoder A neural architecture consisting of an encoder that maps an input sequence to a latent representation and a decoder that generates an output sequence from that representation. The architecture is standard for sequence-to-sequence tasks including machine translation, text summarisation, and morphological generation. *See also: Attention mechanism, Masked language model (MLM).*

Evidential A grammatical category marking the source of information, such as indirect or reported knowledge. Turkish *-mIş* has evidential readings (UD `Evident=Nfh`); translation must choose how to render them. *See:* Section 5.1.

F

Fertility (tokeniser) The number of subword tokens divided by the number of words under a declared word definition and tokenisation protocol. Corpus-total ratios differ from averages of sentence ratios. Report whether words were encoded separately or as running text, and whether special tokens count. High fertility describes fragmentation; neither its cause nor the benefit of script conversion follows from it. *See also: Byte-Pair Encoding (BPE), Subword tokenisation.*

Fine-tuning The process of continuing the training of a pretrained model on additional data for a task or language objective, with or without manual labels, updating all or a subset of model parameters. Fine-tuning is the dominant paradigm for adapting BERT-class models to downstream Turkic NLP tasks. *See also: Transfer learning, LoRA, Parameter-efficient fine-tuning (PEFT).*

FST (finite-state transducer) A computational device that maps one string to another through a finite set of states and transitions, with transitions that may consume or emit epsilon (the empty string). It can define multiple outputs per input and attach weights to paths. FSTs are the core technology of rule-based Turkic analysers such as the Apertium modules and TRmorph. *See:* Chapter 6.

G

Genitive case A grammatical case marking possession or modification. In Turkic languages, the genitive-possessive construction is a two-part structure: the possessor noun takes a genitive suffix and the possessed noun takes a possessive agreement suffix cross-referencing the possessor's person and number. *See also: Case (grammatical).*

GlotLID An open language identifier with more than 2,000 language-and-script labels, used by TurkicNLP (Kargaran et al. 2023). It can confuse closely related languages, especially in short texts. Evaluate its accuracy for the intended languages and input lengths. *See:* Section 3.7.

I

Inflectional morphology The branch of morphology concerned with grammatical forms of a lexical item, expressing categories such as tense, aspect, mood, number, person and case. The boundary between inflection and derivation depends partly on the analysis. Turkic languages have rich inflectional paradigms, particularly on verbs and nouns. *See also: Derivational morphology, Agglutination.*

Inter-annotator agreement The degree to which independent annotators produce consistent labels for the same data. *See: Annotator agreement (Kappa).*

L

Labial harmony (rounding harmony) A subtype of vowel harmony in which the feature [round] spreads from a vowel to subsequent vowels in the word. Turkish rounds high suffix vowels; Kyrgyz also rounds low vowels and writes it, while Kazakh rounding is pronounced but not written (Section 2.2). *See also: Vowel harmony, Palatal harmony.*

Language identification (LID) Automatically determining the language of a text. Closely related Turkic languages written in the same script are frequently confused with each other. *See:* Section 3.7.

Lemma The canonical dictionary form of a word, abstracting away from inflectional variation. The citation convention depends on the resource: a verbal stem and an infinitive may be different lemma conventions. Match the dataset's scheme before scoring. *See also: Morphological analysis, Inflectional morphology.*

Light-verb construction A complex predicate formed by a nominal and a semantically light verb, such as Turkish *yardım et-* 'help' (lit. 'do help'). UD labels the relation `compound:lvc`, but released Turkic treebanks differ in which word is the head. *See:* Section 8.2.

LoRA (Low-Rank Adaptation) A parameter-efficient fine-tuning method that learns an additive weight update as the product of two smaller matrices while keeping the original weight matrix frozen. The chosen rank and adapted modules determine the number of trainable parameters. QLoRA combines it with a quantised base model.

The book's LoRA results are in Chapter 12. *See also: Parameter-efficient fine-tuning (PEFT), Fine-tuning.*

M

Macro-F1 The unweighted mean of per-class F1 scores, so that small classes count as much as large ones; micro-F1 pools all decisions instead. State which one is reported, and for NER whether spans are scored strictly. *See:* Section 9.2.

Masked language model (MLM) A pretraining objective in which selected input positions are prediction targets; depending on the recipe, they may be masked, replaced or left unchanged. The model predicts their original tokens. MLM is the pretraining objective of BERT-class models such as BERTurk, XLM-R and Glot500 (Chapter 12). *See also: Contextual embedding, Fine-tuning.*

Morpheme The minimal unit of meaning or grammatical function. Roots and affixes are both morphemes. Turkic affixation is predominantly suffixal, and a single word may contain five or more morphemes. *See also: Agglutination, Allomorph.*

Morphological analysis Identifying a word form's lemma, part of speech and morphological features, often with alternative readings. Some analysers also provide morpheme segmentation; a UD feature bundle does not itself specify morpheme boundaries. Explicit analysis can support downstream tasks, but end-to-end models need not call a separate analyser. *See also: FST (finite-state transducer), Disambiguation (morphological).*

Morphological generation The inverse of morphological analysis: producing a surface word form from a lemma and a specification of morphological features. An explicit generator is useful in rule-based MT and controlled form production; neural MT and TTS need not contain a separate morphological generator. *See also: Encoder-decoder, FST (finite-state transducer).*

Multi-head attention An extension of the attention mechanism in which multiple attention functions are computed in parallel on linearly projected versions of the input, allowing the model to attend to different aspects of the input simultaneously. The transformer's self-attention layers use multi-head attention. *See also: Attention mechanism.*

N

Named entity recognition (NER) A sequence labelling task that identifies and classifies mentions of named entities (persons, organisations, locations, etc.) in text. Turkic NER resources include KazNERD (Kazakh, 25 entity classes), KyrgyzNER and WikiANN (multilingual silver-standard NER). *See:* Section 9.2.

Nominative case The default, unmarked case in Turkic noun paradigms, used for subjects of sentences and predicate nominals. Because the nominative is often zero-marked, it doubles as the citation form of nouns. *See also: Case (grammatical), Accusative case.*

O

Out-of-vocabulary (OOV) A word or other unit absent from a declared vocabulary. A surface form can be unseen even when its lemma is familiar. Productive morphology creates many possible forms, but observed OOV rates also depend on corpus size, domain and tokenisation. Distinguish unseen word forms from unknown subword tokens (Section 15.2). *See also: Subword tokenisation, Fertility (tokeniser).*

P

Palatal harmony A subtype of vowel harmony in which the feature [back] (or [front]) spreads within a defined harmonic domain. Which vowels participate and which suffixes alternate depends on the language, construction and exceptions. Palatal harmony is the most pervasive form of vowel harmony in Turkic, present in Turkish, Kazakh, Kyrgyz, and most other members of the family. *See also: Vowel harmony, Labial harmony.*

Parameter-efficient fine-tuning (PEFT) A family of methods for adapting pretrained language models to new tasks or languages by updating only a small subset of parameters or adding a small number of new parameters, leaving the bulk of the pretrained weights frozen. They reduce the memory and storage needed to adapt large models; Chapter 12 compares LoRA with full fine-tuning. *See also: LoRA, Fine-tuning.*

Participial clause A clause built around a participial verb form, with a syntactic role that depends on the language and construction. Turkic languages commonly place participial relatives before a noun, alongside finite and other relative strategies. Participial clauses can also serve other functions. *See also: Converb, Dependency parsing.*

Pivot translation Translating through an intermediate language (source → pivot → target) when a direct system or data are lacking. Each leg must be supported, and errors can accumulate or be repaired along the way; compare it with direct translation on the same test set. *See:* Section 10.4.

POS tagging The task of assigning a part-of-speech label to each token in a text. Universal Dependencies defines a set of 17 universal POS tags (UPOS) used across all Turkic treebanks. *See also: UPOS (Universal POS), Dependency parsing, BIO tagging.*

Pro-drop A grammatical property of languages in which overt pronoun subjects are omitted when they can be recovered from verb morphology or context. Null subjects are common in Turkic; their licensing depends on grammar and discourse. Person/number agreement can help identify a subject, but Turkic subject agreement does not encode grammatical gender. *See also: SOV.*

Productive morphology Morphological processes that apply freely to new or nonce words, generating valid forms according to the rules of the language. Turkic inflection and derivation can produce many forms absent from a training corpus. Productivity varies by process and does not mean that any suffix can attach to any stem. *See also: Agglutination, Inflectional morphology.*

Proxy language A language selected as a model routing code, language embedding or transfer source for a target with limited task supervision. This operational choice does not reveal an internal representation learned by the model. *See:* Chapter 16; also Chapter 7 and Chapter 8.

R

Retrieval-augmented generation (RAG) Generation conditioned on retrieved evidence. Evaluate retrieval relevance, answer correctness, citation support, language fidelity and abstention separately.

Romanisation The representation of a text written in a non-Latin script using Latin characters. For Turkic NLP, transliteration arises in Cyrillic-to-Latin conversion (Kazakh, Uzbek) and Arabic-to-Latin conversion (Uyghur, Ottoman Turkish). *See:* Section 3.4.

Round trip (transliteration) Converting text into another script and back and checking whether the original is recovered. It tests reversibility, not correctness of the intermediate text: a lossy mapping fails it, and a consistent error can pass it. *See:* Section 3.4.

S

Scrambling (word order) The syntactic phenomenon in which arguments of a verb can appear in multiple linear orders without changing the basic propositional meaning, though information structure (topic, focus) typically changes. Turkic languages allow considerable word-order variation around an SOV base; this flexibility complicates dependency parsing and semantic role labelling. *See also: SOV, Pro-drop.*

SentencePiece A tokeniser framework supporting BPE and unigram language-model algorithms. It is not a synonym for the unigram algorithm. Record the algorithm and normalisation configuration.

Silver data Automatically produced annotation, for example from a tagger or annotation projection. It may supplement manually checked gold data or be used where gold training data are unavailable. *See:* Section 8.5.

SOV Subject-Object-Verb: a typological label for languages in which the canonical order of major clause constituents places the subject first, the object second, and the verb last. SOV is the usual base order in Turkic, with contact-induced exceptions as well as discourse and construction effects (Section 8.1). *See also:* Scrambling (word order).

Subword tokenisation A tokenisation strategy whose vocabulary can contain whole words, word parts and individual characters or bytes, depending on the method. BPE, WordPiece and unigram language-model tokenisation can represent unseen words as sequences of known units. Unknown-token behaviour depends on character coverage and fallback rules. Subword tokenisation does not change whether a whole word was seen in training (Section 15.2). *See also: Byte-Pair Encoding (BPE), Fertility (tokeniser), Out-of-vocabulary (OOV).*

Suffix template The ordered slot-structure specifying the sequence of morphological categories that can be realised as suffixes on a word stem. A common Turkic verbal order is stem → derivation → voice → ability → negation → tense/aspect/mood → person agreement (*konuş-tur-a-ma-dı* 'could not make speak', Section 2.1). Templates are construction-specific generalisations, not fixed slots in every word (Section 5.1). *See also: Agglutination, FST (finite-state transducer).*

T

Temperature sampling Sampling training languages with probability proportional to $n_i^{1/T}$, where n_i is the data size of language i. $T = 1$ follows the data sizes, larger T moves towards uniform; small languages then repeat their few examples more often. *See:* Section 16.4.

Transfer learning A machine learning paradigm in which a model trained on one task or dataset is adapted to a different (often related) task or dataset, exploiting shared structure. In Turkic NLP, cross-lingual transfer from well-resourced languages (Turkish, Kazakh) to lower-resourced relatives (Kyrgyz, Uzbek) is a primary strategy for building systems with limited annotated data. *See also: Fine-tuning, Zero-shot transfer.*

Transliteration Conversion between writing systems under a declared mapping. Romanisation specifically targets Latin script. Mappings may be lossy or context-sensitive; preserve originals and offsets.

Treebank A corpus of sentences annotated with syntactic structure, such as constituency trees or dependency graphs. The release-specific inventory in Chapter 8 lists the Turkic UD resources, including the UD-Turkic parallel treebanks. *See also: Dependency parsing, UPOS (Universal POS).*

U

UPOS (Universal POS) The set of 17 coarse-grained, cross-linguistically applicable part-of-speech tags defined by the Universal Dependencies framework: `NOUN`, `VERB`, `ADJ`, `ADV`, `PRON`, `DET`, `ADP`, `AUX`, `CCONJ`, `SCONJ`, `PART`, `INTJ`, `NUM`, `PUNCT`, `SYM`, `X`, and `PROPN`. *See also: POS tagging, Treebank.*

V

Vowel harmony A phonological process in which participating vowels within a harmonic domain are constrained by one or more phonological features, most commonly frontness/backness (palatal harmony) and/or rounding (labial harmony). Harmony is widespread in Turkic, but its domains, strength, exceptions and spelling vary; standard written Uzbek has largely fixed suffix vowels (Section 2.2). *See also: Palatal harmony, Labial harmony, Allomorph.*

Z

Zero-shot transfer The application of a model trained on one language (or task) to a different language (or task) without any task-specific fine-tuning on the target language. The model may still have seen target-language text during pretraining (Section 16.1); zero-shot here means no target task-labelled training. *See also: Transfer learning, Fine-tuning.*